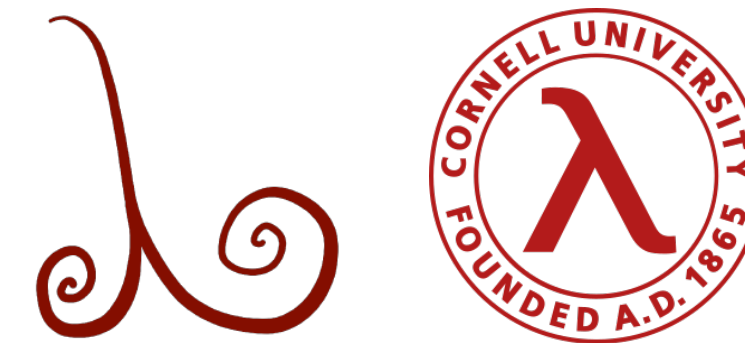




Capra lab @ Cornell

# Specifying Hardware Communication as Programs

Ernest Ng, Nikil Shyamsunder, Francis Pham, Adrian Sampson, Kevin Laeuffer

MIT FLAME Lab Seminar  
July 8, 2026

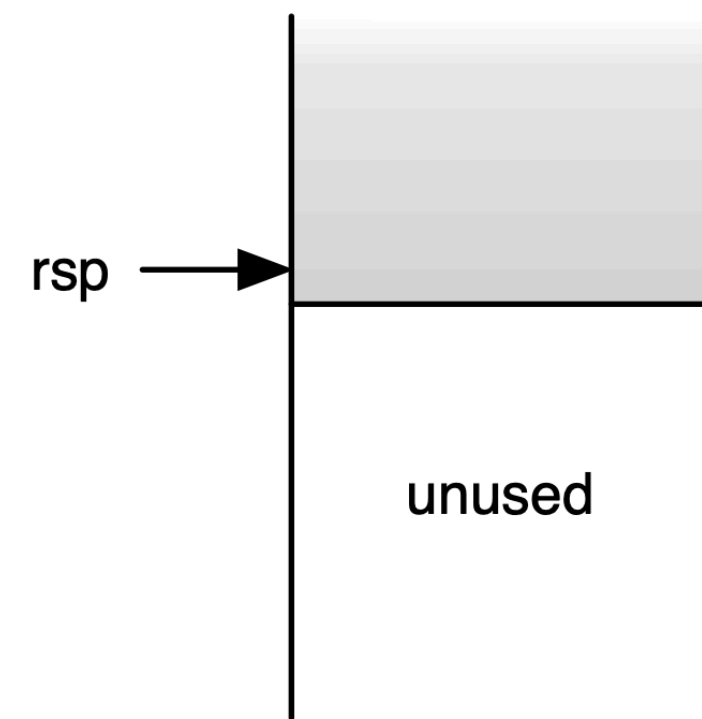
# Calling Conventions in Software

Standardized contract about how to invoke functions

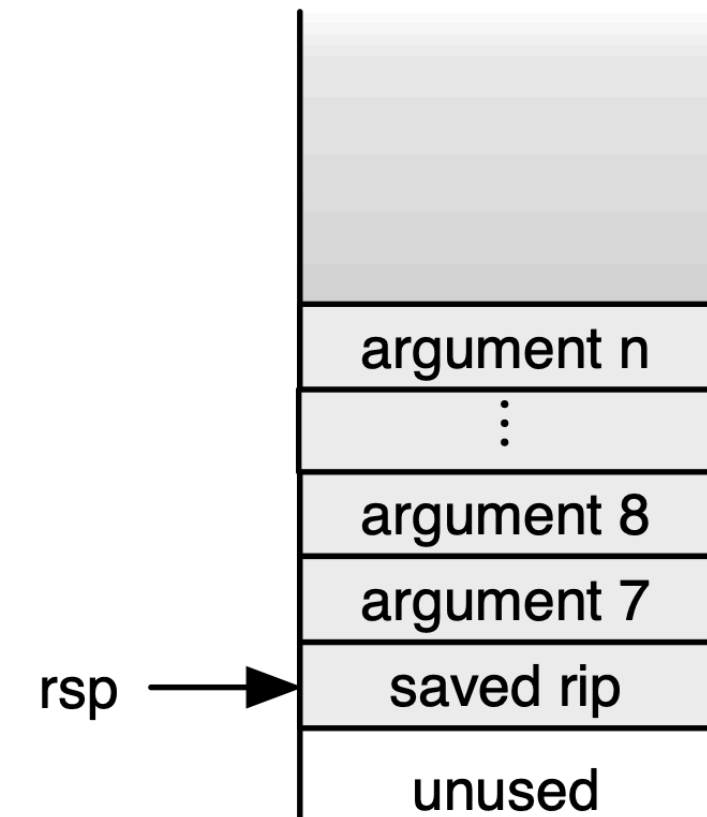


# Calling Conventions in Software

Given the calling convention,  
there is exactly one way to invoke a function



*before function call*



*immediately after  
call instruction*

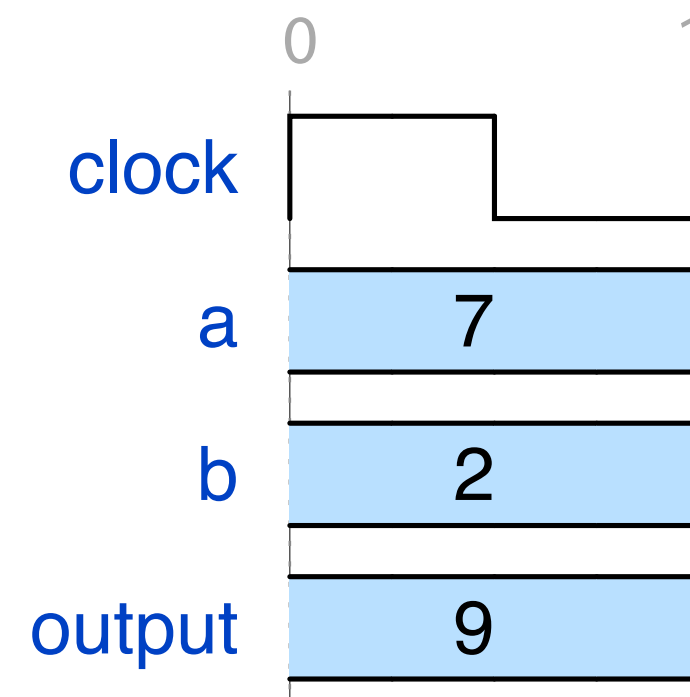
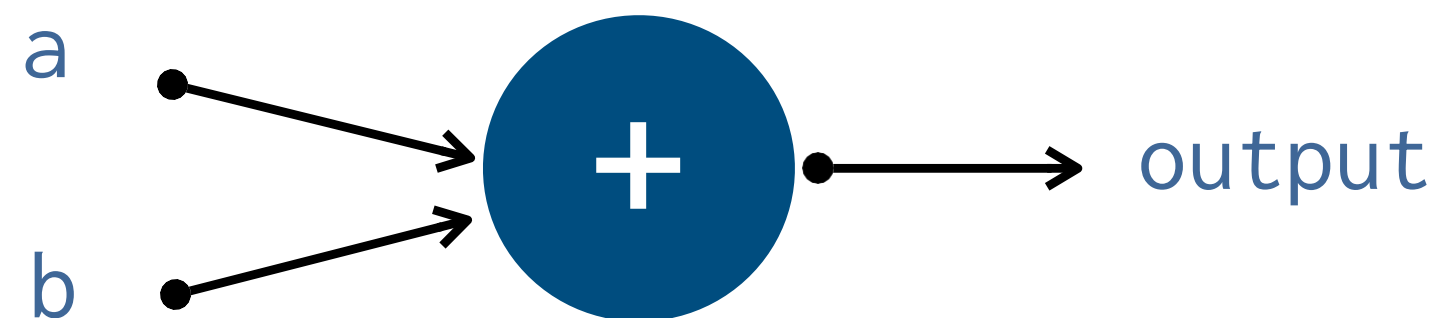
# Calling Conventions in *Hardware*?

# Calling Conventions in *Hardware*?

**There is no one way to “call” a HW module!**

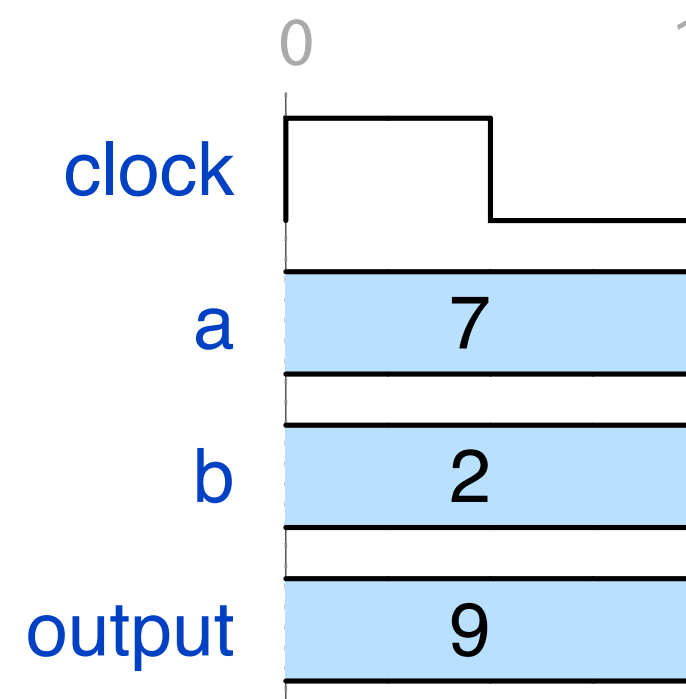
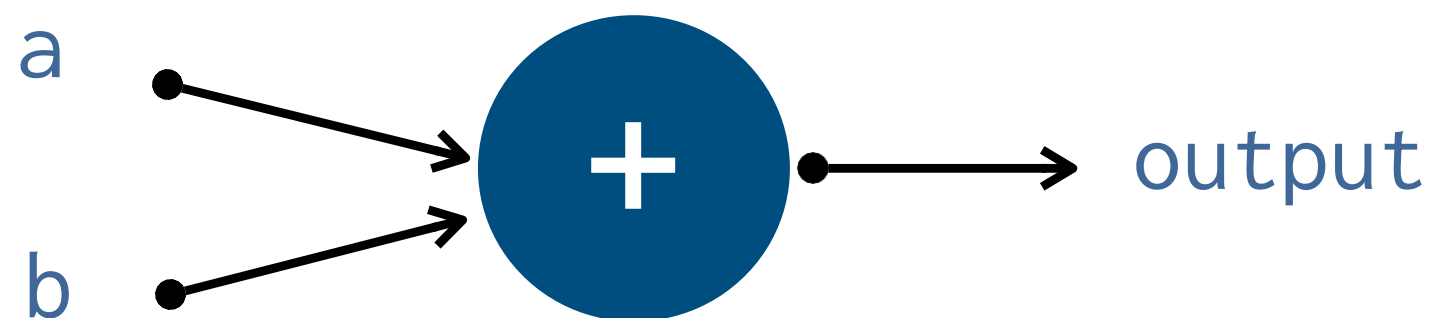
# “Calling” an adder in hardware

## **Combinational** Adder



# “Calling” an adder in hardware

## **Combinational** Adder

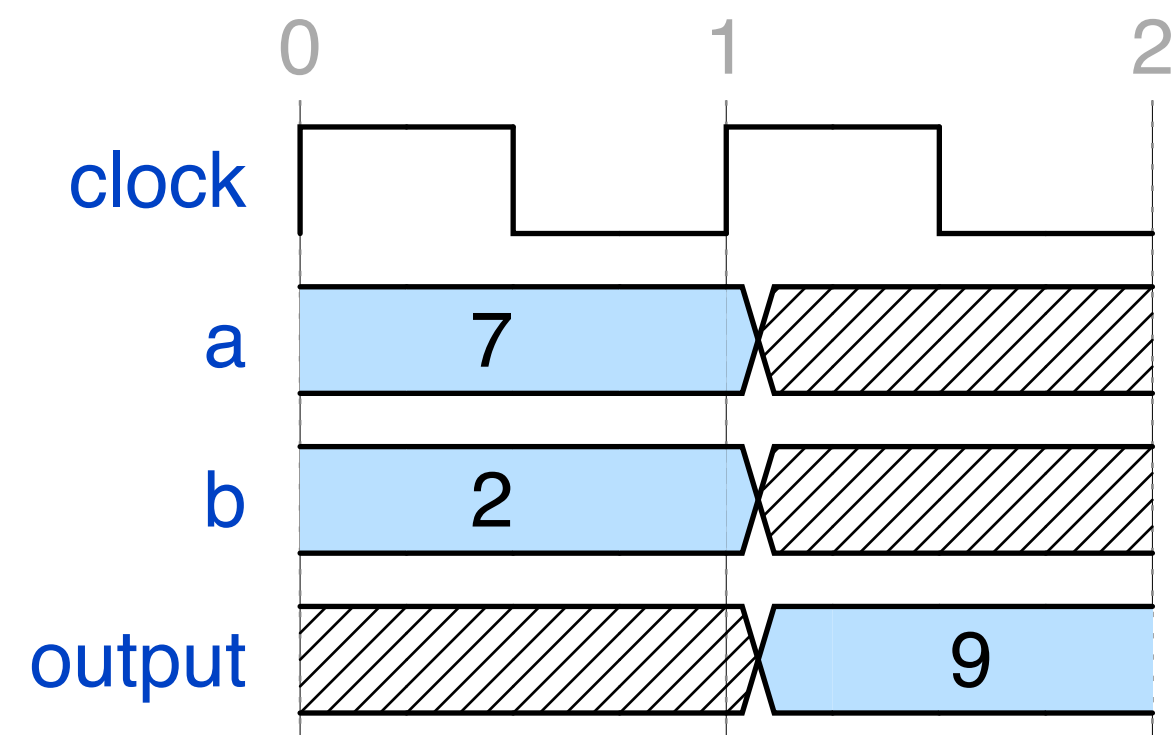
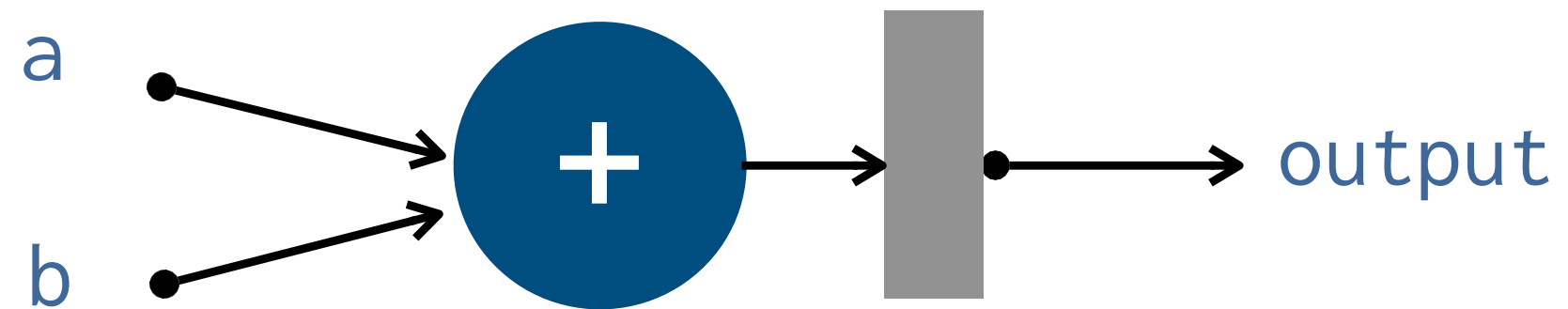


“If I drive  $a = 7, b = 2,$

I get  $output = 9$  in the **same** clock cycle”

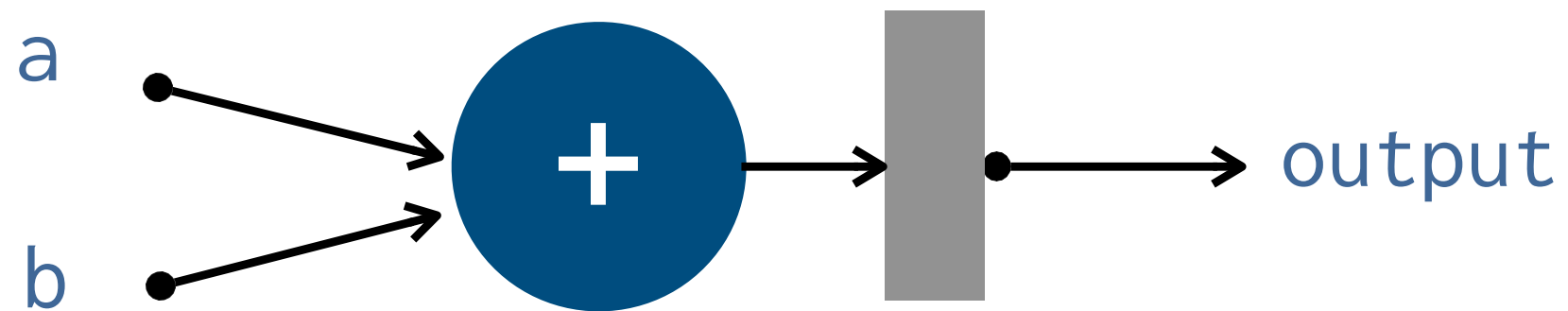
# "Calling" an adder in hardware

## *Sequential* Adder



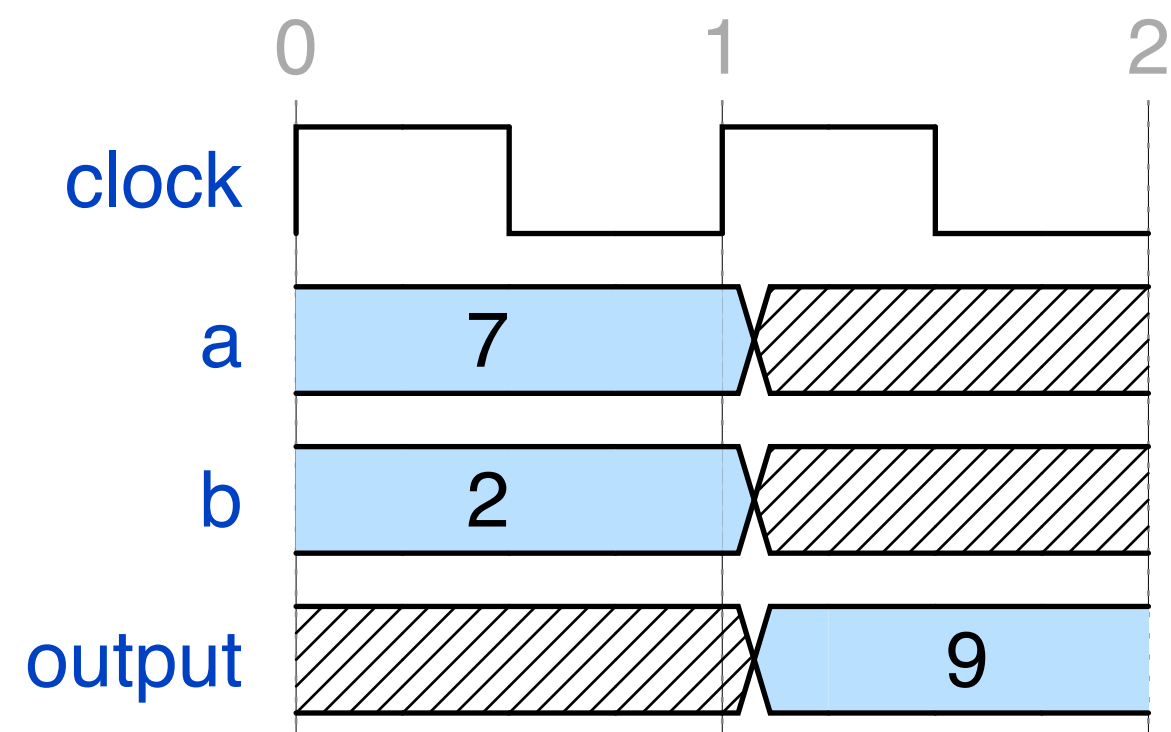
# "Calling" an adder in hardware

## *Sequential* Adder



"If I drive  $a = 7$ ,  $b = 2$ ,

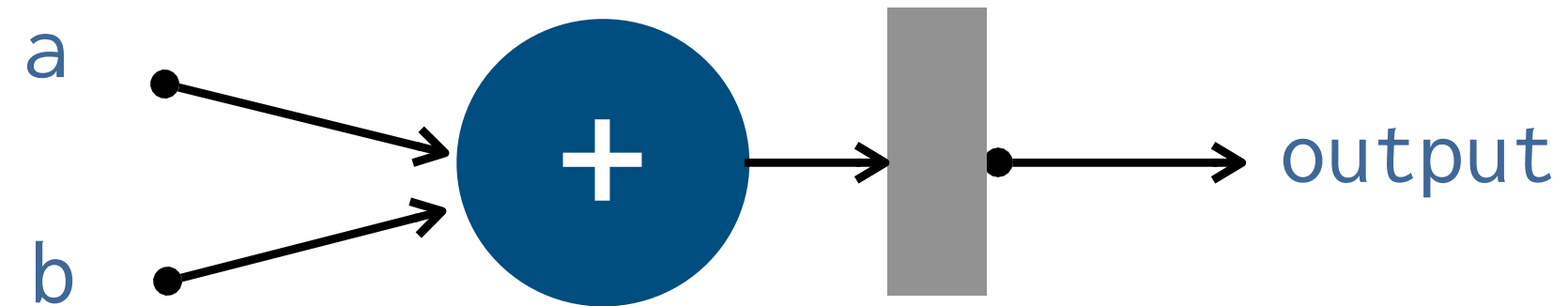
I get  $\text{output} = 9$  **after** one clock cycle"



“Calling” a hardware module  
depends on how the  
module is implemented!

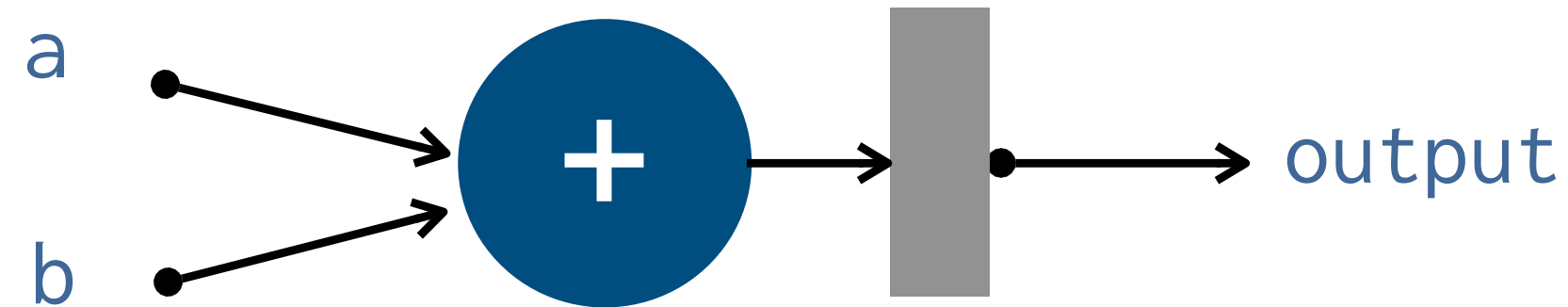
# The abstraction gap in hardware

Hardware runs at the level of individual clock cycles & signals,  
but we care about high-level **transactions**!



# The abstraction gap in hardware

Hardware runs at the level of individual clock cycles & signals,  
but we care about high-level **transactions**!

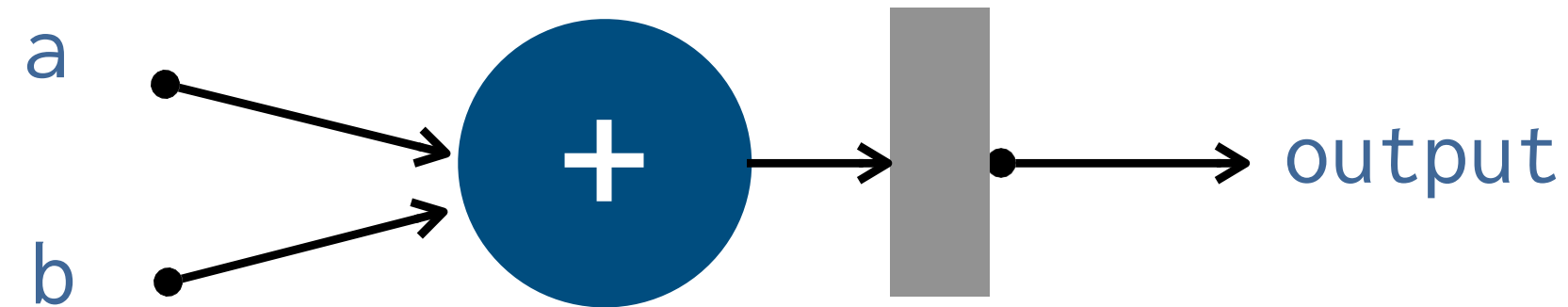


## Transactions

“Adding 7 and 2  
results in 9”

# The abstraction gap in hardware

Hardware runs at the level of individual clock cycles & signals,  
but we care about high-level **transactions**!



## Transactions

“Adding 7 and 2  
results in 9”

## Signals

“If I drive  $a = 7, b = 2$ ,  
I get  $output = 9$  after one clock cycle”

“Calling” a hardware module  
involves **communication protocols**



(I/O signals changing  
over clock cycles)

# **Example protocol: Ready-Valid Handshake**

# Ready-Valid Handshake

Mediates data transfer between a sender & a receiver



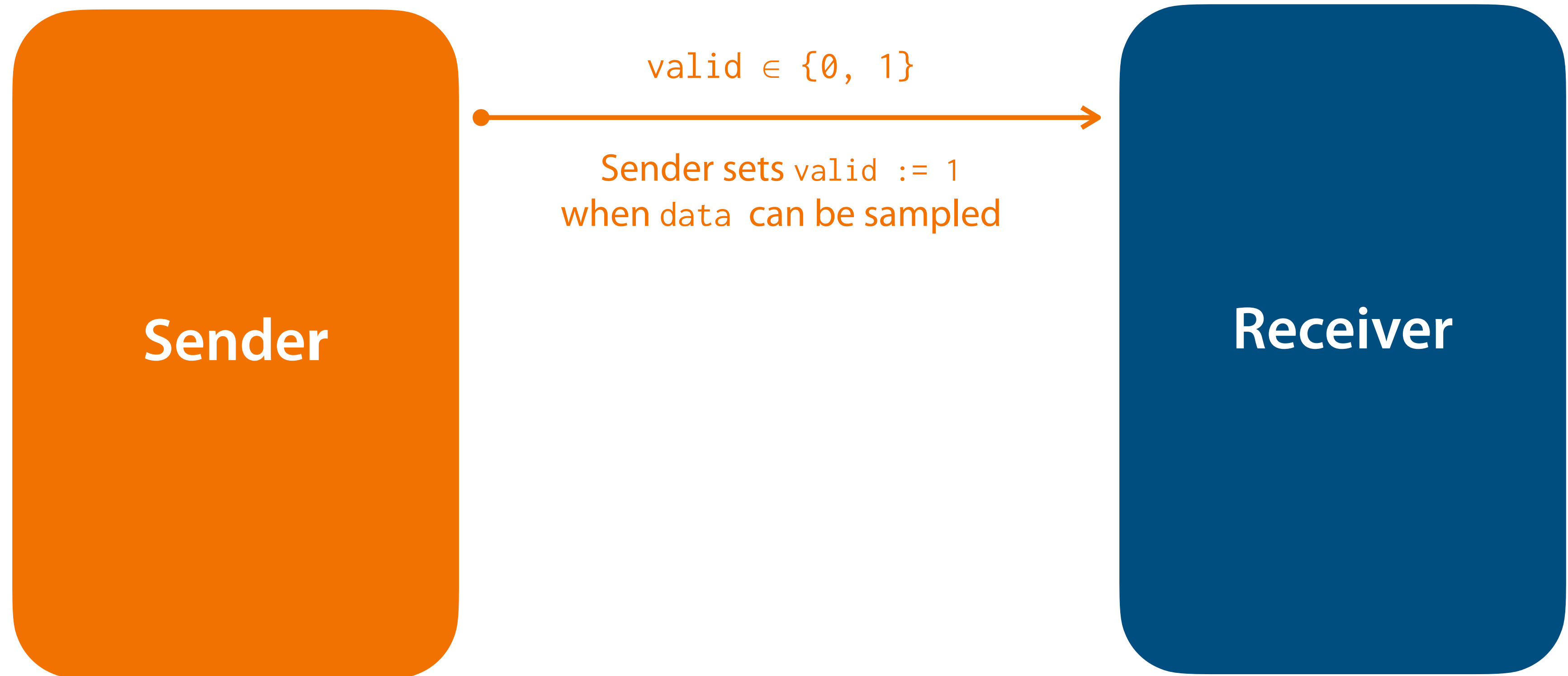
The diagram illustrates a Ready-Valid Handshake. It consists of two main components: a Sender and a Receiver. The Sender is represented by a large orange rounded rectangle on the left, and the Receiver is represented by a large dark blue rounded rectangle on the right. The word "Sender" is written in white text inside the orange rectangle, and the word "Receiver" is written in white text inside the blue rectangle. The two rectangles are positioned side-by-side, separated by a gap, representing the two parties involved in the data transfer process.

Sender

Receiver

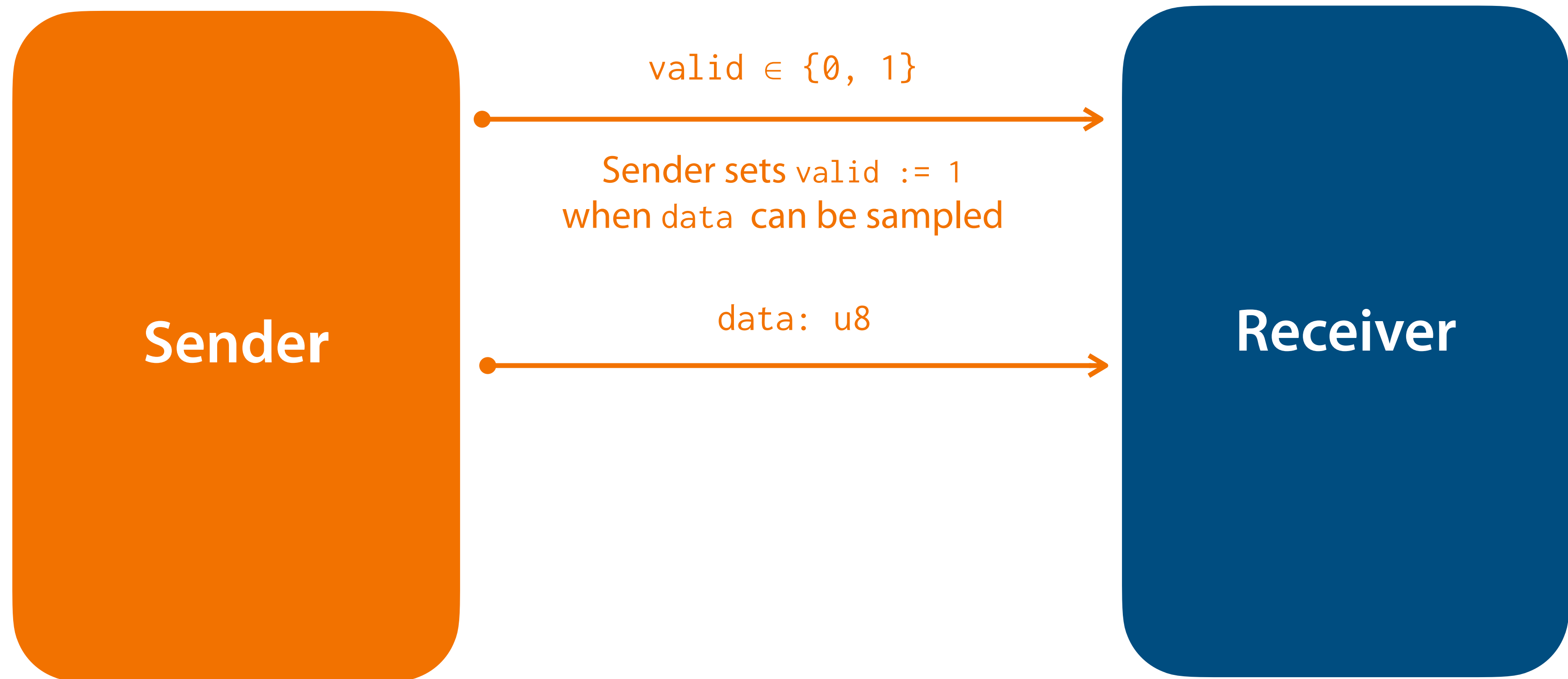
# Ready-Valid Handshake

Mediates data transfer between a sender & a receiver



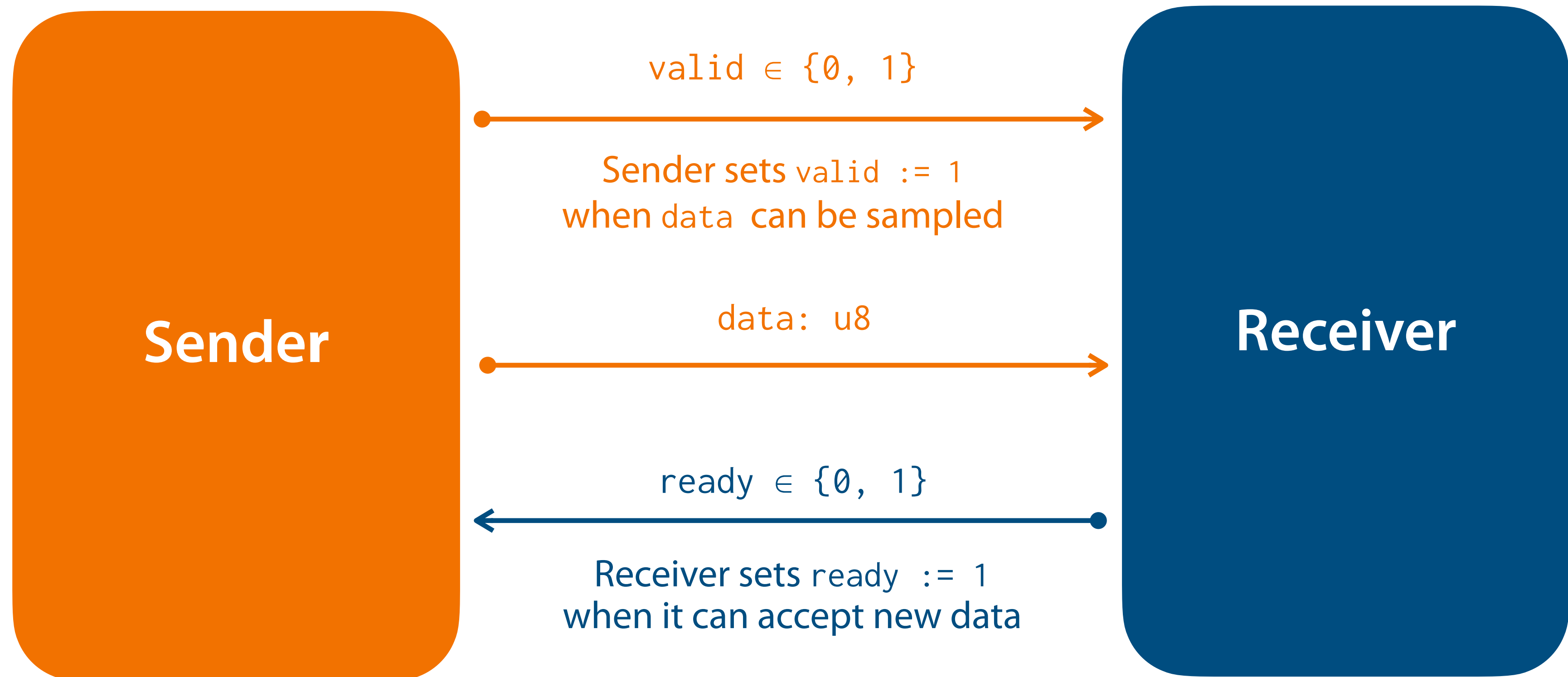
# Ready-Valid Handshake

Mediates data transfer between a sender & a receiver



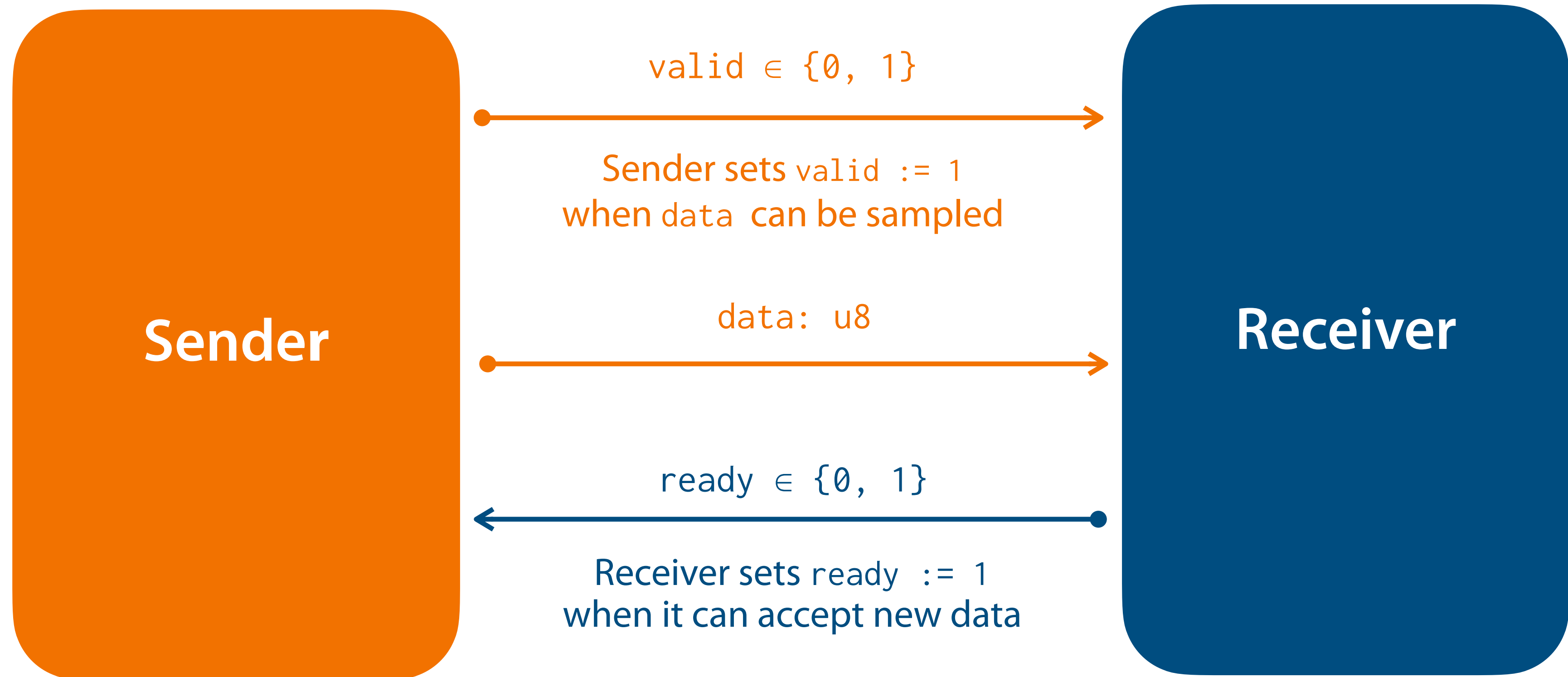
# Ready-Valid Handshake

Mediates data transfer between a sender & a receiver



# Ready-Valid Handshake

Invariant: **data** is only sent when **ready** & **valid** are both 1 during the same clock cycle



# **Problem:**

These protocols are often stated implicitly (in English),  
making testing & debugging hard!

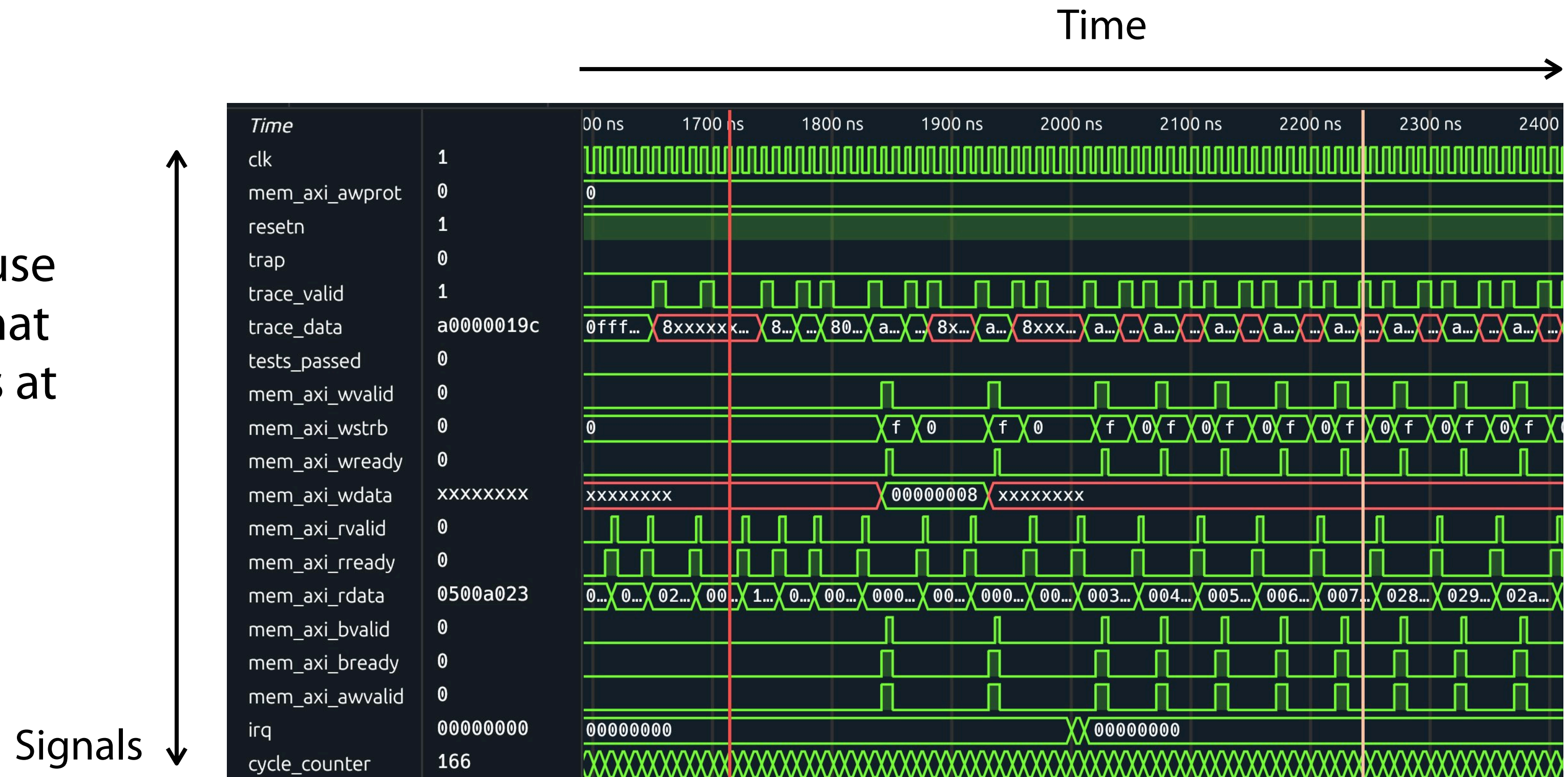
# Problem:

These protocols are often stated implicitly (in English),  
making testing & debugging hard!



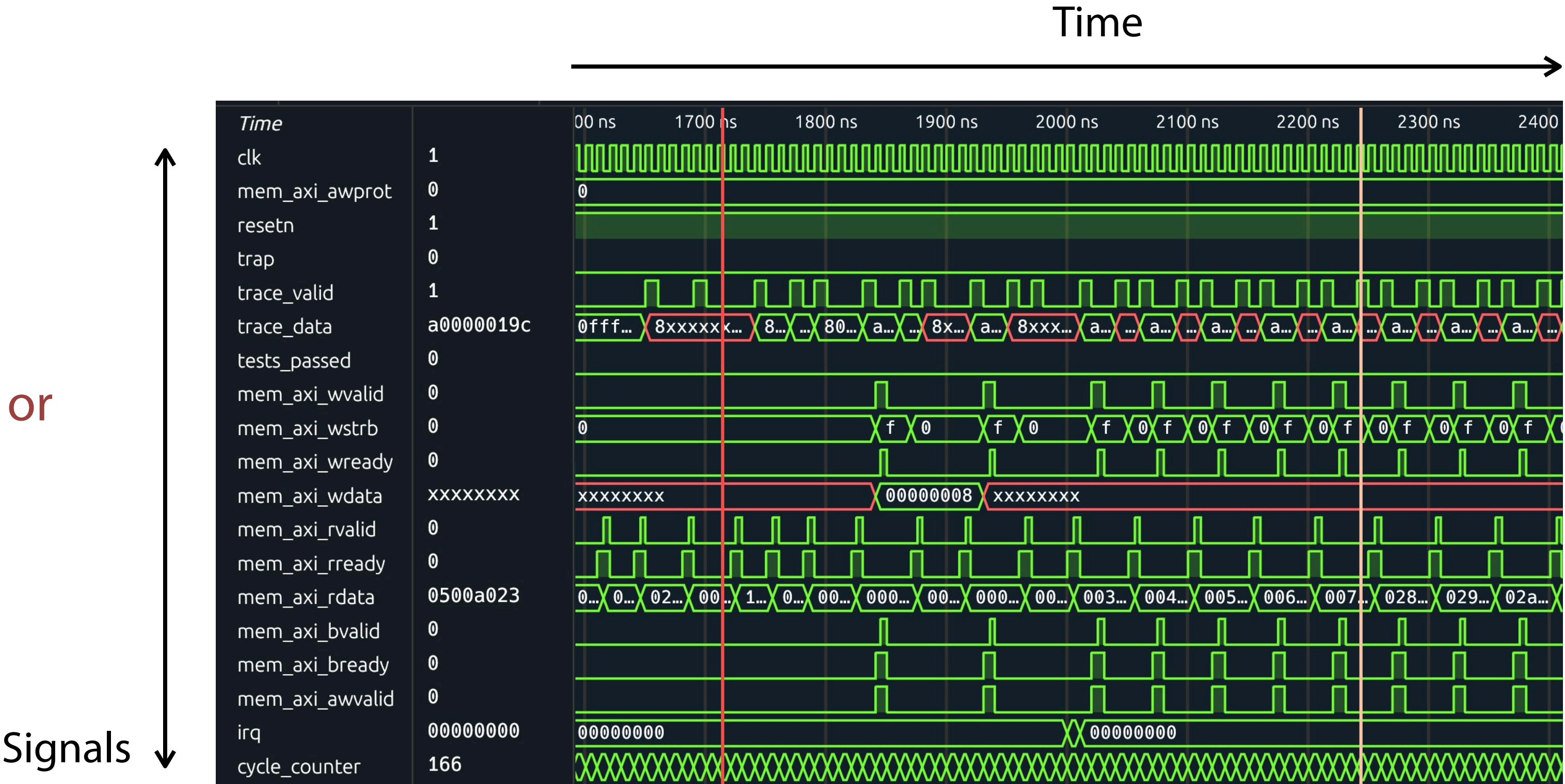
# Debugging hardware is hard

Hardware designers use **waveform viewers** that display signals' values at each cycle



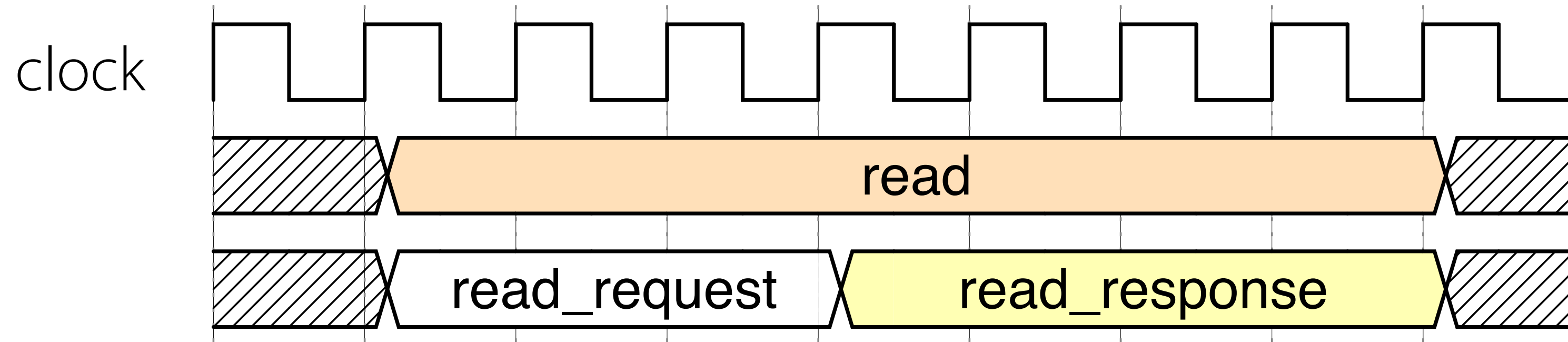
# Debugging hardware is hard

**Problem:** hard to identify *when* a transaction occurred, or *how long* it took!

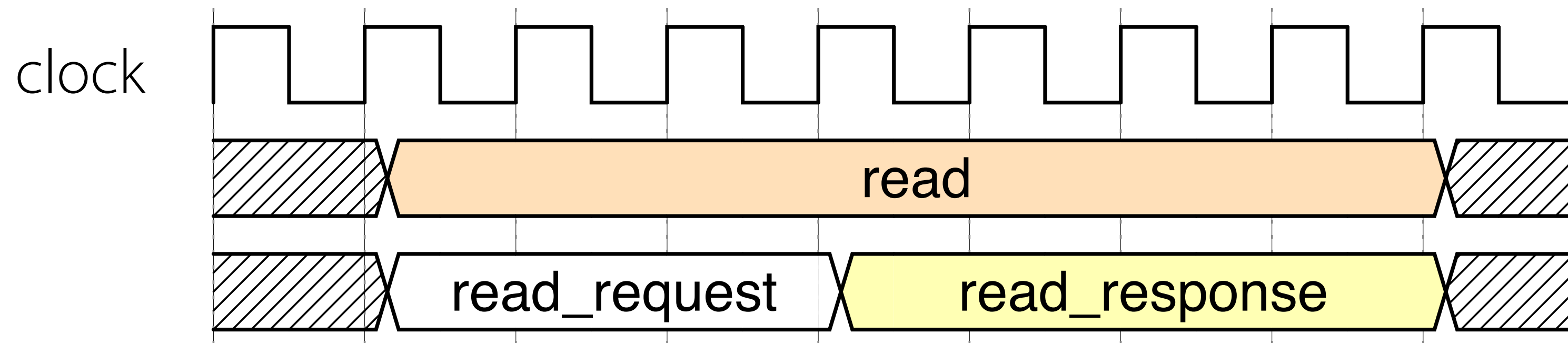


**What if we had waveform viewers  
that looked like this instead?**

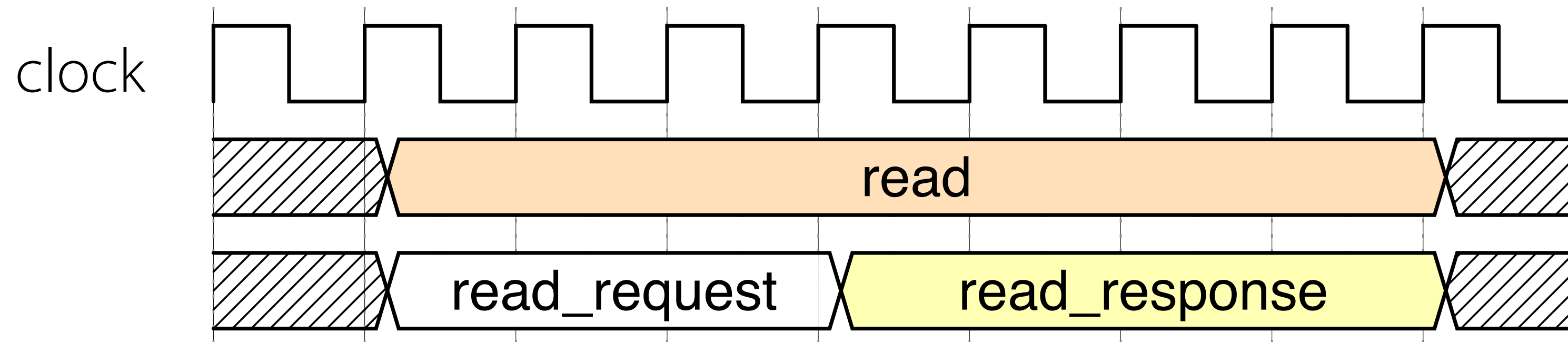
# What if we had waveform viewers that looked like this instead?



# Problem: need to reconstruct transactions from signals



# Problem: need to reconstruct transactions from signals



I'll show you later how to do this!

# Thesis

# Thesis

1. Hardware communication protocols can be specified as **programs**! We design a DSL, Paso, which enables this.

# Thesis

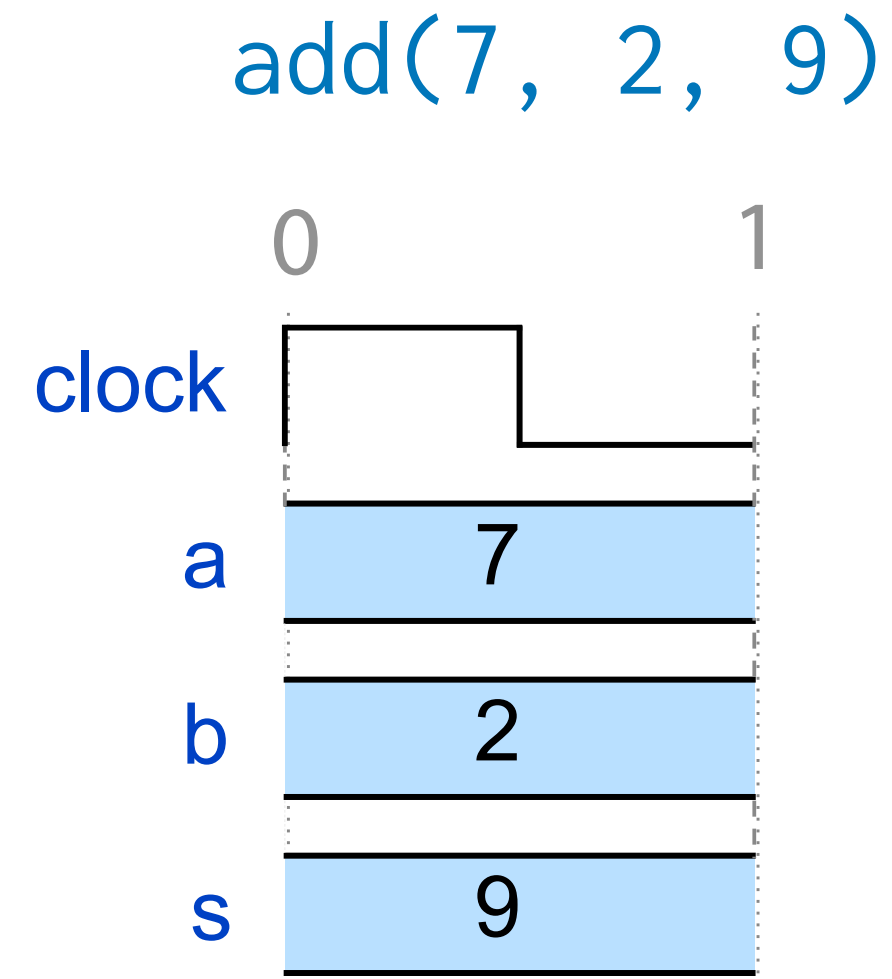
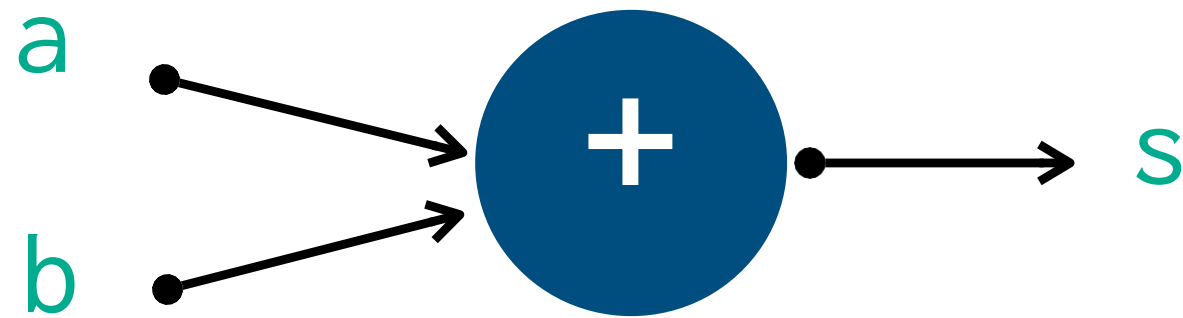
1. Hardware communication protocols can be specified as **programs**! We design a DSL, Paso, which enables this.
2. Using Paso, we can build PL tools that address pain-points with testing & debugging hardware.

# **Paso by example:**

## **Two protocol specs for an adder**

# A combinational adder

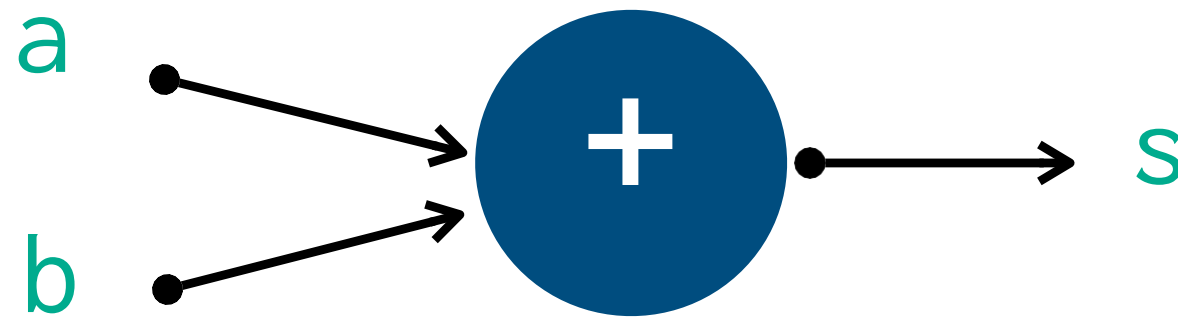
Note: we're not checking functional correctness,  
we are only interested in the adder's communication interface!



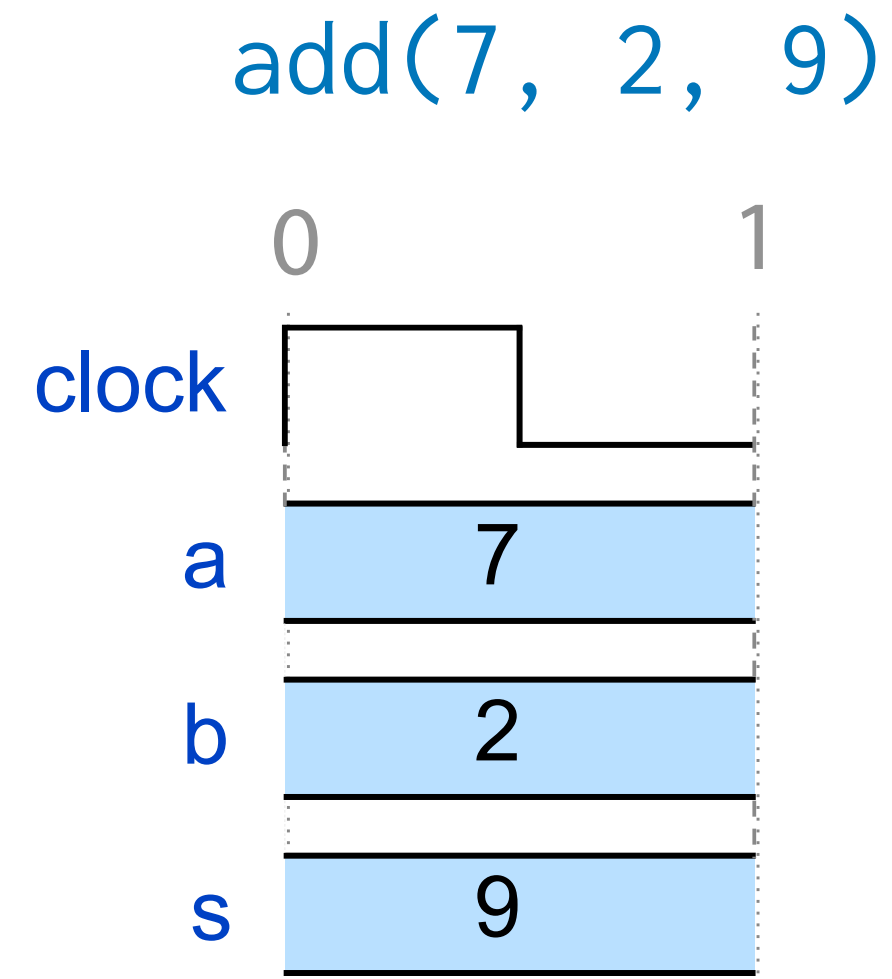
```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  assert_eq(DUT.s, s);  
  step();  
}
```

# A combinational adder

Note: we're not checking functional correctness,  
we are only interested in the adder's communication interface!



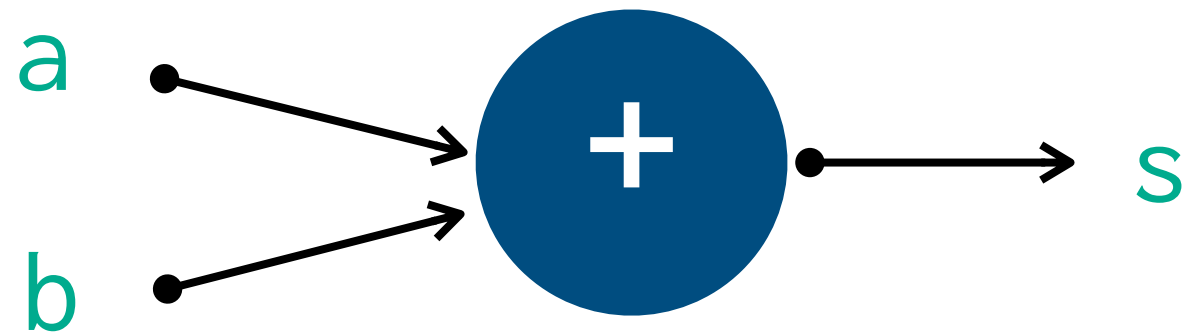
```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  assert_eq(DUT.s, s);  
  step();  
}
```



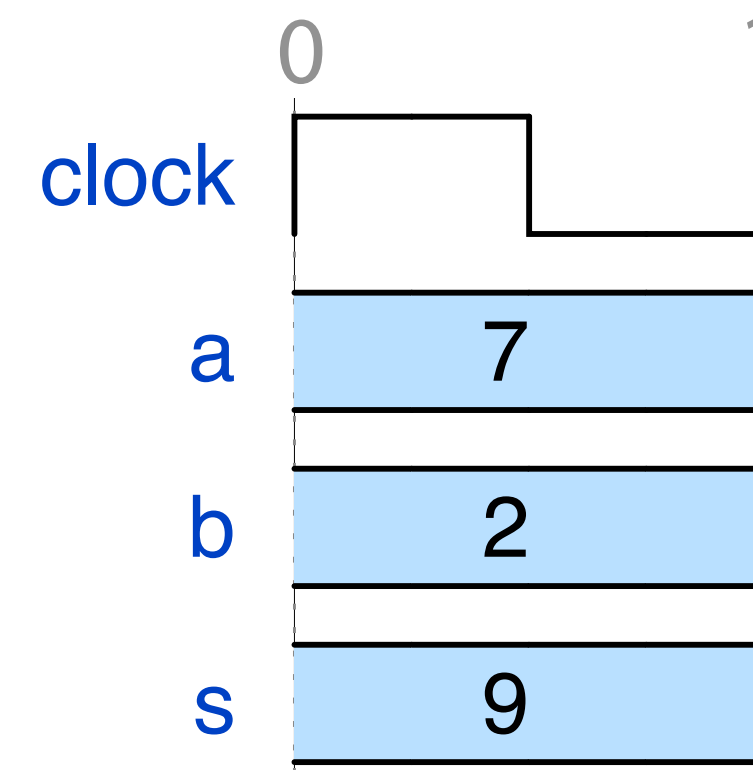
Notation: We specify  
expected inputs & outputs  
to the transaction,  
akin to a unit test

# A combinational adder

Note: we're not checking functional correctness,  
we are only interested in the adder's communication interface!



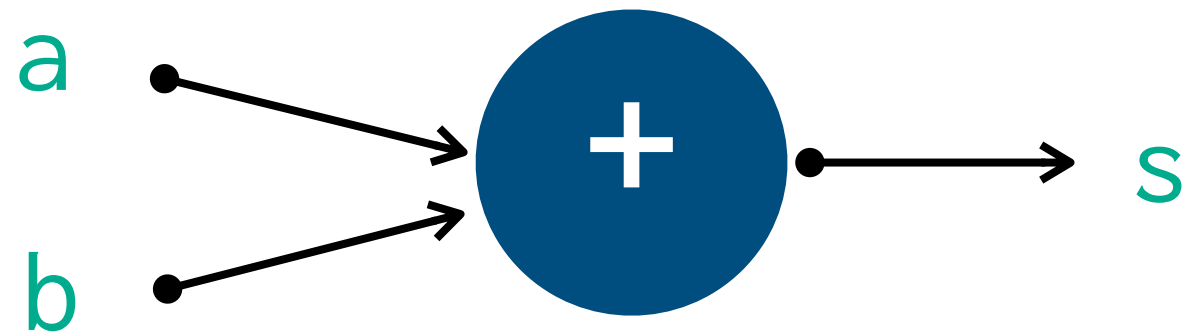
add(7, 2, 9)



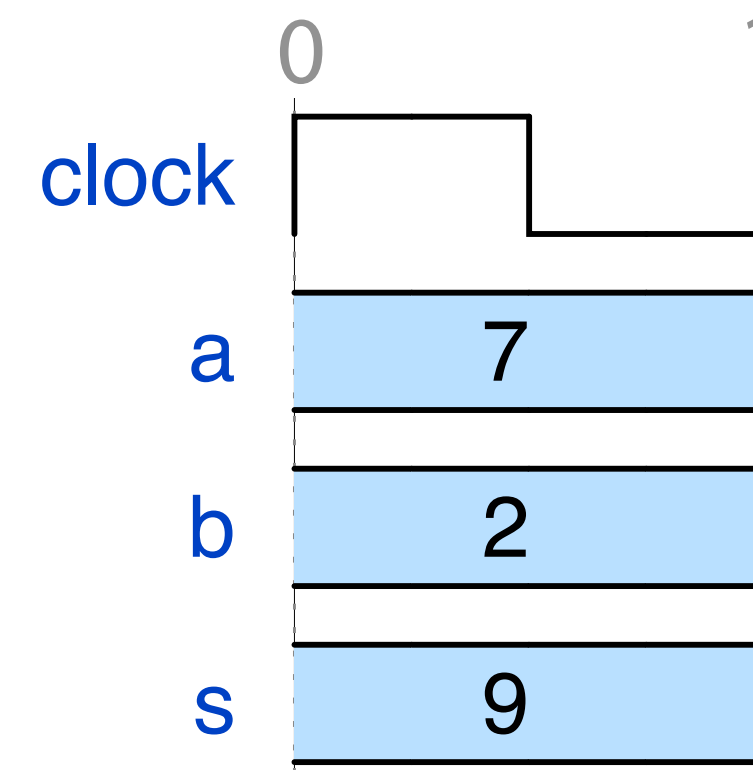
```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;      •————•      Apply values onto input ports  
  assert_eq(DUT.s, s);                      DUT.a & DUT.b  
  step();  
}
```

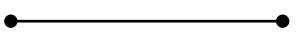
# A combinational adder

Note: we're not checking functional correctness,  
we are only interested in the adder's communication interface!



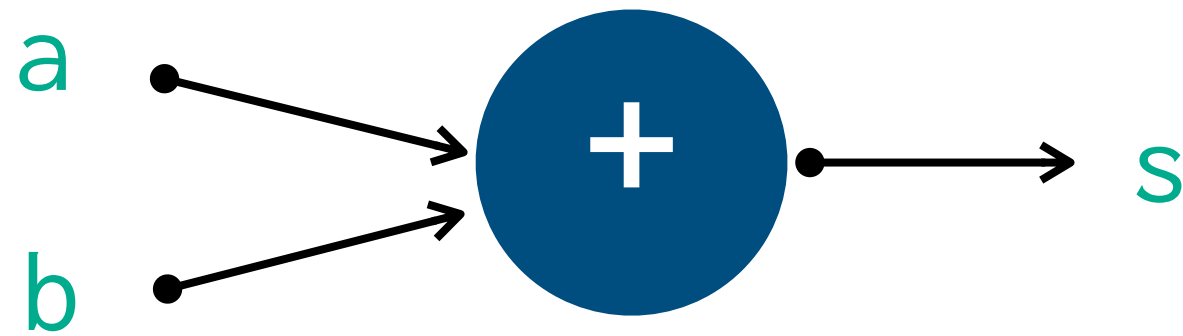
add(7, 2, 9)



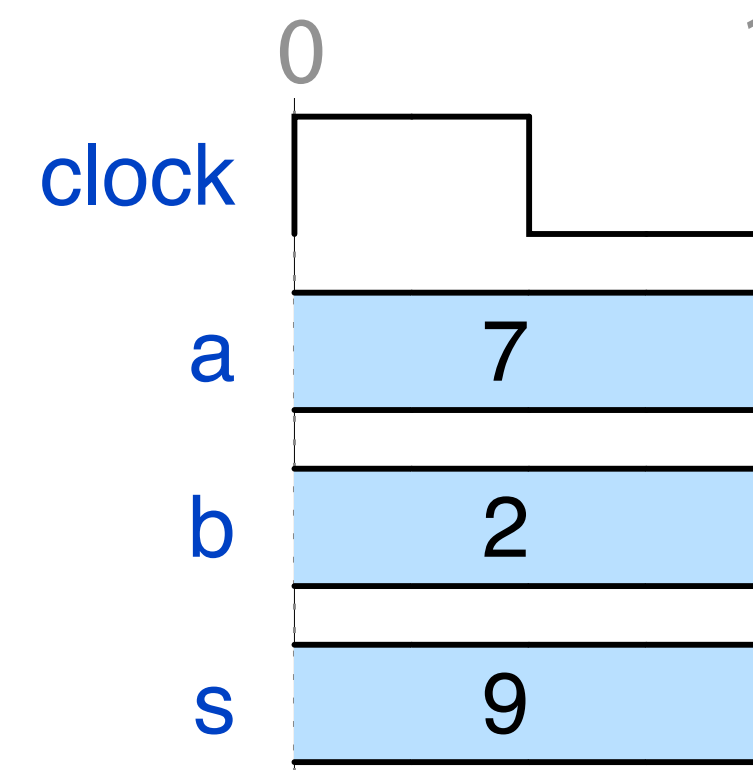
```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  assert_eq(DUT.s, s);  Check that output port DUT.s  
  step();  
  contains the value s  
}
```

# A combinational adder

Note: we're not checking functional correctness,  
we are only interested in the adder's communication interface!



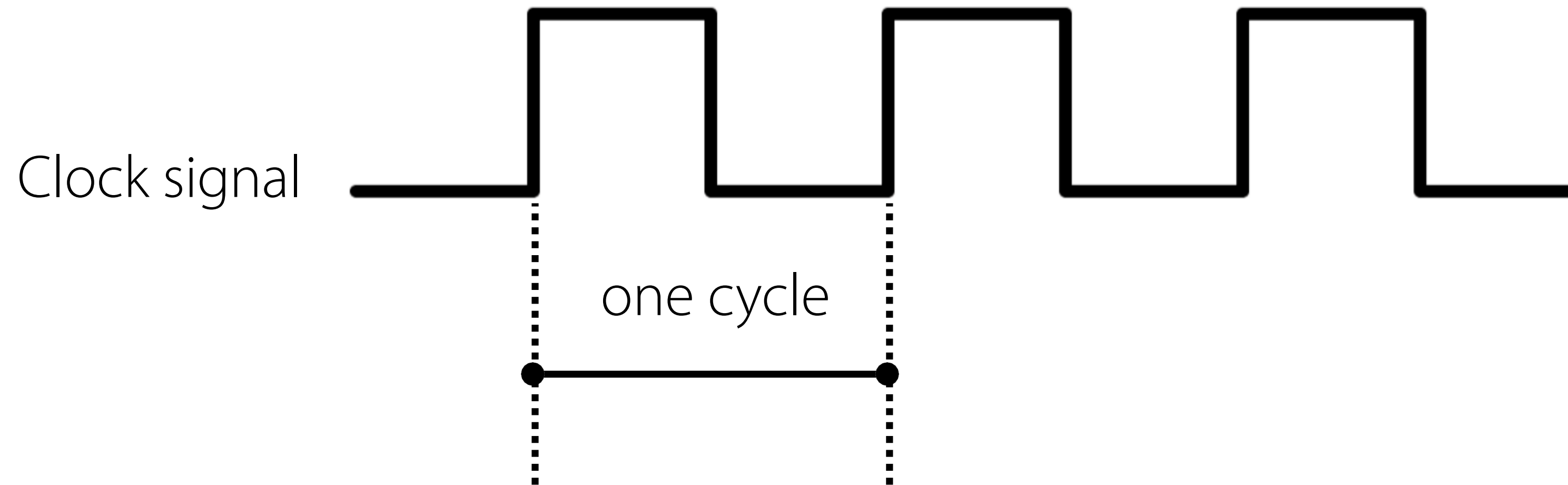
add(7, 2, 9)



```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  assert_eq(DUT.s, s);  
  step(); ————— Finish clock cycle  
}
```

# Aside: the `step()` primitive

`step()` advances the clock by one cycle



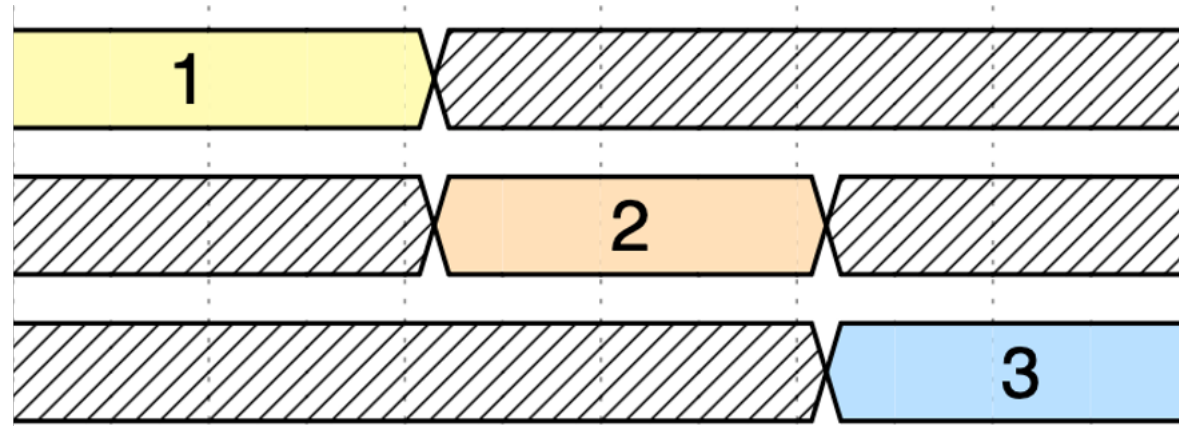
**Can we optimize our adder?**

Yes, through **pipelining**!

# Yes, through **pipelining**!

## Sequential

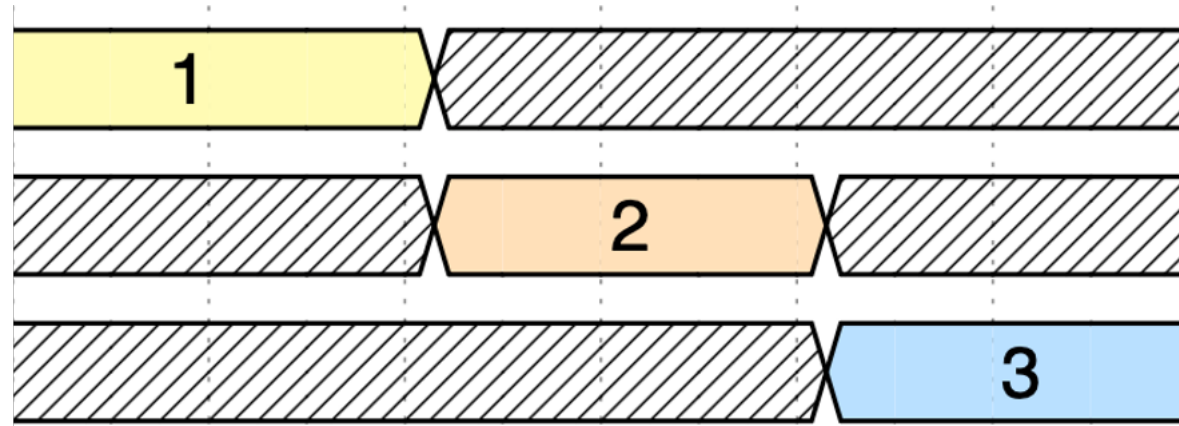
Processes **one** input at a time



# Yes, through **pipelining**!

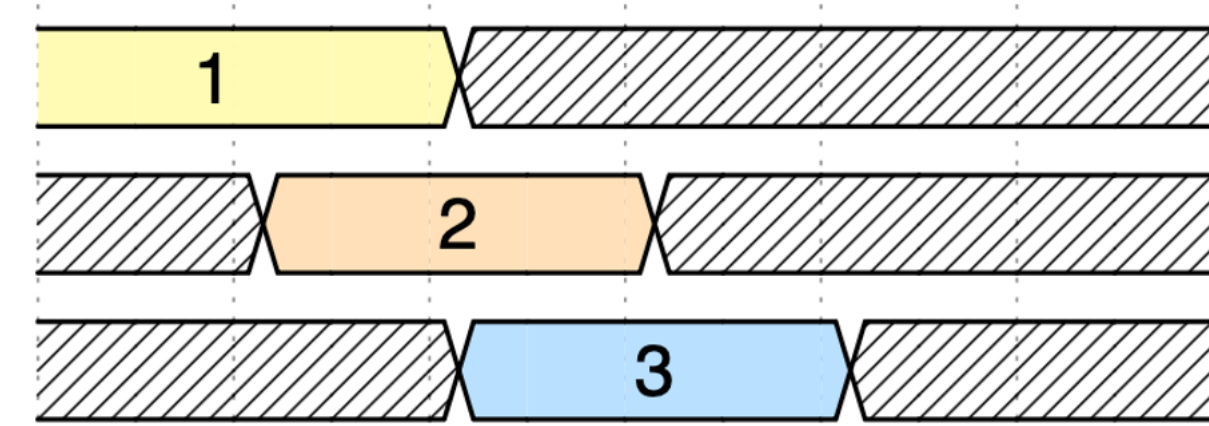
## Sequential

Processes **one** input at a time



## Pipelined

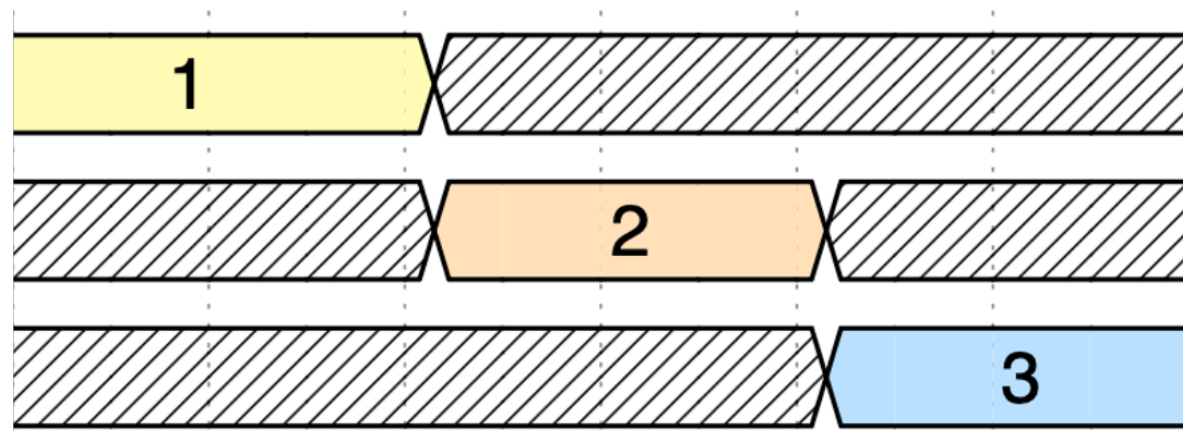
Processes **multiple** inputs at a time



# Yes, through **pipelining**!

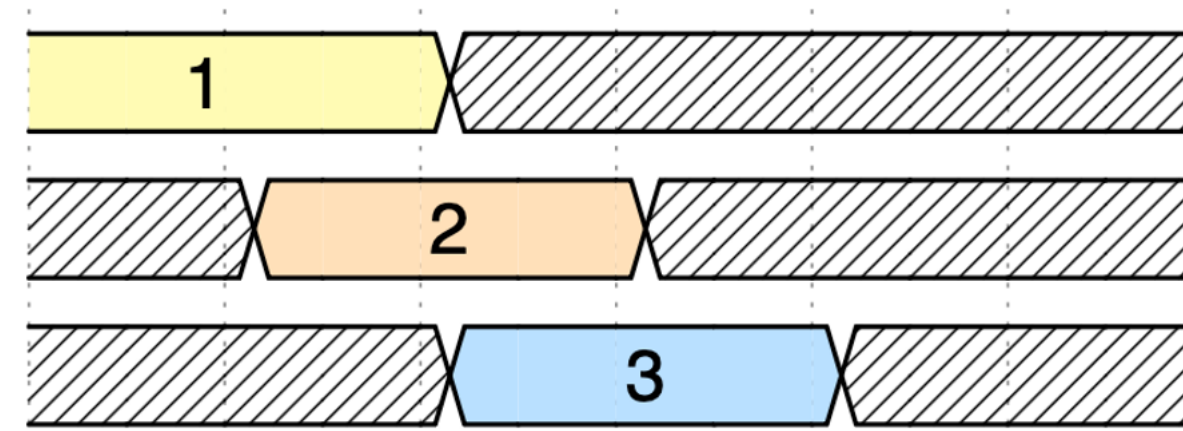
## Sequential

Processes **one** input at a time



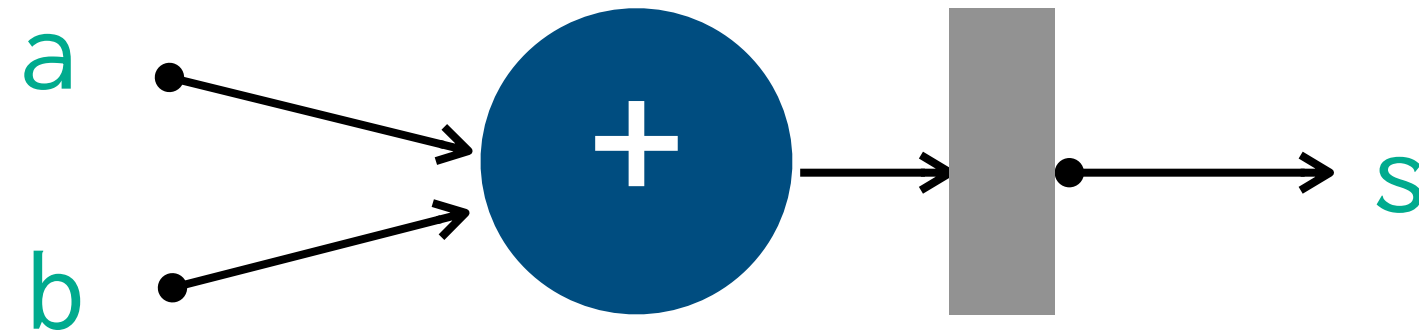
## Pipelined

Processes **multiple** inputs at a time



**fork()** allows for concurrent protocol execution in our DSL

# A **naïve** attempt at a pipelined adder



Combinational

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;
```

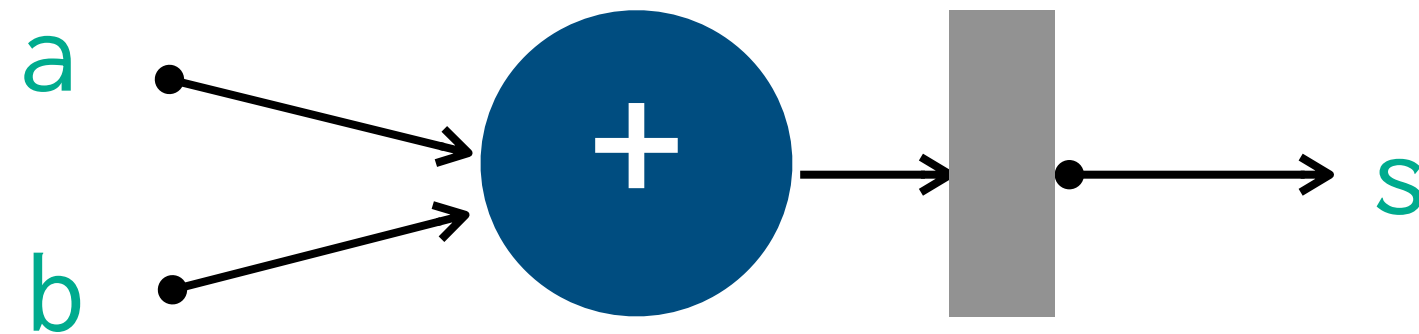
```
    assert_eq(DUT.s, s);  
    step();
```

```
}
```

Pipelined (**naïve**)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;  
    step();  
    fork();  
    assert_eq(DUT.s, s);  
    step();  
}
```

# A **naïve** attempt at a pipelined adder



## Combinational

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;
```

```
    assert_eq(DUT.s, s);  
    step();
```

```
}
```

## Pipelined (**naïve**)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;
```

```
+ step();
```

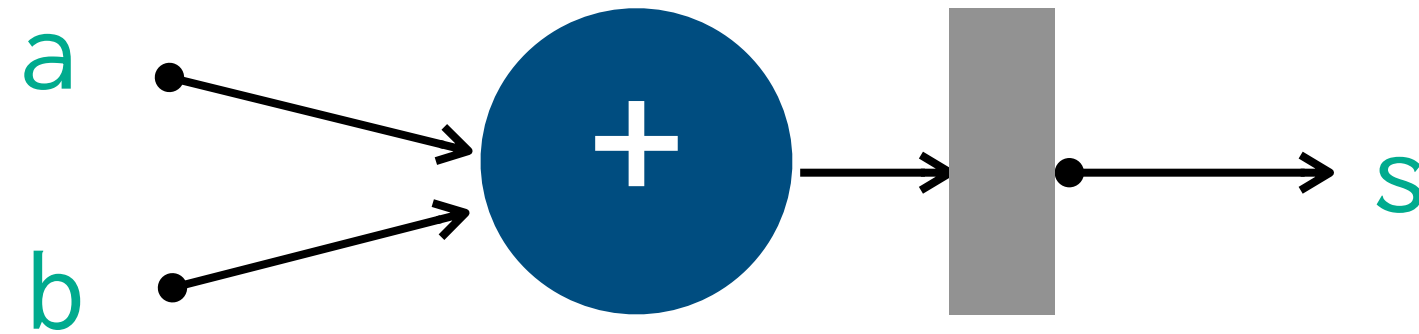
```
+ fork();
```

```
    assert_eq(DUT.s, s);
```

```
    step();
```

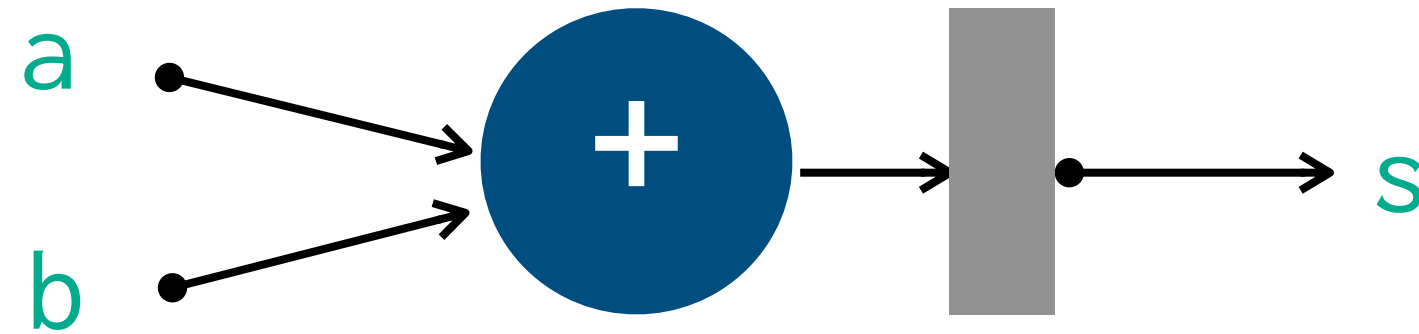
```
}
```

# A **naïve** attempt at a pipelined adder

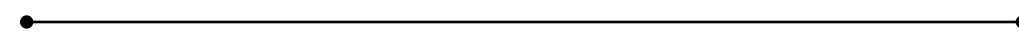


```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

# A **naïve** attempt at a pipelined adder

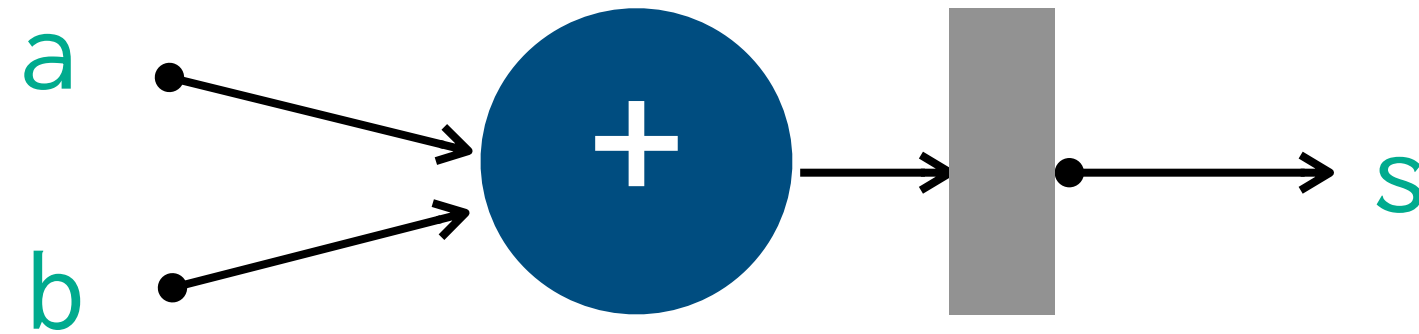


```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```



`fork()`: allows another transaction to begin concurrently in the same cycle (pipelining!)

# A **naïve** attempt at a pipelined adder



```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

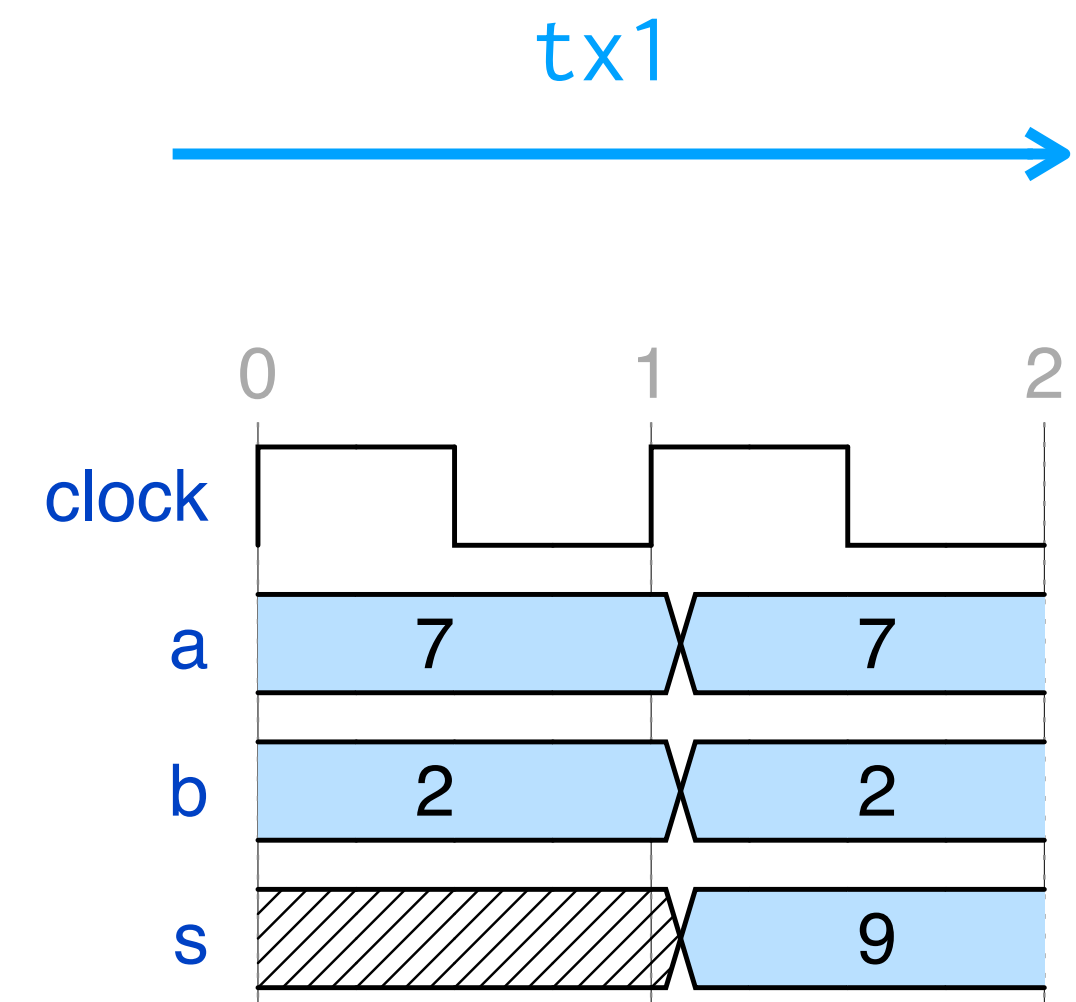
•————• 2nd `step()` for a total latency of 2 cycles

# A **naïve** attempt at a pipelined adder

tx1: add(7, 2, 9)

Pipelined (naïve)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```



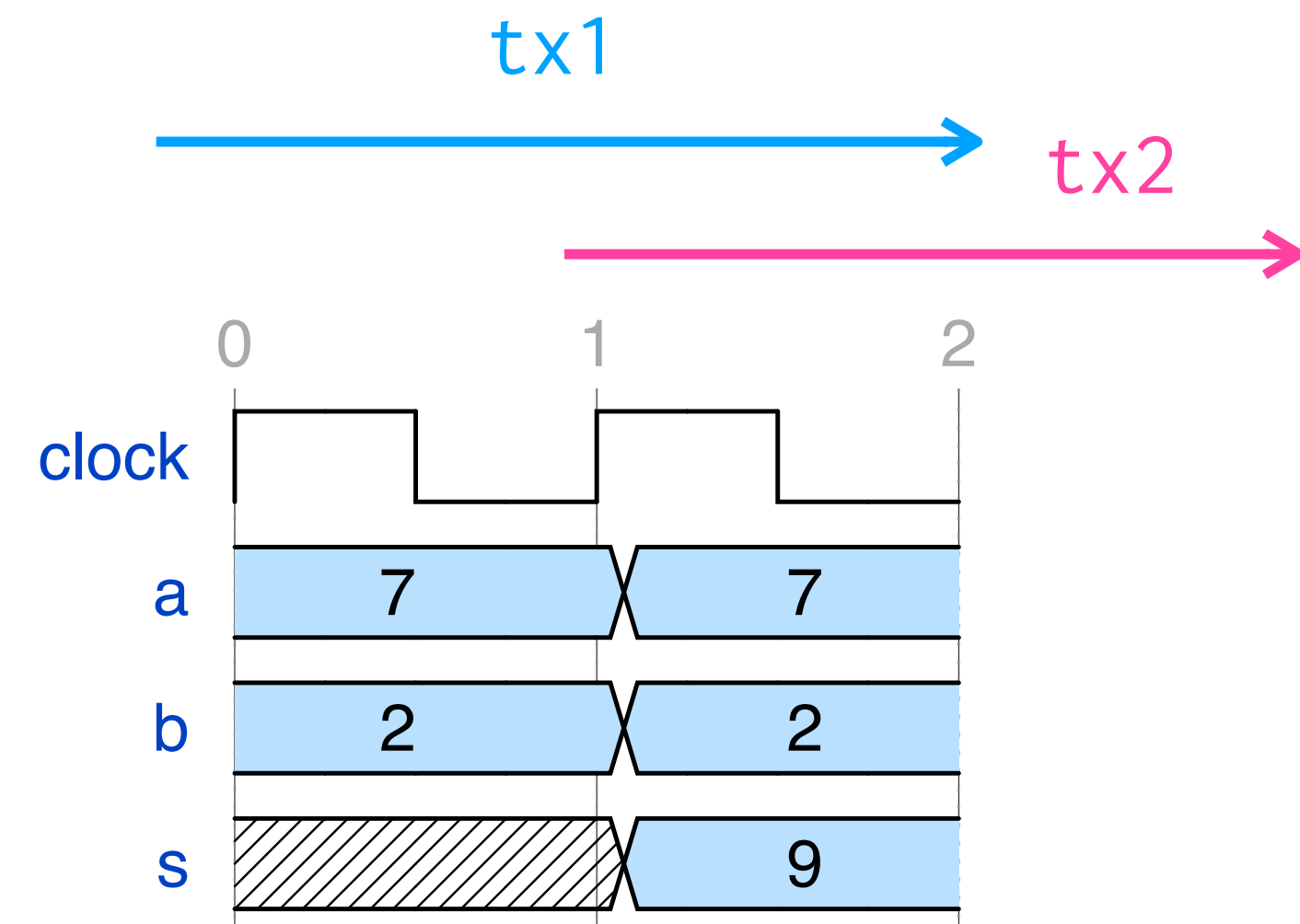
# A **naïve** attempt at a pipelined adder

Pipelined (naïve)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

tx1: add(7, 2, 9)

tx2: add(2, 4, 6)



# A **naïve** attempt at a pipelined adder

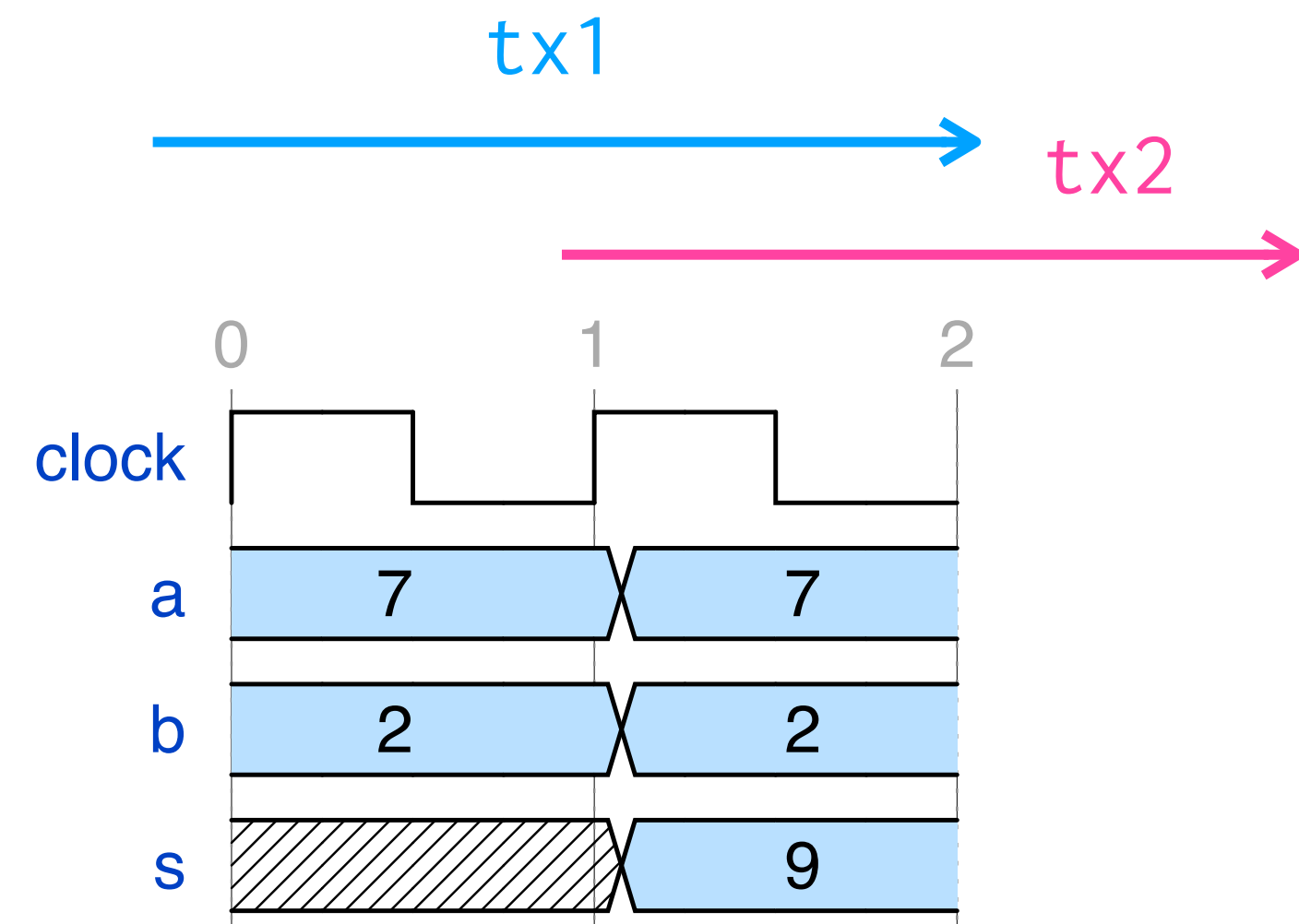
Pipelined (naïve)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

Problem: **tx2** wants to drive values onto **DUT.a** & **DUT.b**, but **tx1** still has control over these ports!

**tx1**: add(7, 2, 9)

**tx2**: add(2, 4, 6)



# A **naïve** attempt at a pipelined adder

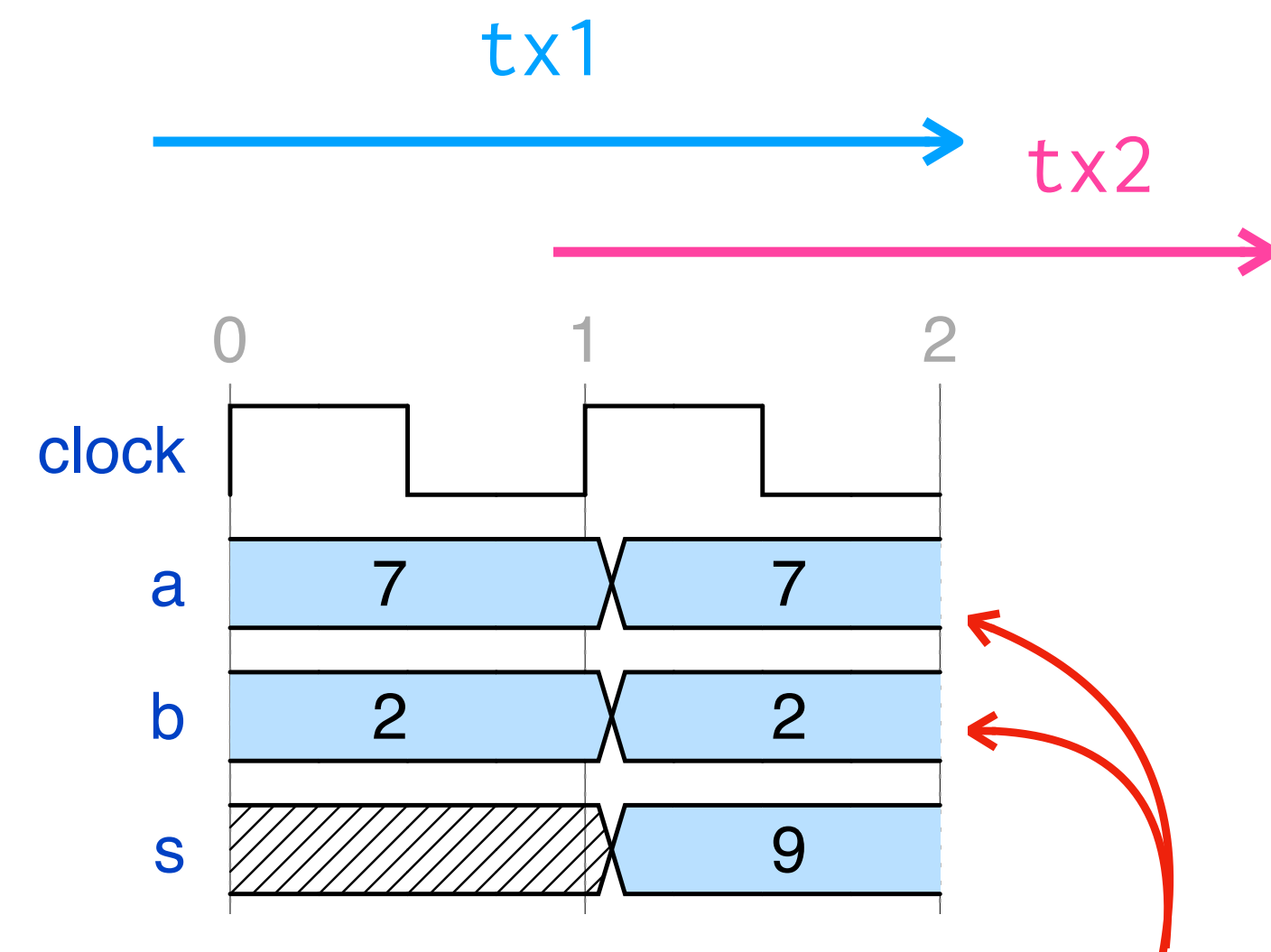
Pipelined (naïve)

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

Problem: **tx2** wants to drive values onto **DUT.a** & **DUT.b**, but **tx1** still has control over these ports!

**tx1**: add(7, 2, 9)

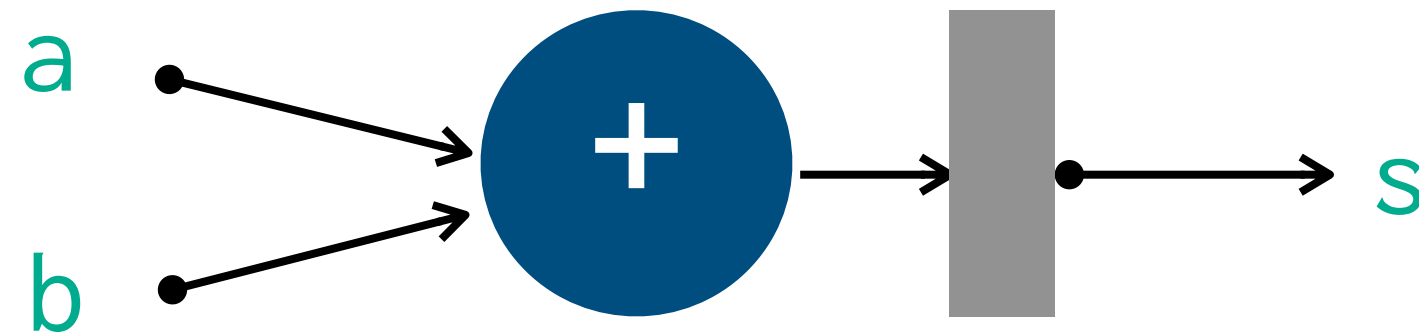
**tx2**: add(2, 4, 6)



**tx2** wants to assign

**DUT.a := 2; DUT.b := 4**  
**here but is unable to do so!**

# A pipelined adder, take two



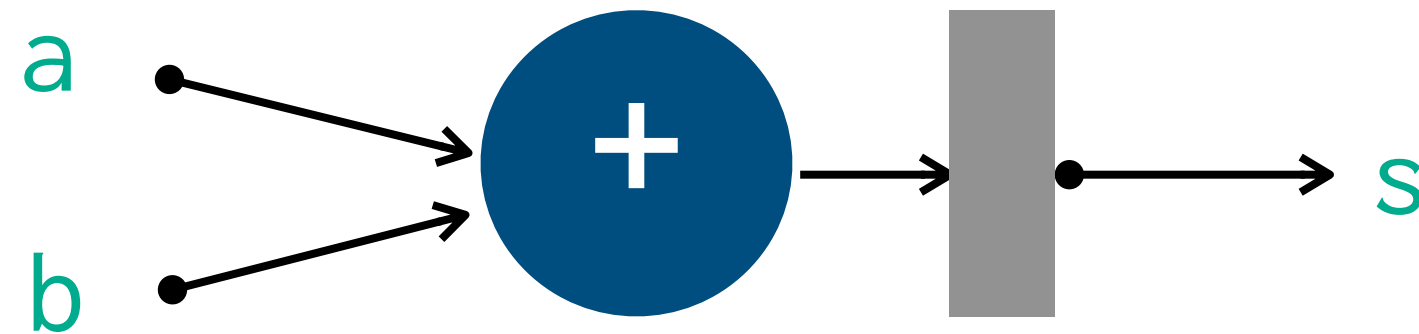
## Combinational

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;  
  
    assert_eq(DUT.s, s);  
    step();  
}
```

## Pipelined

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;  
    step();  
    DUT.a := X; DUT.b := X;  
    fork();  
    assert_eq(DUT.s, s);  
    step();  
}
```

# A pipelined adder, take two



## Combinational

```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;
```

```
    assert_eq(DUT.s, s);  
    step();  
}
```

## Pipelined

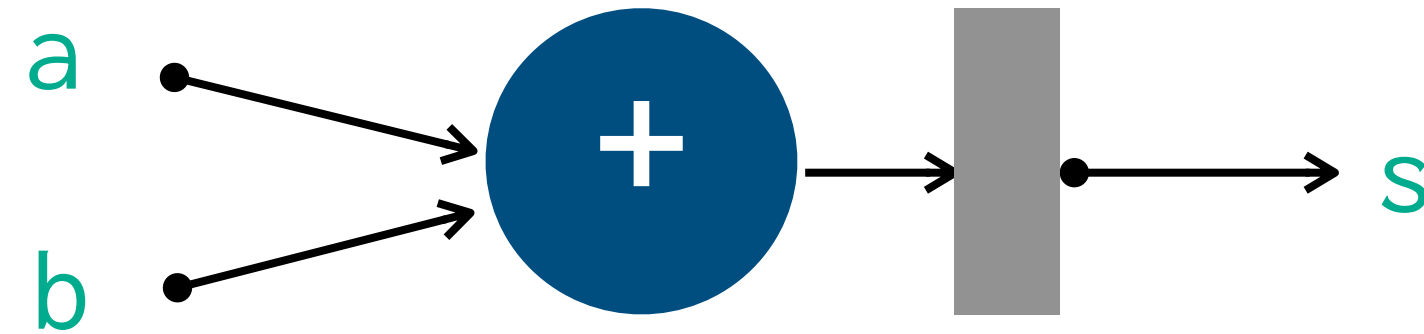
```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
    DUT.a := a; DUT.b := b;
```

```
    + step();  
    + DUT.a := X; DUT.b := X;  
    + fork();
```

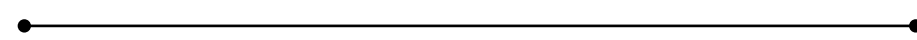
← new!

```
    assert_eq(DUT.s, s);  
    step();  
}
```

# A pipelined adder

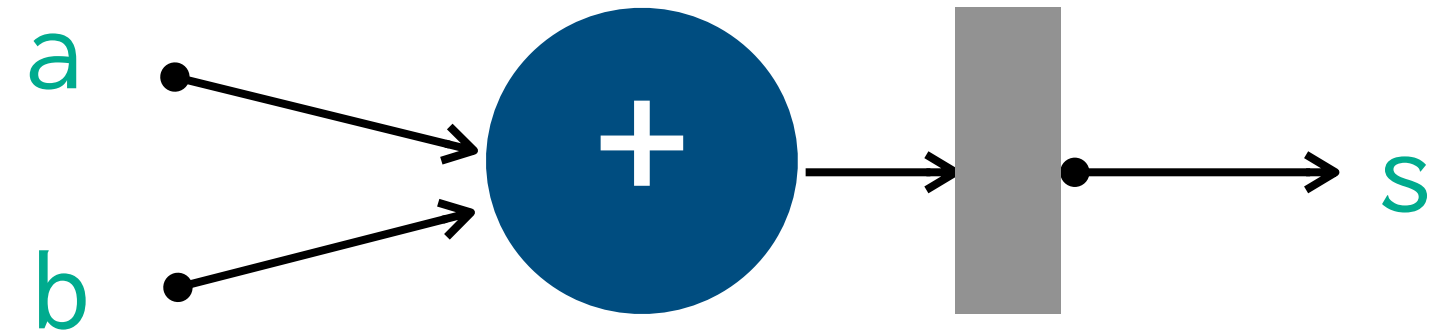


```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  DUT.a := X; DUT.b := X;  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```



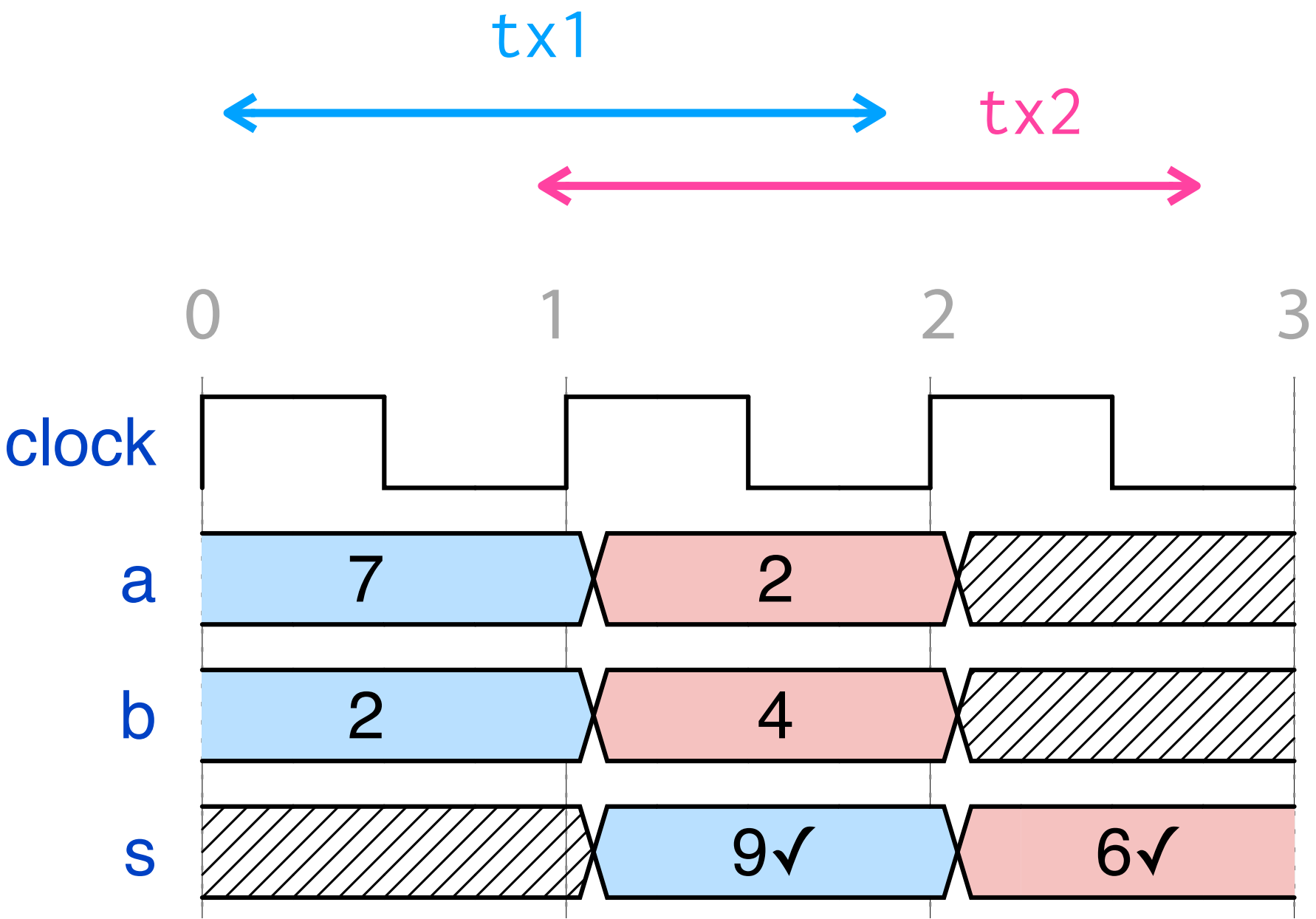
DontCare assignments: release constraints on ports  
(Value on ports `DUT.a` & `DUT.b` no longer matter)

# A pipelined adder



```
prot add<DUT: Adder>(a: u32, b: u32, s: u32) {  
  DUT.a := a; DUT.b := b;  
  step();  
  DUT.a := X; DUT.b := X;  
  fork();  
  assert_eq(DUT.s, s);  
  step();  
}
```

tx1: add(7, 2, 9)  
tx2: add(2, 4, 6)



# **Second example: A Ready-Valid Handshake in Paso**

# **A Ready-Valid Handshake in Paso**

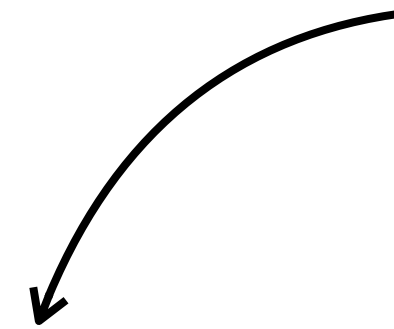
# A Ready-Valid Handshake in Paso

```
prot send_data<DUT: ReadyValid>(data: u8) {
```

```
}
```

# A Ready-Valid Handshake in Paso

Parameterized over DUTs (Designs Under Test)  
that implement the ReadyValid interface



```
prot send_data<DUT: ReadyValid>(data: u8) {
```

```
}
```

# A Ready-Valid Handshake in Paso

```
prot send_data<DUT: ReadyValid>(data: u8) {  
    DUT.valid := 1;  Indicate that we have  
    semantically meaningful data to send  
  
}
```

# A Ready-Valid Handshake in Paso

```
prot send_data<DUT: ReadyValid>(data: u8) {  
    DUT.valid := 1;  
    DUT.data  := data; ←————— Indicate that the data parameter  
                                is the payload to be sent  
}
```

# A Ready-Valid Handshake in Paso

```
prot send_data<DUT: ReadyValid>(data: u8) {  
    DUT.valid := 1;  
    DUT.data := data;  
    while (DUT.ready  $\neq$  1) {  
        step();  
    }  
}
```

Wait until **ready** becomes 1  
(i.e. till the receiver becomes ready)

# A Ready-Valid Handshake in Paso

```
prot send_data<DUT: ReadyValid>(data: u8) {  
    DUT.valid := 1;  
    DUT.data := data;  
    while (DUT.ready  $\neq$  1) {  
        step();  
    }  
    step();  
}
```

← The actual data transfer  
takes one more clock cycle

# A Ready-Valid Handshake in Paso

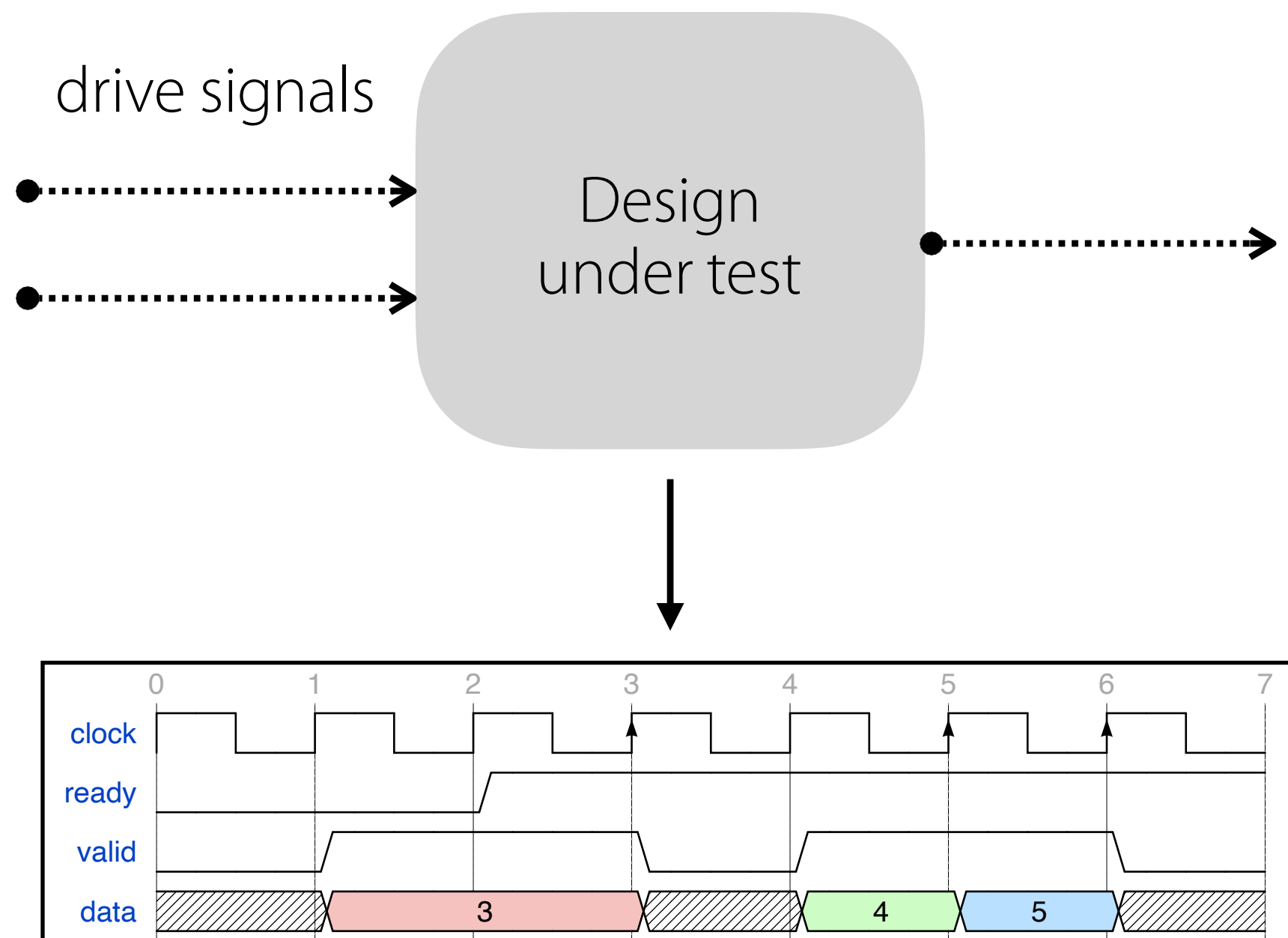
Simple imperative semantics, just like software!

```
prot send_data<DUT: ReadyValid>(data: u8) {  
    DUT.valid := 1;  
    DUT.data := data;  
    while (DUT.ready  $\neq$  1) {  
        step();  
    }  
    step();  
}
```

# **What can we do with one single protocol?**

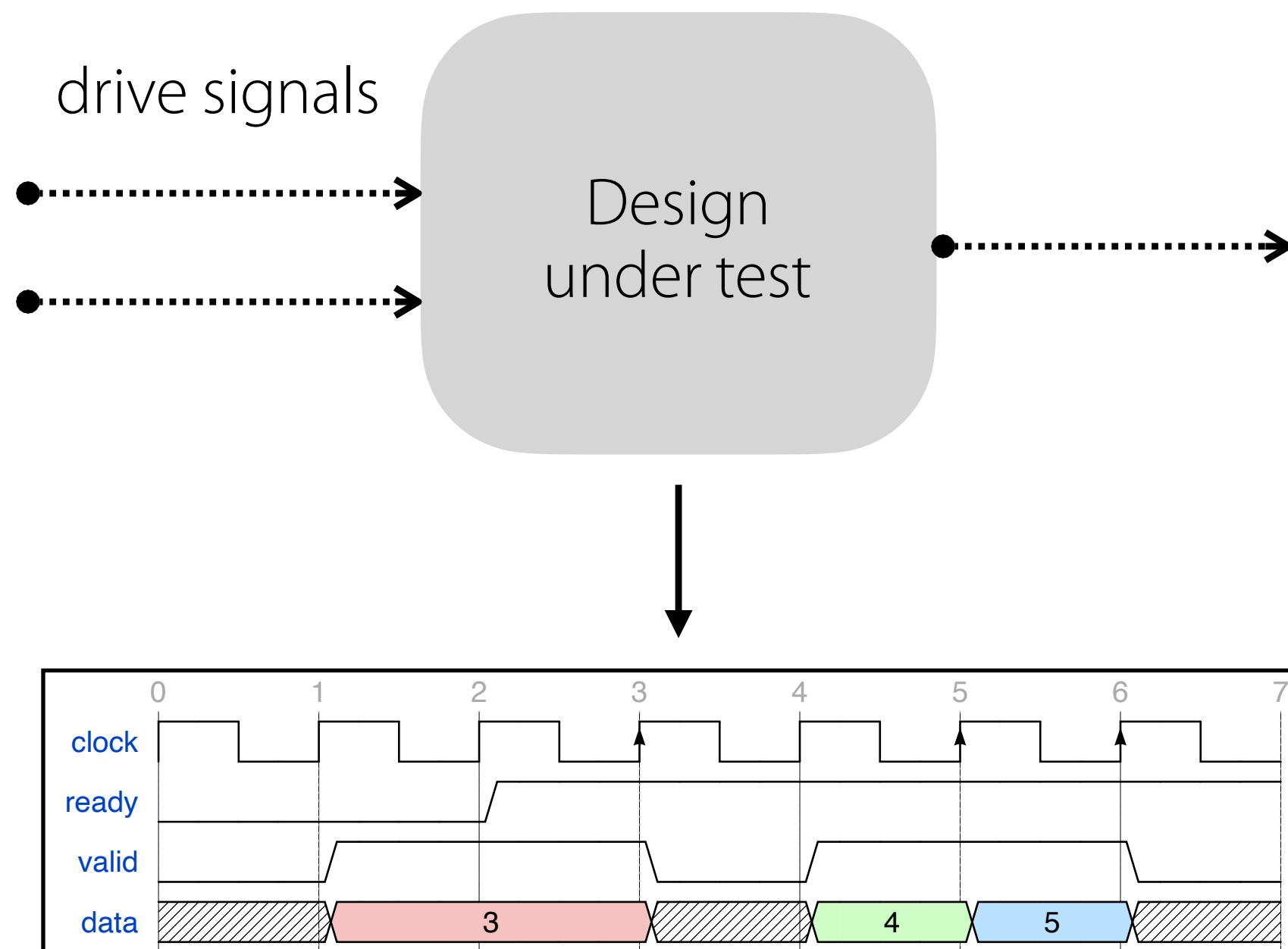
# What can we do with one single protocol?

1. Run concrete tests  
(drive hardware modules)

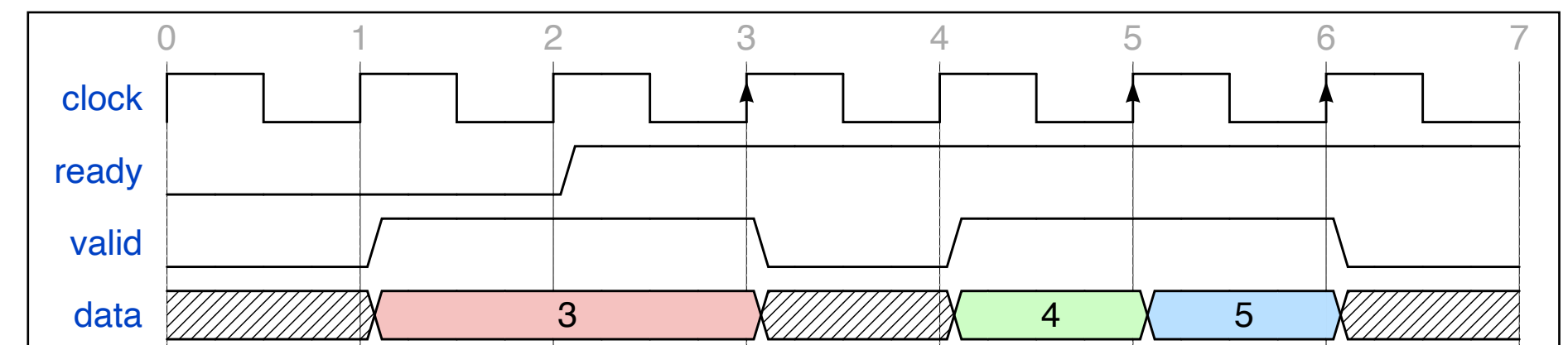


# What can we do with one single protocol?

1. Run concrete tests  
(drive hardware modules)



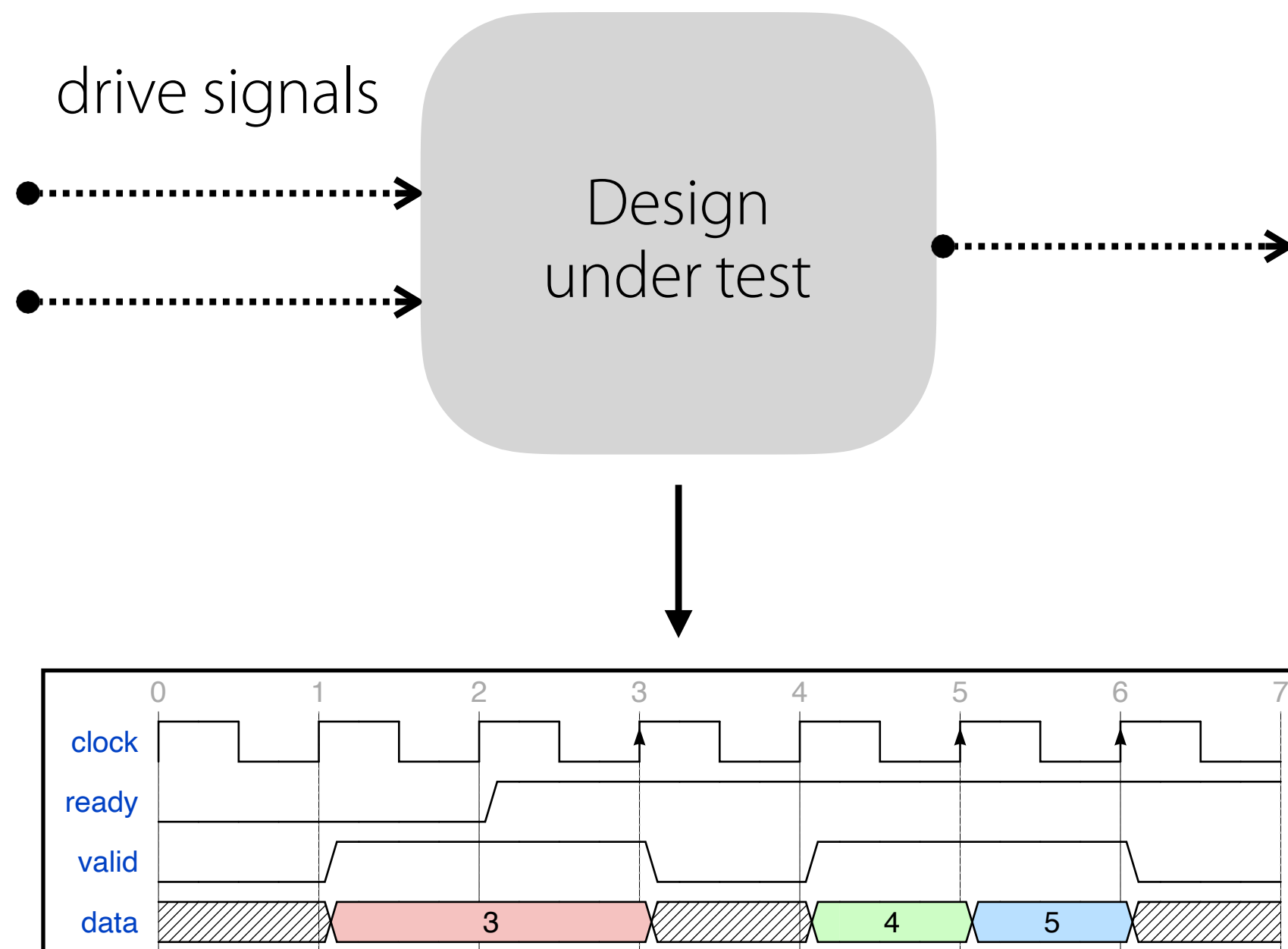
2. Infer transactions from waveforms



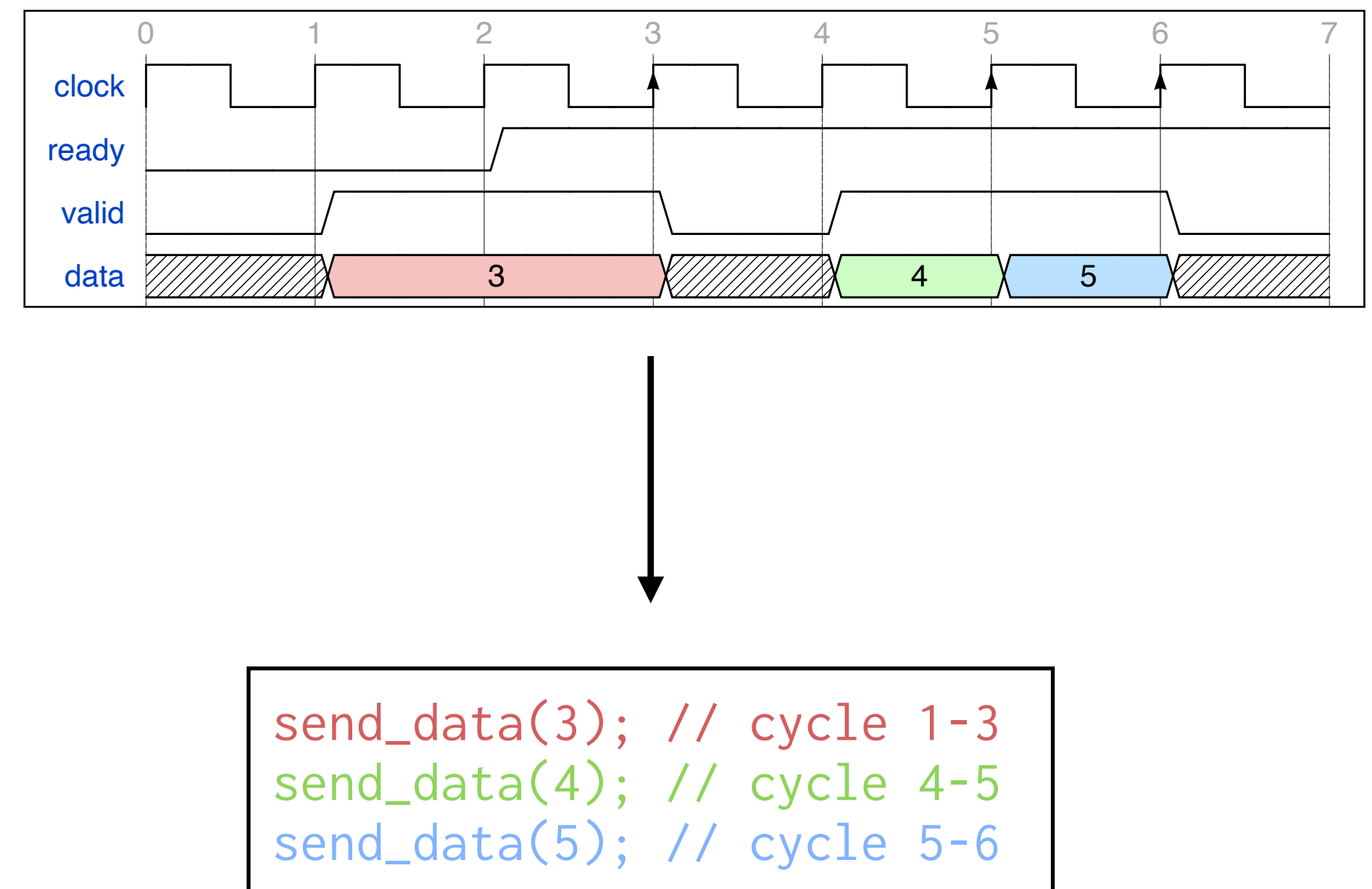
```
send_data(3); // cycle 1-3
send_data(4); // cycle 4-5
send_data(5); // cycle 5-6
```

# Our DSL: write one protocol spec, do both!

1. Run concrete tests  
(drive hardware modules)



2. Infer transactions from waveforms



Paso comes with two tools:

- an **interpreter** (runs tests)
- a ***reconstructor*** (infers transactions from signals)

# Interpreter

Hardware module  
implementation

Transactions to execute

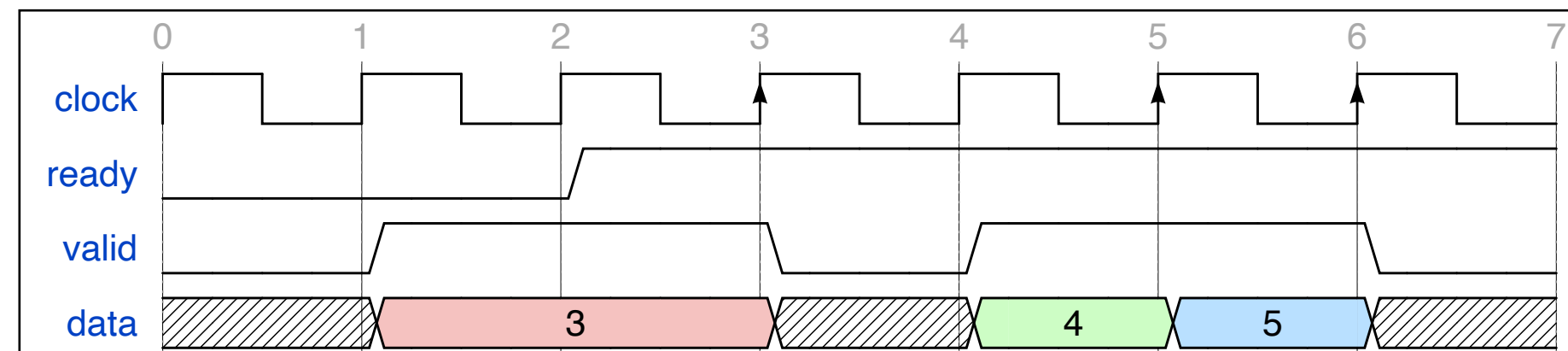
Paso protocol spec



```
send_data(3);  
send_data(4);  
send_data(5);
```

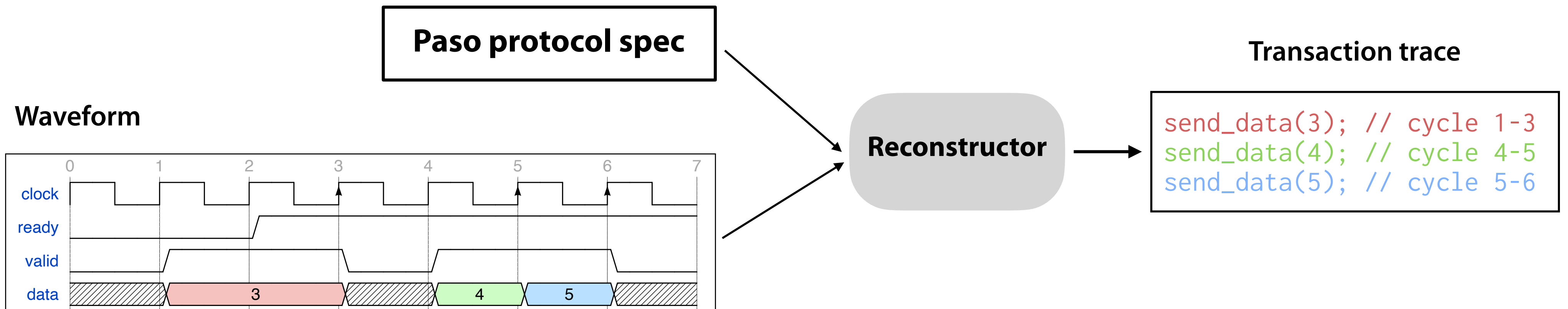
Interpreter

Waveform



# Reconstructor

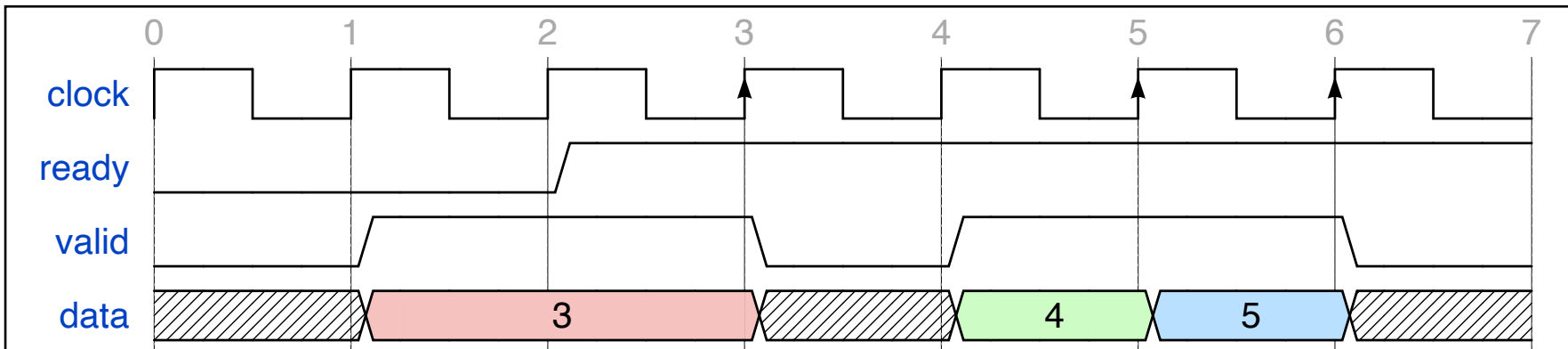
(the cool part!)



# Applying the Reconstructor on the Ready-Valid Example

Given the protocol definition and a waveform,  
the reconstructor infers a series of transactions that are consistent with the waveform

```
prot send_data<DUT: ReadyValid>(data: u8) { ... }
```



**Reconstructor**

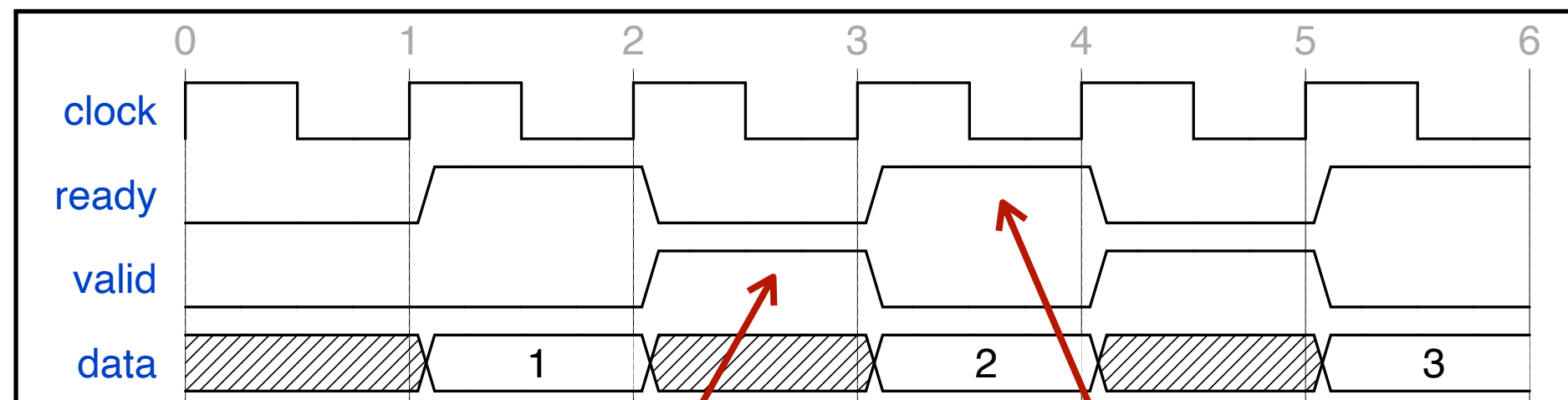
```
send_data(3); // cycle 1-3  
send_data(4); // cycle 4-5  
send_data(5); // cycle 5-6
```

**Abstraction level raised from signals to transactions!**

# Applying the Reconstructor on the Ready-Valid Example

If no transactions could have resulted in the waveform trace,  
the reconstructor warns the user

```
prot send_data<DUT: ReadyValid>(data: u8) { ... }
```



ready & valid are never both 1  
during the same cycle!  
(i.e. data transfer never occurs)

**Reconstructor**

Error: no matching  
transactions

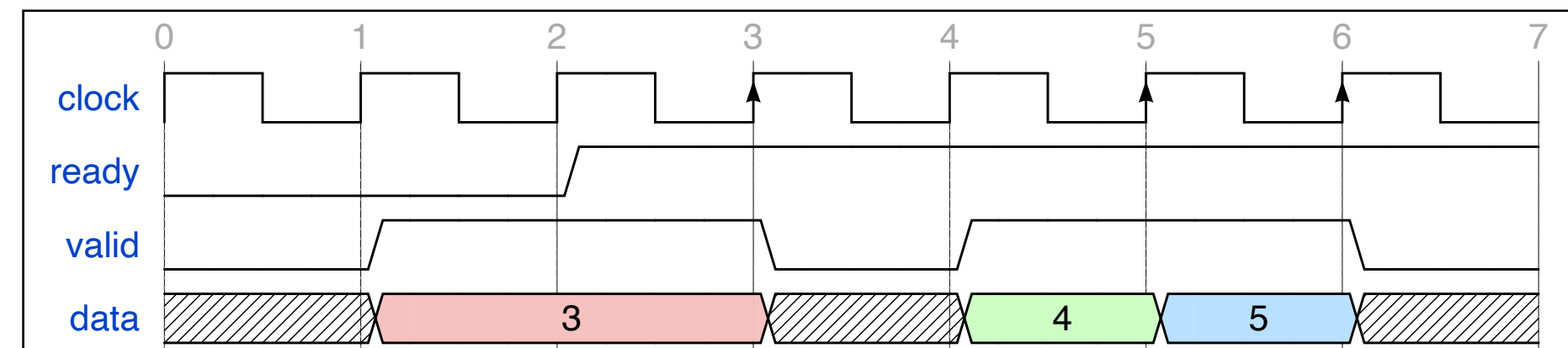
# Round-trip property

Interpreter

Waveform

User-supplied  
transactions

```
send_data(3);  
send_data(4);  
send_data(5);
```



Reconstructor

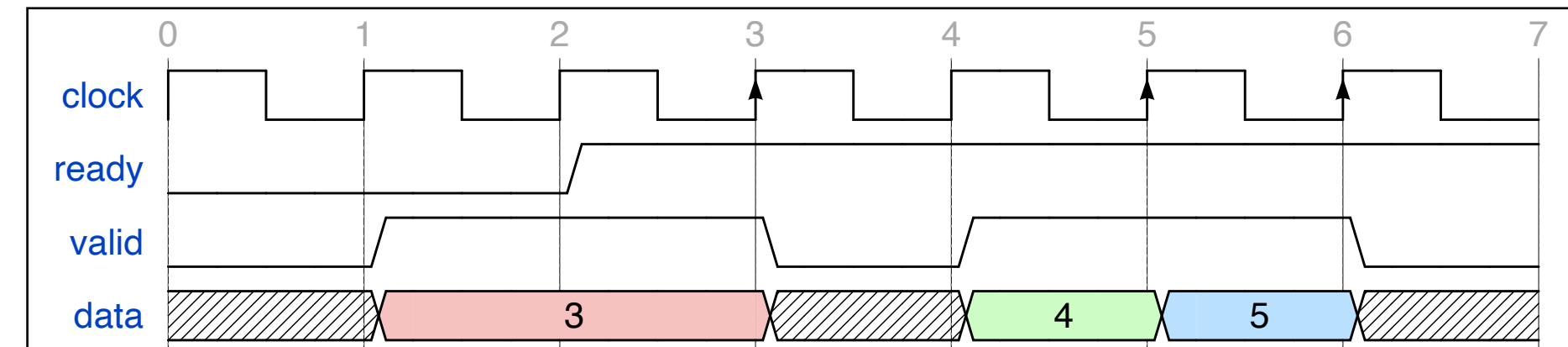
# Round-trip property

Interpreter

Waveform

User-supplied  
transactions

```
send_data(3);  
send_data(4);  
send_data(5);
```



Reconstructor

Designing the  
reconstructor involves  
“reversing” Paso’s semantics!

# How does the reconstructor work?

# Assignments as Constraints

`foo := 0`

means

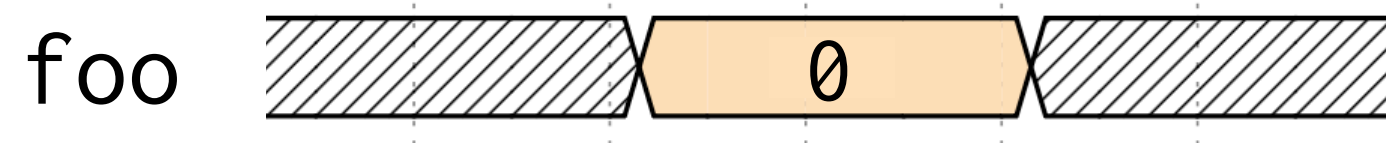
“Check if the current waveform value of `foo` is equal to **0**”

# Assignments as Constraints

`foo := 0`

means

“Check if the current waveform value of `foo` is equal to **0**”



Matches!

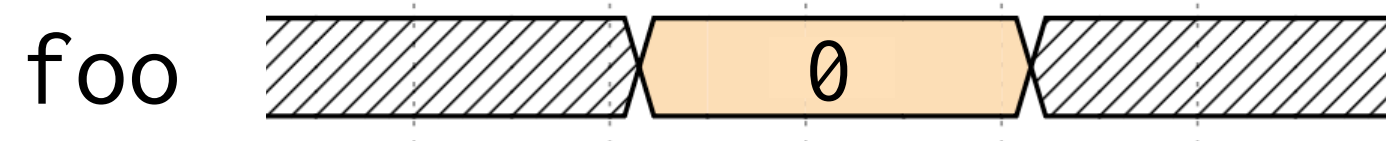


# Assignments as Constraints

`foo := 0`

means

“Check if the current waveform value of `foo` is equal to `0`”



Matches!



Inconsistent!

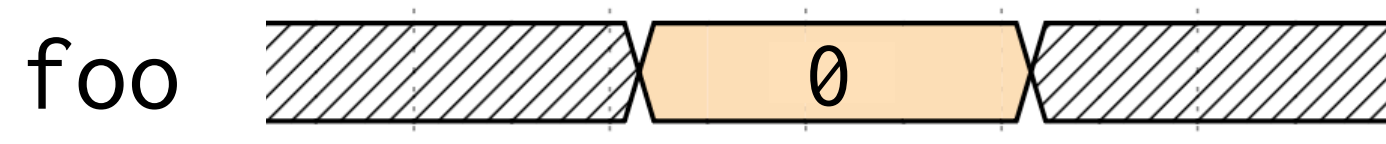


# Assignments as Constraints

`foo := 0`

means

“Check if the current waveform value of `foo` is equal to `0`”



Matches!



Inconsistent!



Treat `foo == 0` as a constraint for the rest of the protocol  
(or until `foo` is reassigned)

## Assignment in protocol body

LHS := RHS



## Constraint in reconstructor

RHS = waveform value of LHS  
at the current clock cycle

## Assignment in protocol body

LHS := RHS



## Constraint in reconstructor

RHS = waveform value of LHS  
at the current clock cycle

foo := 0



0 = waveform(foo)

foo := a



a = waveform(foo)

## Assignment in protocol body

LHS := RHS



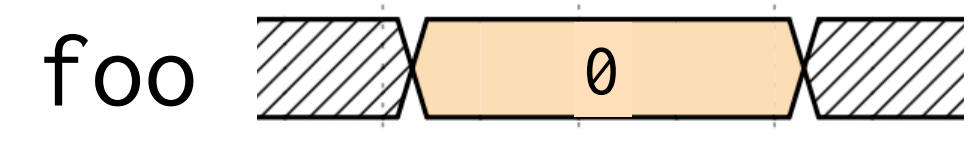
## Constraint in reconstructor

RHS = waveform value of LHS  
at the current clock cycle

foo := 0



0 = 0

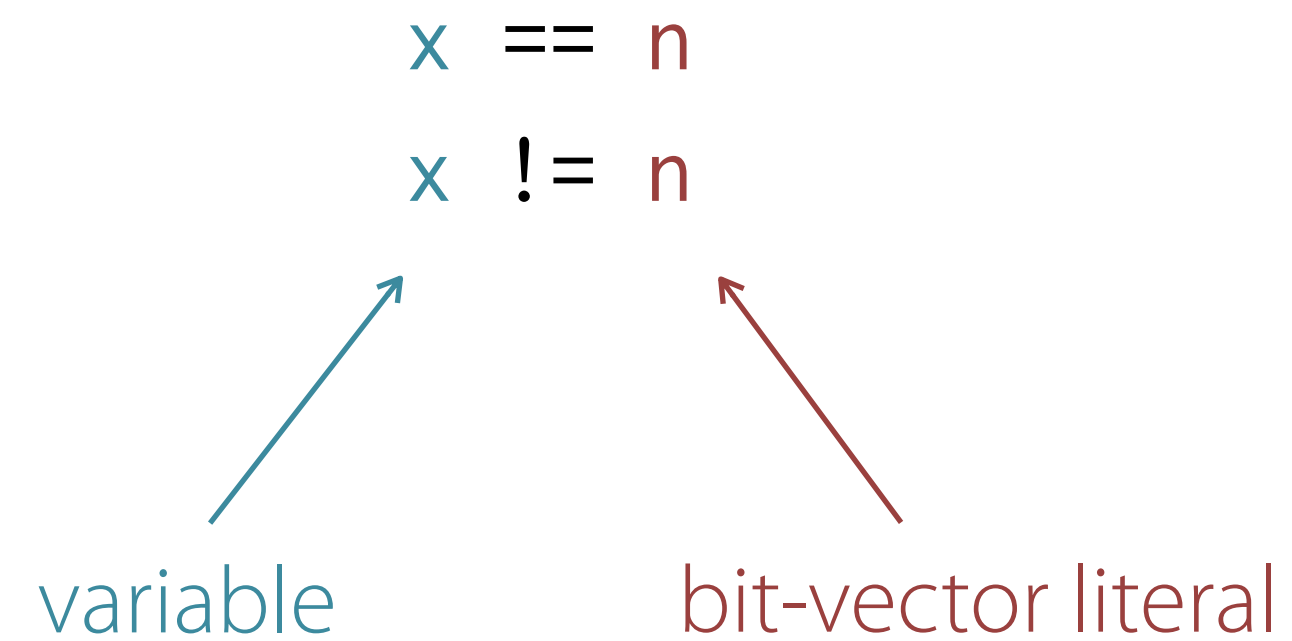


foo := a



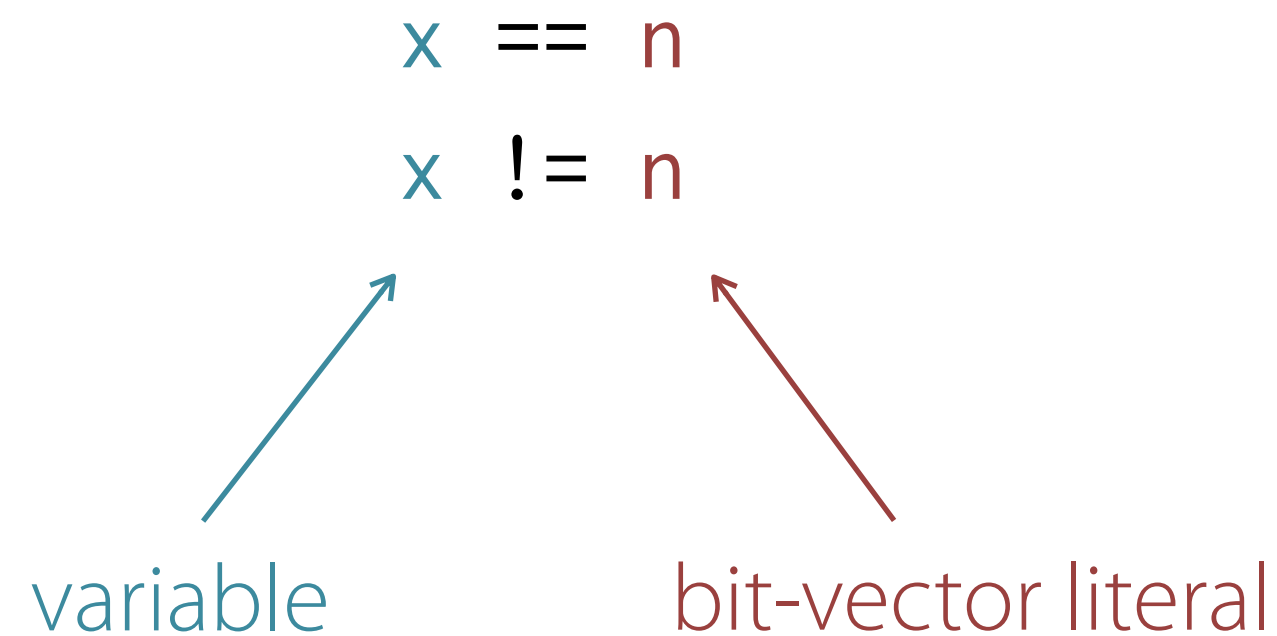
a = 0

# No SMT solvers needed!



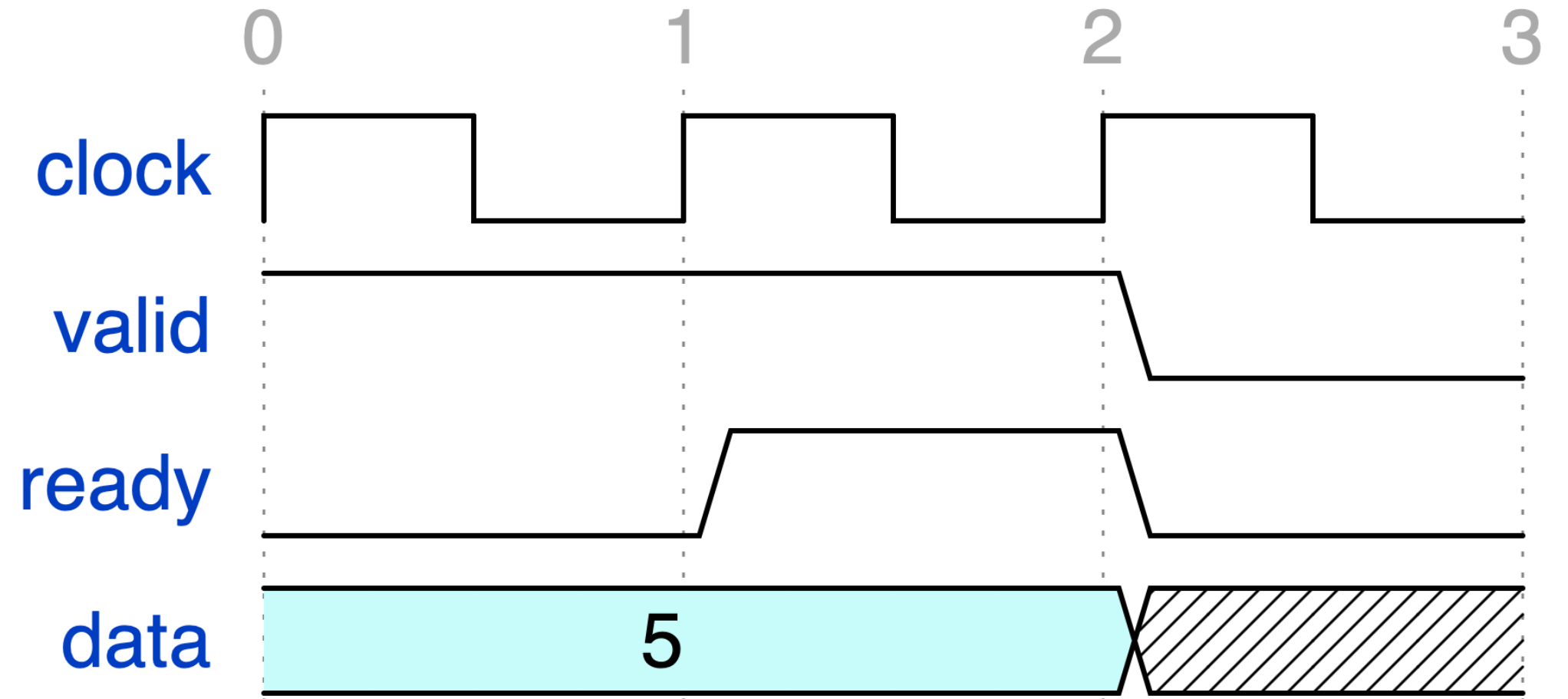
# No SMT solvers needed!

All constraints in our DSL are equality constraints where one side is a constant  
 $\implies$  can be solved efficiently (by inspecting waveform) without relying on solvers



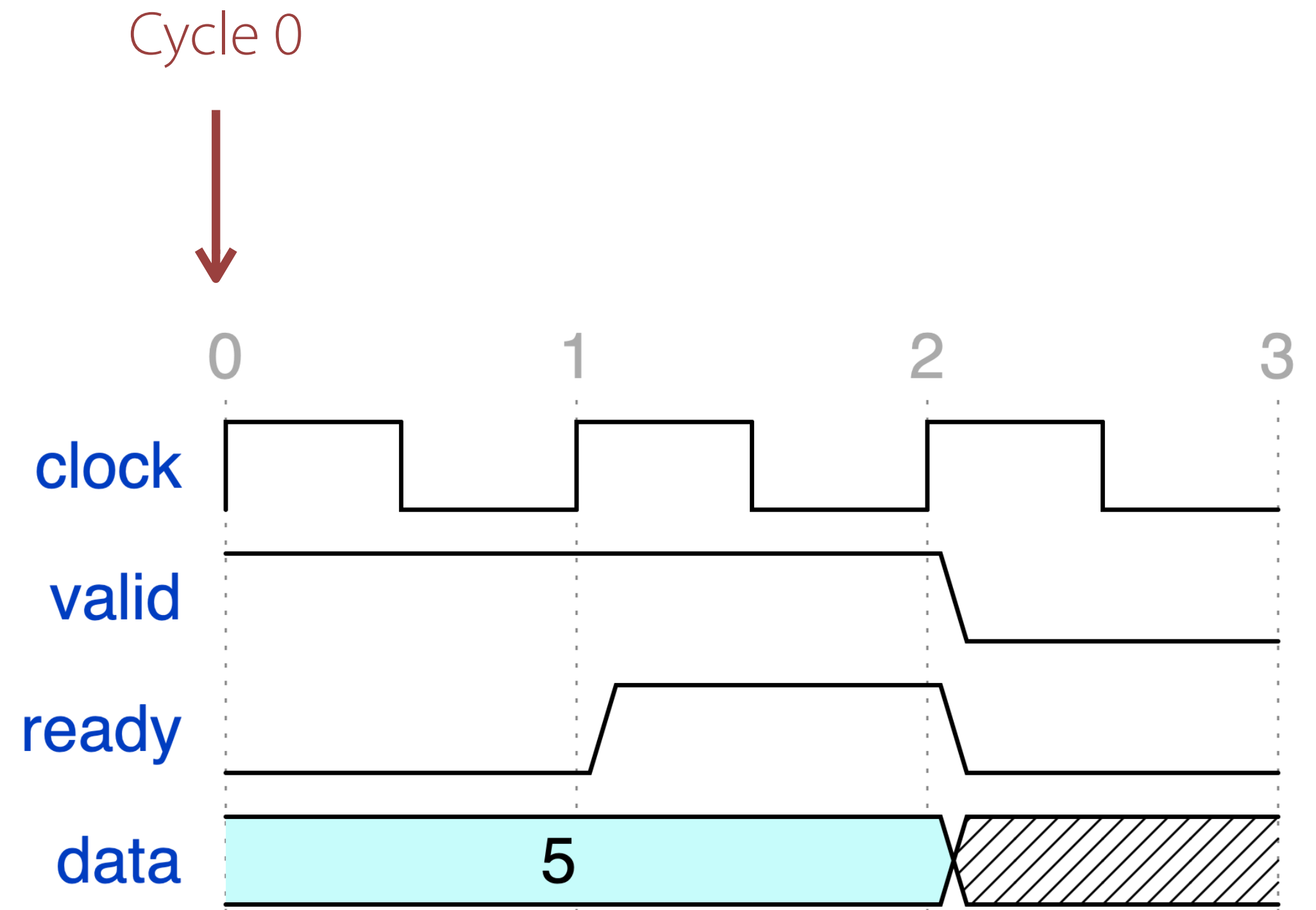
# Idea: symbolically execute protocols & compare against waveform

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



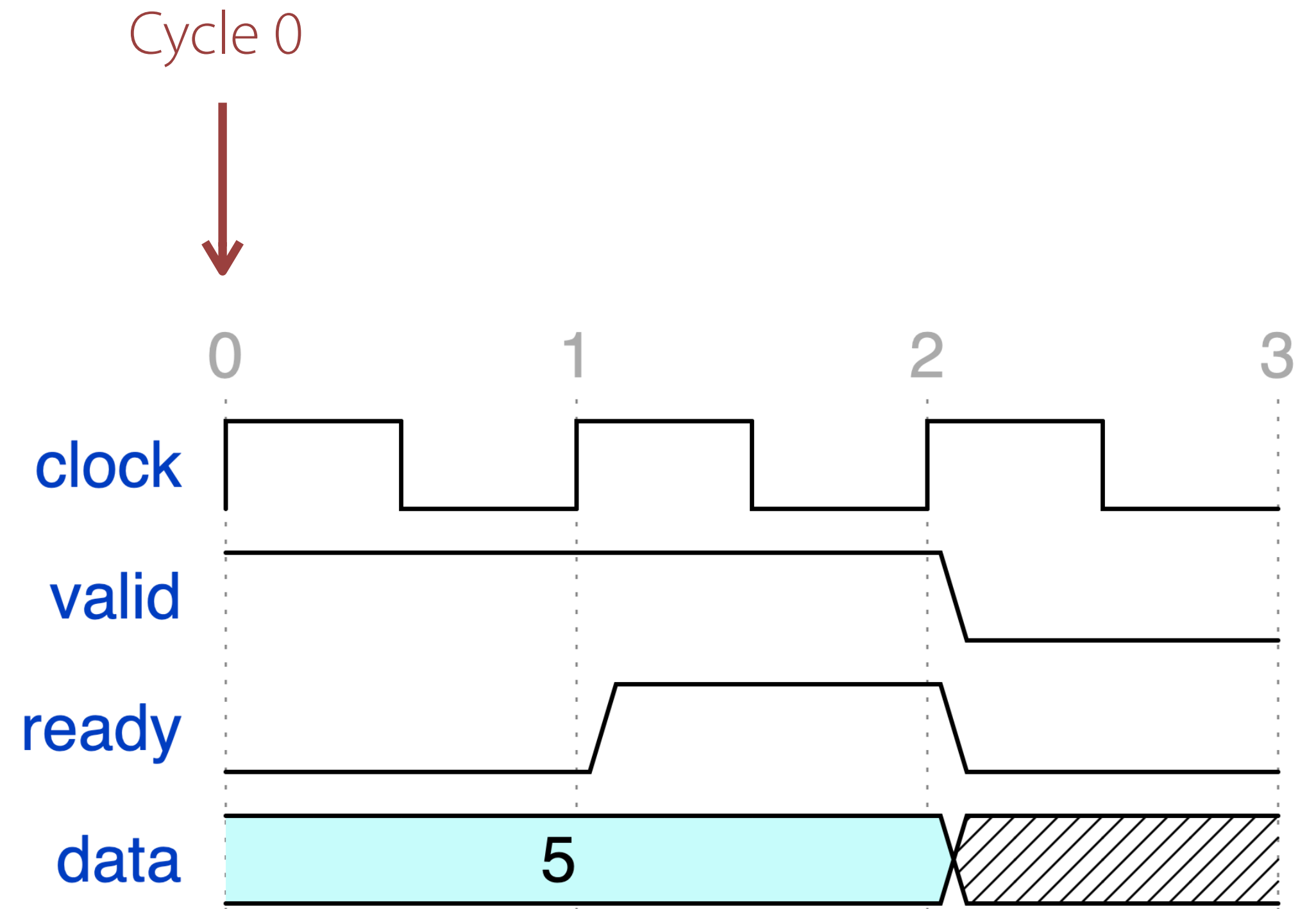
Constraints:  $\emptyset$

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



Constraints:  $\emptyset$

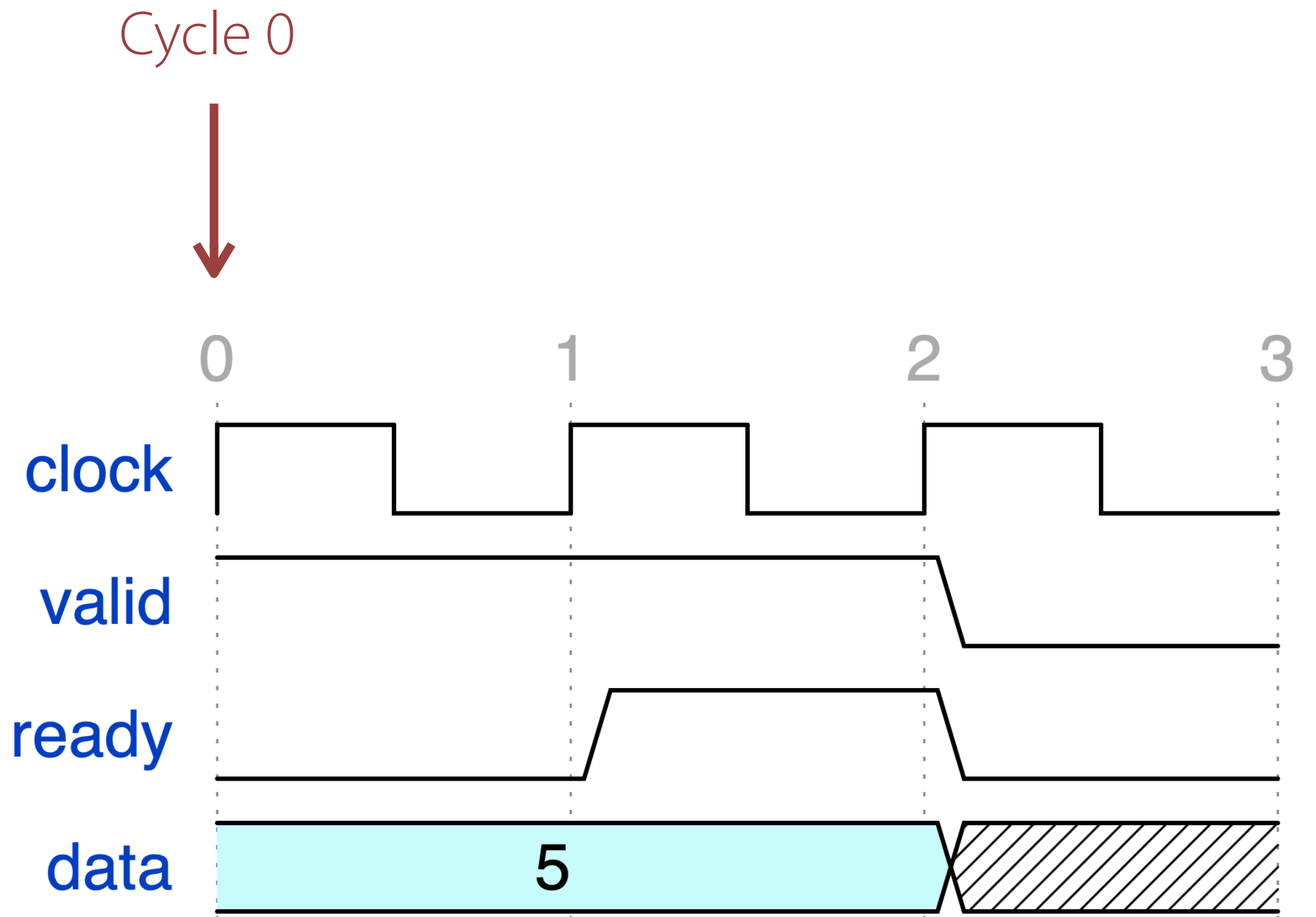
```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



Constraints:  $\emptyset$

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```

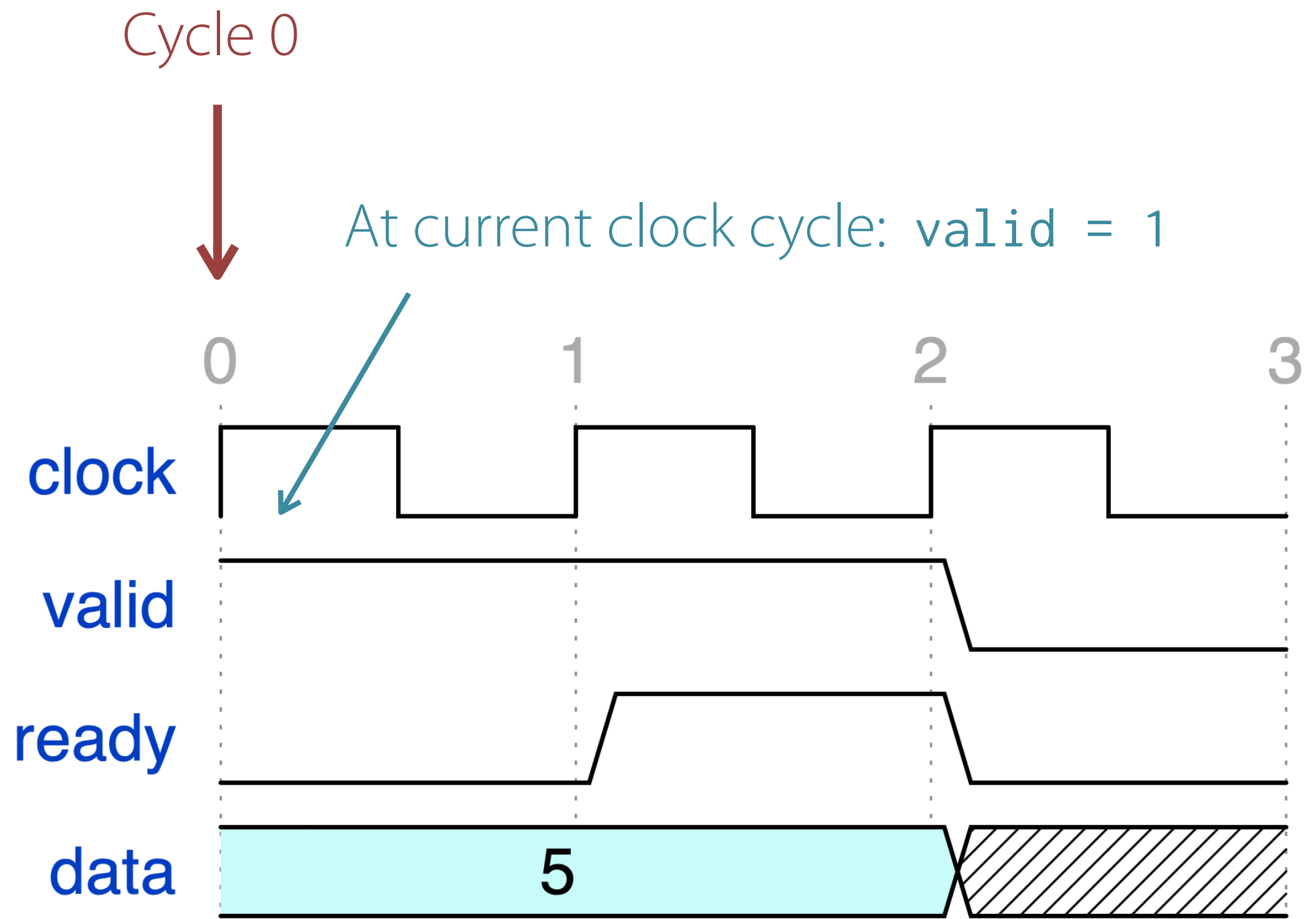
Check if current waveform  
value of `valid`  $=?$  1



Constraints:  $\emptyset$

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```

Check if current waveform  
value of `valid`  $=?$  1



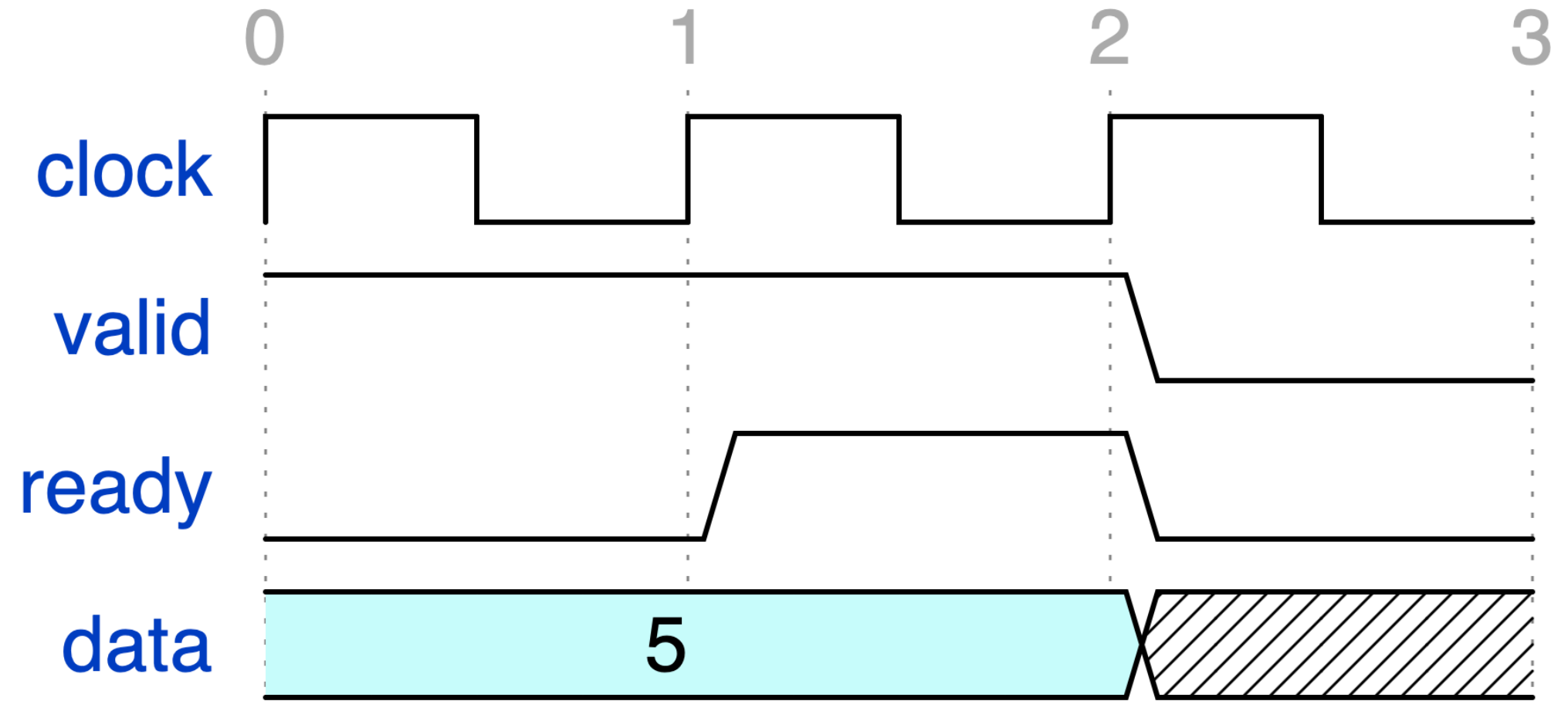
Constraints: { 1 = 1 }

Cycle 0

DUT.valid := 1 is consistent w/ waveform!  
Add a new constraint.

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```

Check if current waveform  
value of valid =? 1

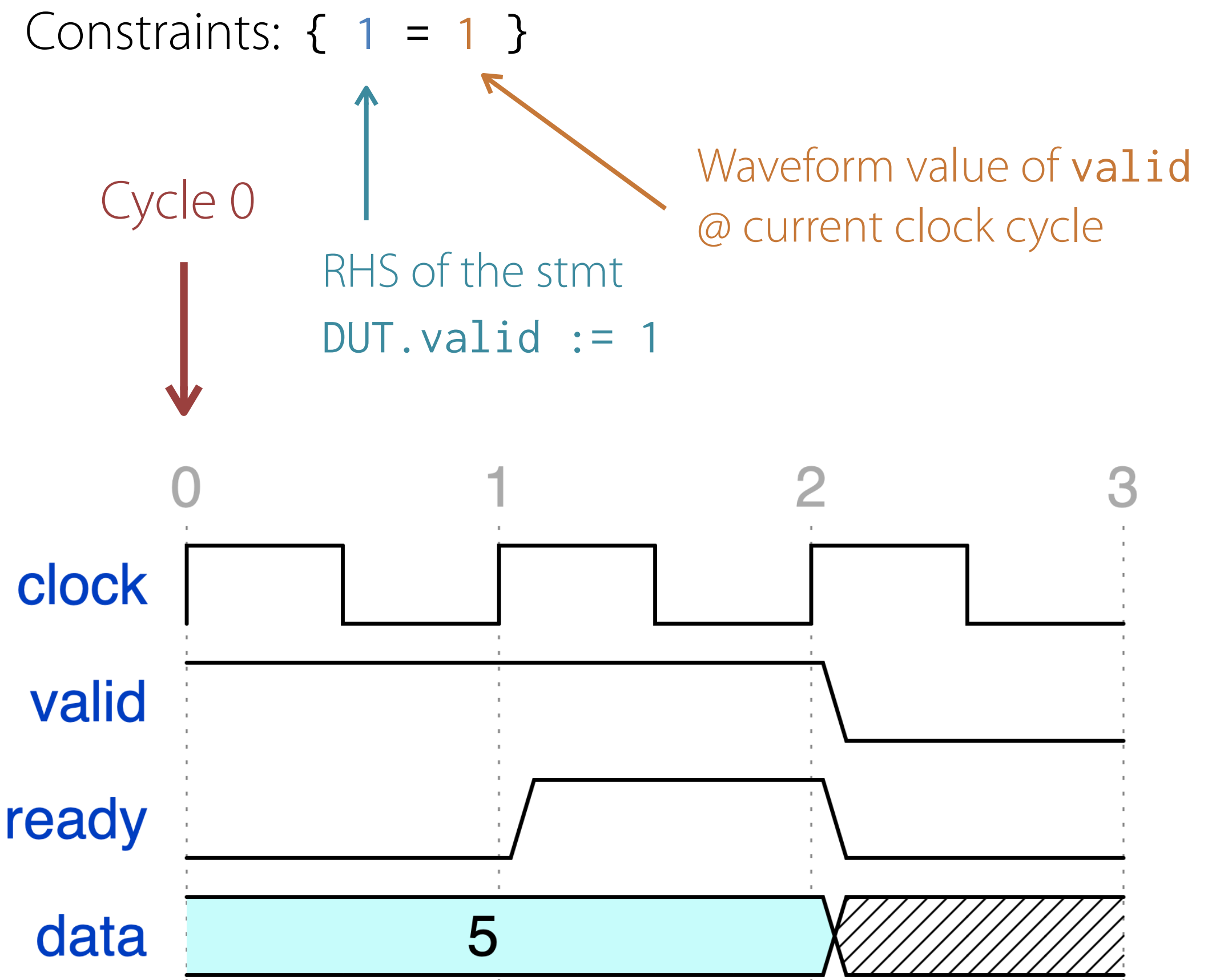


```

prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}

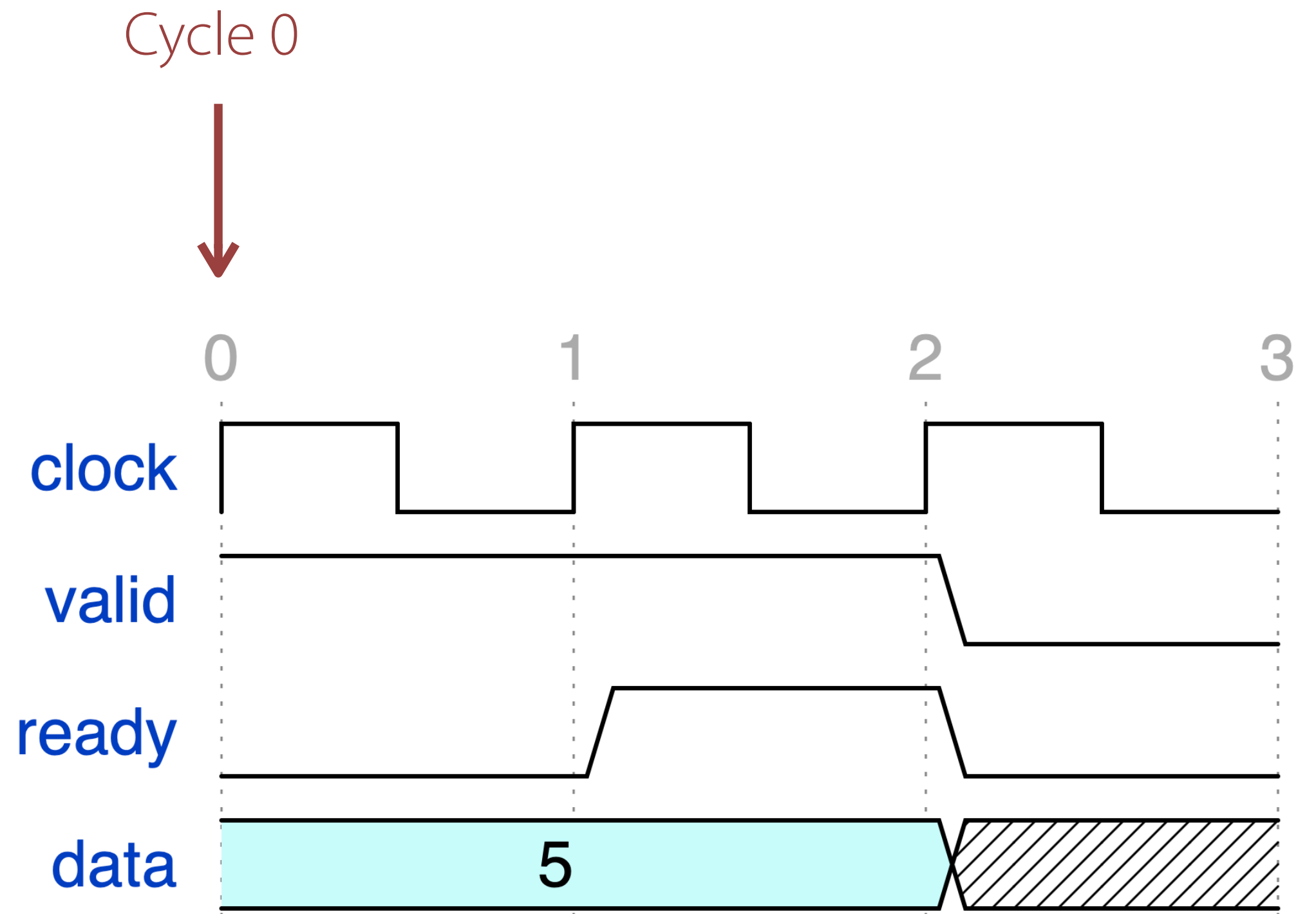
```

Check if current waveform  
value of `valid` =? 1



Constraints: { 1 = 1 }

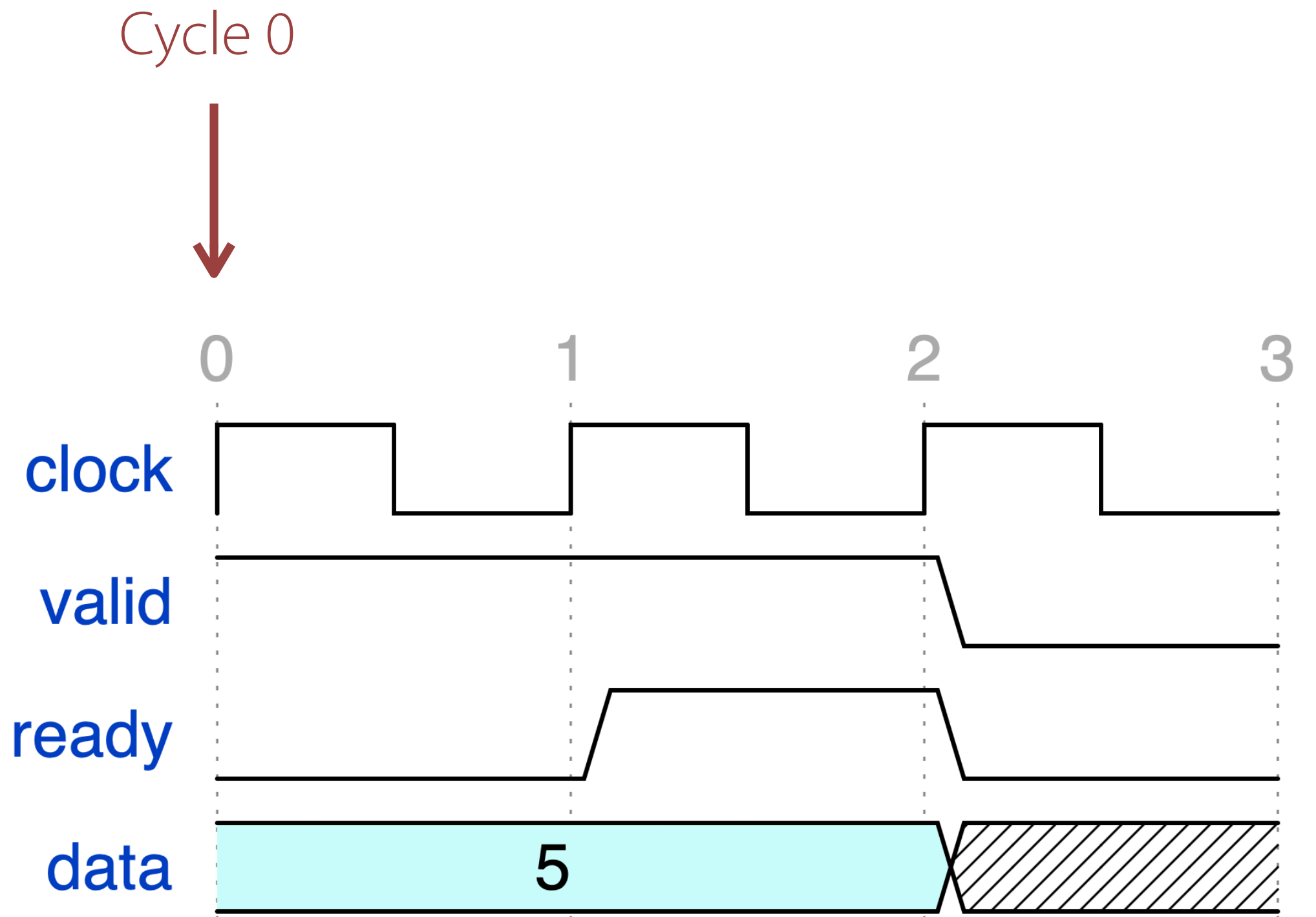
```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



Constraints: { 1 = 1 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```

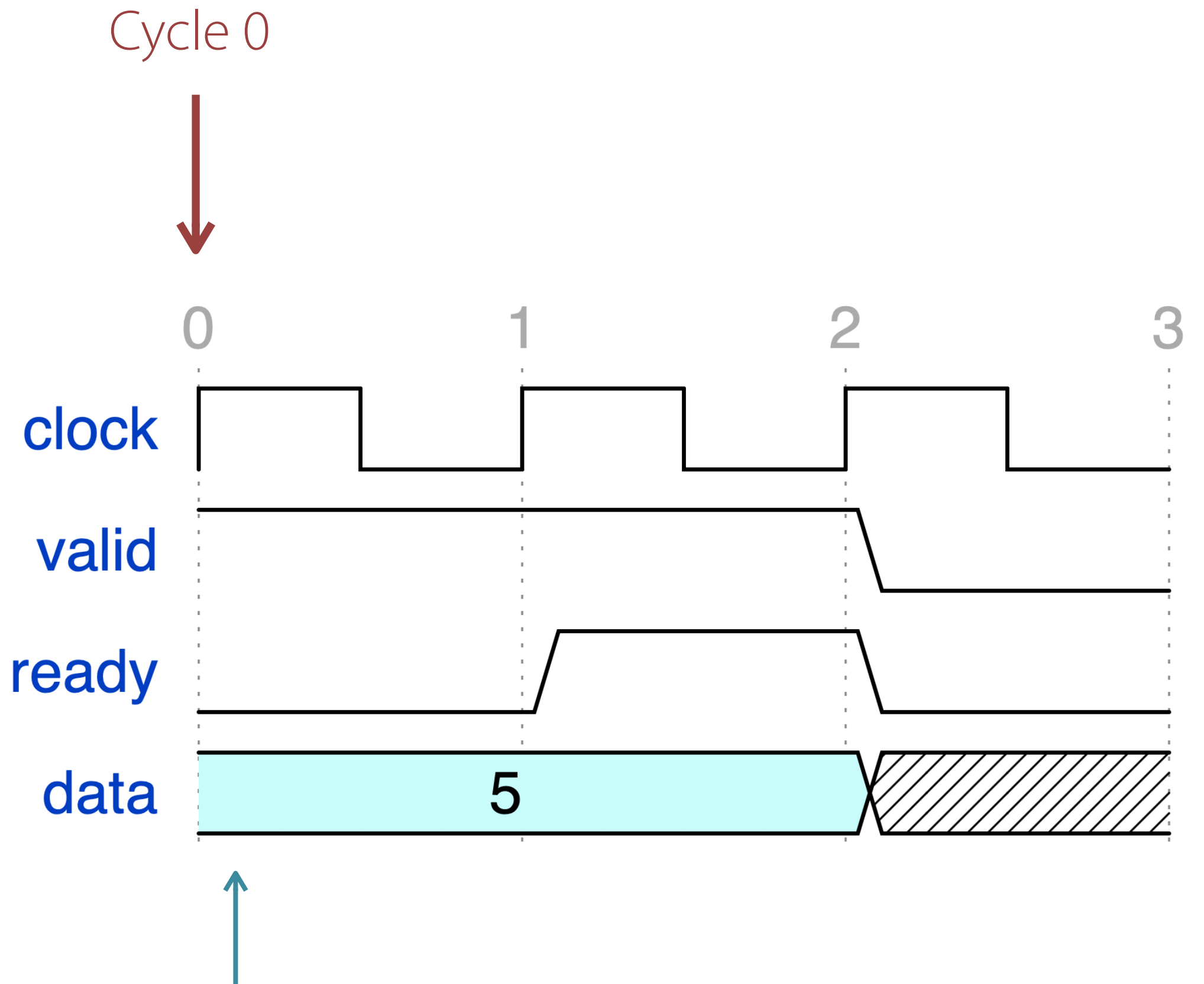
Look up current value of RHS  
(data) from waveform



Constraints: { 1 = 1 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```

Look up current value of RHS  
(data) from waveform



At current clock cycle: data = 5

Constraints: { 1 = 1, data = 5 }

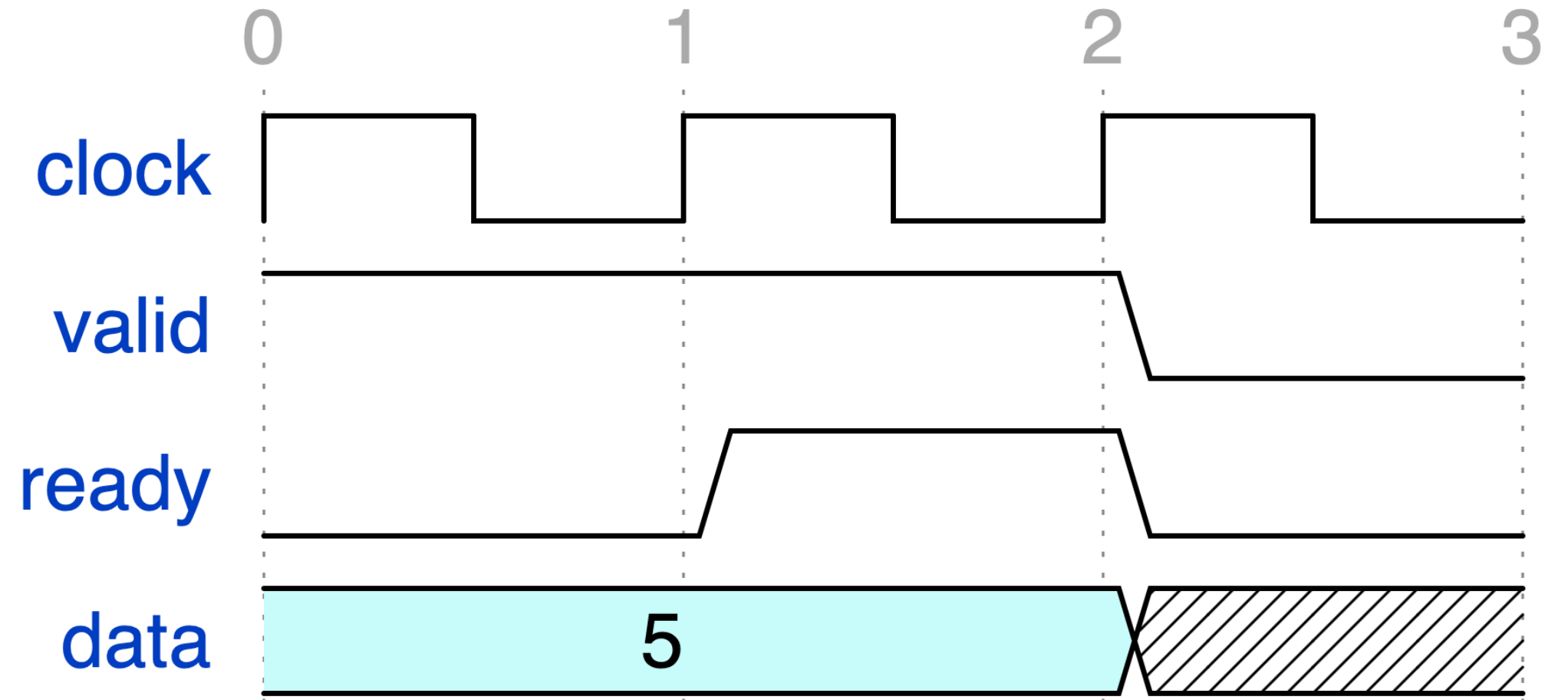


Add a new constraint data = 5

Cycle 0



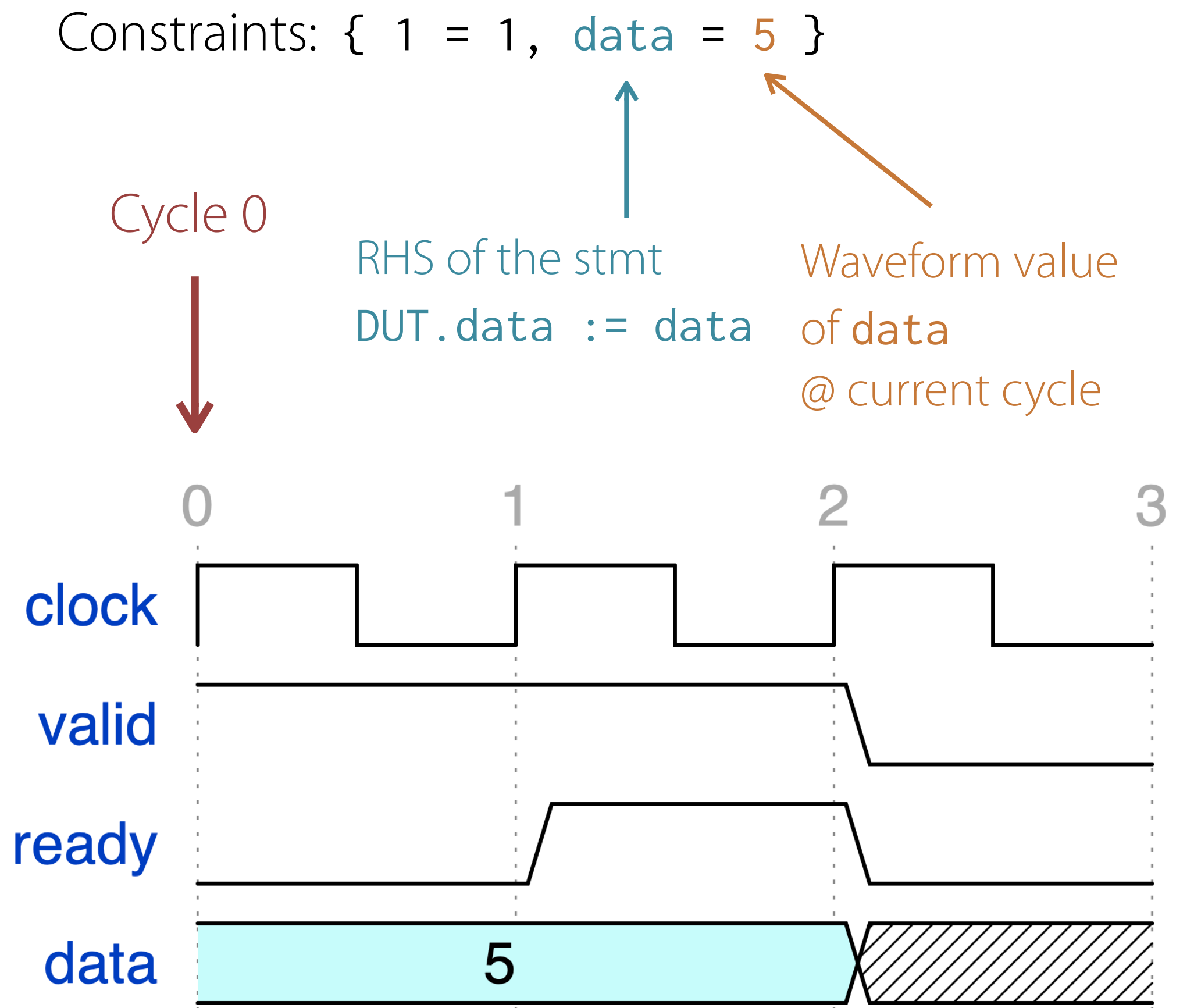
```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



```

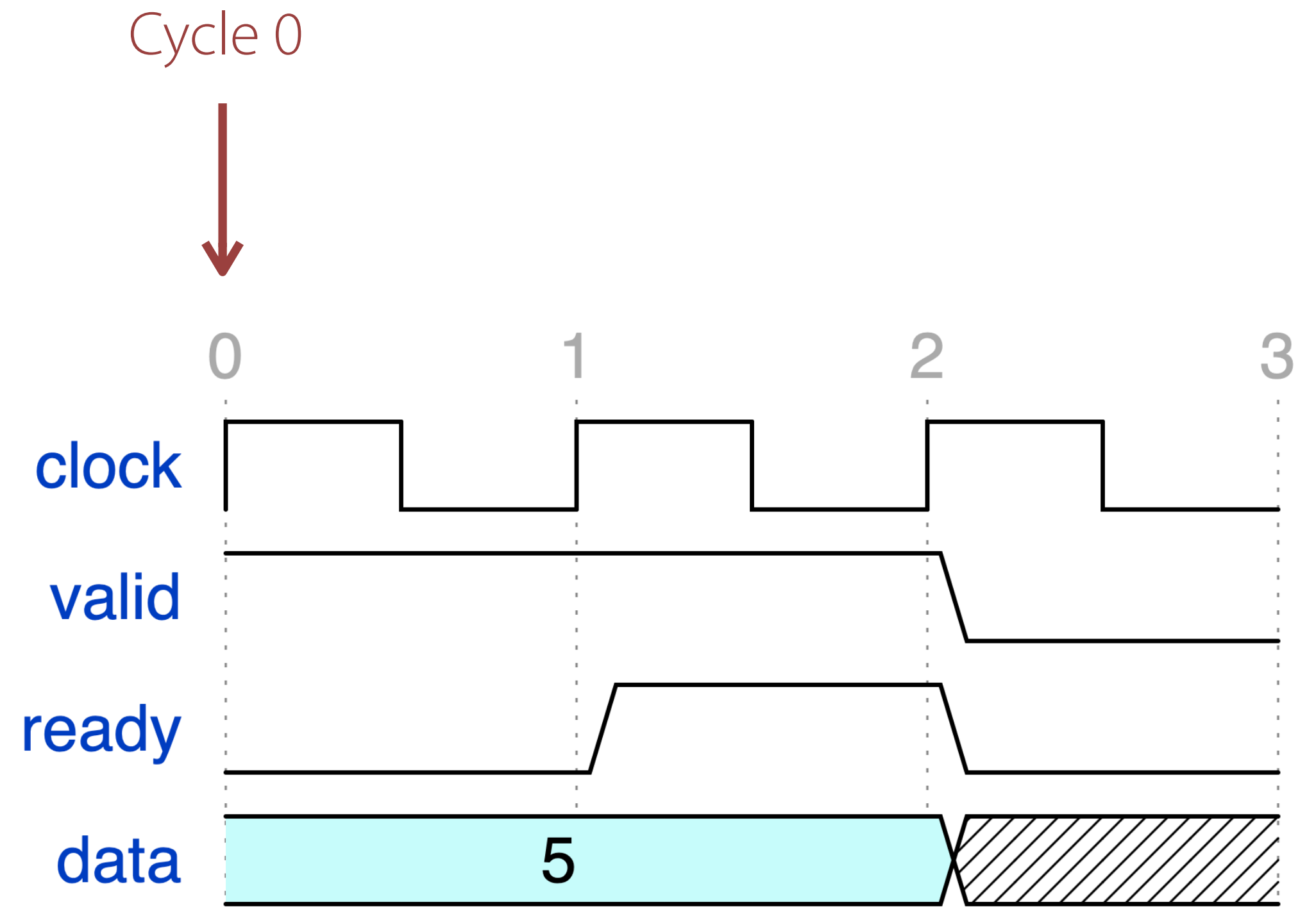
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}

```



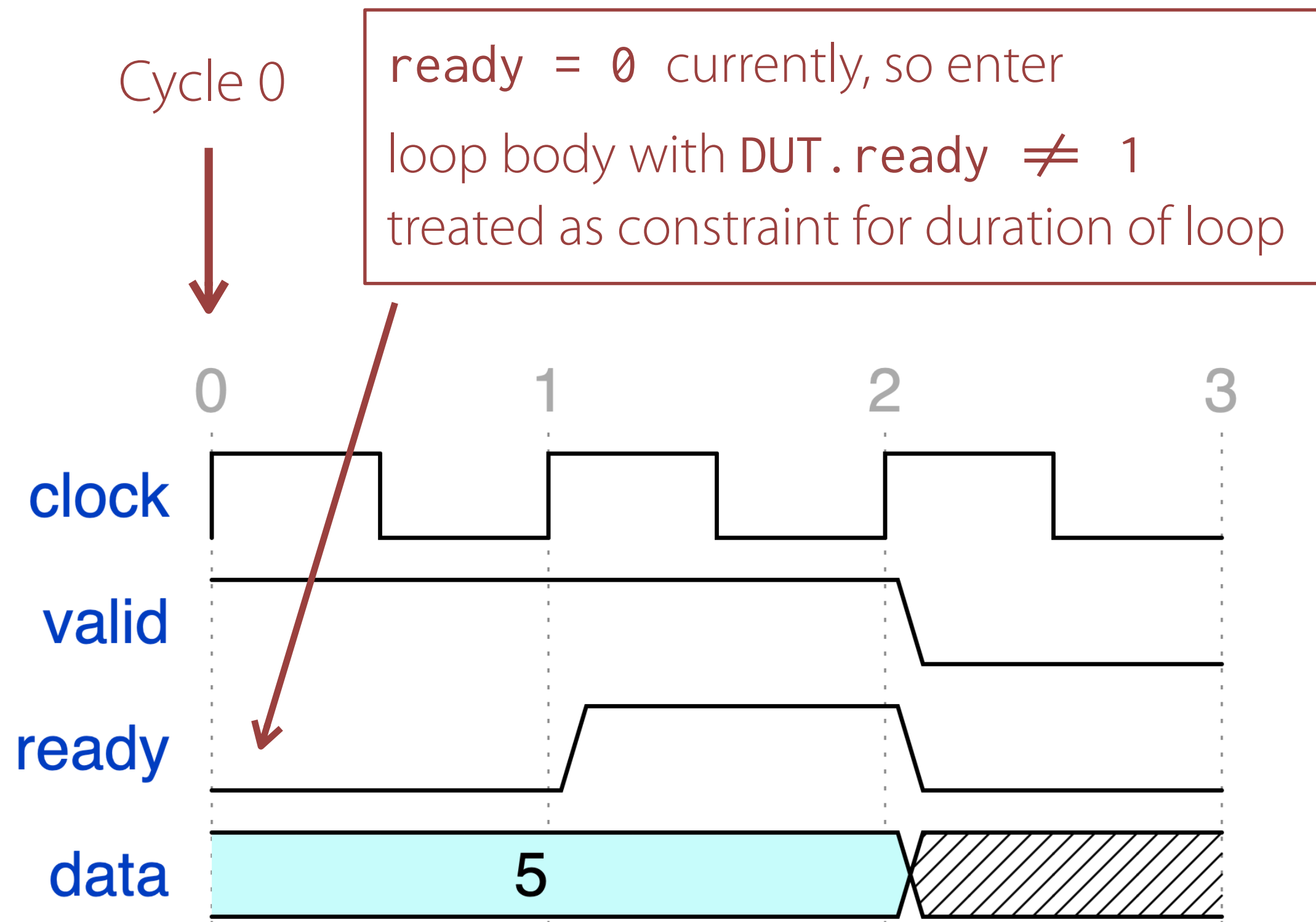
Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



Constraints: { 1 = 1, data = 5, 1  $\neq$  0 }

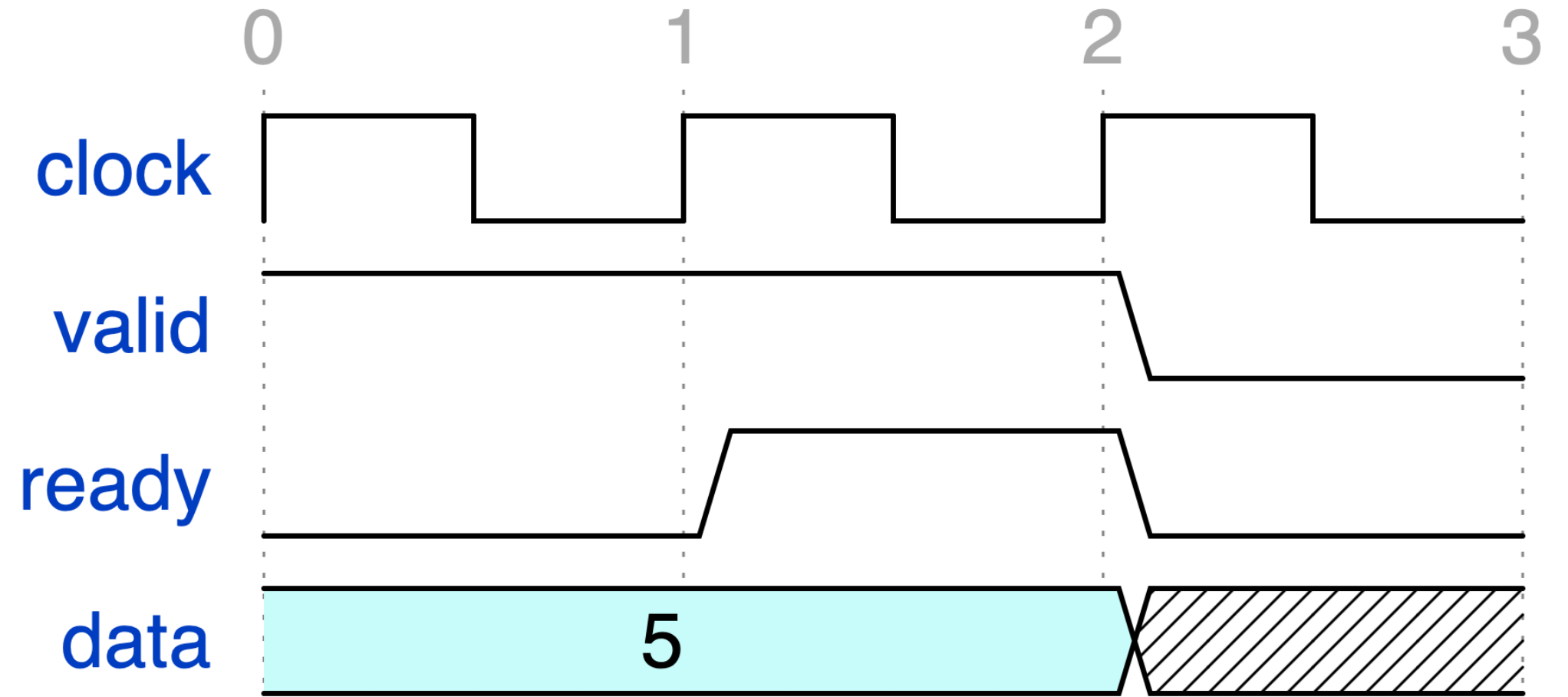


Add a new constraint 1  $\neq$  0  
representing the loop guard

Cycle 0

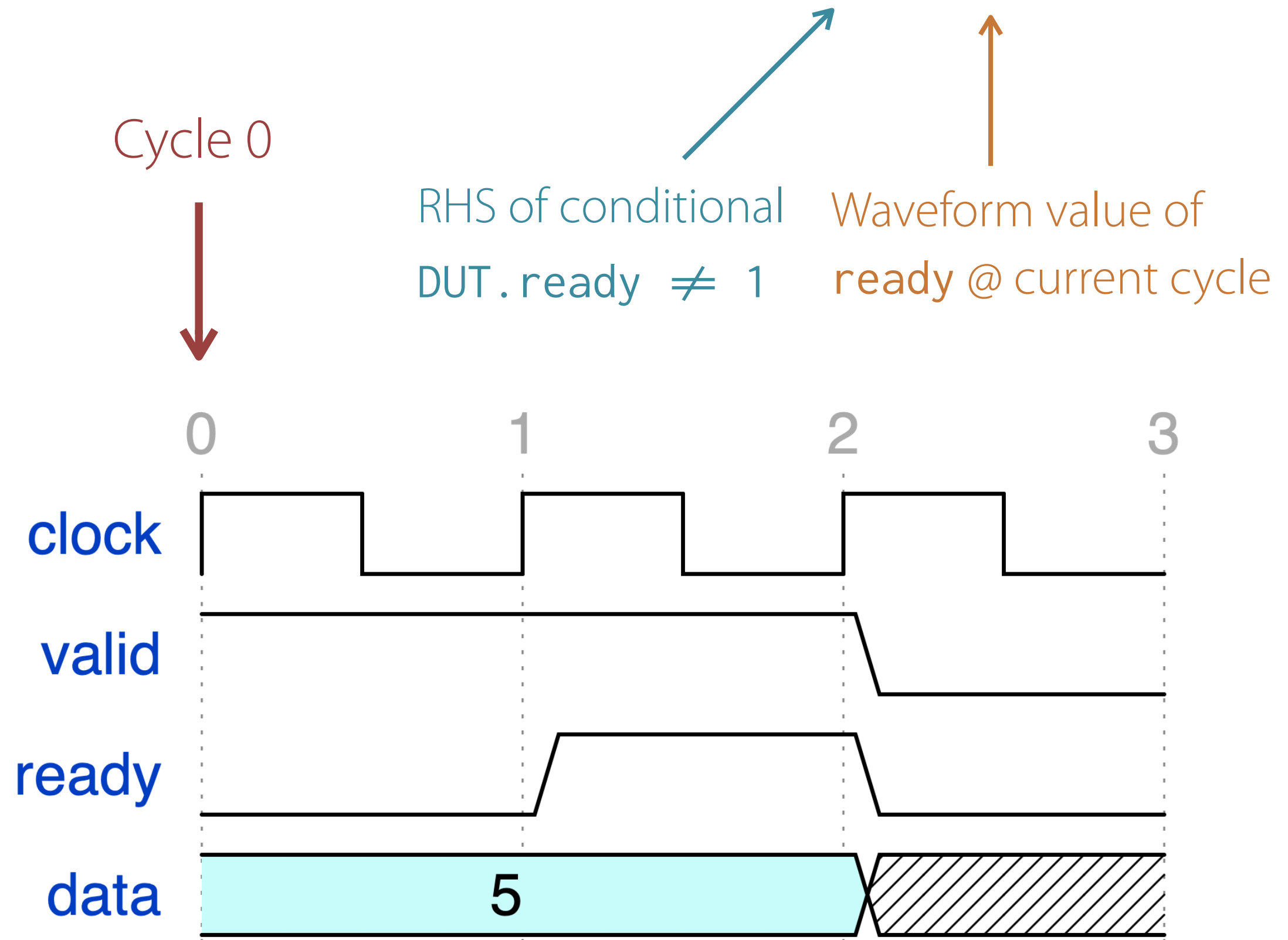


```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



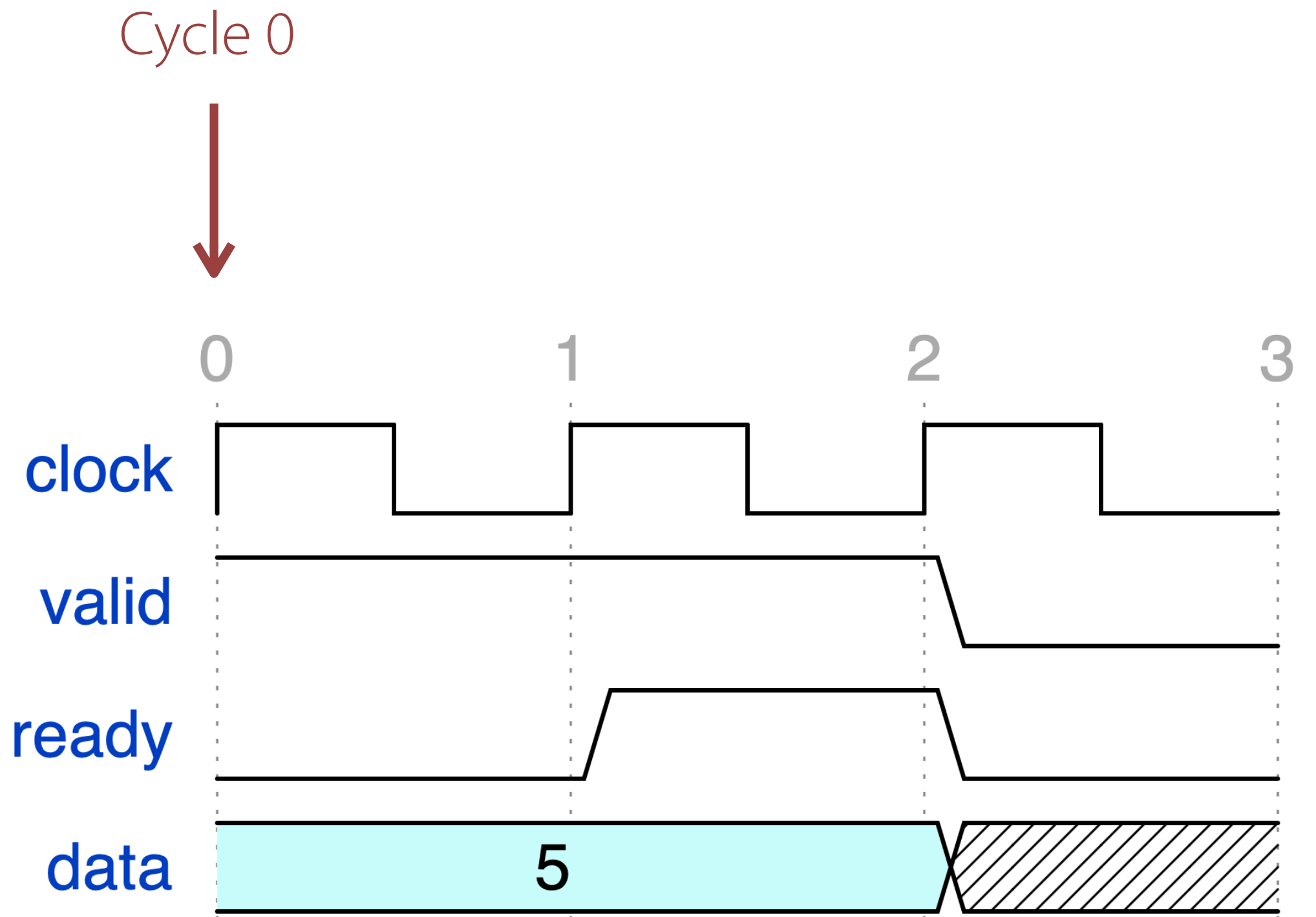
Constraints: { 1 = 1, data = 5, 1  $\neq$  0 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



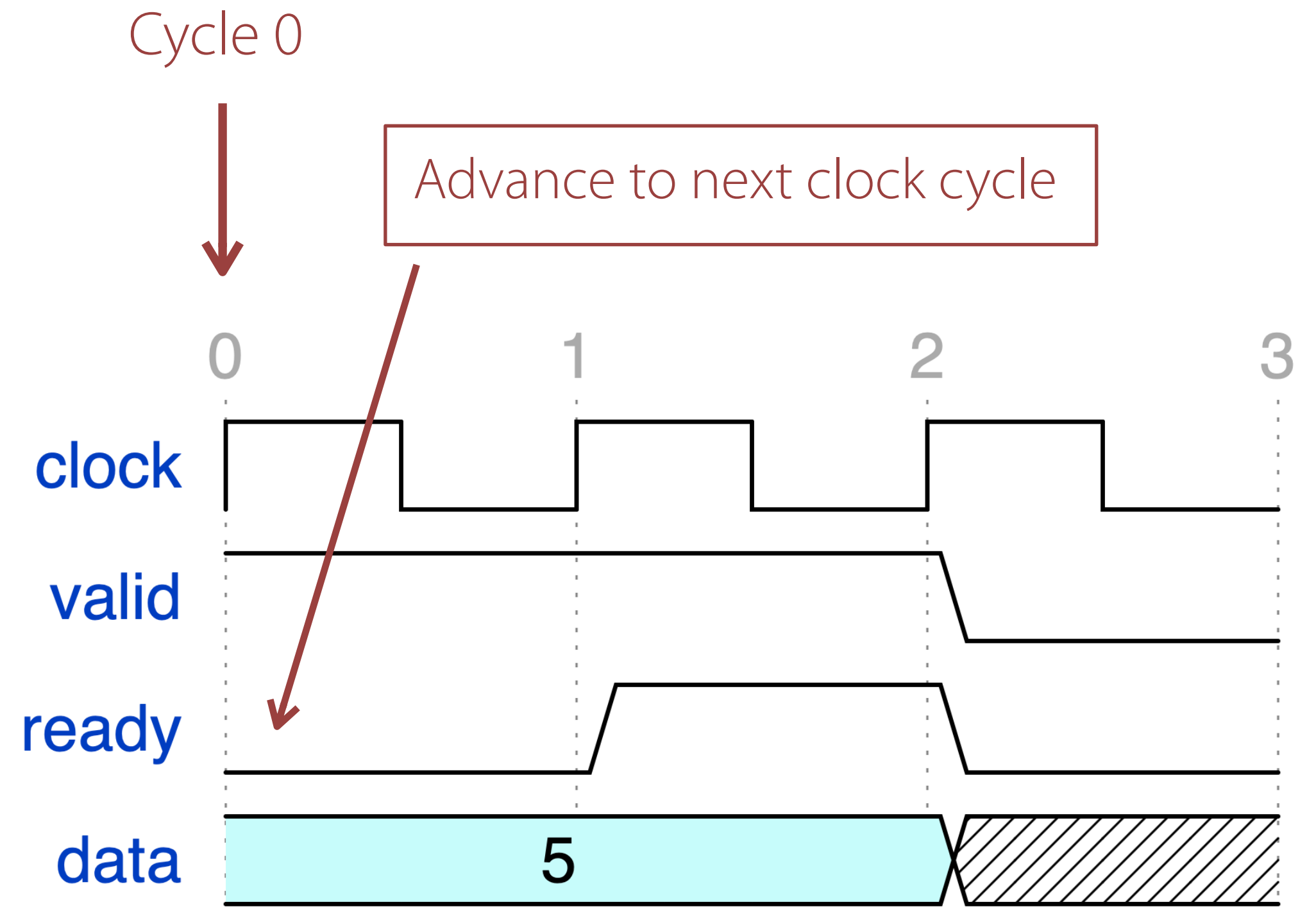
Constraints: { 1 = 1, data = 5, 1 ≠ 0 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step(); ← Evaluate loop body
  }
  step();
}
```



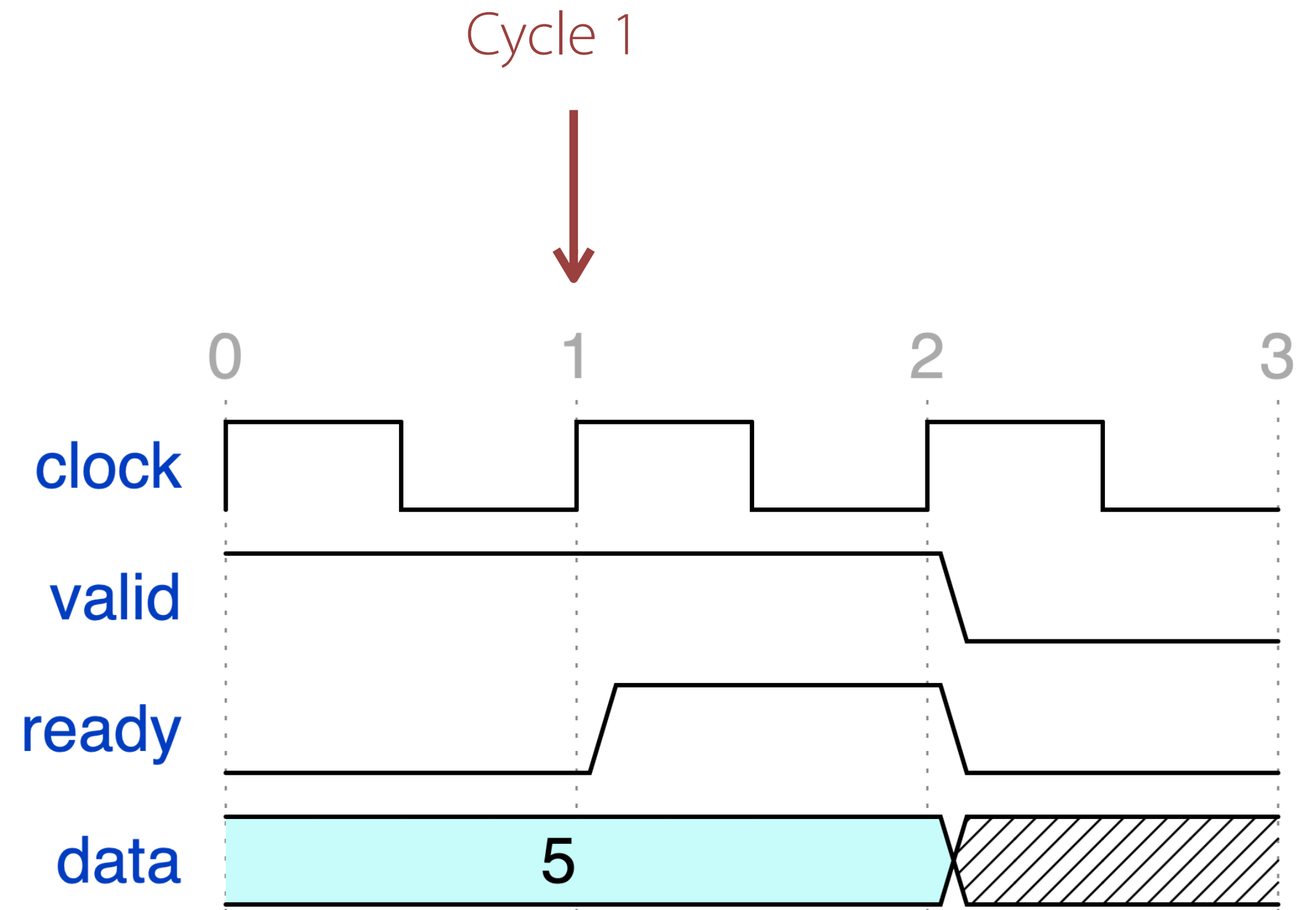
Constraints: { 1 = 1, data = 5, 1 ≠ 0 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



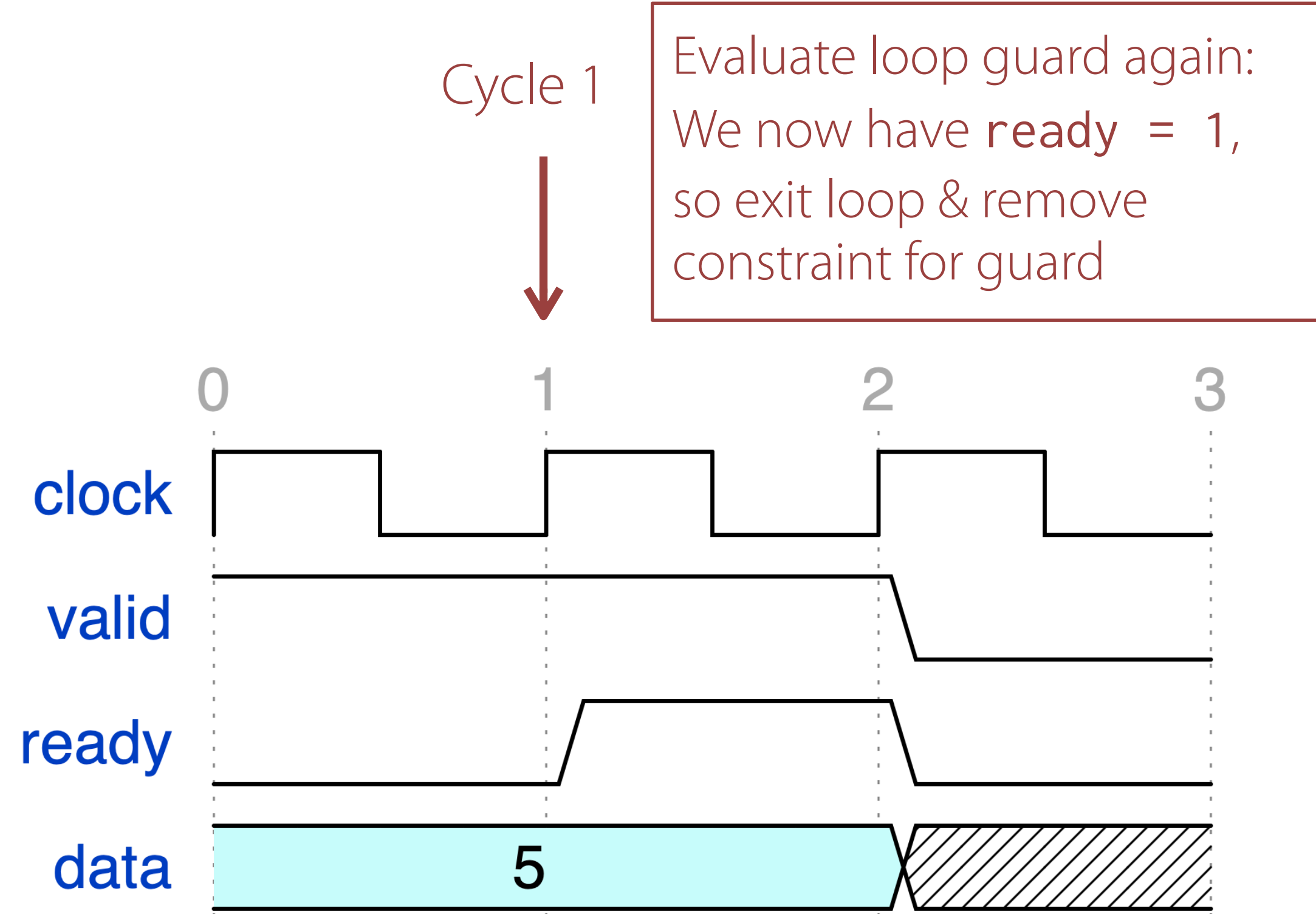
Constraints: { 1 = 1, data = 5, 1 ≠ 0 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



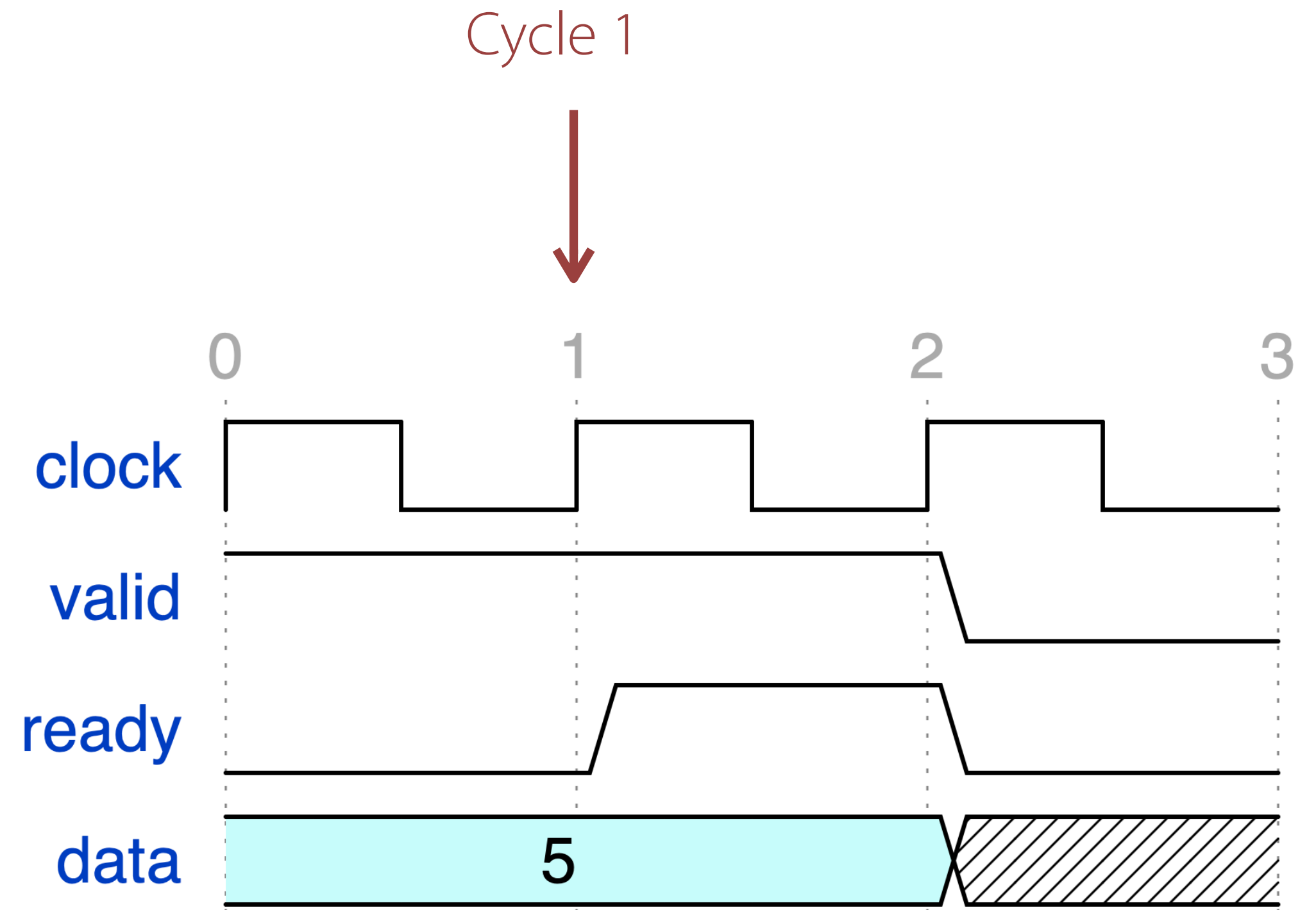
Constraints: { 1 = 1, data = 5, 1 ≠ 0 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```

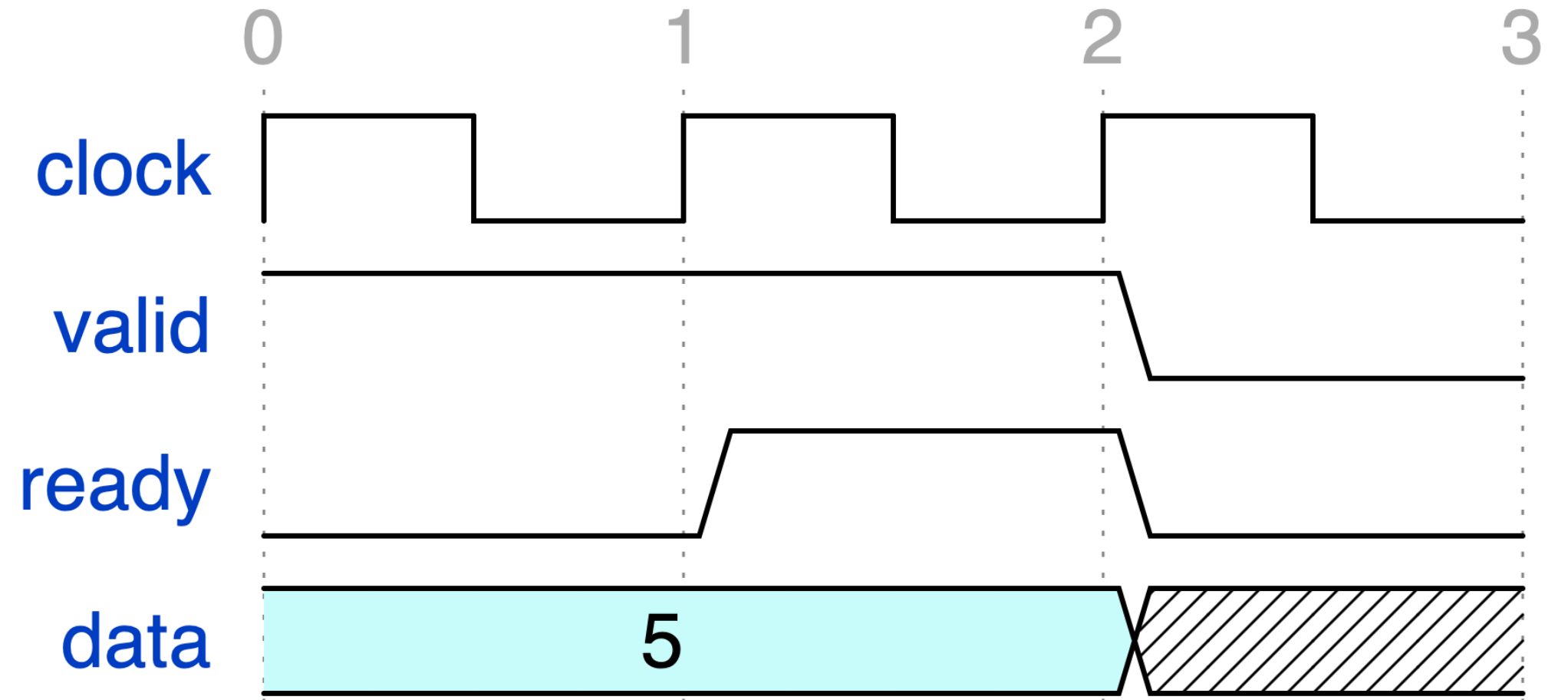


Constraints: { 1 = 1, data = 5 }

Cycle 1

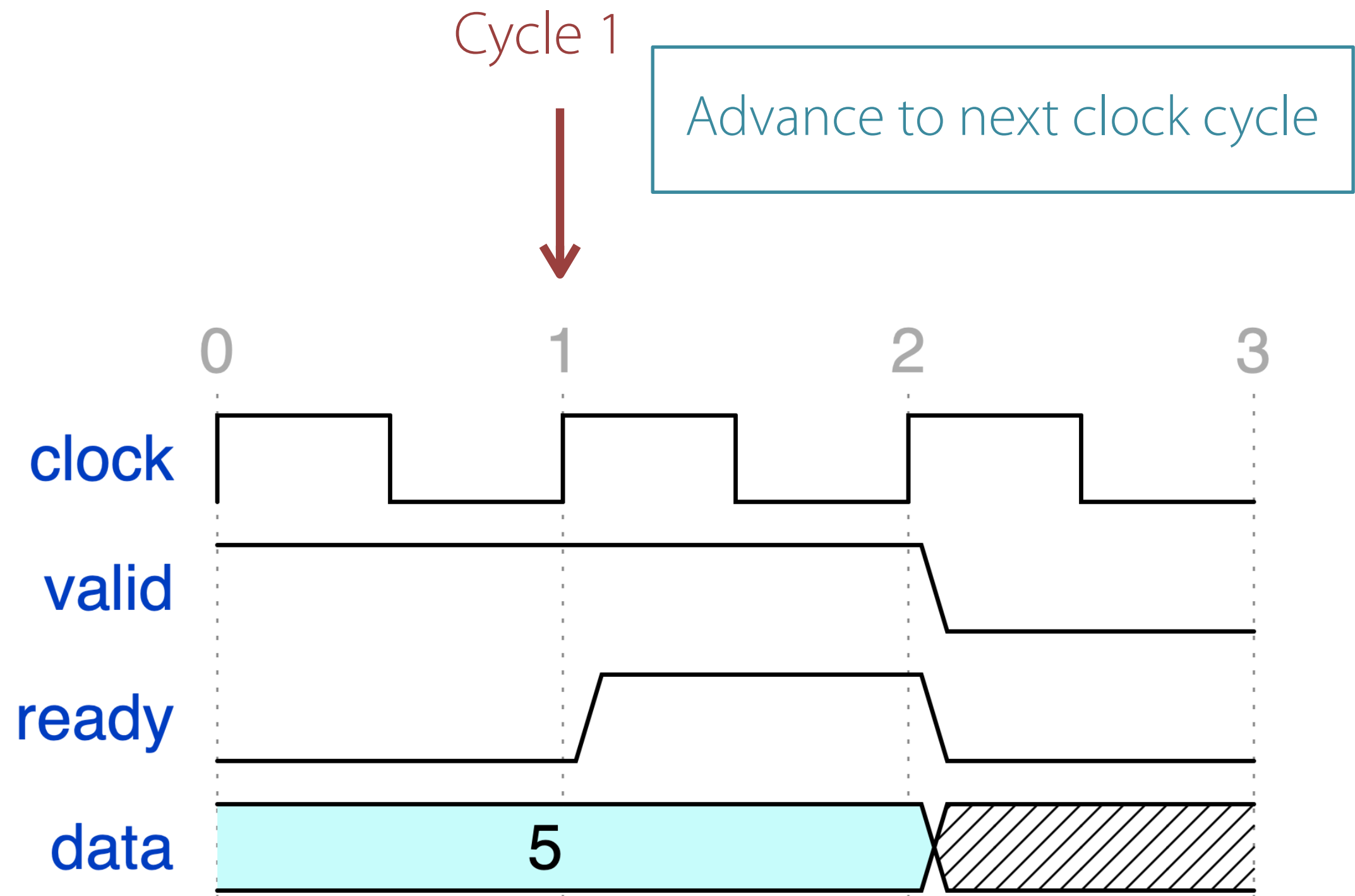
Constraint 0  $\neq$  1 removed

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}
```



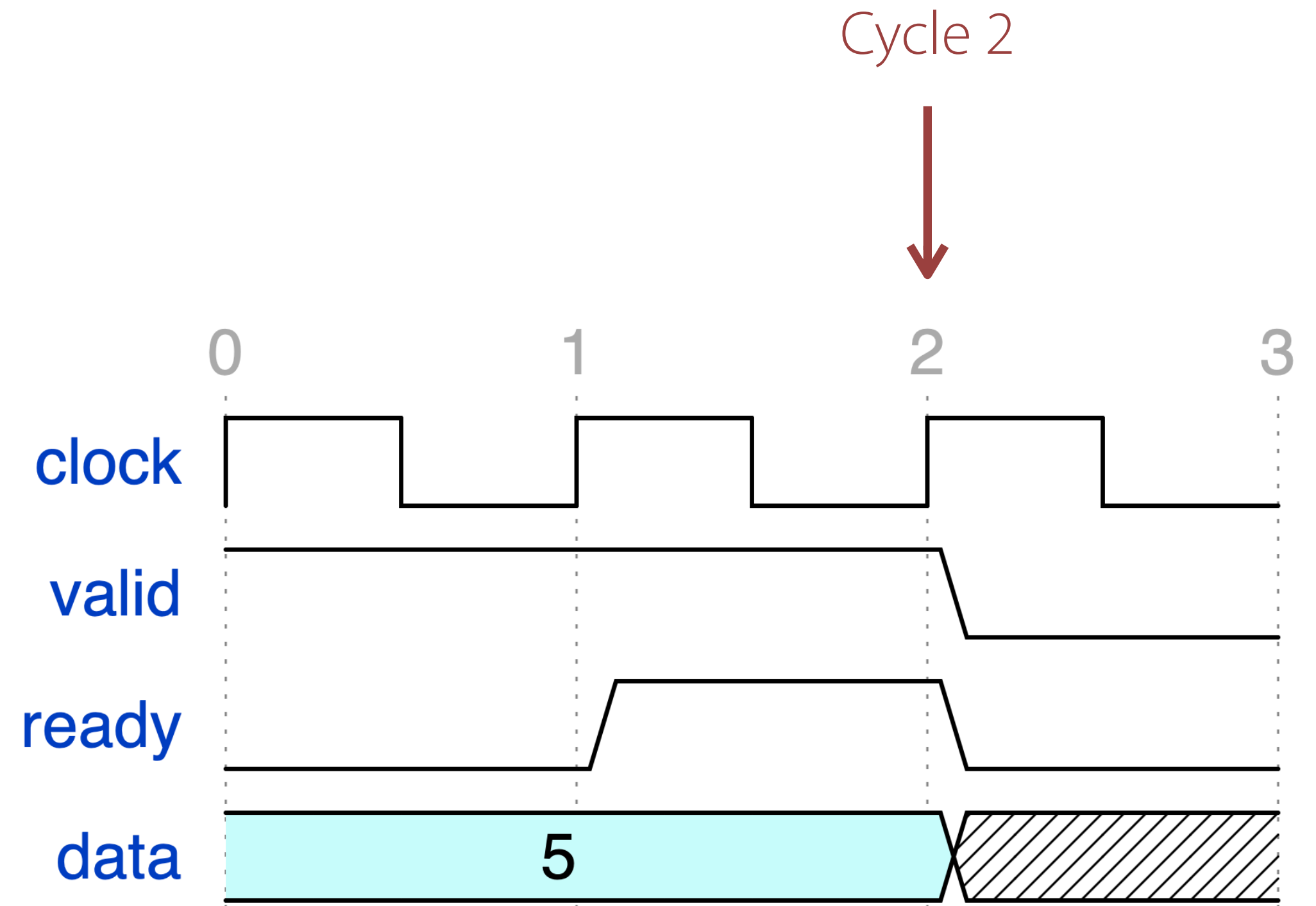
Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



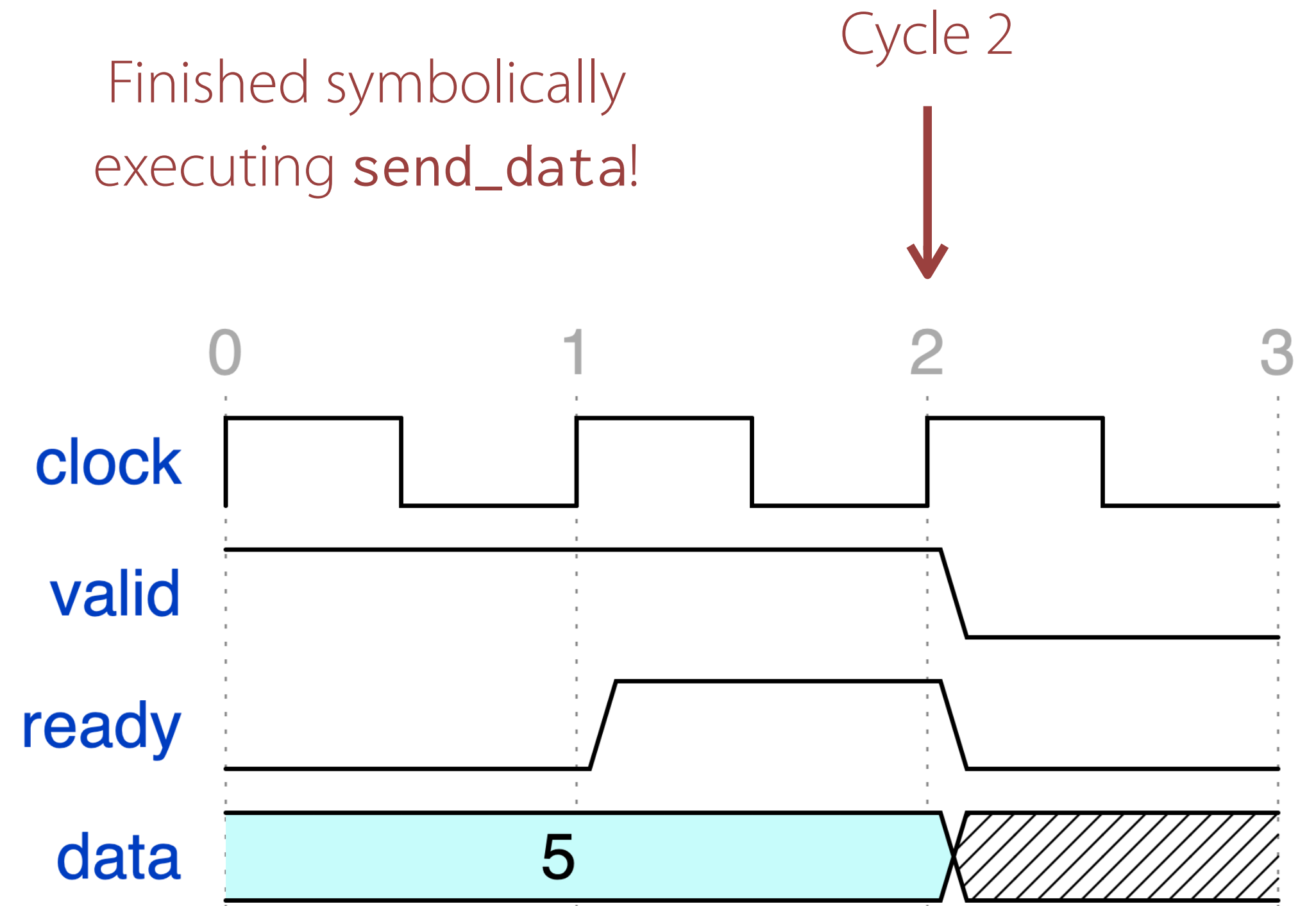
Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



Constraints: { 1 = 1, data = 5 }

```
prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready ≠ 1) {
    step();
  }
  step();
}
```



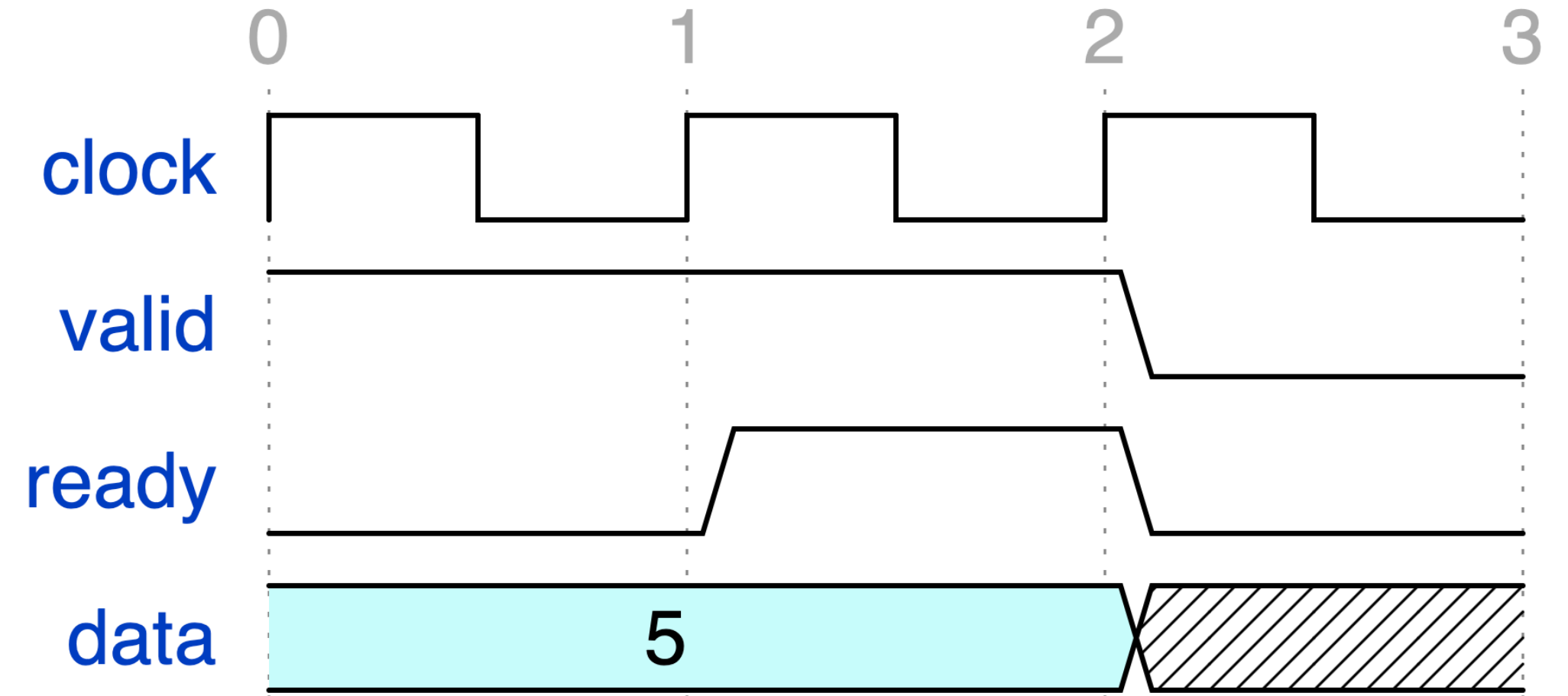
```

prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}

```

Inferred transaction trace:

`send_data(5);` // cycles 0-2



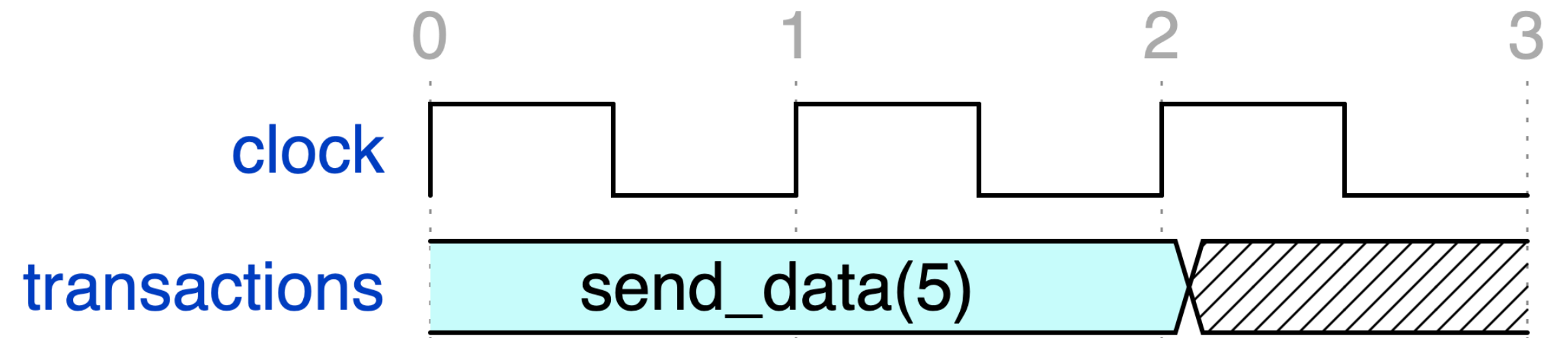
```

prot send_data<DUT: ReadyValid>(
  data: u8
) {
  DUT.valid := 1;
  DUT.data  := data;
  while (DUT.ready  $\neq$  1) {
    step();
  }
  step();
}

```

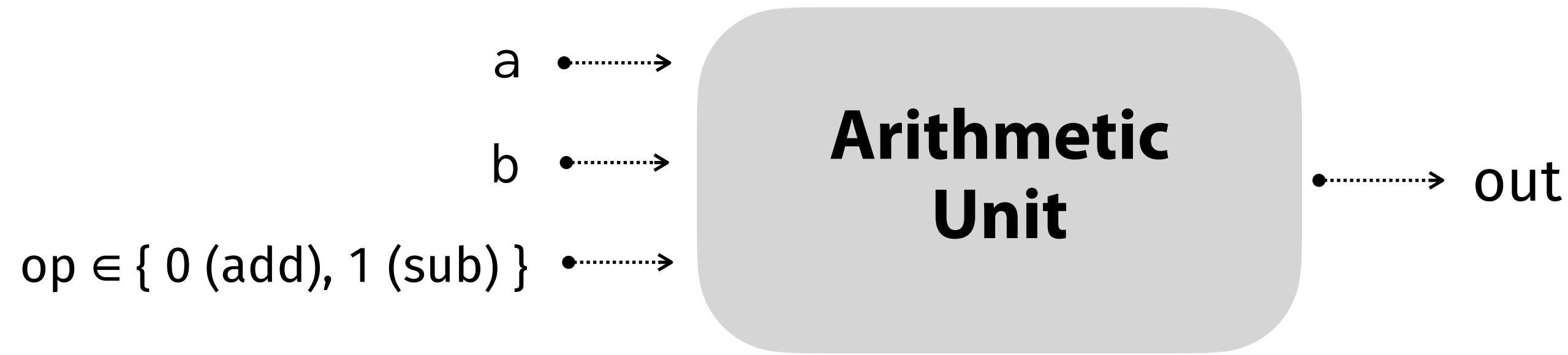
Inferred transaction trace:

`send_data(5);` // cycles 0-2



# **A reconstructor example with multiple protocols**

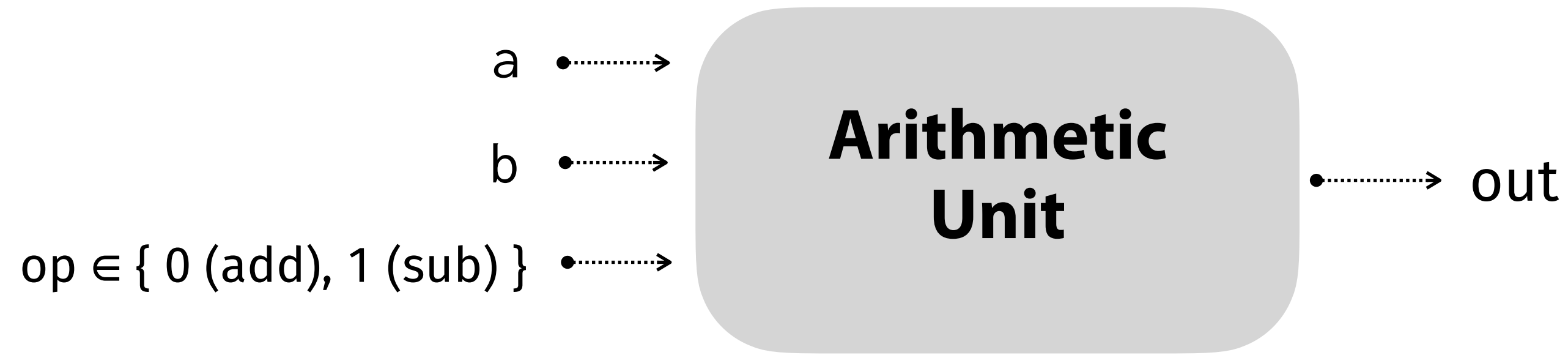
# Example: Arithmetic Unit



```
prot add<DUT: ALU>(a, b, out) {  
  DUT.a := a; DUT.b := b;  
  DUT.op := 0;  
  step();  
  DUT.a := X; DUT.b := X;  
  DUT.op := X;  
  fork();  
  step();  
  assert_eq(DUT.out, out);  
}
```

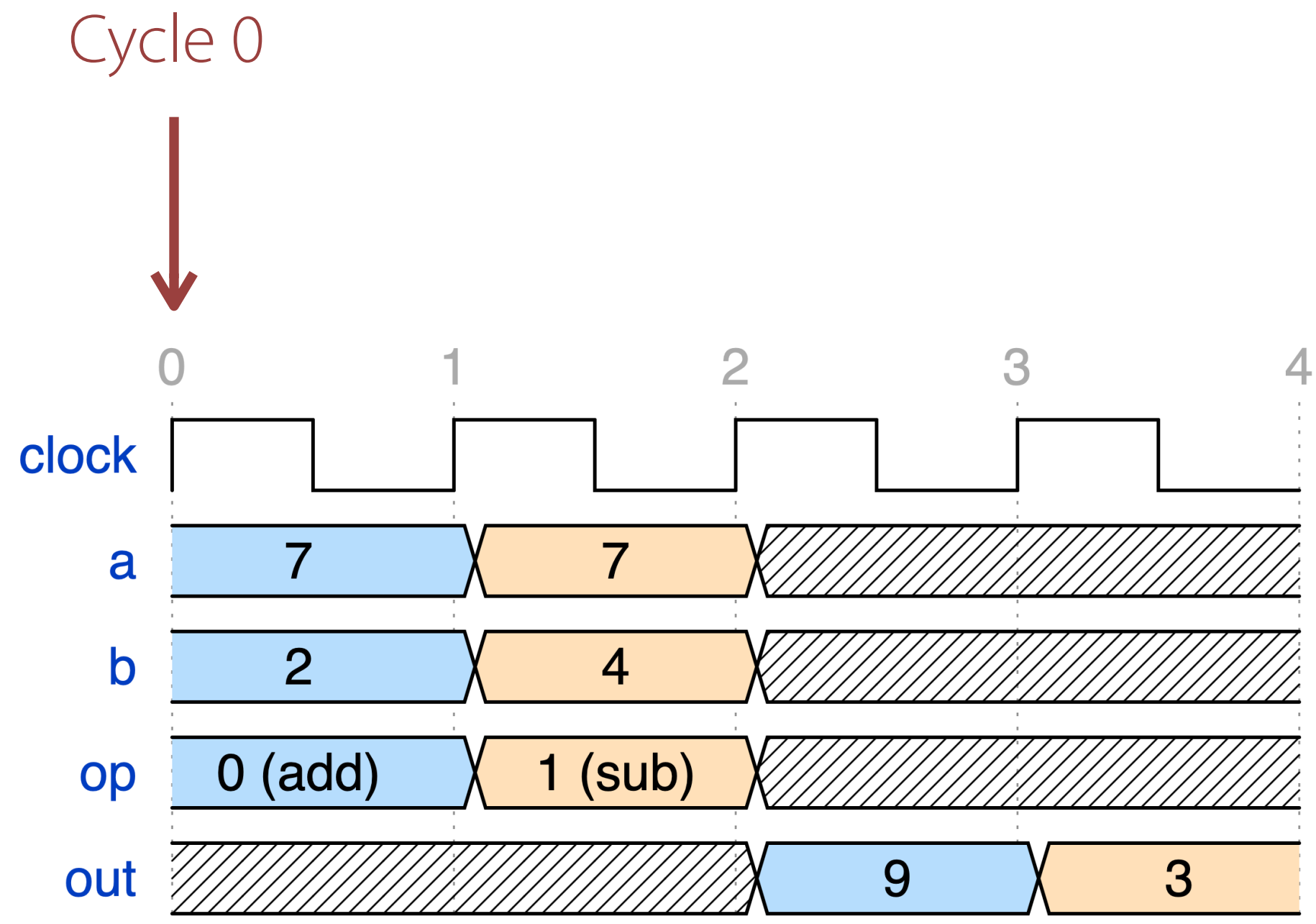
```
prot sub<DUT: ALU>(a, b, out) {  
  DUT.a := a; DUT.b := b;  
  DUT.op := 1;  
  step();  
  DUT.a := X; DUT.b := X;  
  DUT.op := X;  
  fork();  
  step();  
  assert_eq(DUT.out, out);  
}
```

# Example: Arithmetic Unit



```
prot add<DUT: ALU>(a, b, out) {  
  DUT.a := a; DUT.b := b;  
  DUT.op := 0;  
  step();  
  DUT.a := X; DUT.b := X;  
  DUT.op := X;  
  fork();  
  step();  
  assert_eq(DUT.out, out);  
}
```

```
prot sub<DUT: ALU>(a, b, out) {  
  DUT.a := a; DUT.b := b;  
  DUT.op := 1;  
  step();  
  DUT.a := X; DUT.b := X;  
  DUT.op := X;  
  fork();  
  step();  
  assert_eq(DUT.out, out);  
}
```



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

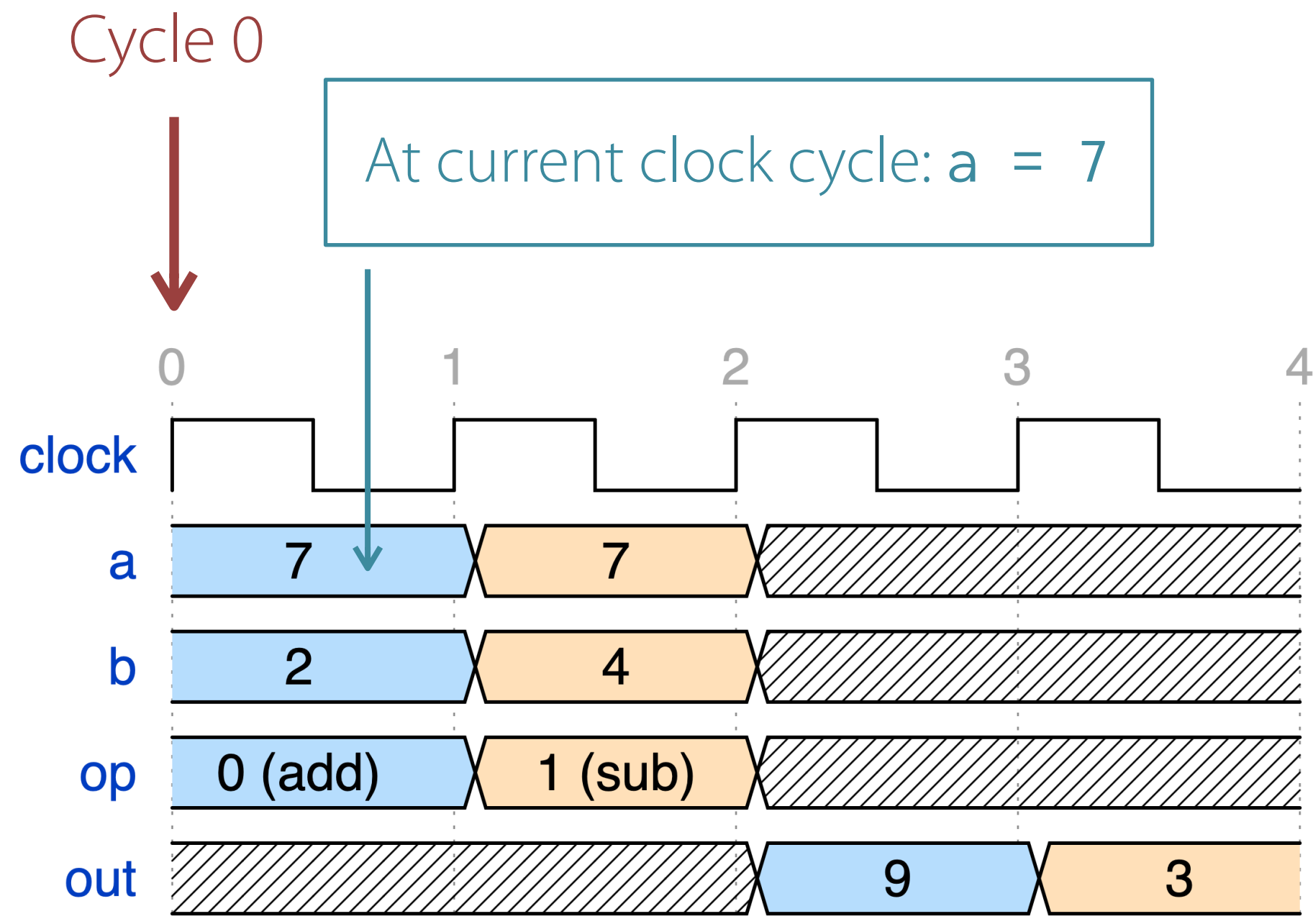
```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

add(a=?, b=?, out=?)

Constraints:  $\emptyset$

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

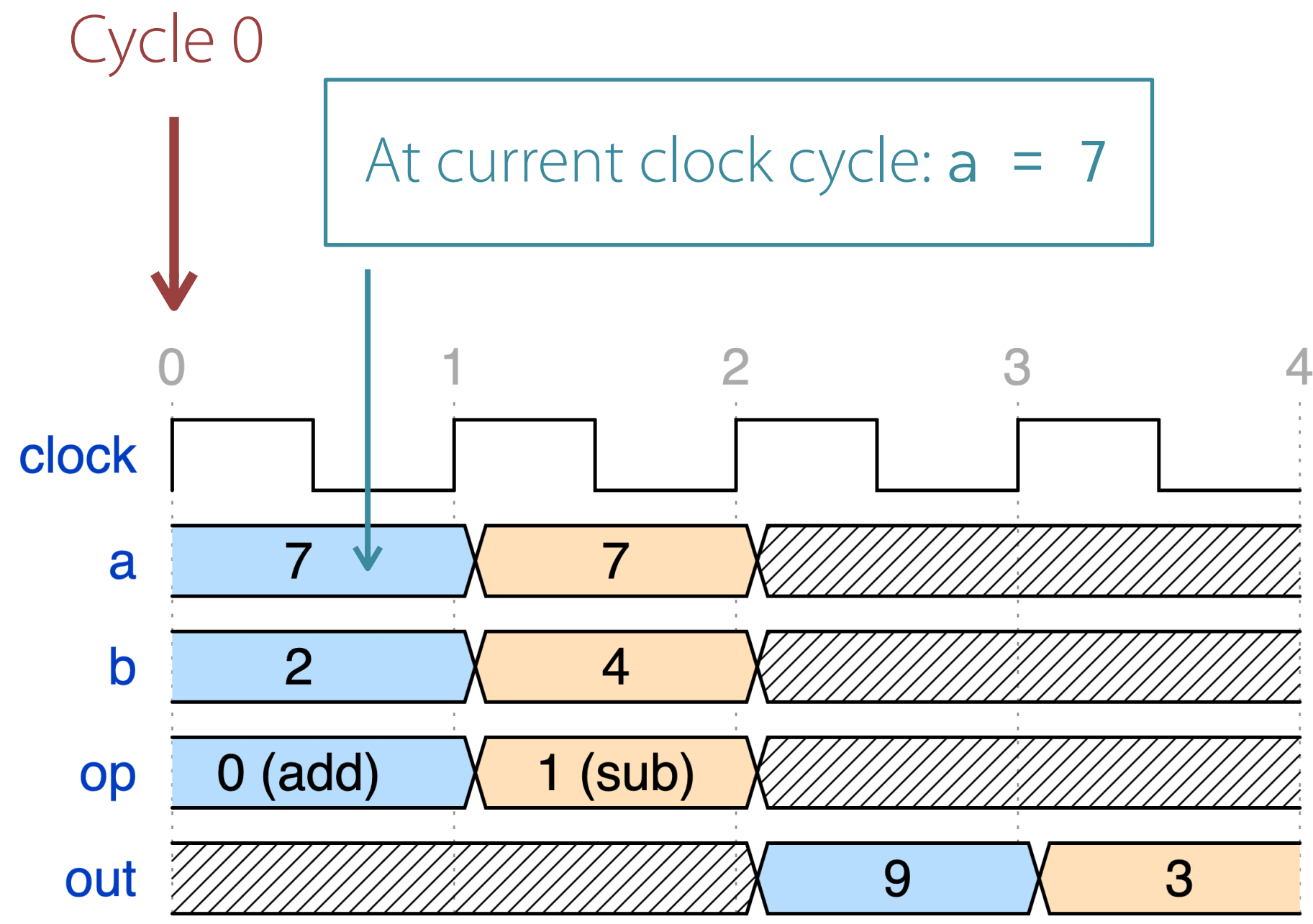
```

add(a=?, b=?, out=?)

Constraints:  $\emptyset$

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

```

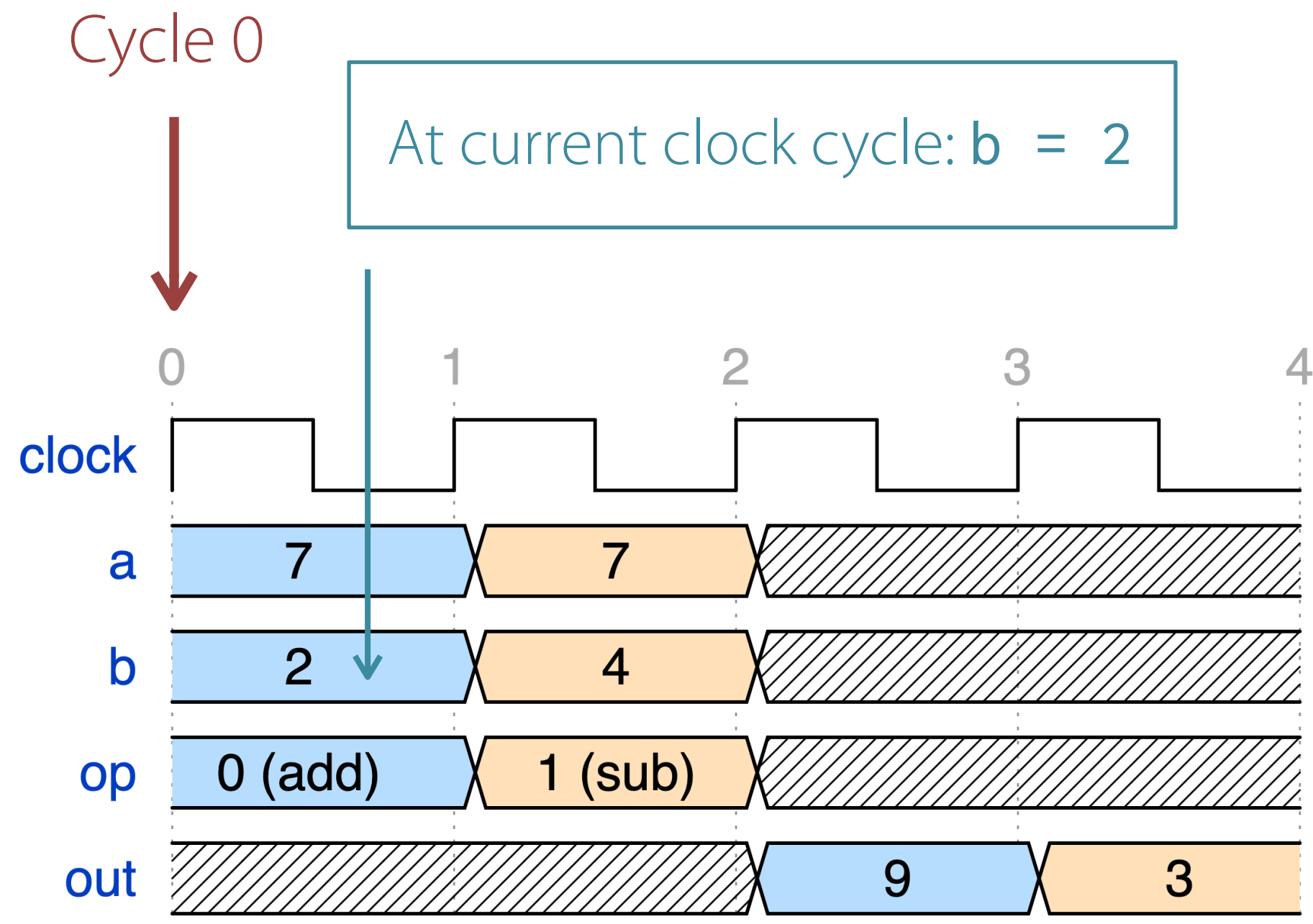
add(a=7, b=?, out=?)

Constraints: {  $a = 7$  }

New!

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

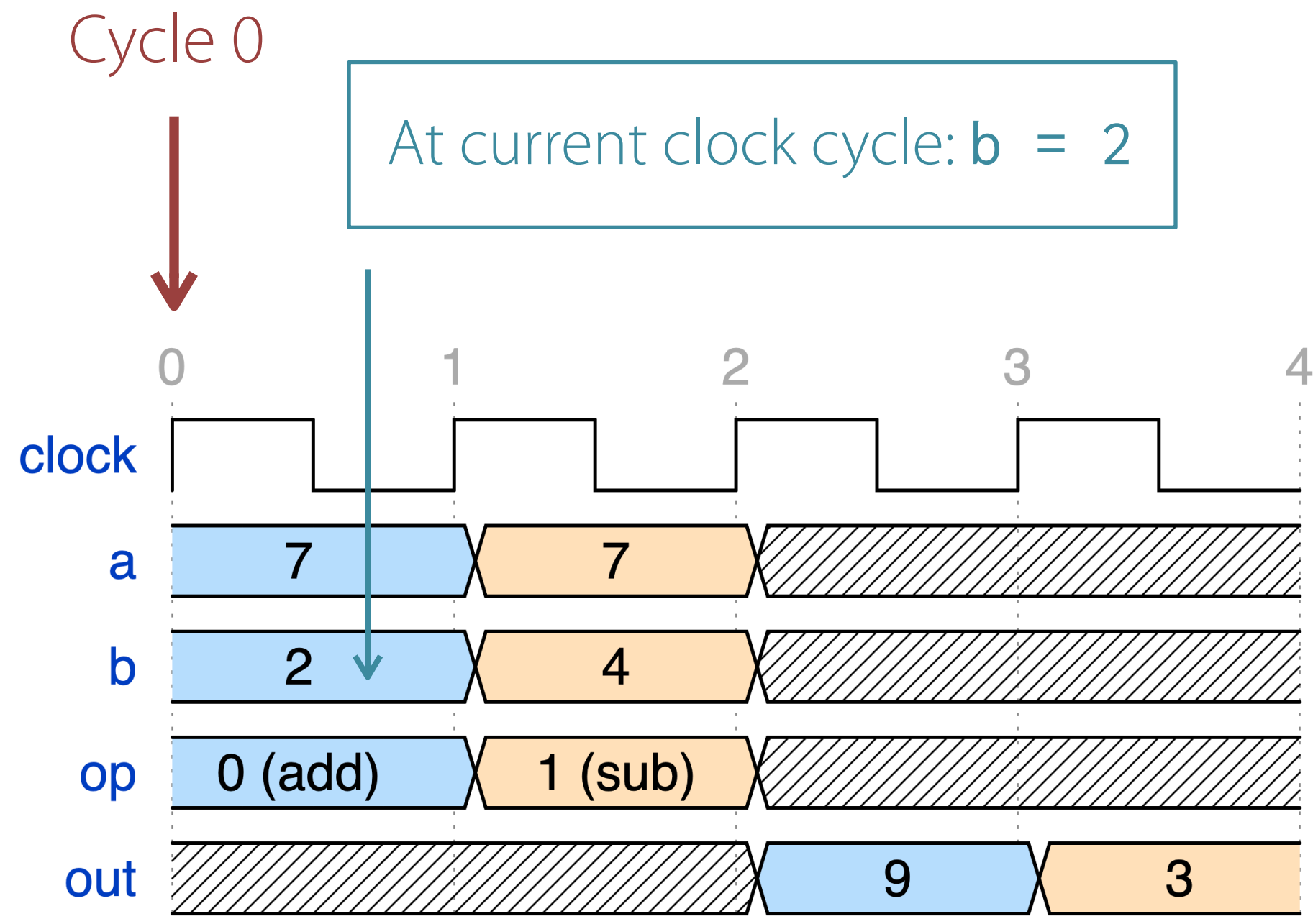
prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

```

add(a=7, b=?, out=?)  
 Constraints: {  $a = 7$  }

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

```

add(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2$  }

New!

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$

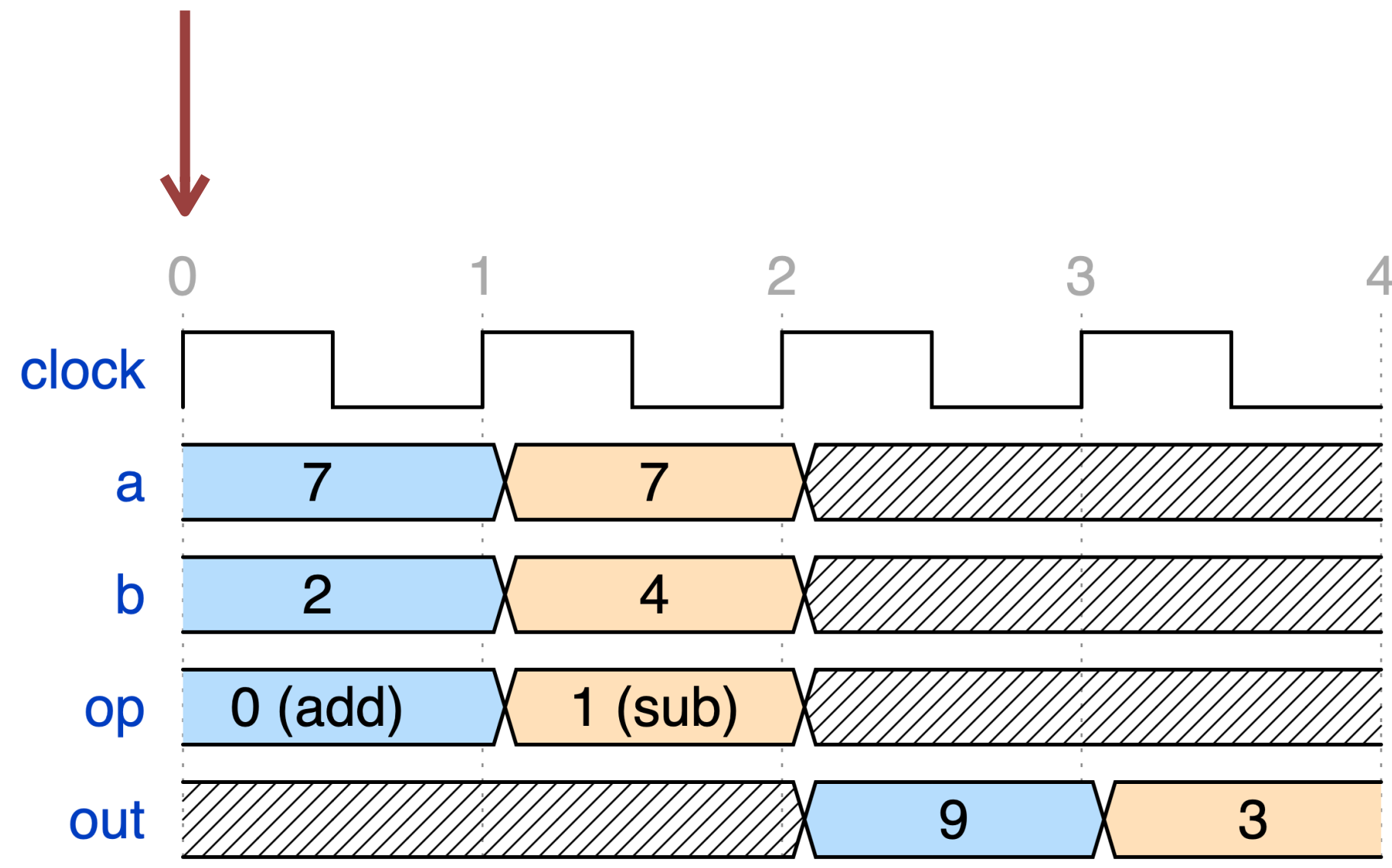
Check if current waveform  
value of `op` =? 0



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

Cycle 0



add(a=7, b=2, out=?)

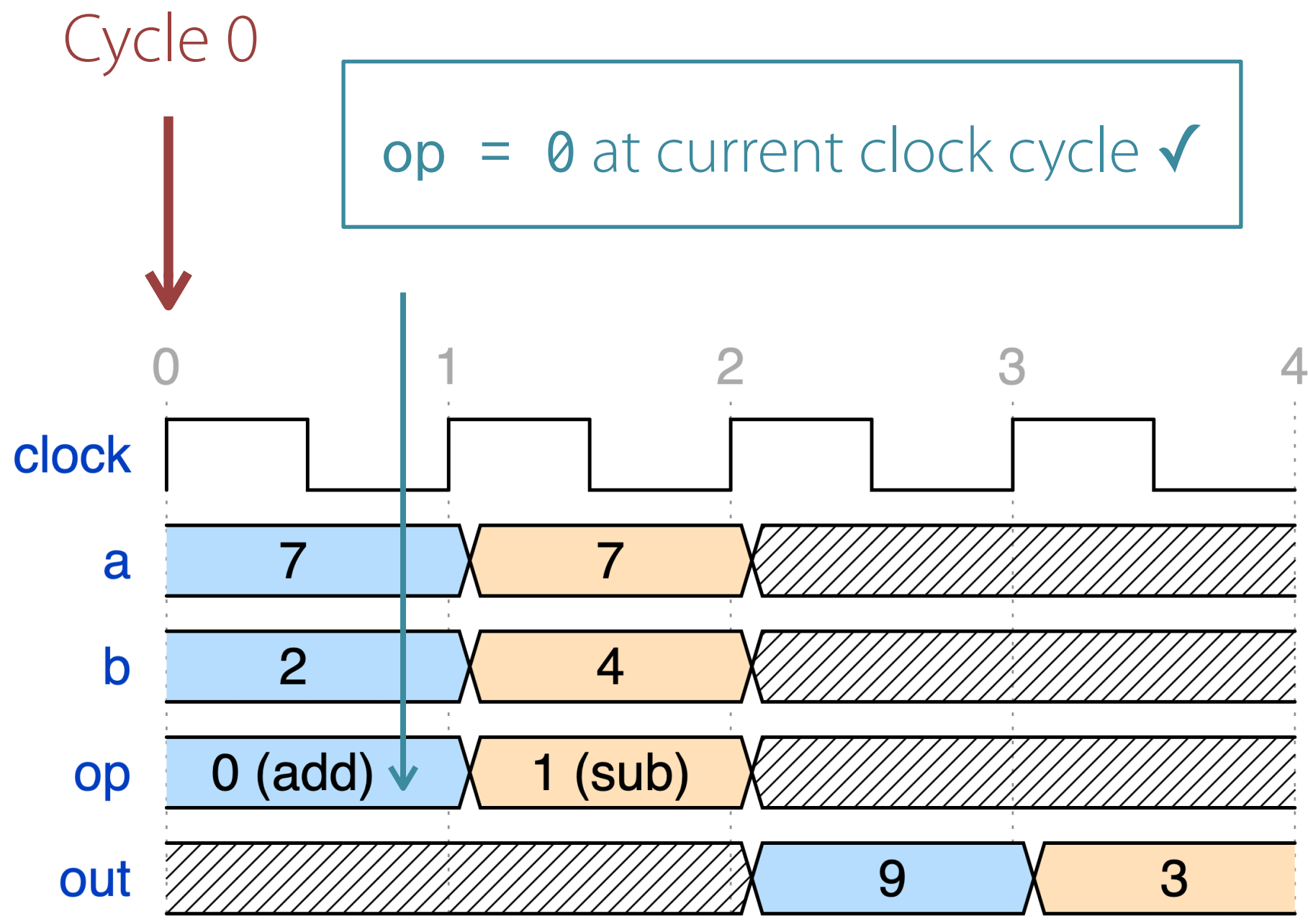
Constraints: { a = 7, b = 2 }



sub(a=?, b=?, out=?)

Constraints:  $\emptyset$

Check if current waveform  
value of `op` =? 0



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

add(a=7, b=2, out=?)

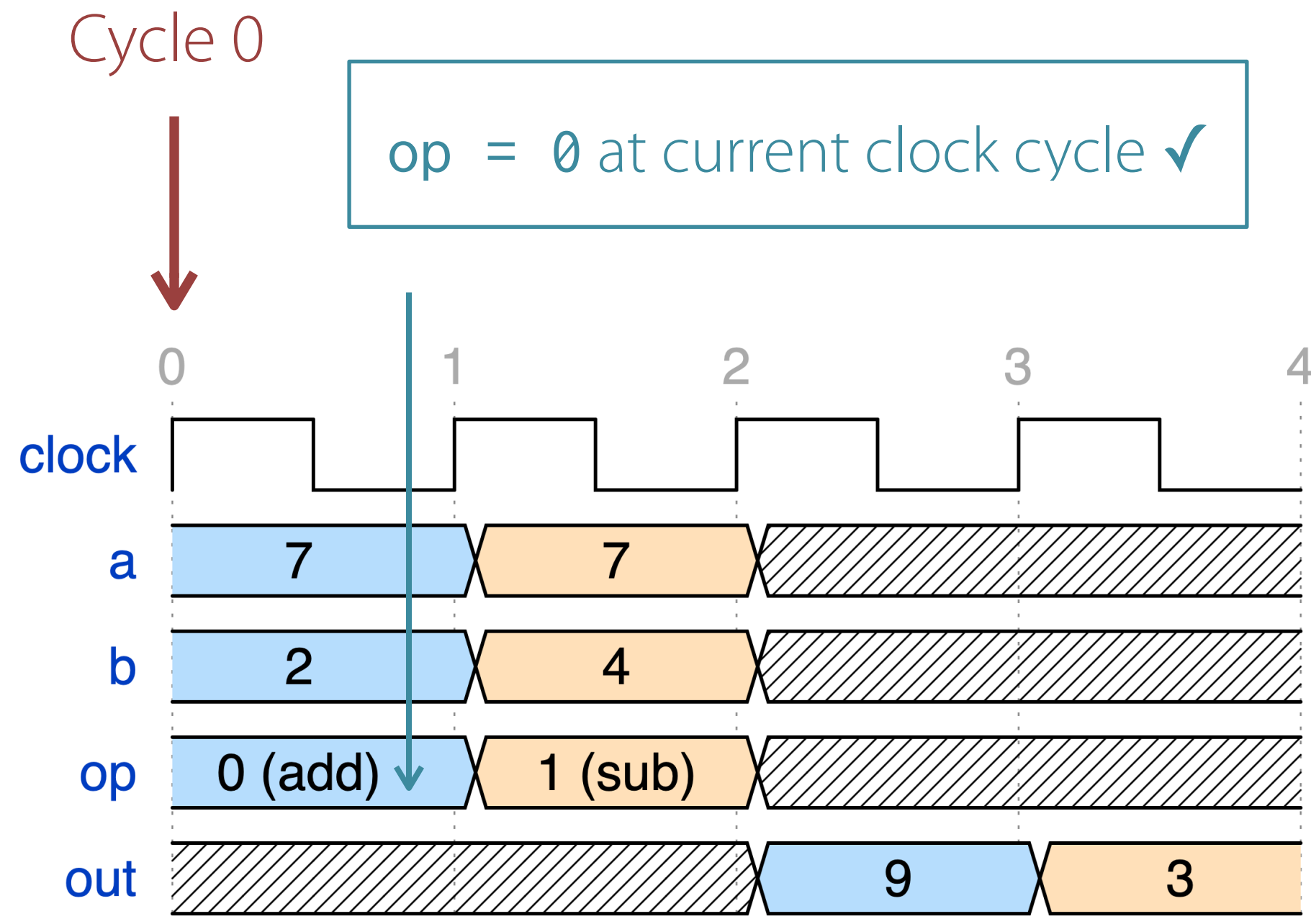
Constraints: { a = 7, b = 2 }

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$

Check if current waveform  
value of `op` =? 0

`op` = 0 at current clock cycle ✓



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

`add(a=7, b=2, out=?)`

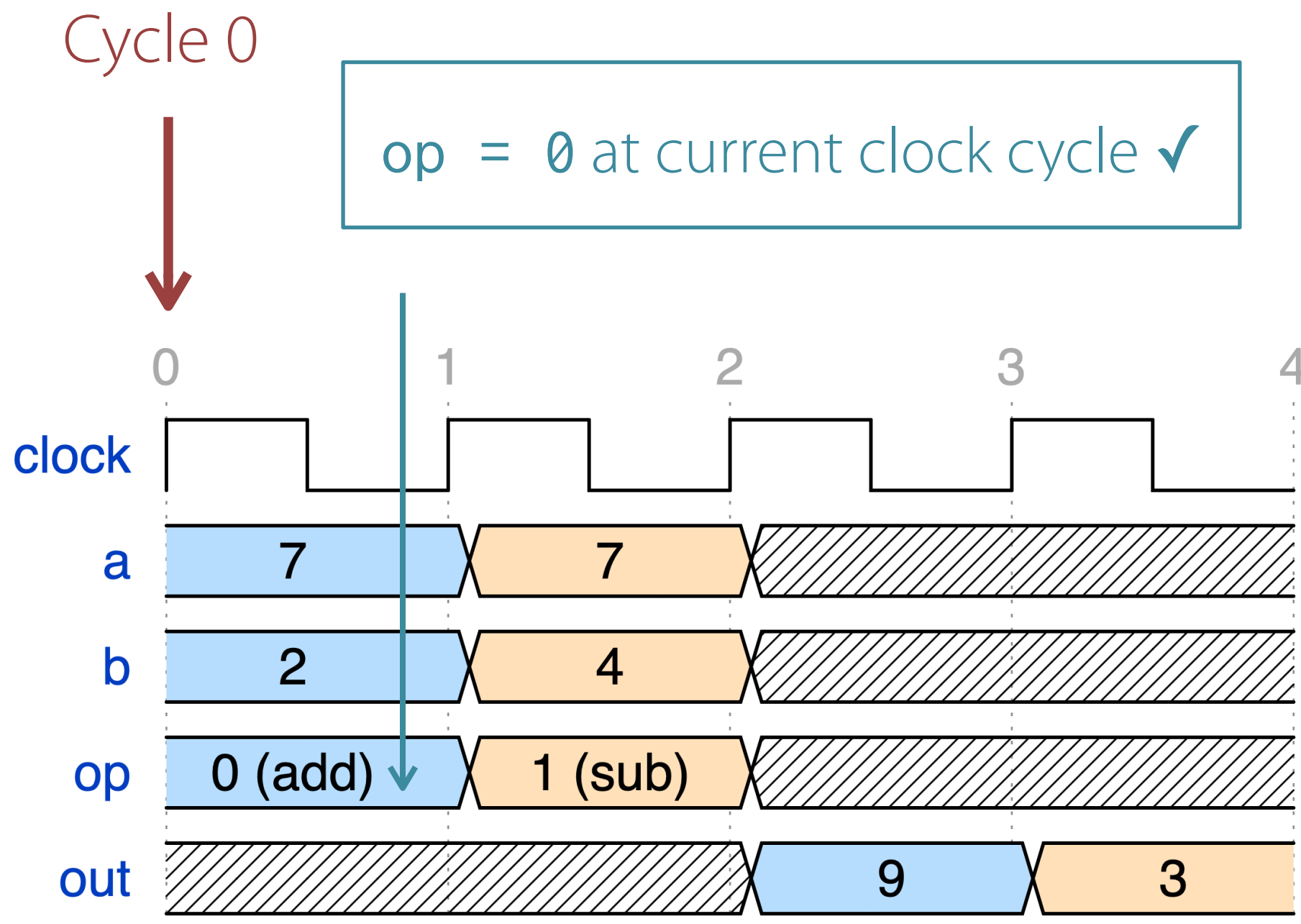
Constraints: { `a = 7`, `b = 2`, `0 = 0` }

New!

`sub(a=?, b=?, out=?)`

Constraints:  $\emptyset$

Check if current waveform  
value of `op` =? 0



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

Waveform value of `op`  
@ current cycle

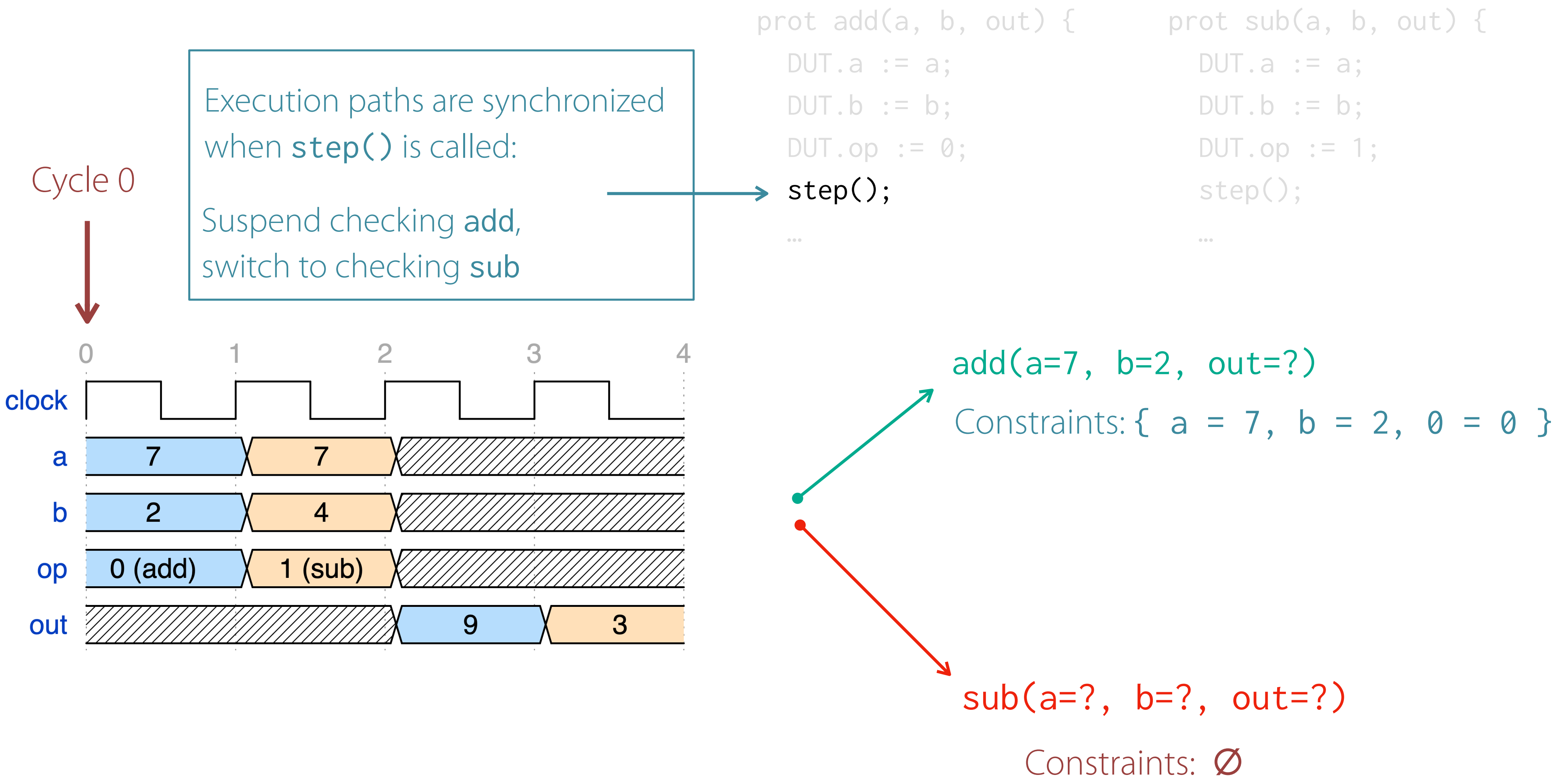
add(a=7, b=2, out=?)

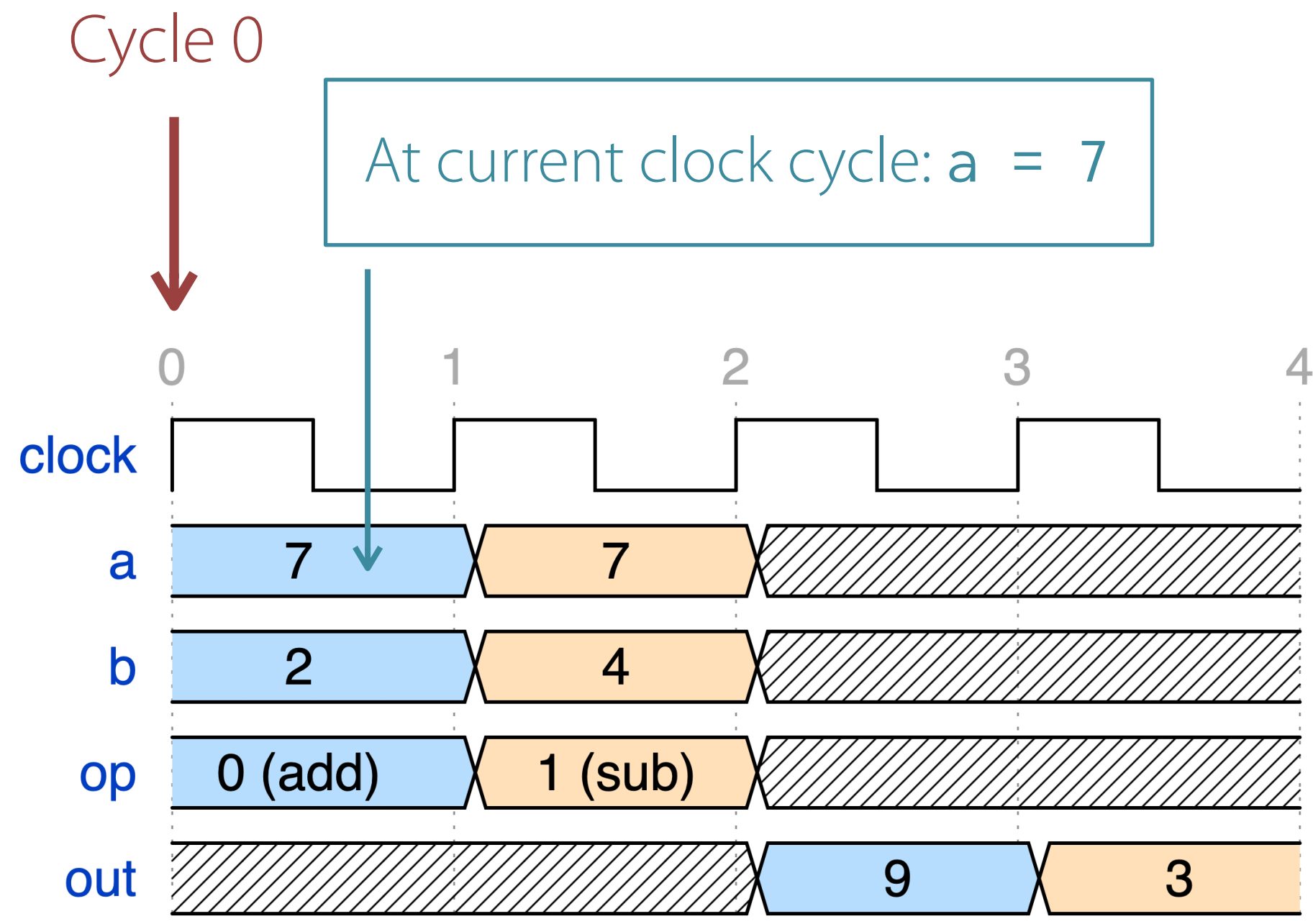
Constraints: { a = 7, b = 2, 0 = 0 }

RHS of DUT.op := 0

sub(a=?, b=?, out=?)

Constraints:  $\emptyset$





```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

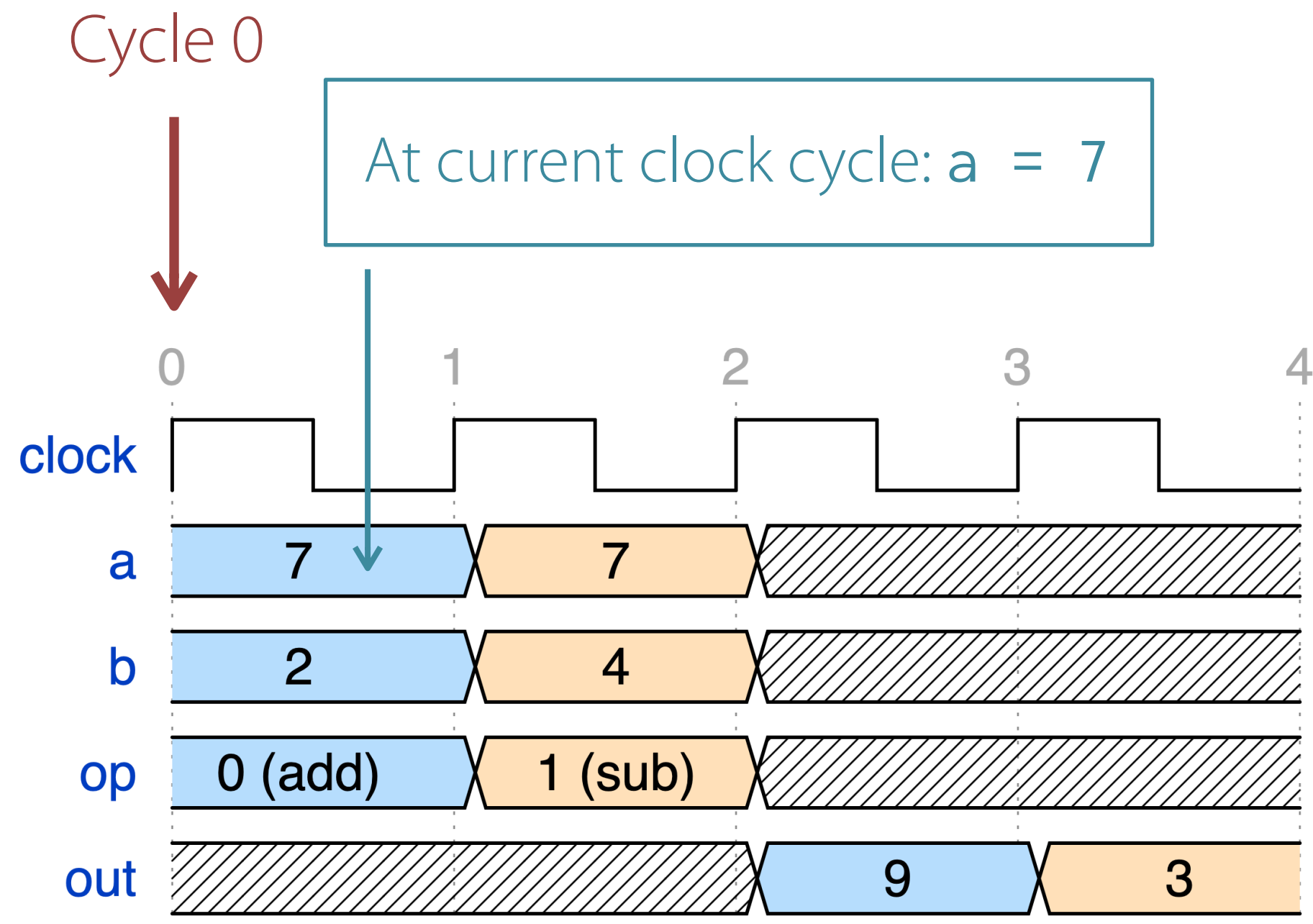
```

add(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2, 0 = 0$  }

sub(a=7, b=?, out=?)

Constraints:  $\emptyset$



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

```

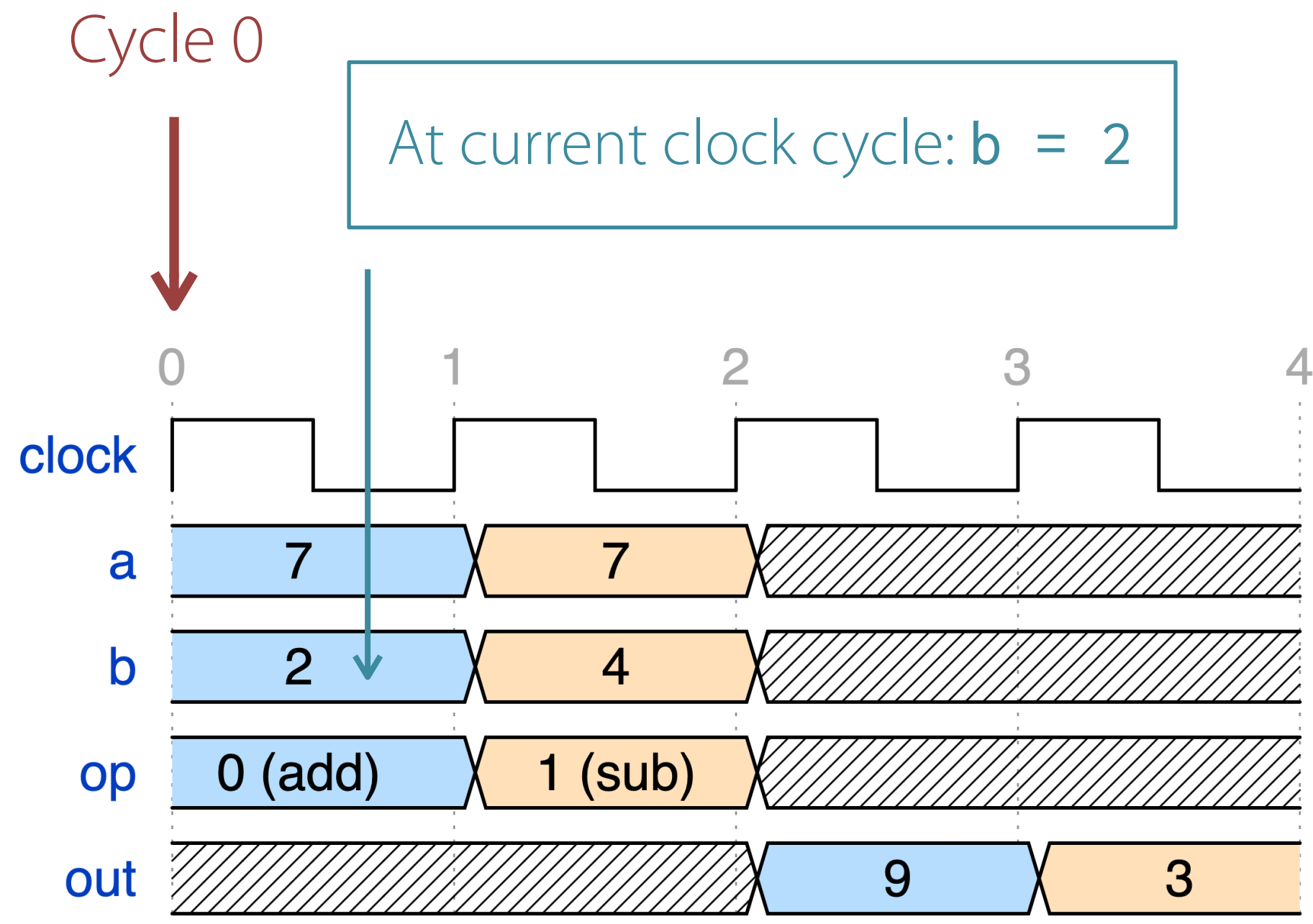
add(a=7, b=2, out=?)

Constraints: {  $a = 7$ ,  $b = 2$ ,  $0 = 0$  }

sub(a=7, b=?, out=?)

Constraints: {  $a = 7$  }

New!



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

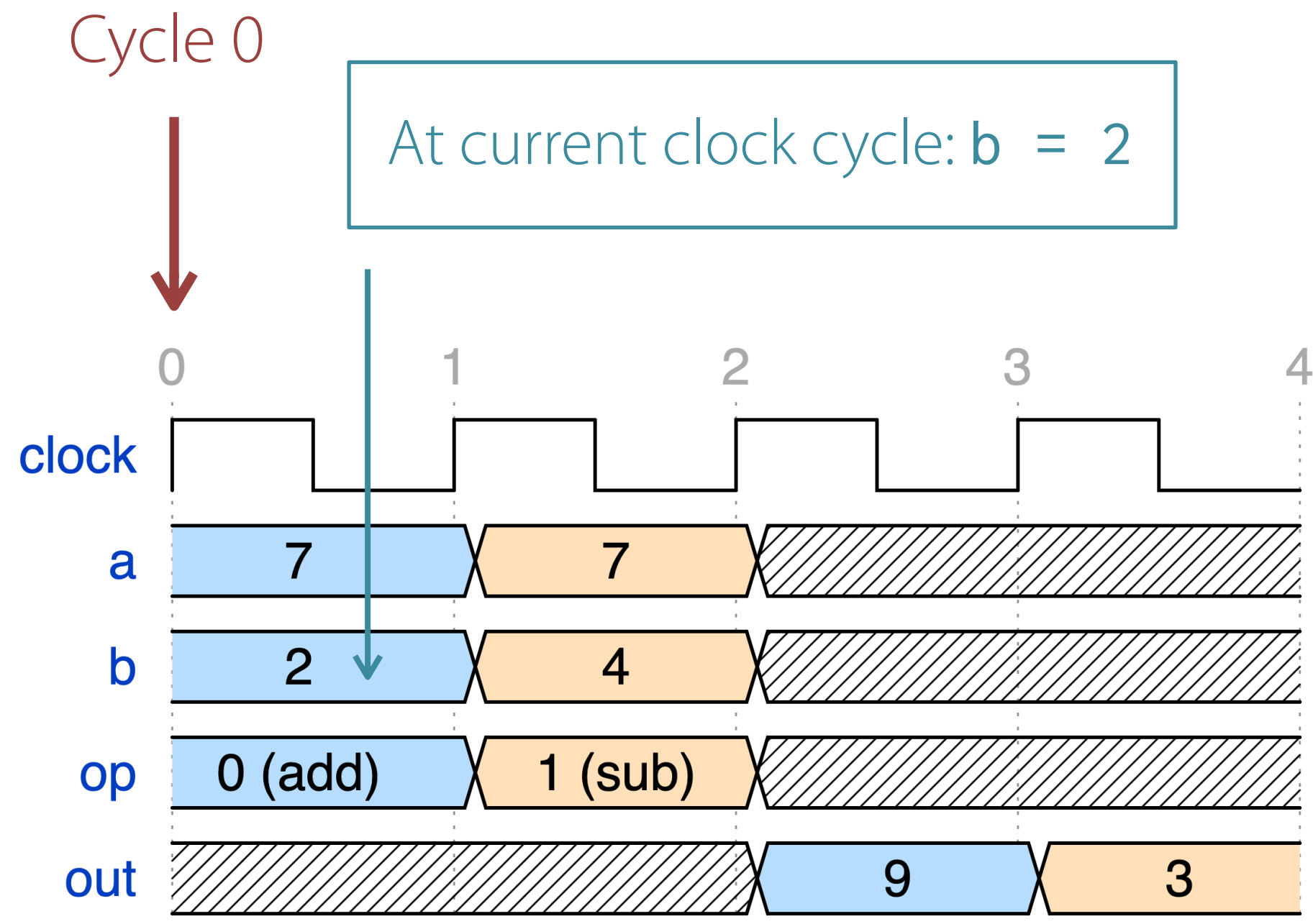
```

add(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2, 0 = 0$  }

sub(a=7, b=?, out=?)

Constraints: {  $a = 7$  }



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...

```

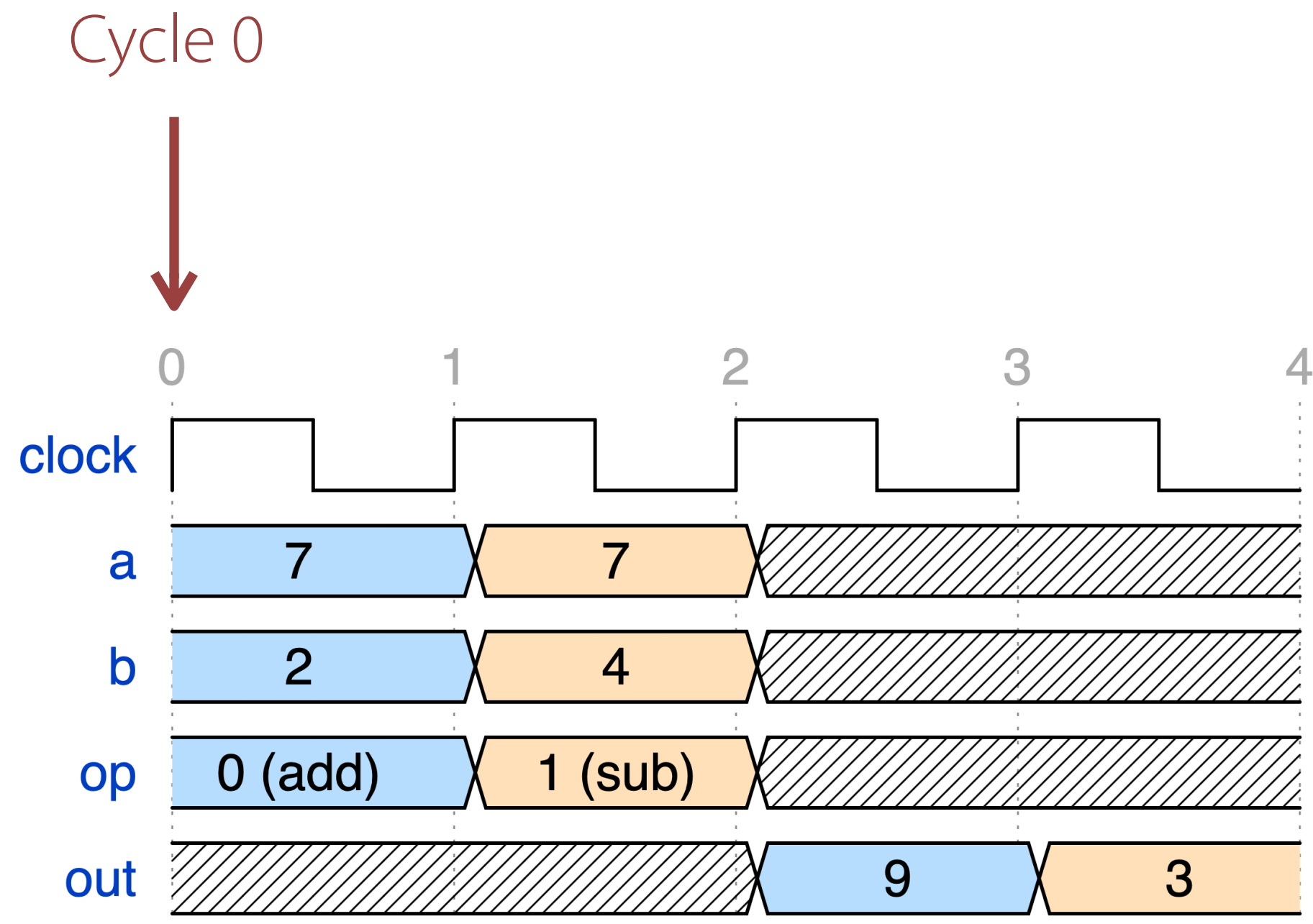
add(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2, 0 = 0$  }

sub(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2$  }

New!



Check if current waveform  
value of `op` =? 1

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...
}

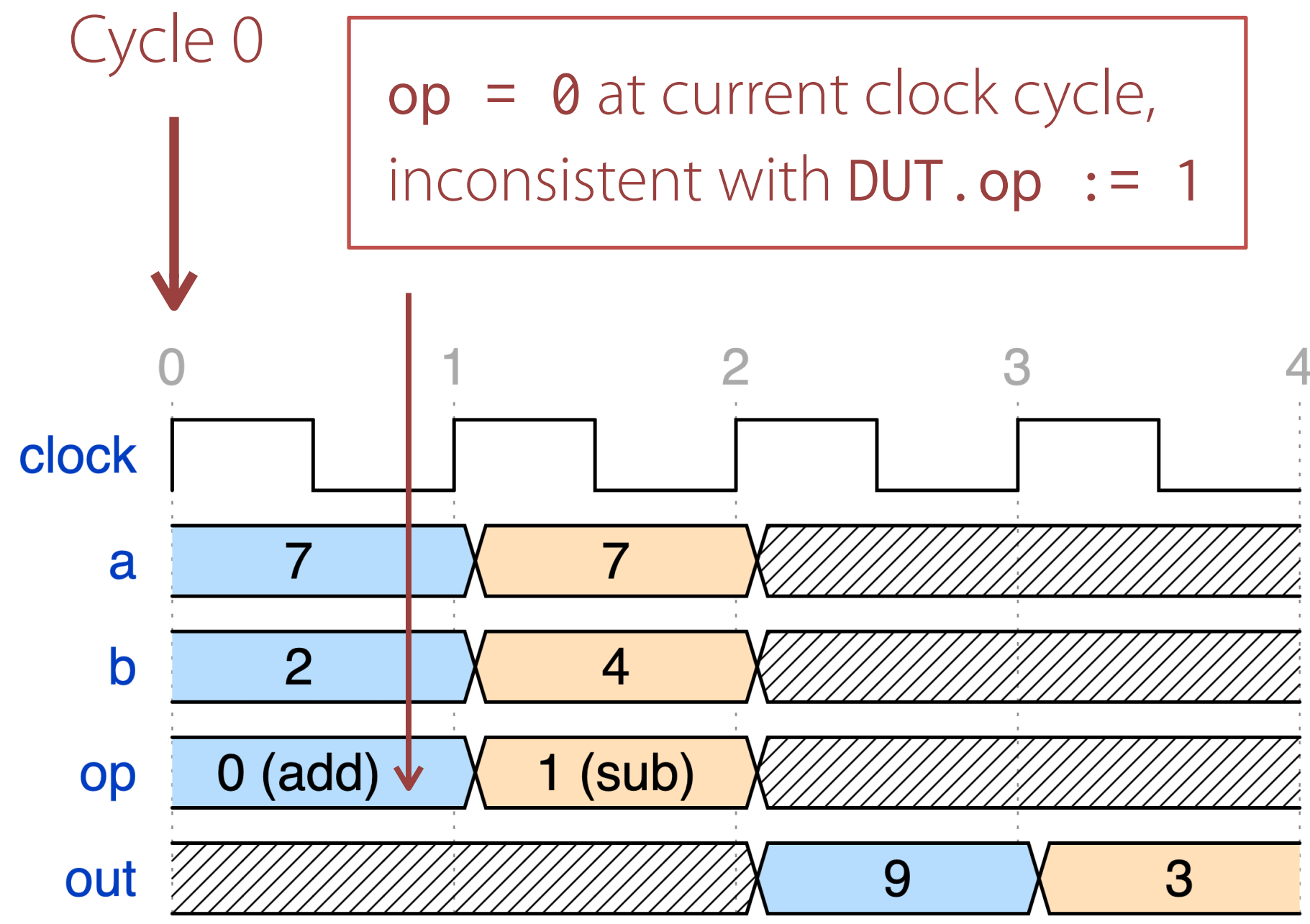
```

add(a=7, b=2, out=?)

Constraints: { a = 7, b = 2, 0 = 0 }

sub(a=7, b=2, out=?)

Constraints: { a = 7, b = 2 }



Check if current waveform  
value of op =? 1

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...
}

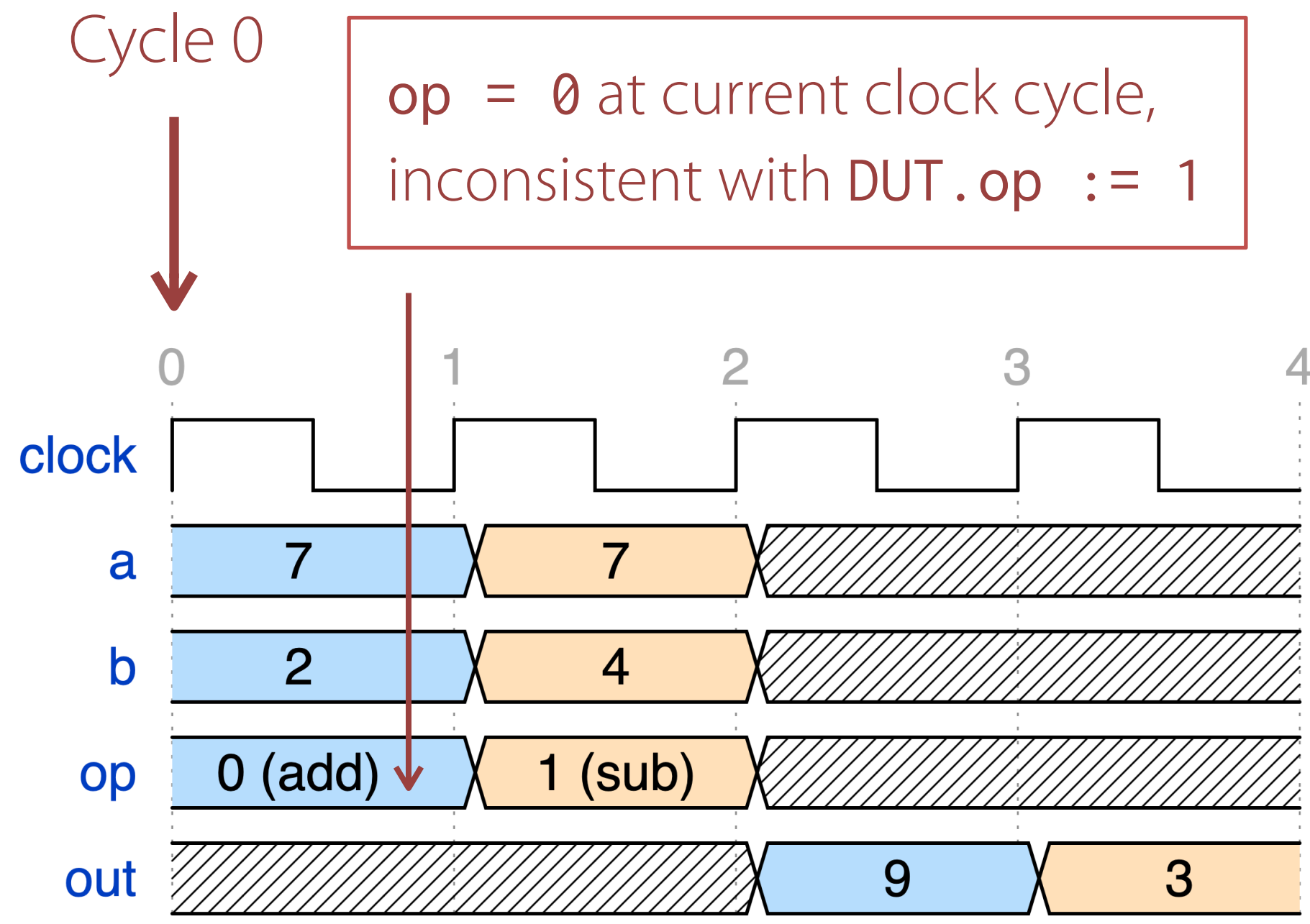
```

add(a=7, b=2, out=?)

Constraints: { a = 7, b = 2, 0 = 0 }

sub(a=7, b=2, out=?)

Constraints: { a = 7, b = 2 }



Check if current waveform  
value of  $op = ? 1$

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...
}

```

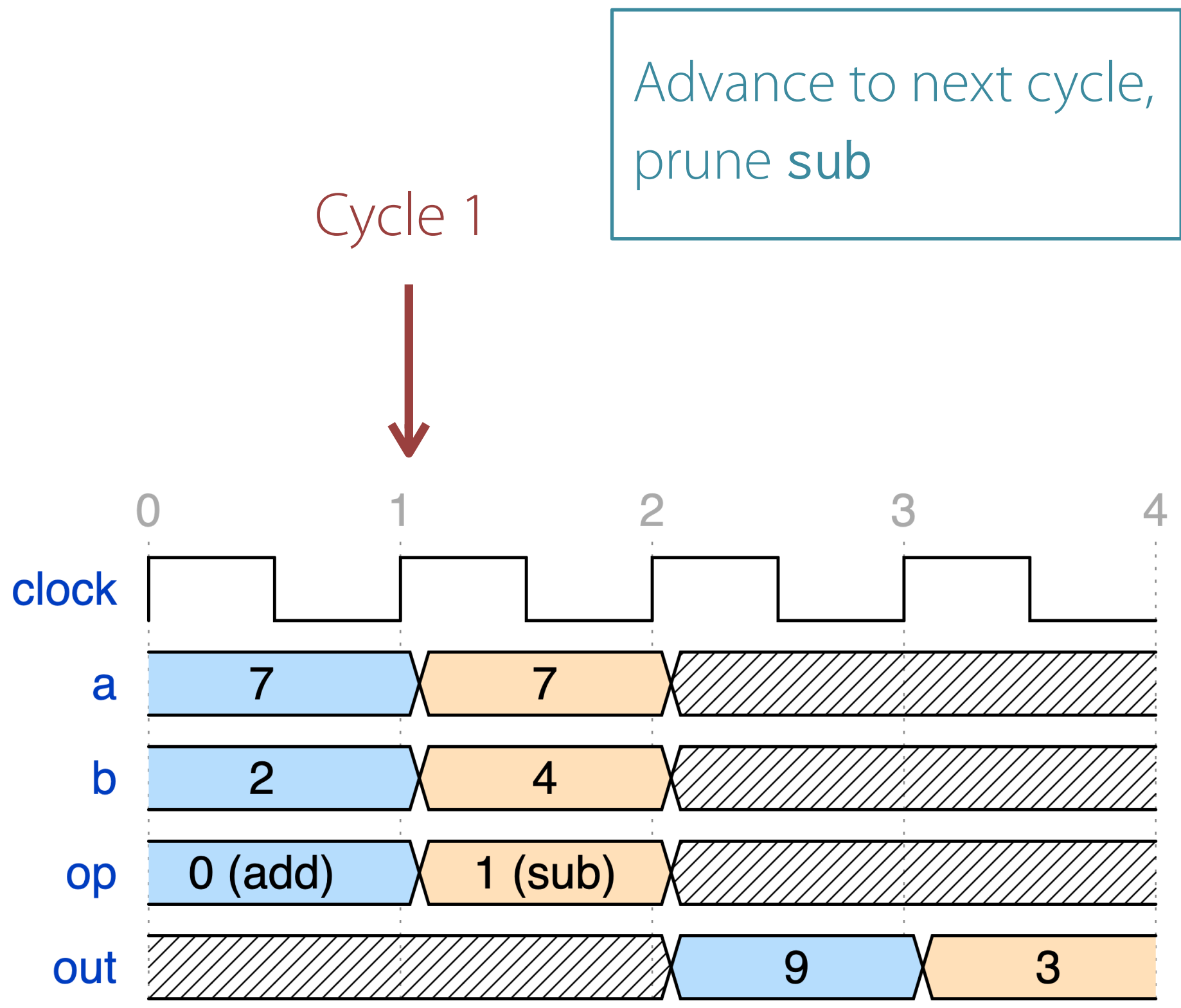
add(a=7, b=2, out=?)

Constraints: {  $a = 7, b = 2, 0 = 0$  }

sub(a, b, out)

**X**

Can't have occurred during  
cycle 0! This path is pruned



```

prot add(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  ...
}

```

```

prot sub(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  ...
}

```

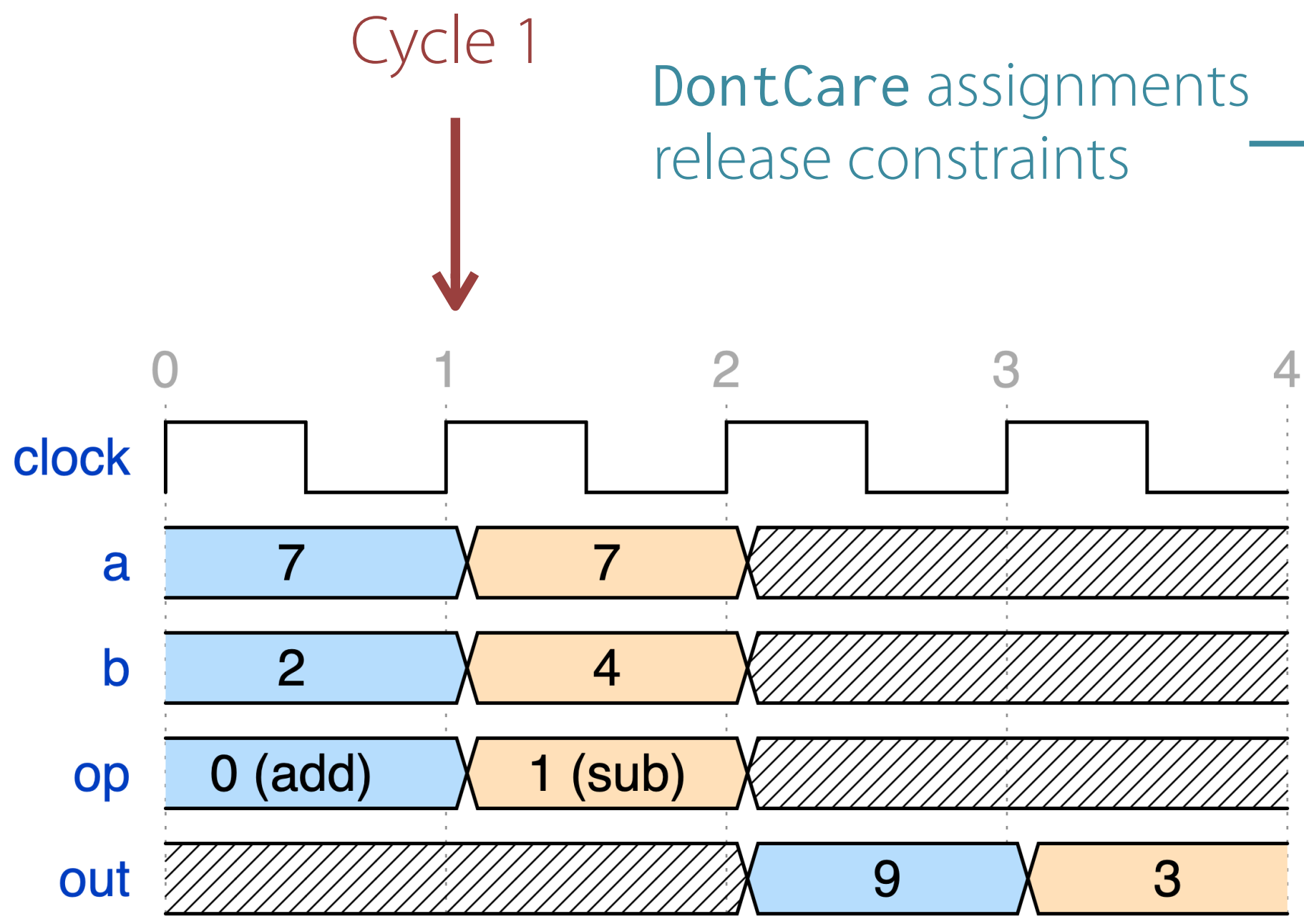
add(a=7, b=2, out=?)

Constraints: { a = 7, b = 2, 0 = 0 }

sub(a, b, out)

**X**

Can't have occurred during cycle 0! This path is pruned

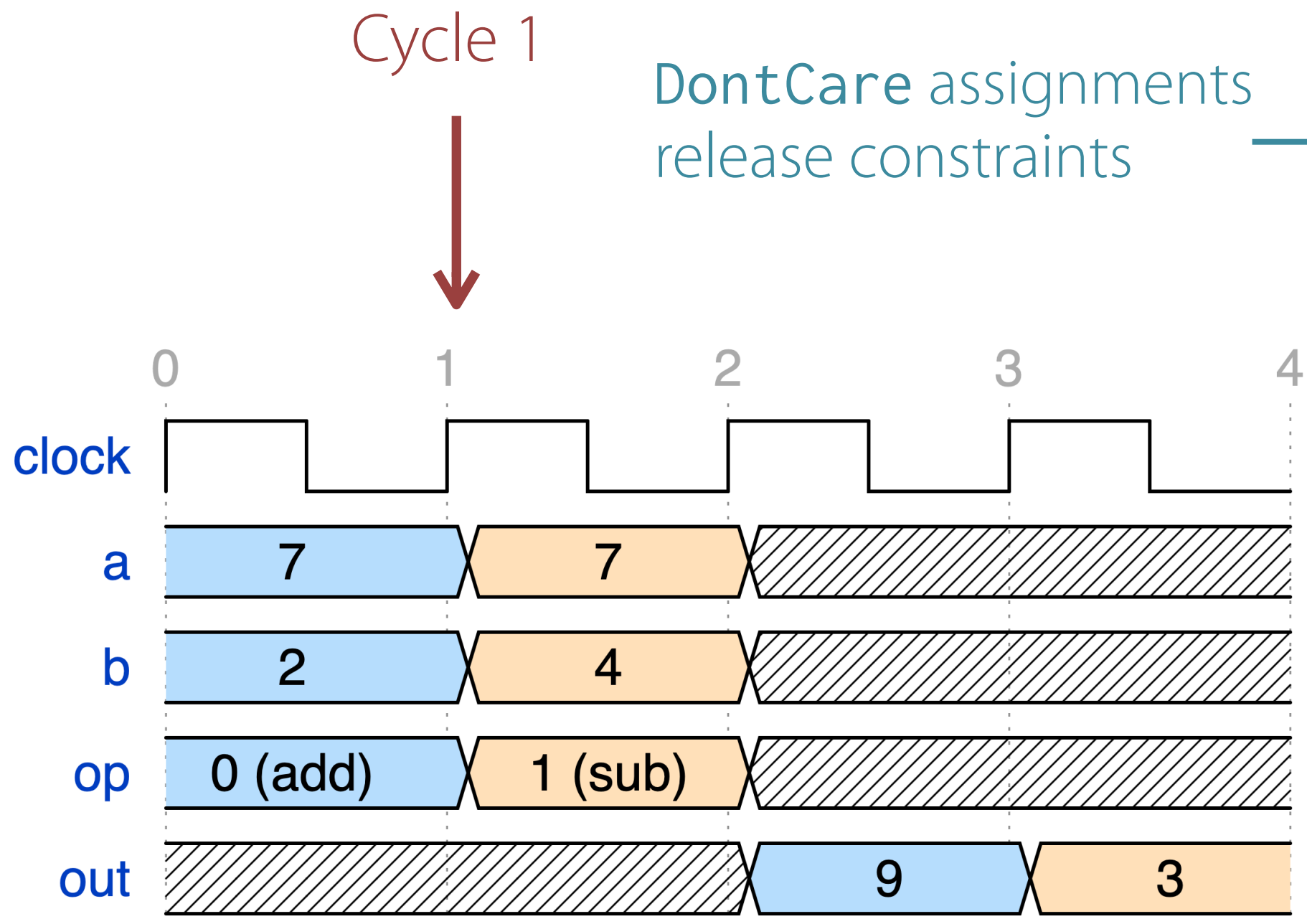


```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}
  
```

add(a=7, b=2, out=?)

Constraints: { a = 7, b = 2, 0 = 0 }



```

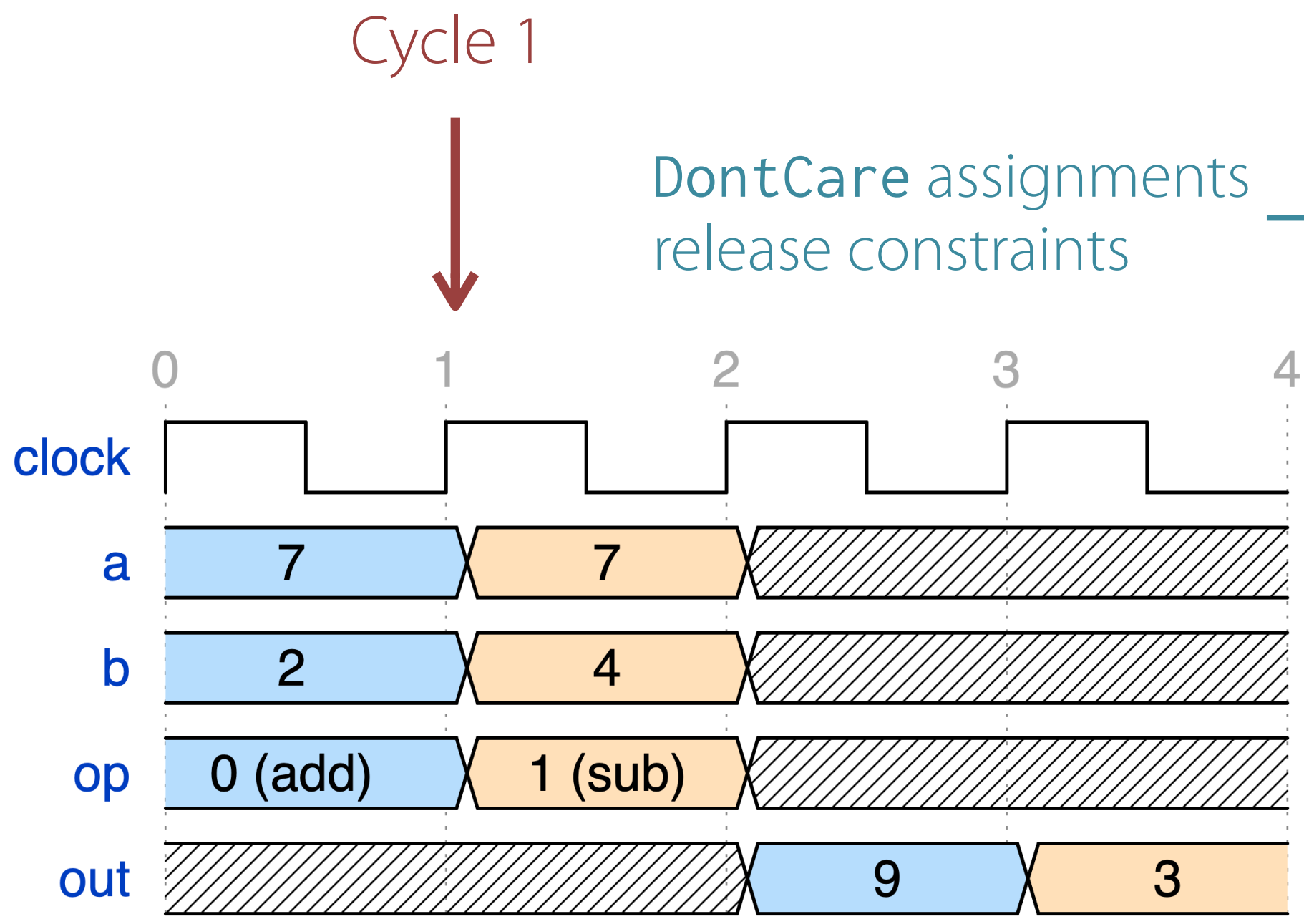
prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}

```

add(a=7, b=2, out=?)

Constraints: { b = 2, 0 = 0 }

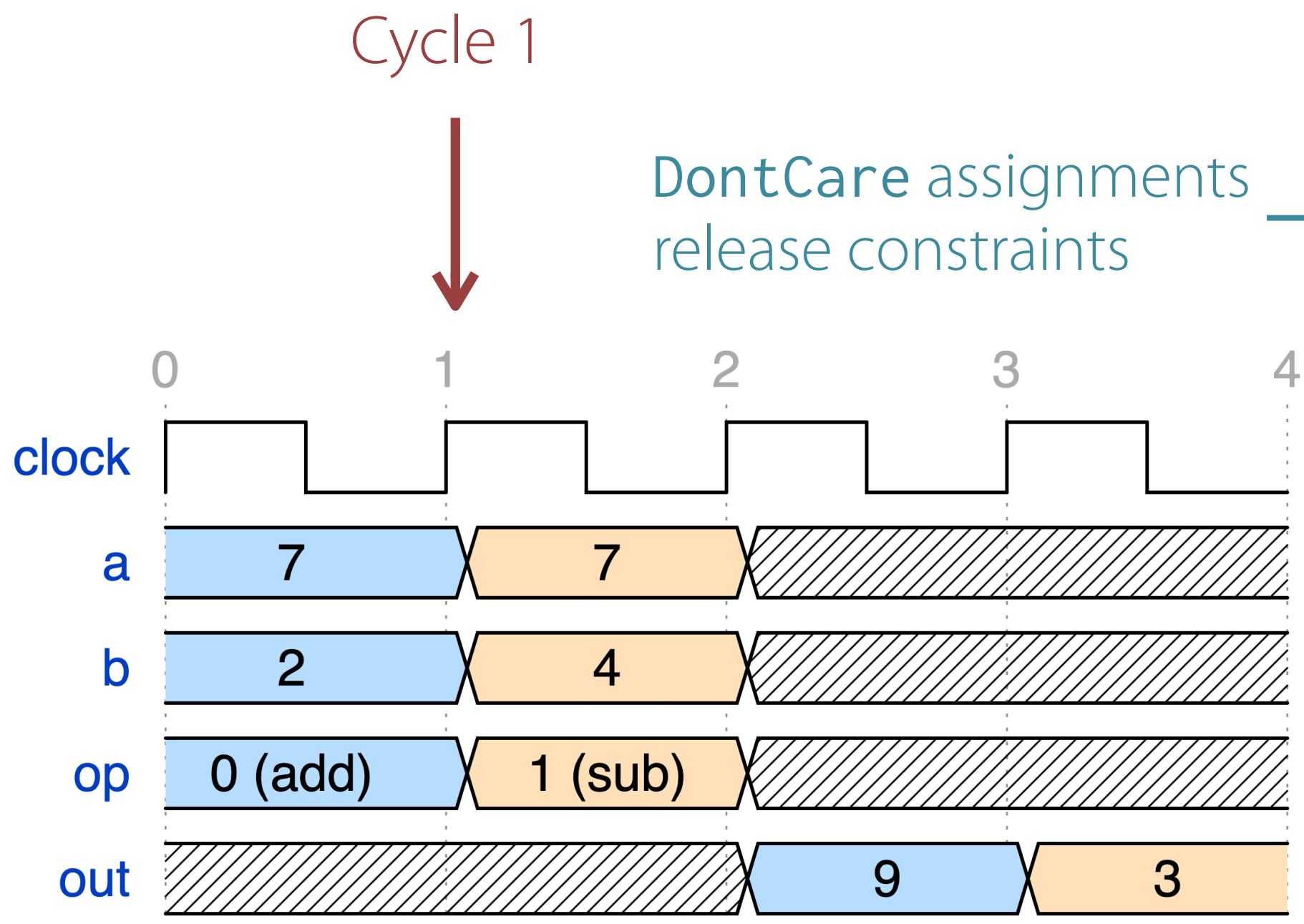
Constraint a = 7 removed



```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}
  
```

add(a=7, b=2, out=?)  
Constraints: { b = 2, 0 = 0 }



```

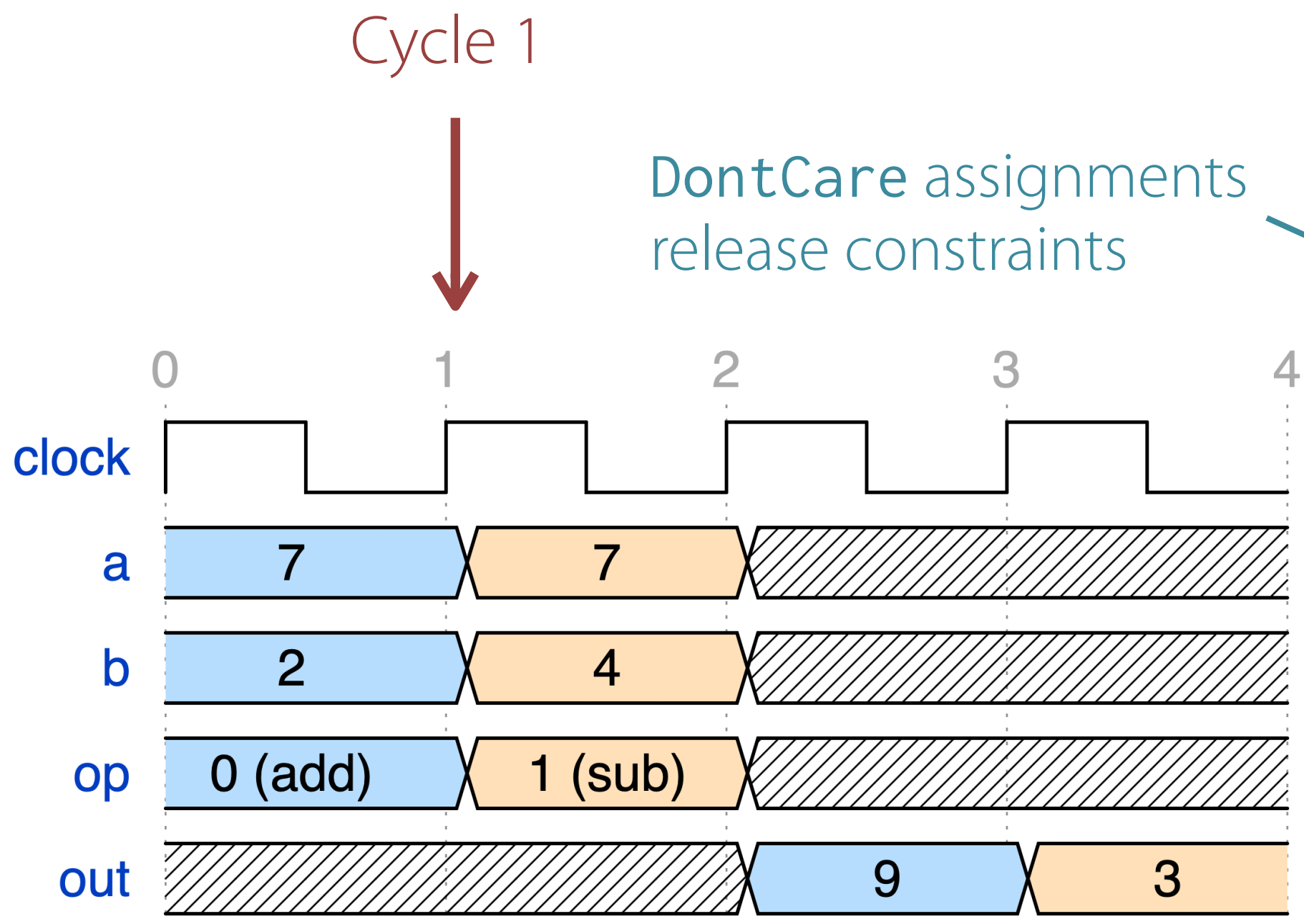
prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}

```

add(a=7, b=2, out=?)

Constraints: { 0 = 0 }

Constraint b = 2 removed



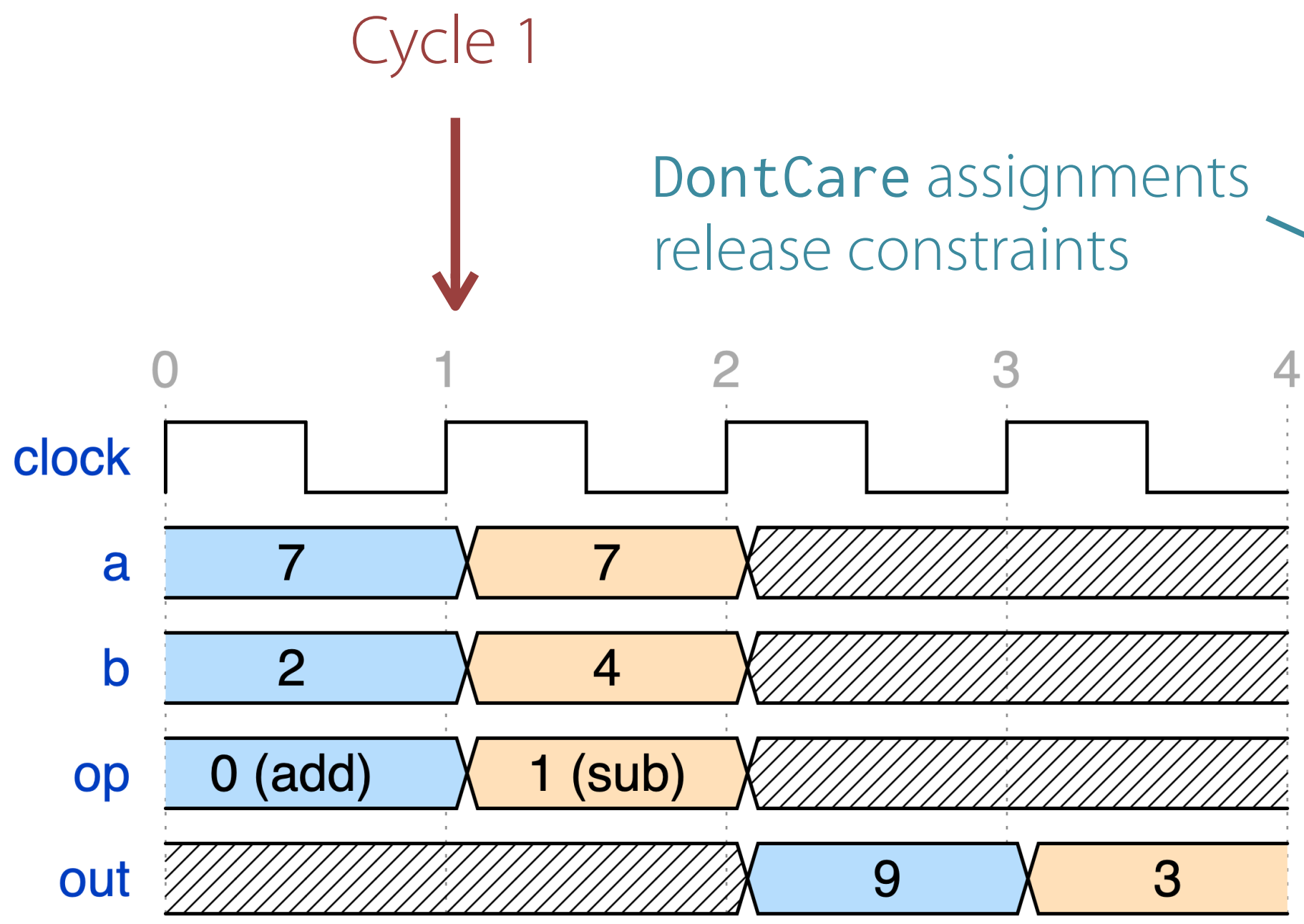
```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}

```

add(a=7, b=2, out=?)

Constraints: { 0 = 0 }



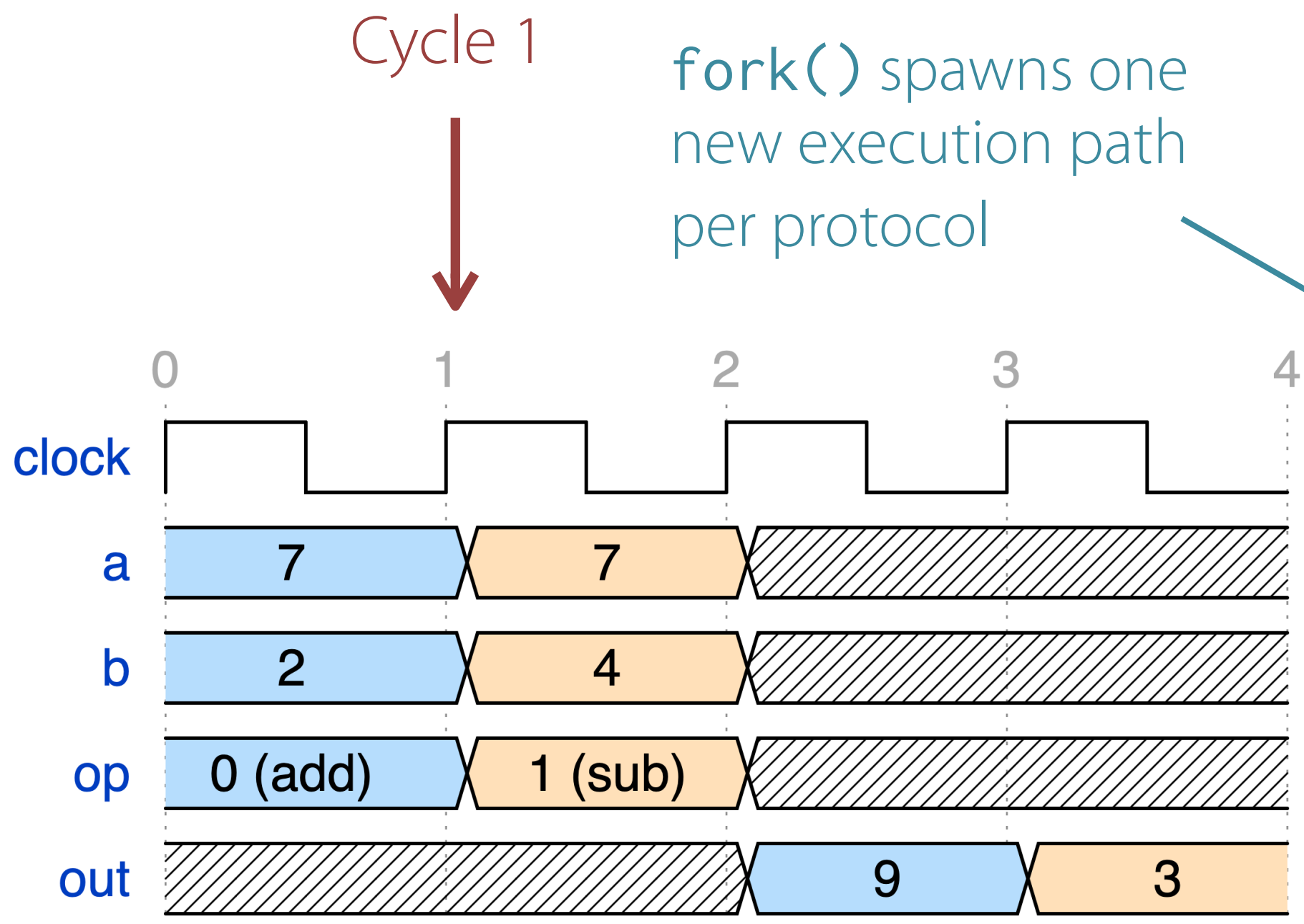
```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}
  
```

add(a=7, b=2, out=?)

Constraints:  $\emptyset$

Constraint 0 = 0 removed

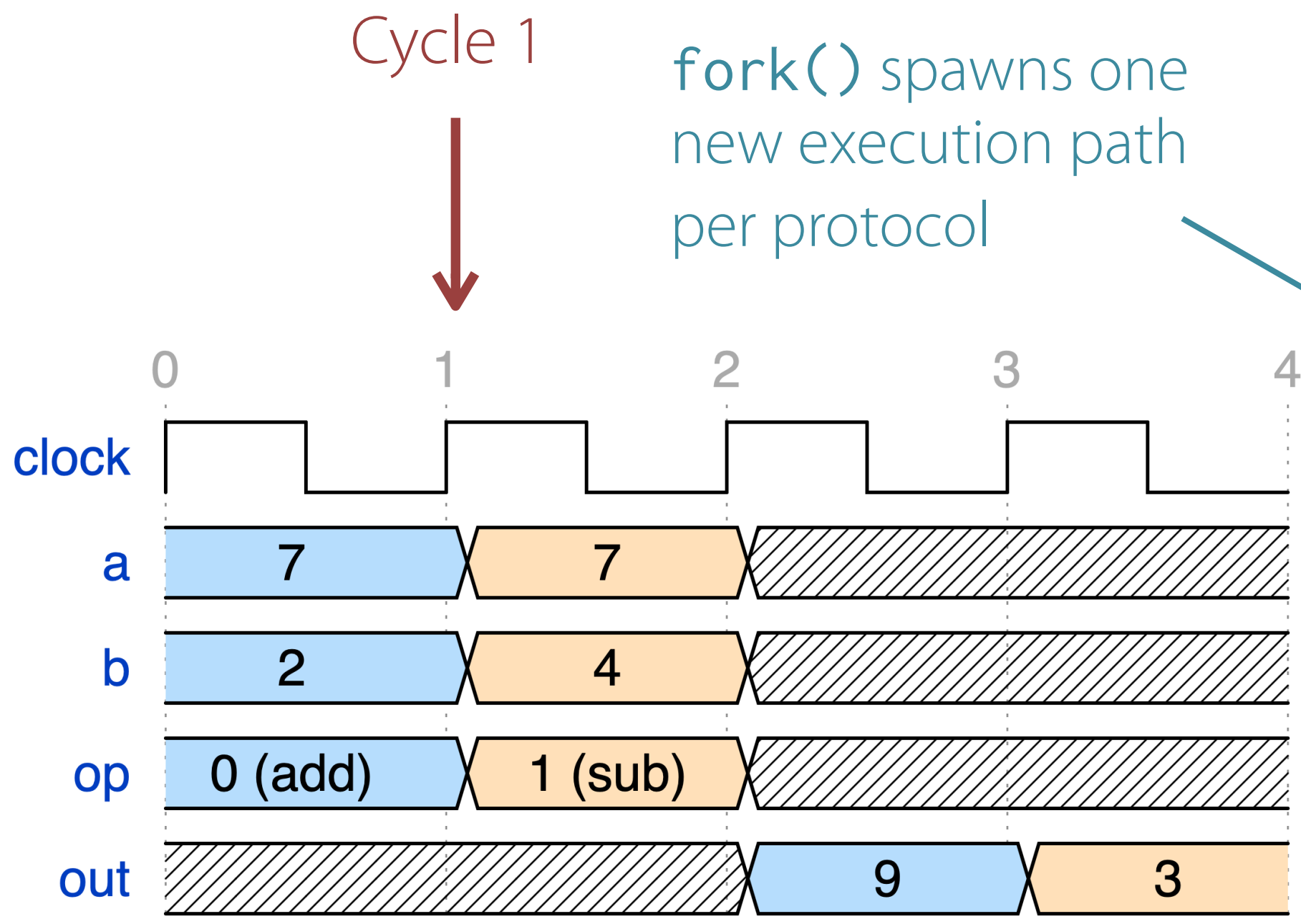


```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}
  
```

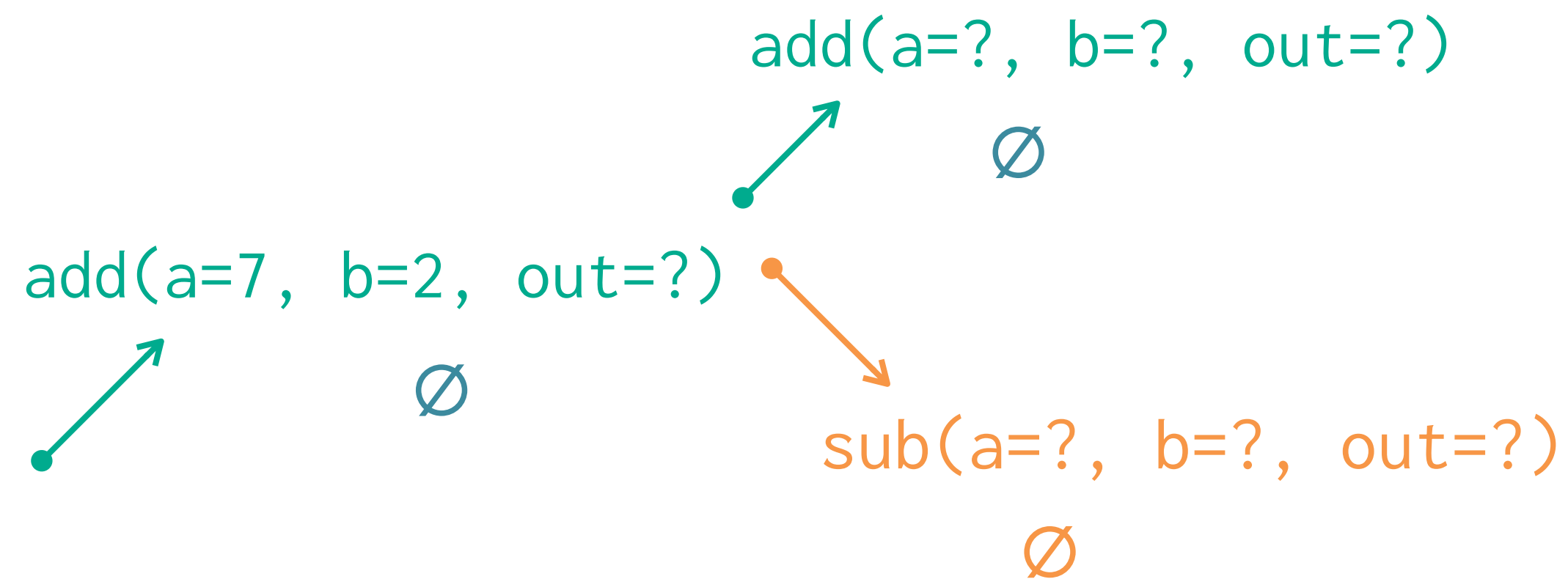
add(a=7, b=2, out=?)

∅

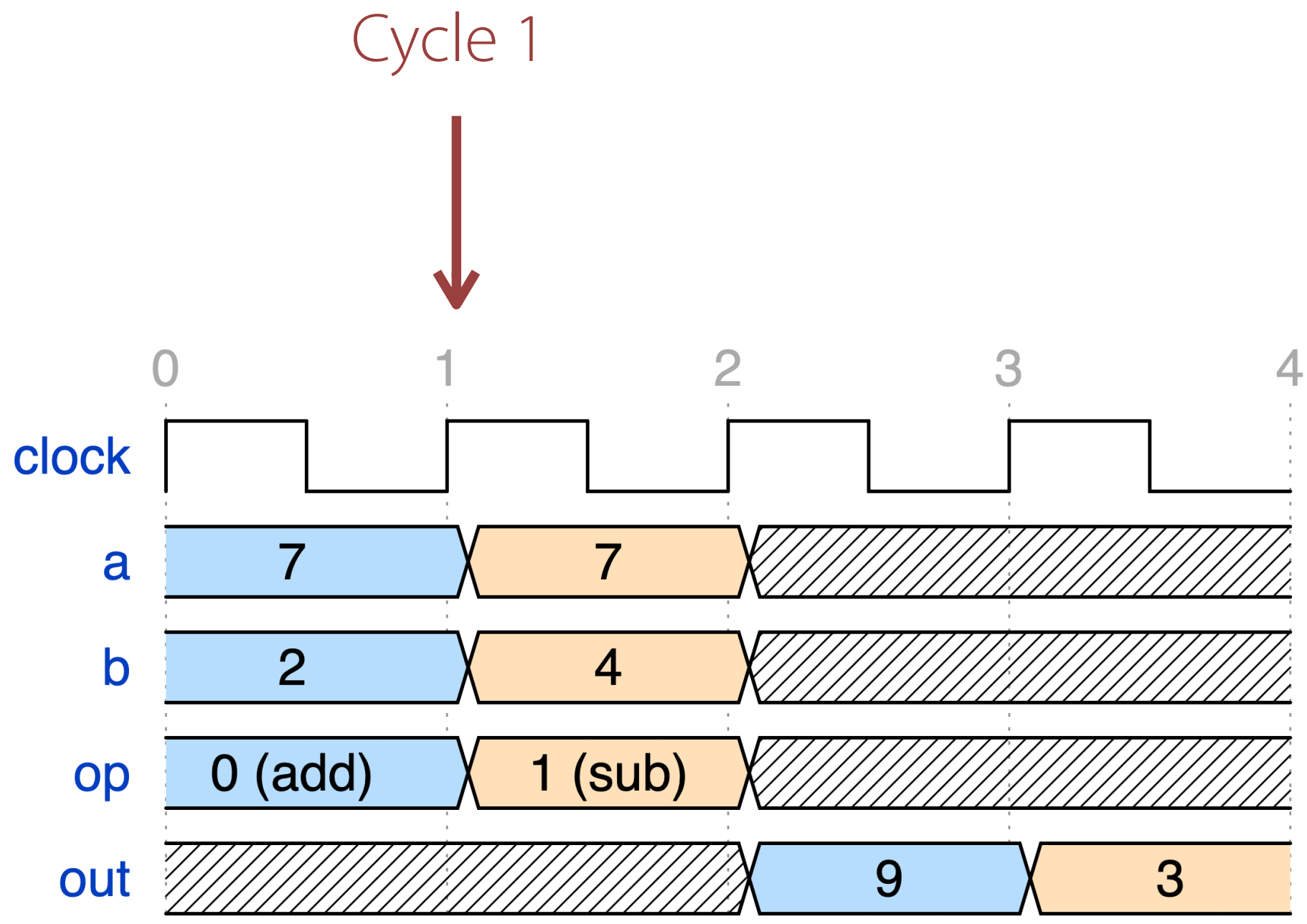


```

prot add<DUT: ALU>(a, b, out) {
    DUT.a := a;
    DUT.b := b;
    DUT.op := 0;
    step();
    DUT.a := X;
    DUT.b := X;
    DUT.op := X;
    fork();
    step();
    assert_eq(DUT.out, out);
}
  
```

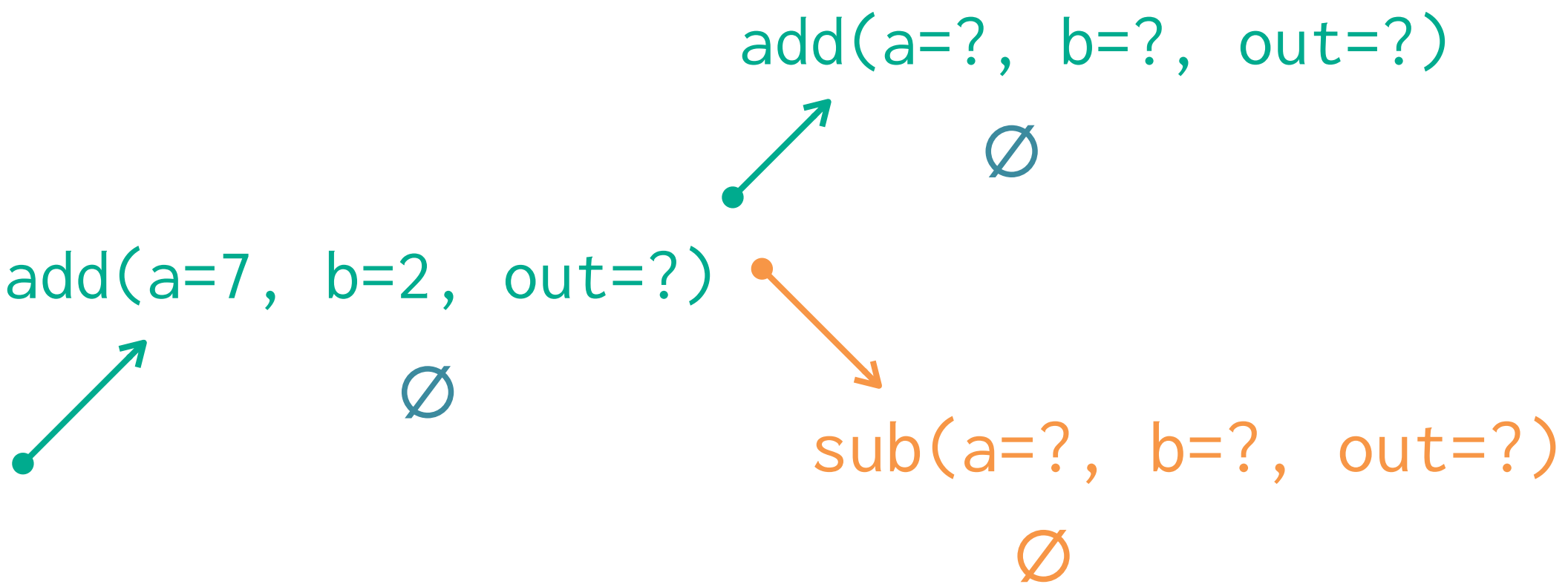


Skipping ahead...

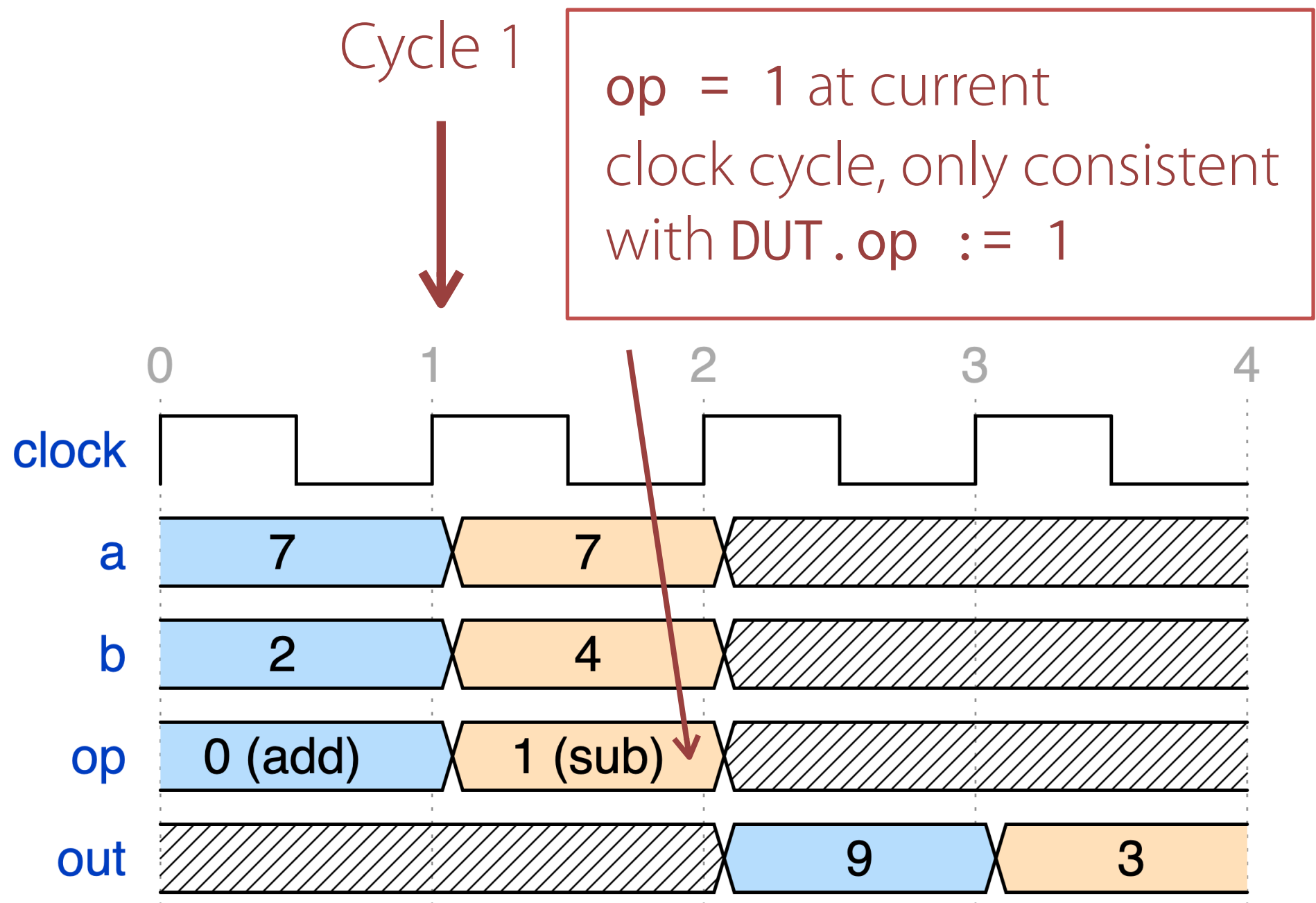


```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

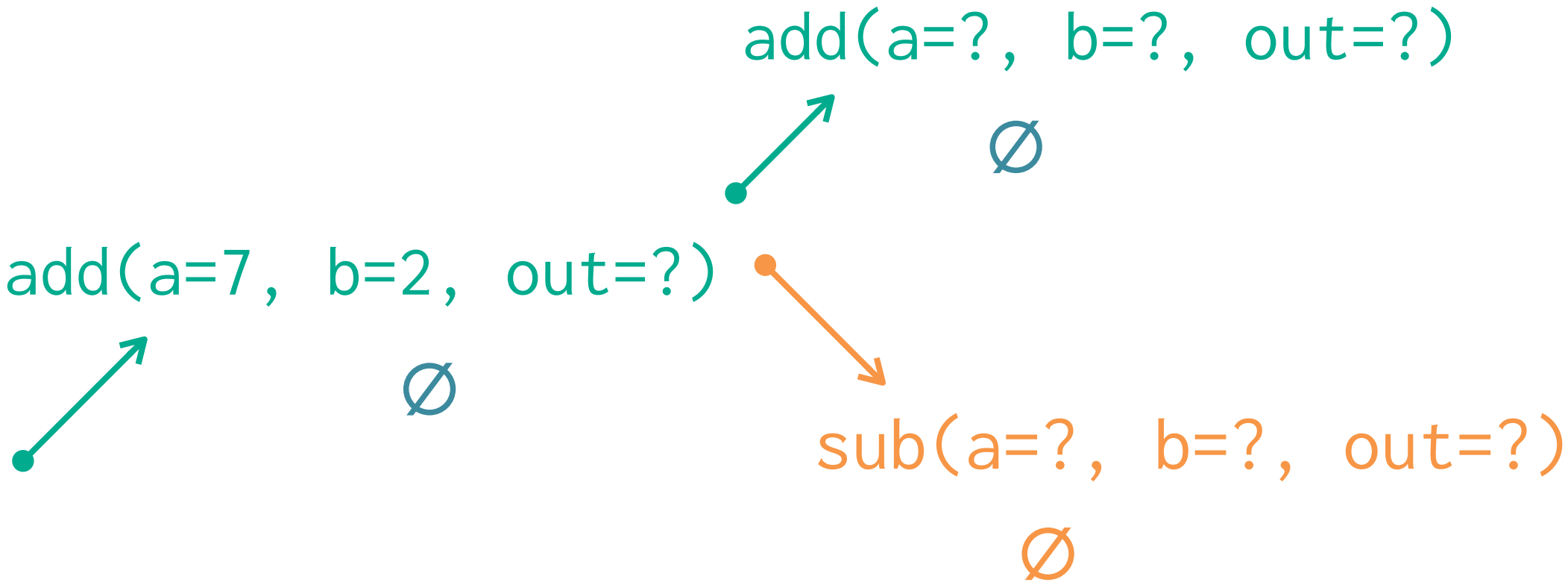


Skipping ahead...

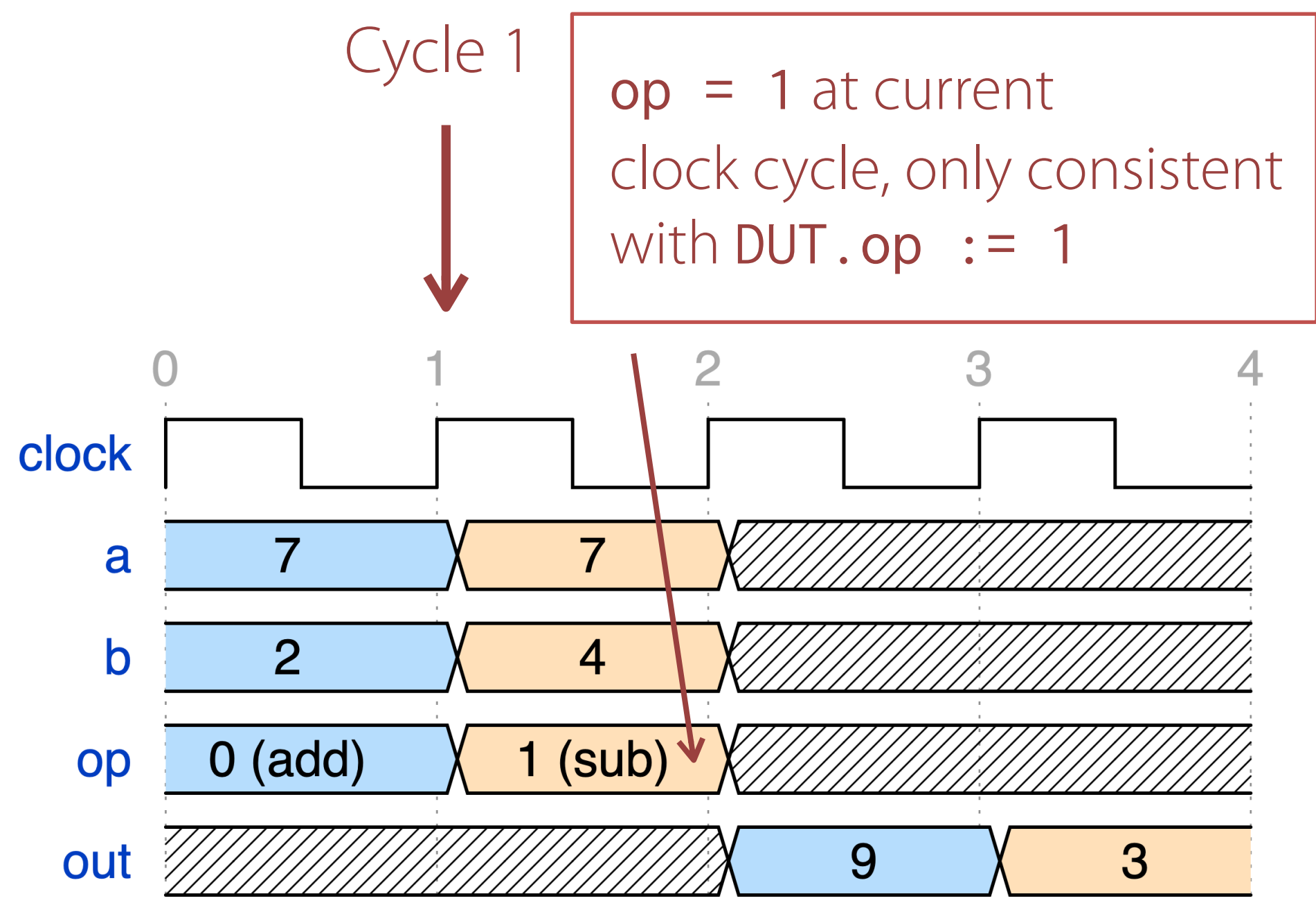


```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```



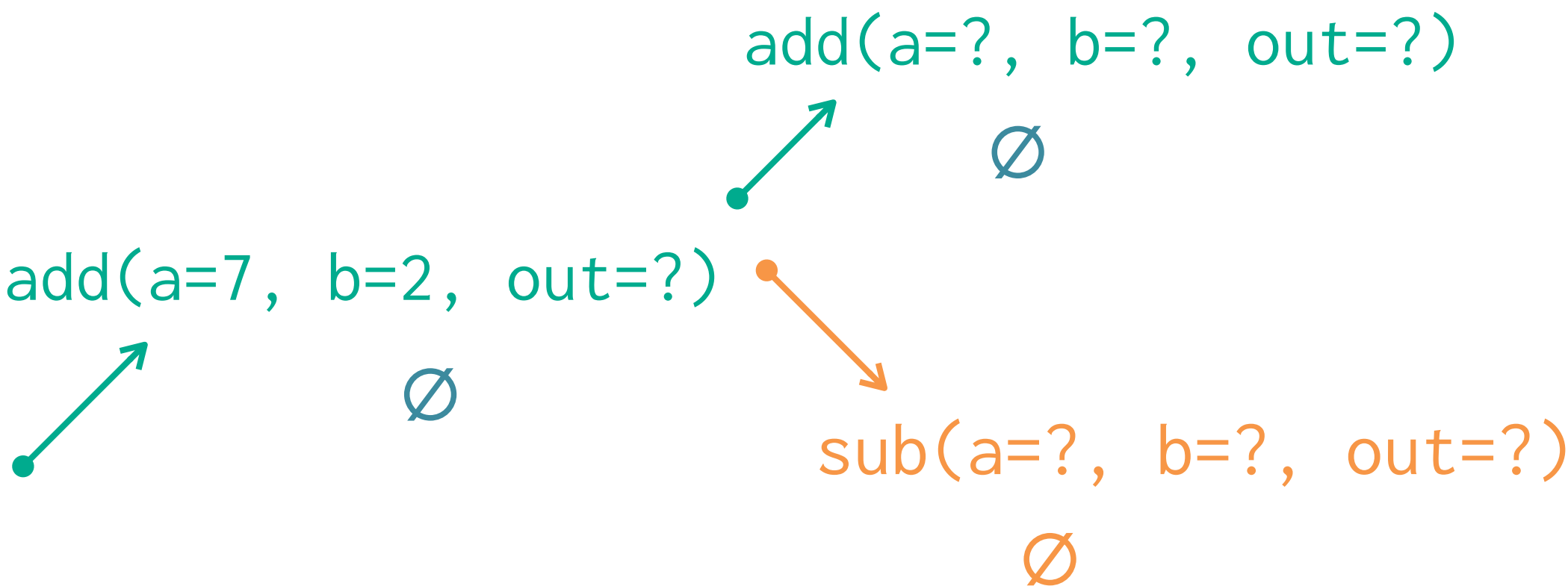
Skipping ahead...



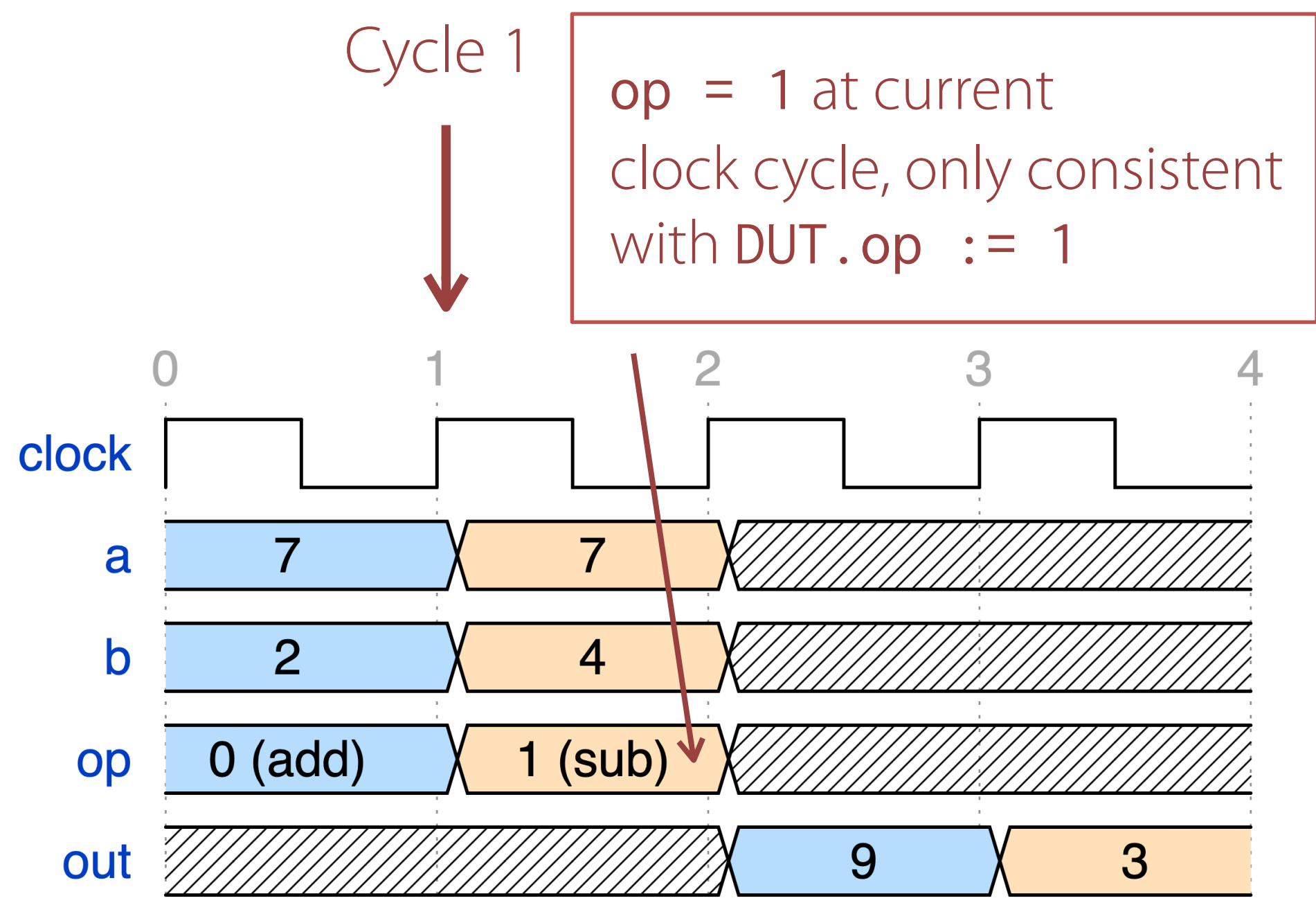
```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

Inconsistent!



# Skipping ahead...

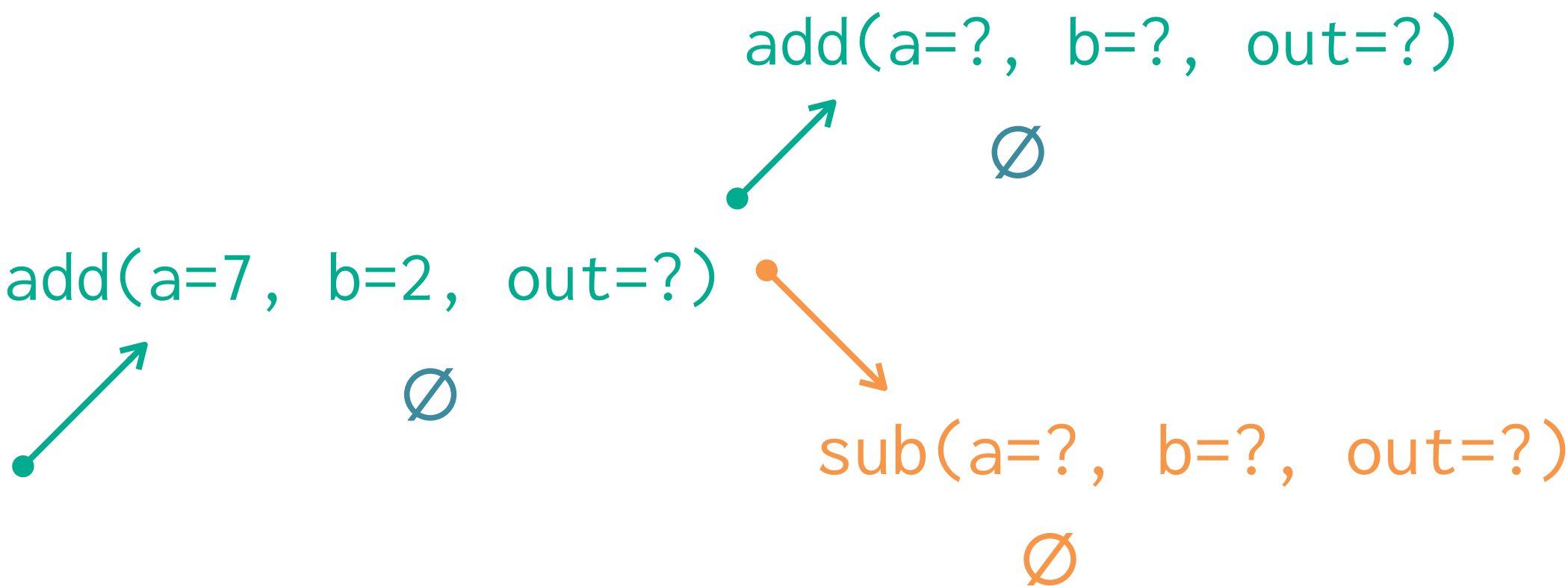


```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

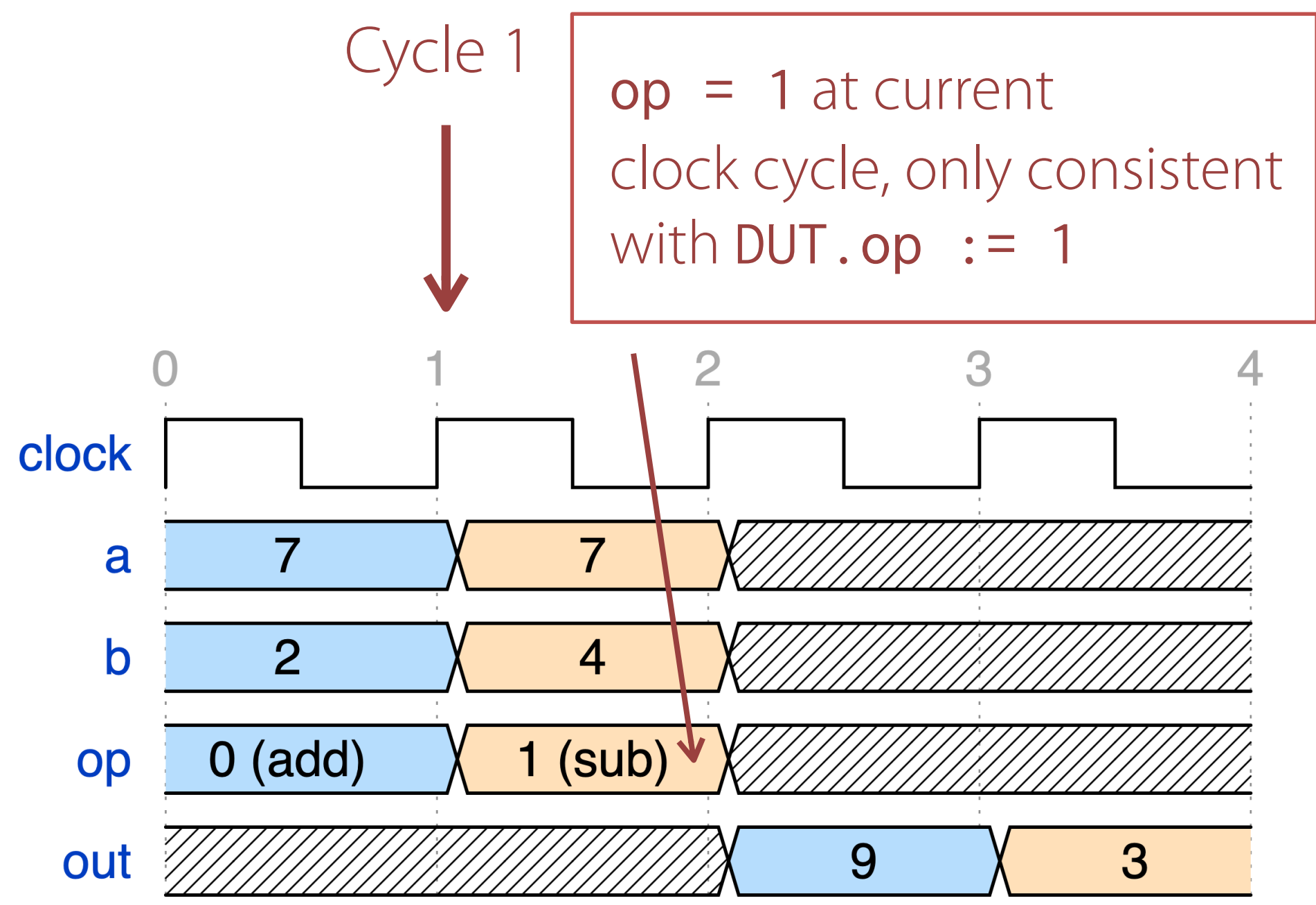
**Inconsistent!**

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

**Consistent**



Skipping ahead...



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

Inconsistent!

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

Consistent

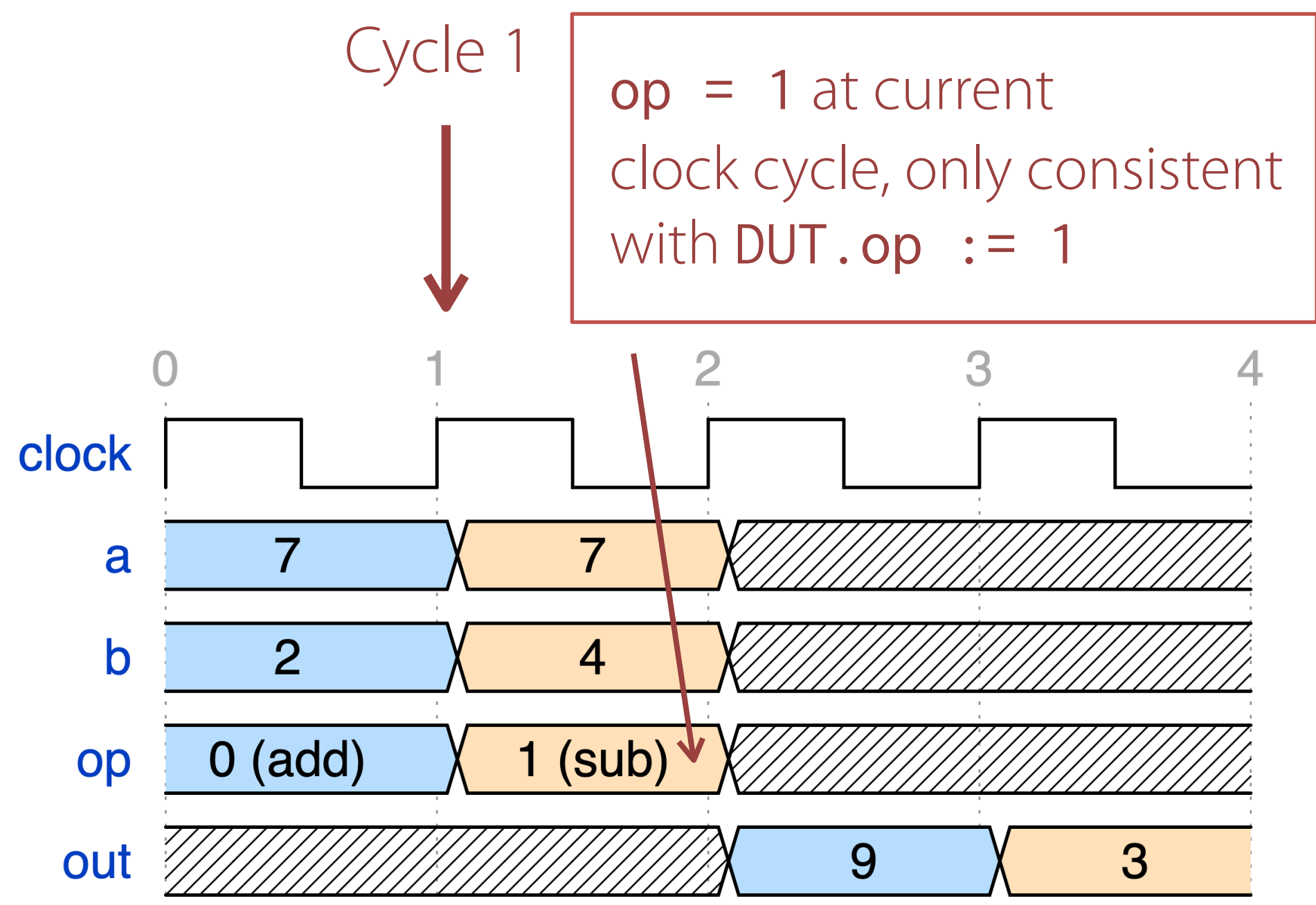
Can't have occurred during cycle 1! This path is pruned

add(a=7, b=4, out=?) X

add(a=7, b=2, out=?)

sub(a=?, b=?, out=?)

Skipping ahead...



```
prot add(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 0;  
  step();  
  ...  
}
```

Inconsistent!

```
prot sub(a, b, out) {  
  DUT.a := a;  
  DUT.b := b;  
  DUT.op := 1;  
  step();  
  ...  
}
```

Consistent

Can't have occurred during cycle 1! This path is pruned

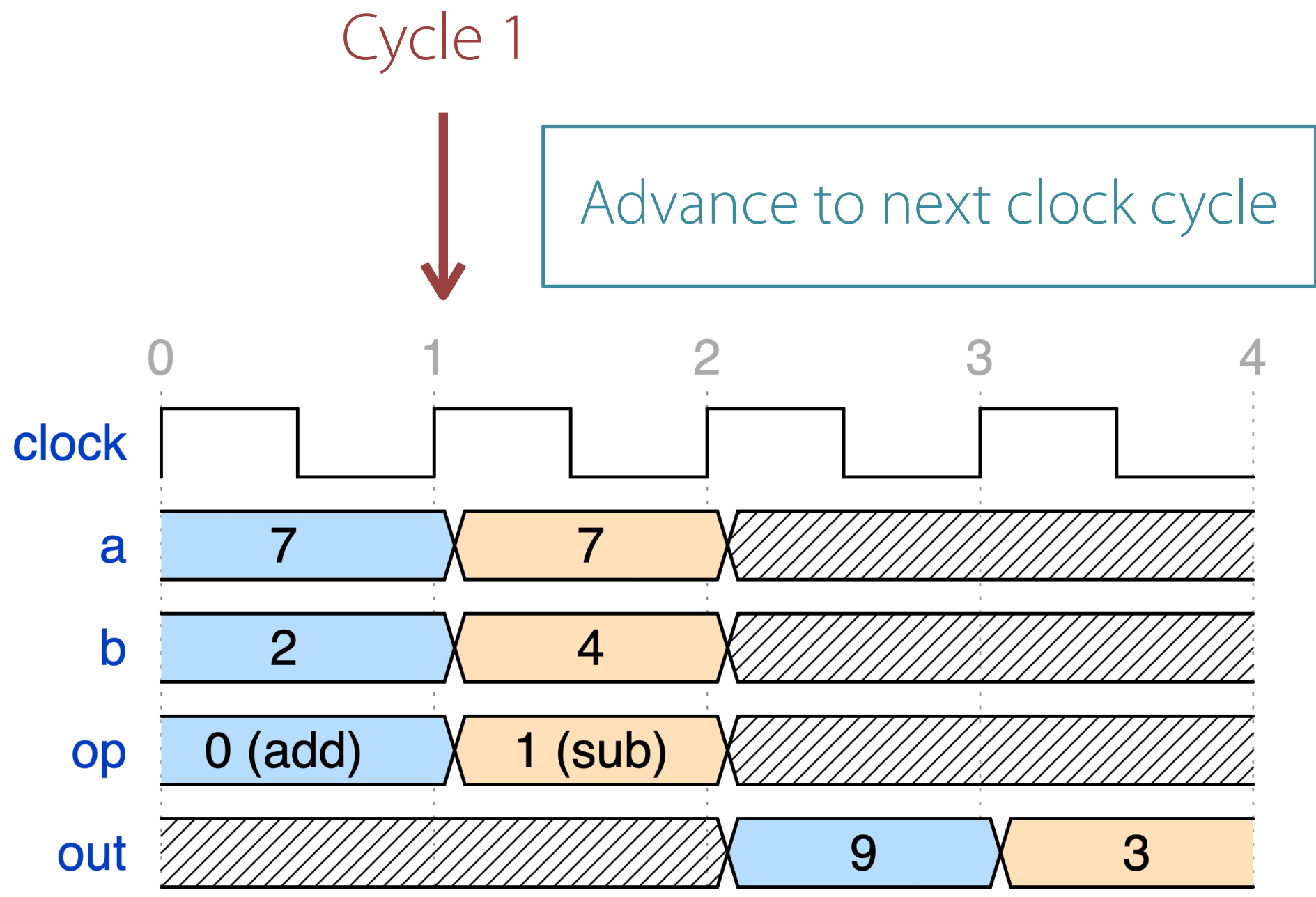
add(a=7, b=4, out=?) X

add(a=7, b=2, out=?)

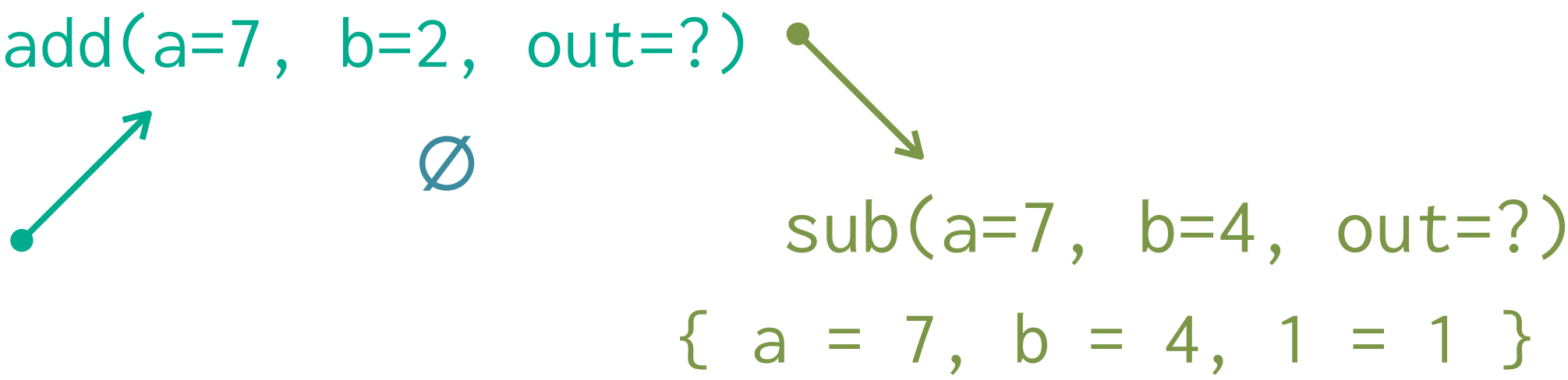
sub(a=7, b=4, out=?)

{ a = 7, b = 4, 1 = 1 }

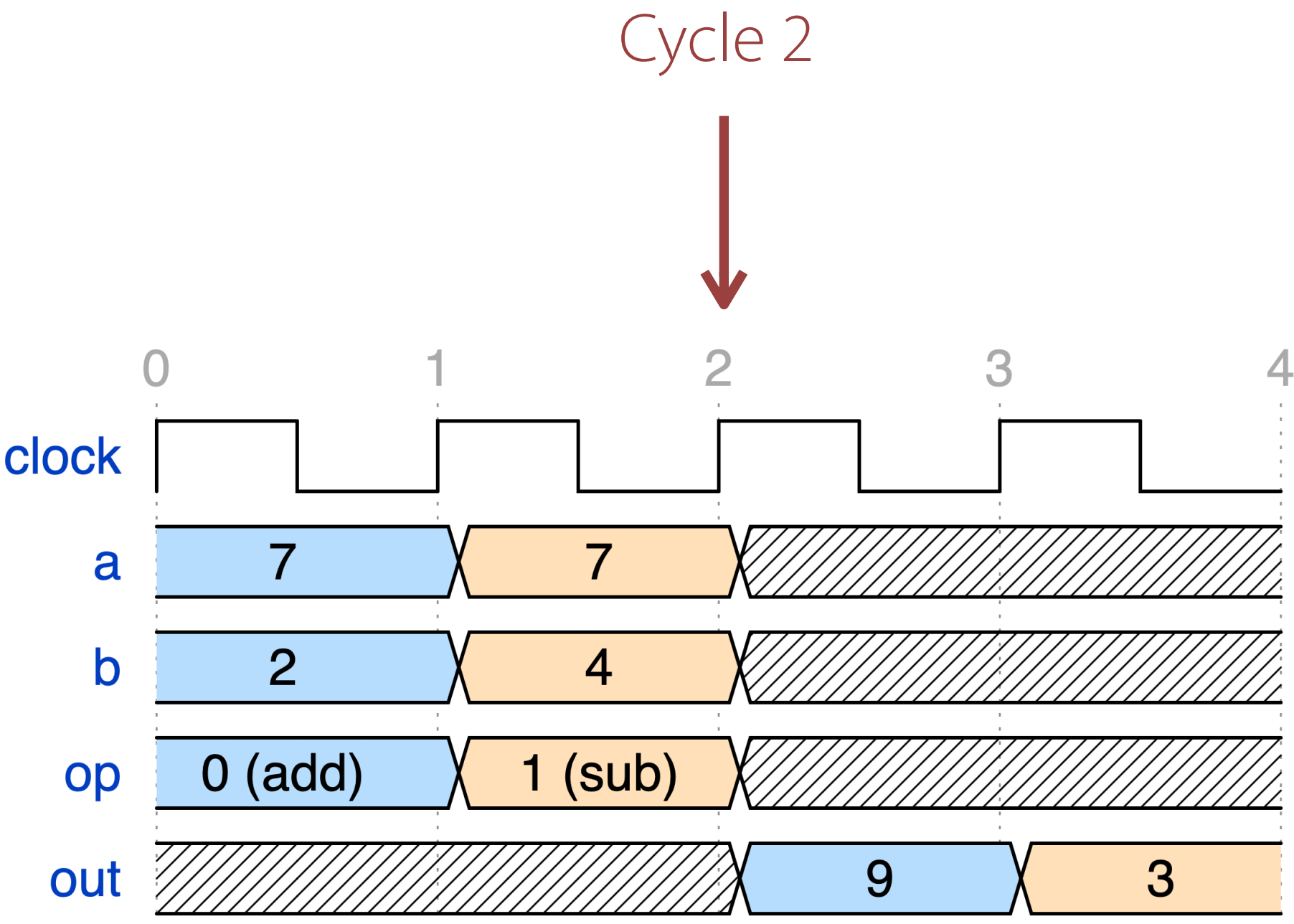
Parent add transaction continues...



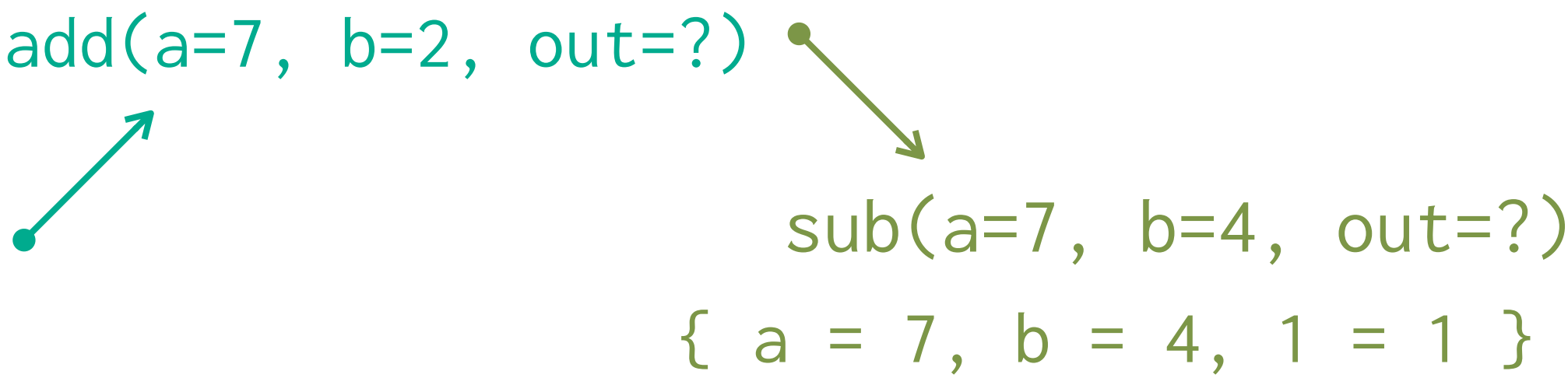
```
prot add<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 0;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```



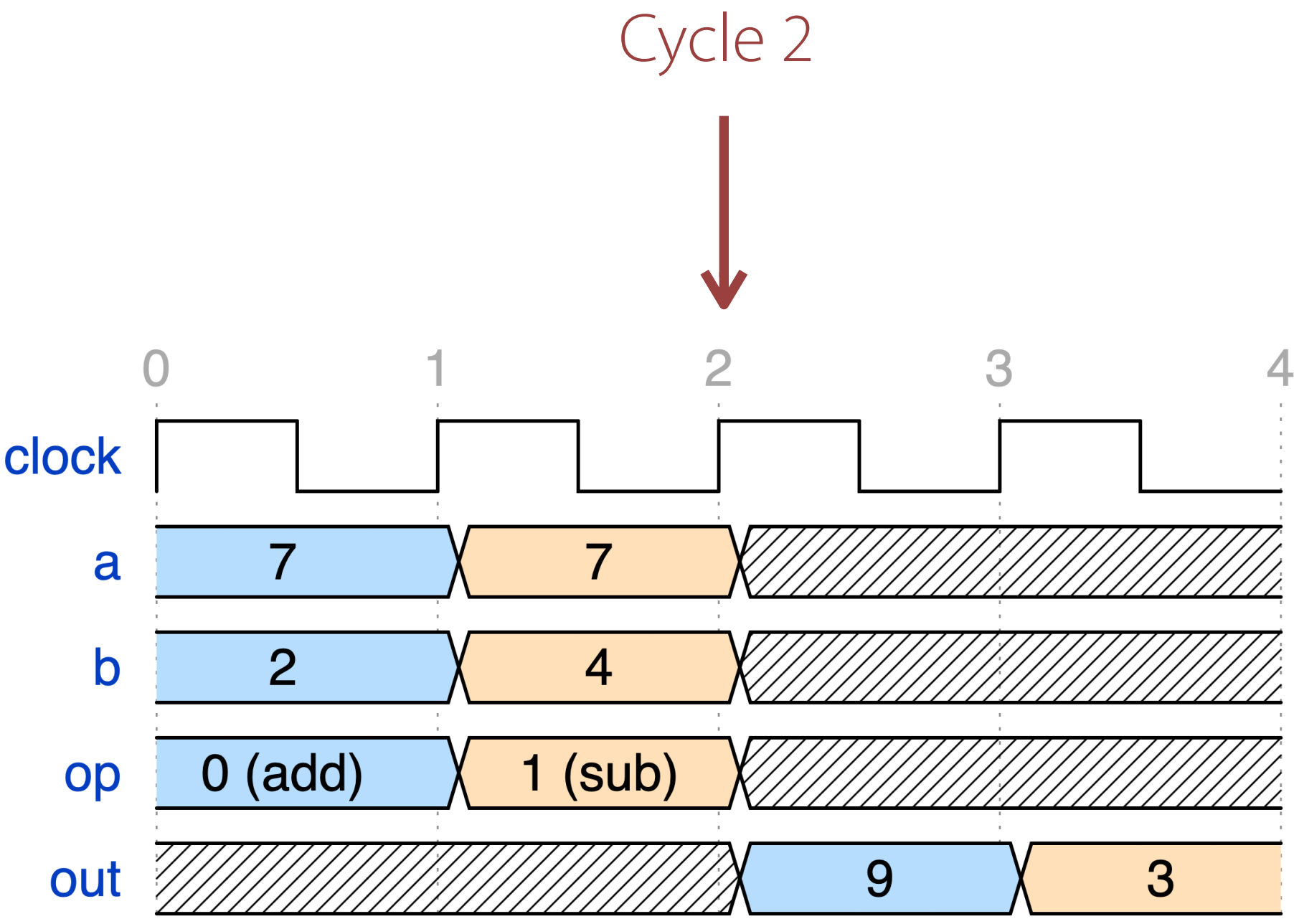
Parent add transaction continues...



```
prot add<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 0;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```

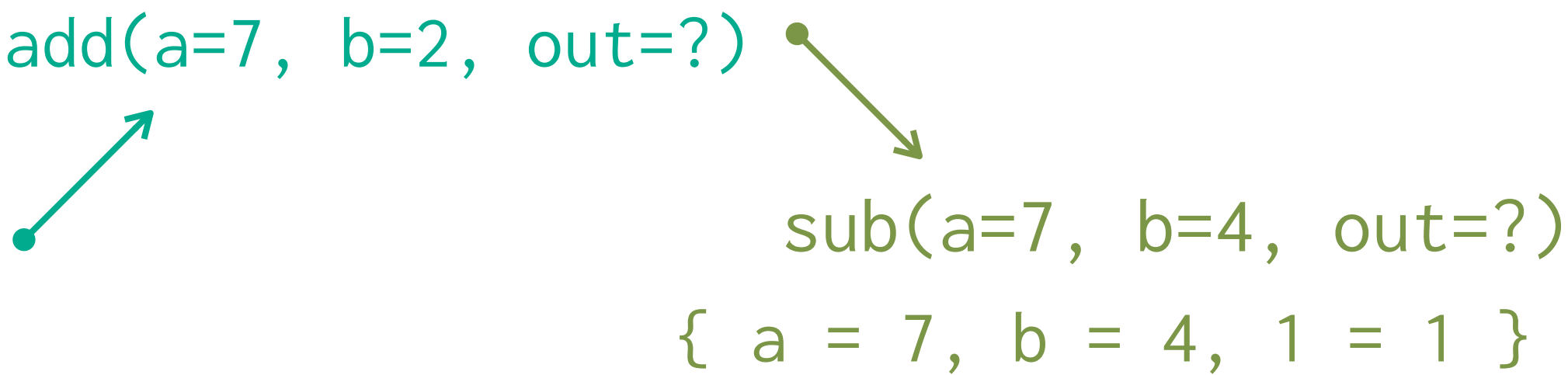


Parent add transaction continues...

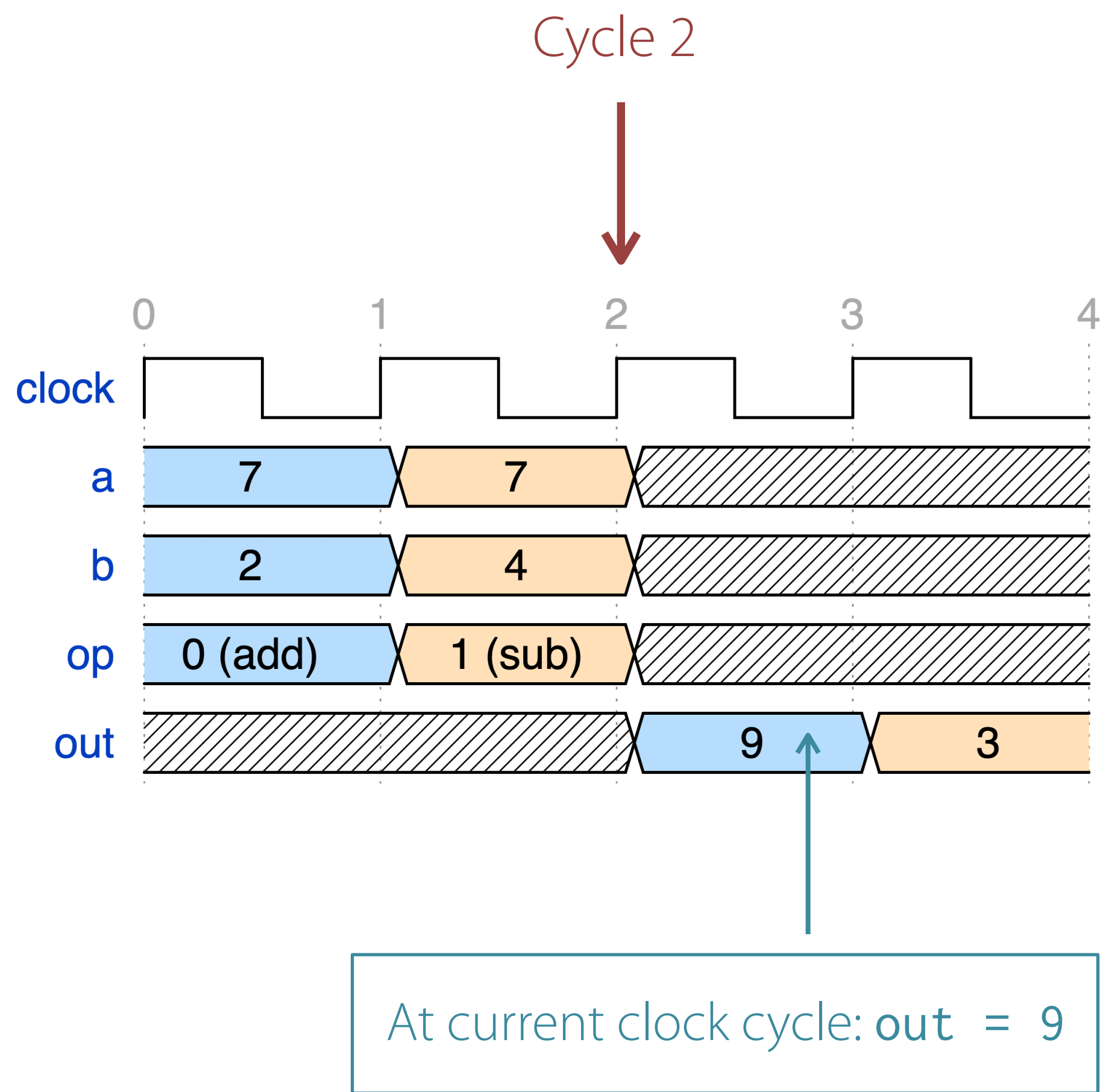


```
prot add<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 0;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```

Value of out inferred from waveform

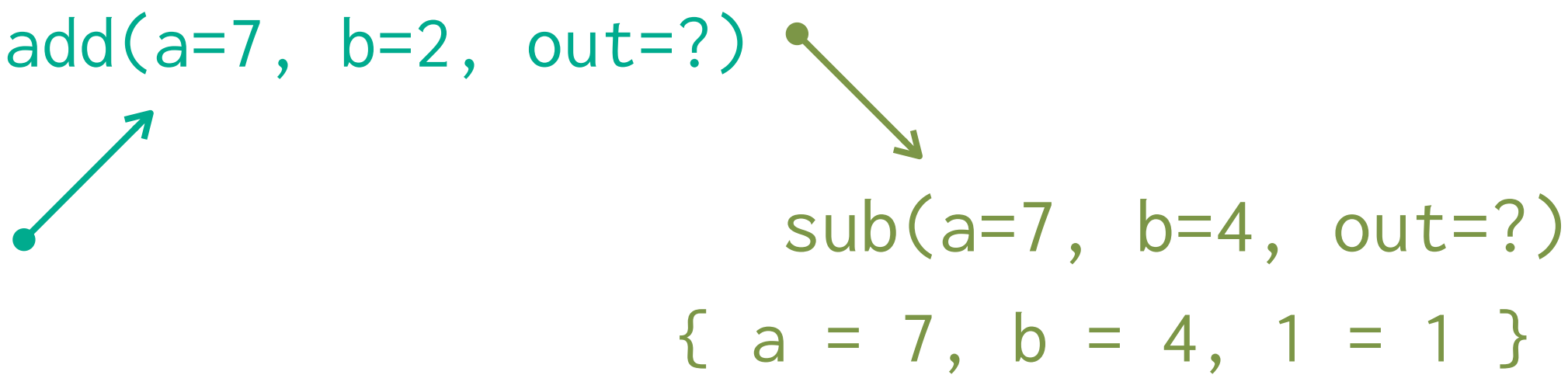


Parent add transaction continues...

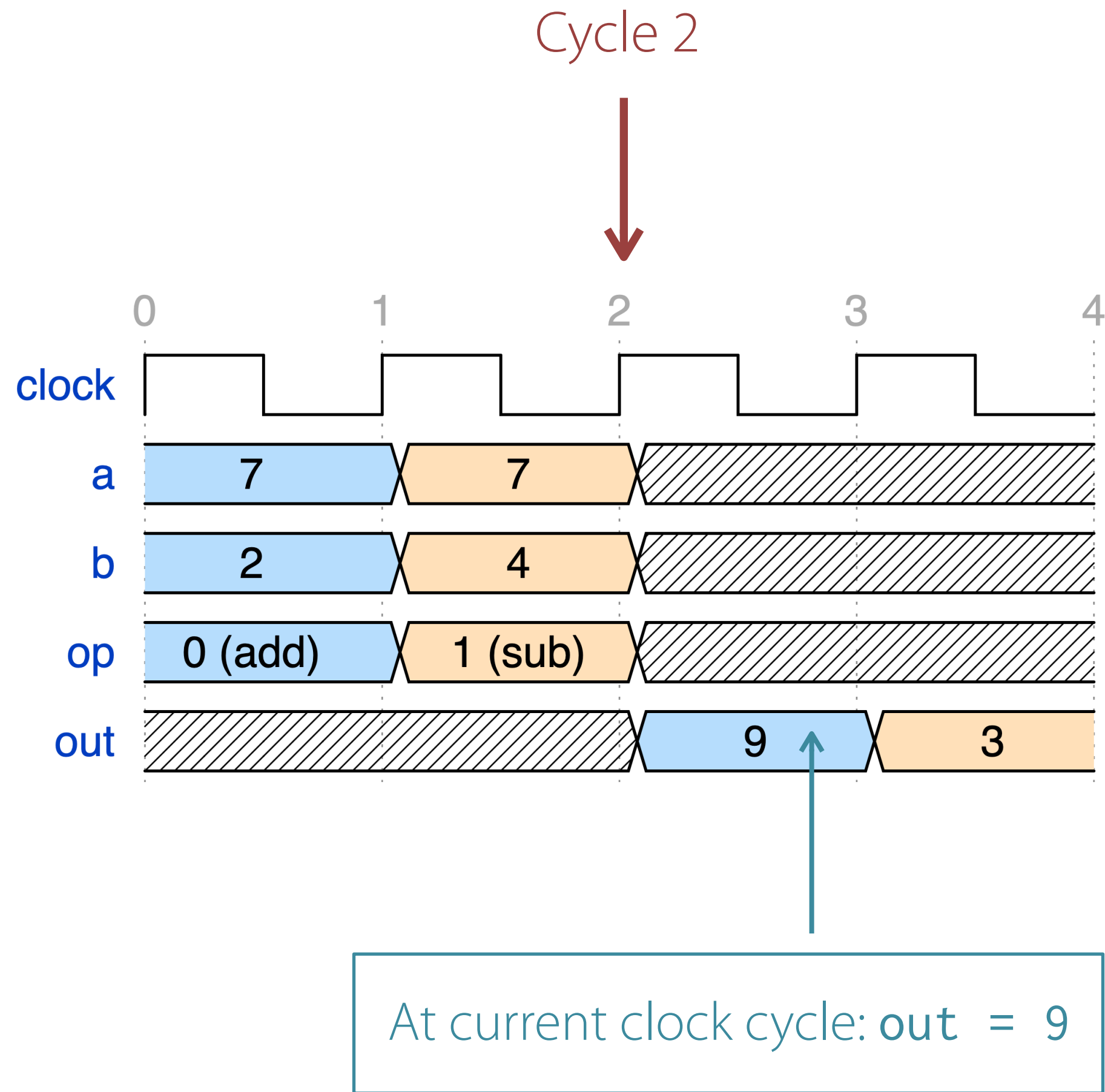


```
prot add<DUT: ALU>(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 0;
  step();
  DUT.a := X;
  DUT.b := X;
  DUT.op := X;
  fork();
  step();
  assert_eq(DUT.out, out);
}
```

Value of out inferred from waveform



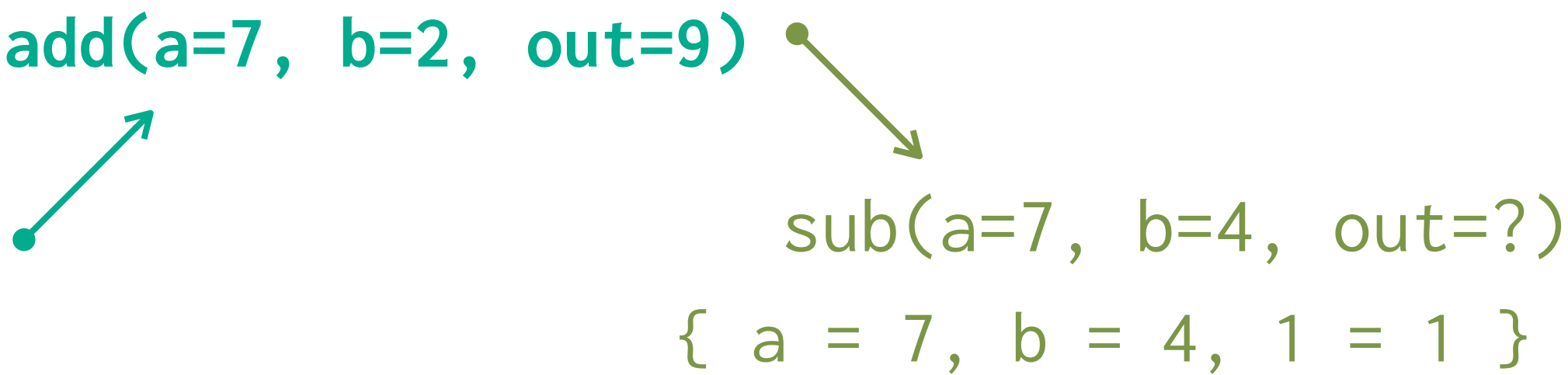
Parent add transaction continues...



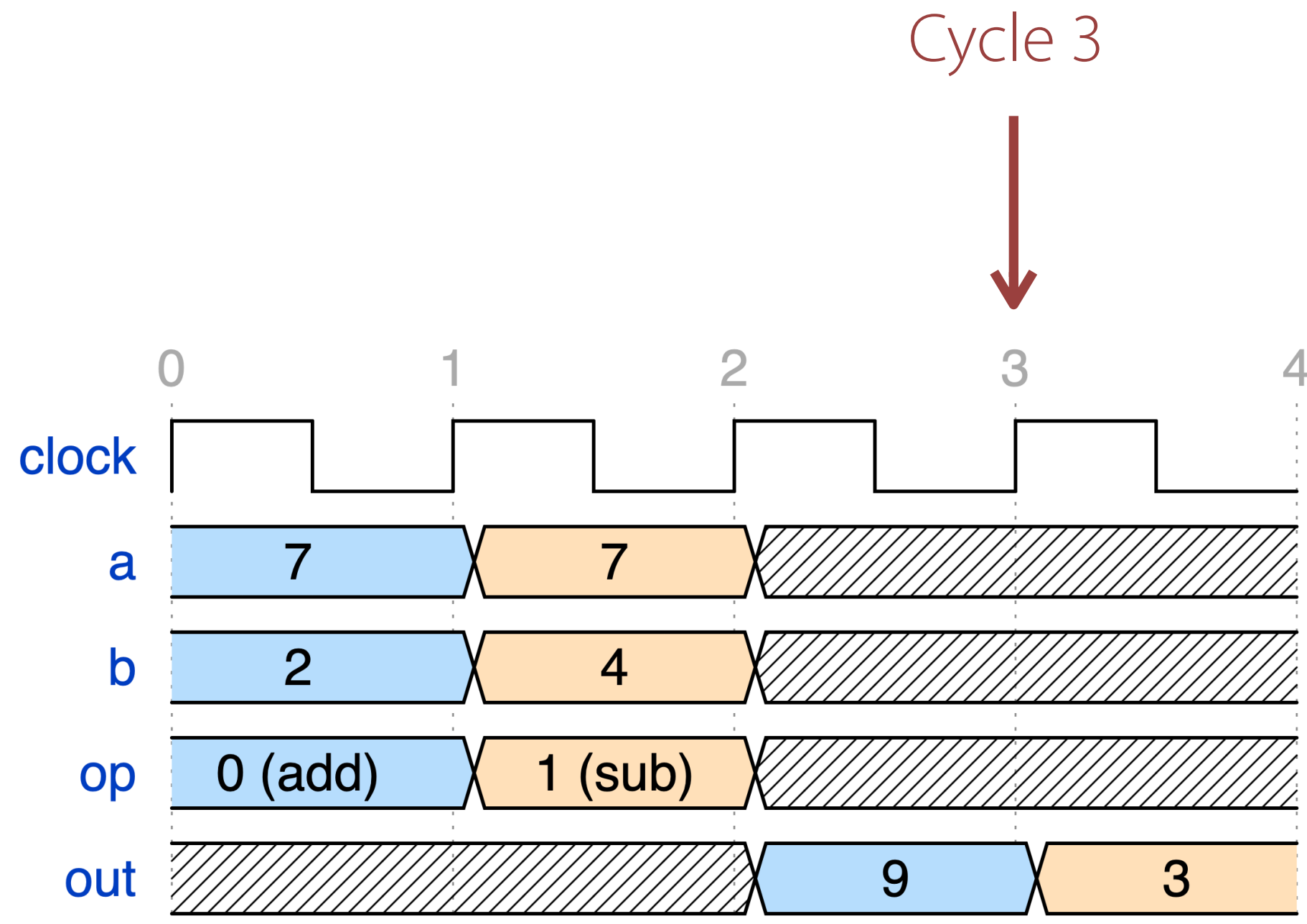
```
prot add<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 0;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```

Value of out inferred from waveform

Transaction complete!



## Skipping ahead to end of sub transaction...



```
prot sub<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 1;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```

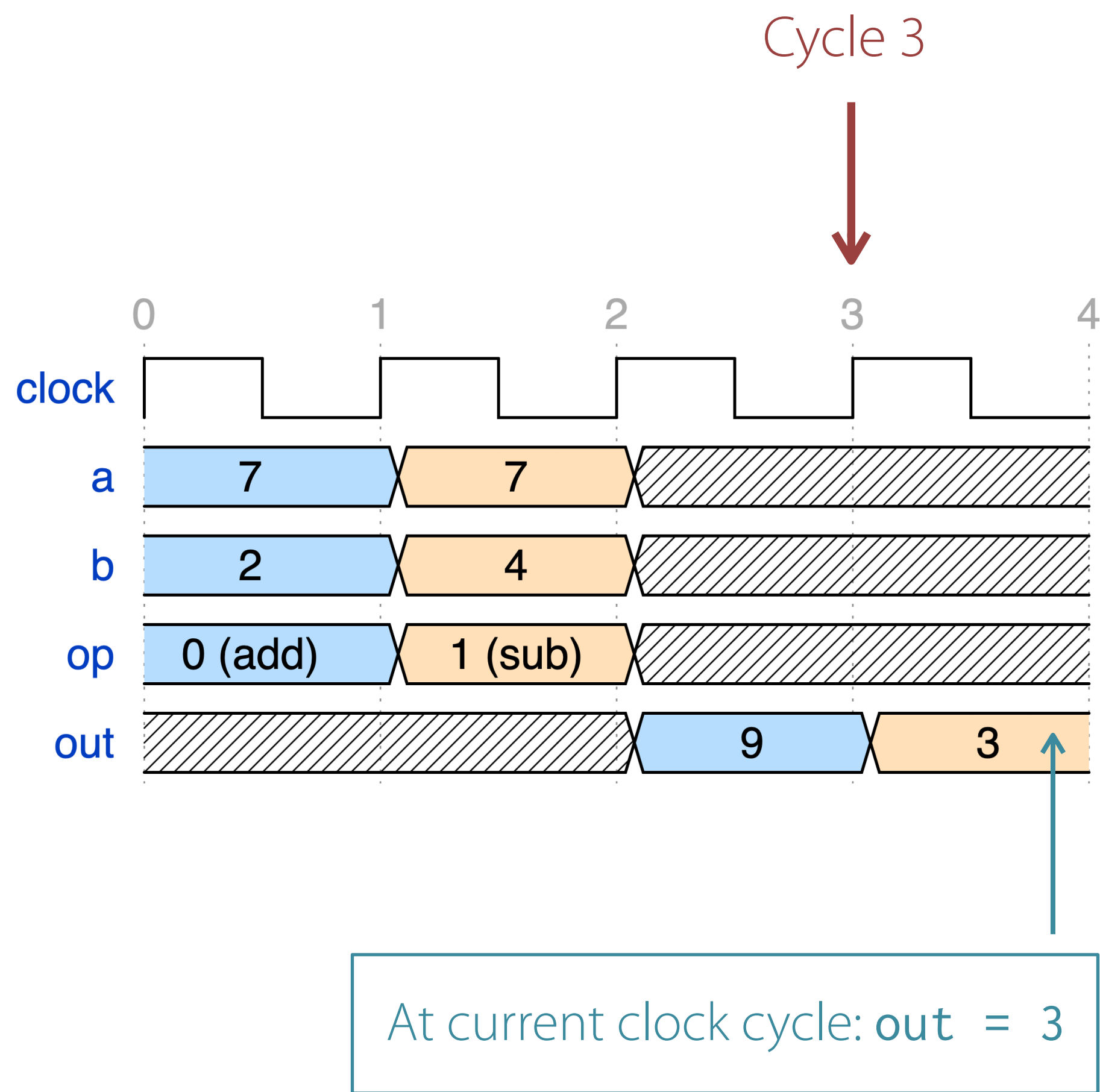
Value of **out** inferred from waveform

**Transaction complete!**

add(a=7, b=2, out=9)

sub(a=7, b=4, out=?)

Skipping ahead to end of sub transaction...



```
prot sub<DUT: ALU>(a, b, out) {
  DUT.a := a;
  DUT.b := b;
  DUT.op := 1;
  step();
  DUT.a := X;
  DUT.b := X;
  DUT.op := X;
  fork();
  step();
  assert_eq(DUT.out, out);
}
```

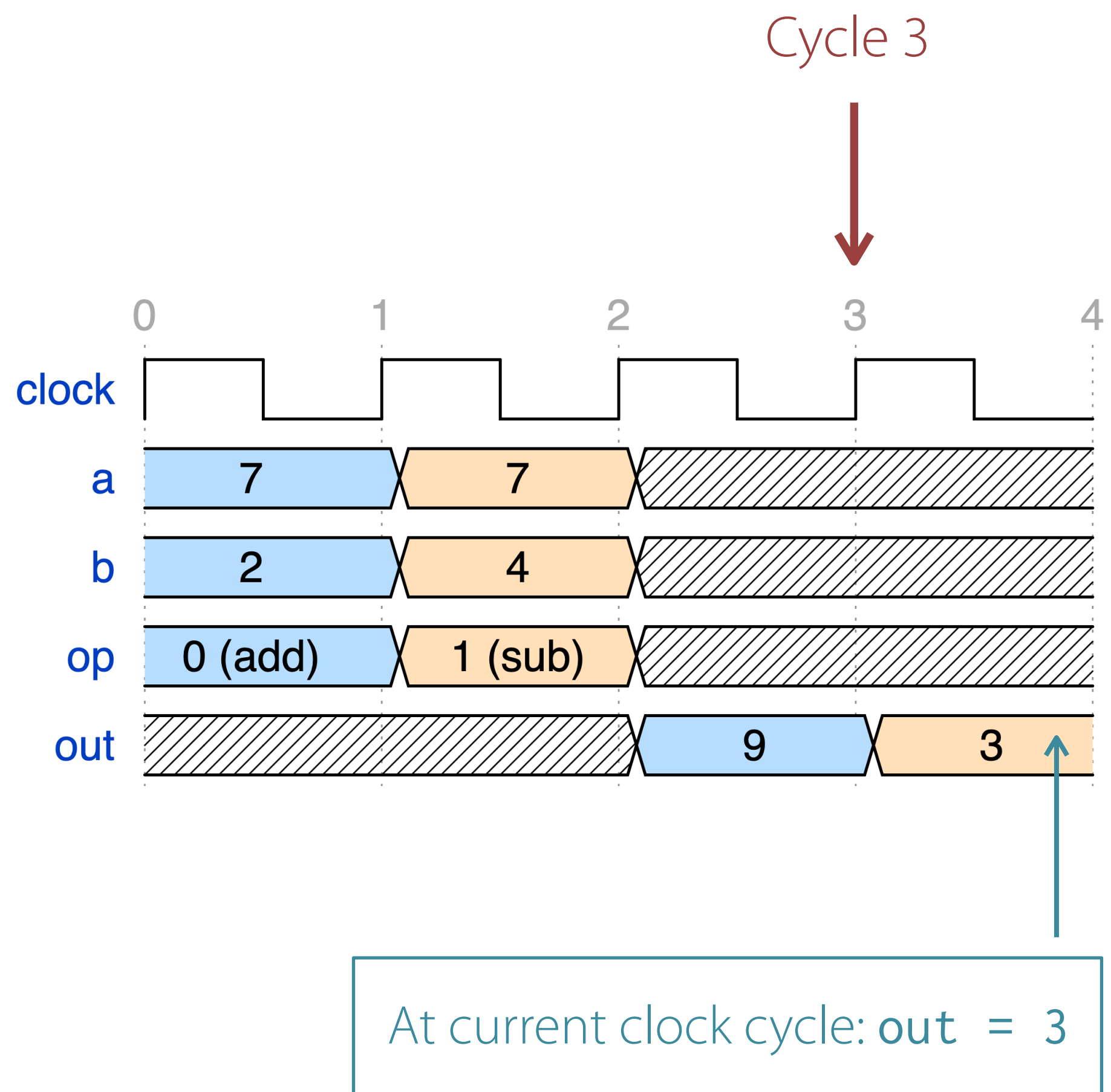
Value of out inferred from waveform

Transaction complete!

add(a=7, b=2, out=9)

sub(a=7, b=4, out=?)

**Skipping ahead to end of sub transaction...**



```
prot sub<DUT: ALU>(a, b, out) {  
    DUT.a := a;  
    DUT.b := b;  
    DUT.op := 1;  
    step();  
    DUT.a := X;  
    DUT.b := X;  
    DUT.op := X;  
    fork();  
    step();  
    assert_eq(DUT.out, out);  
}
```

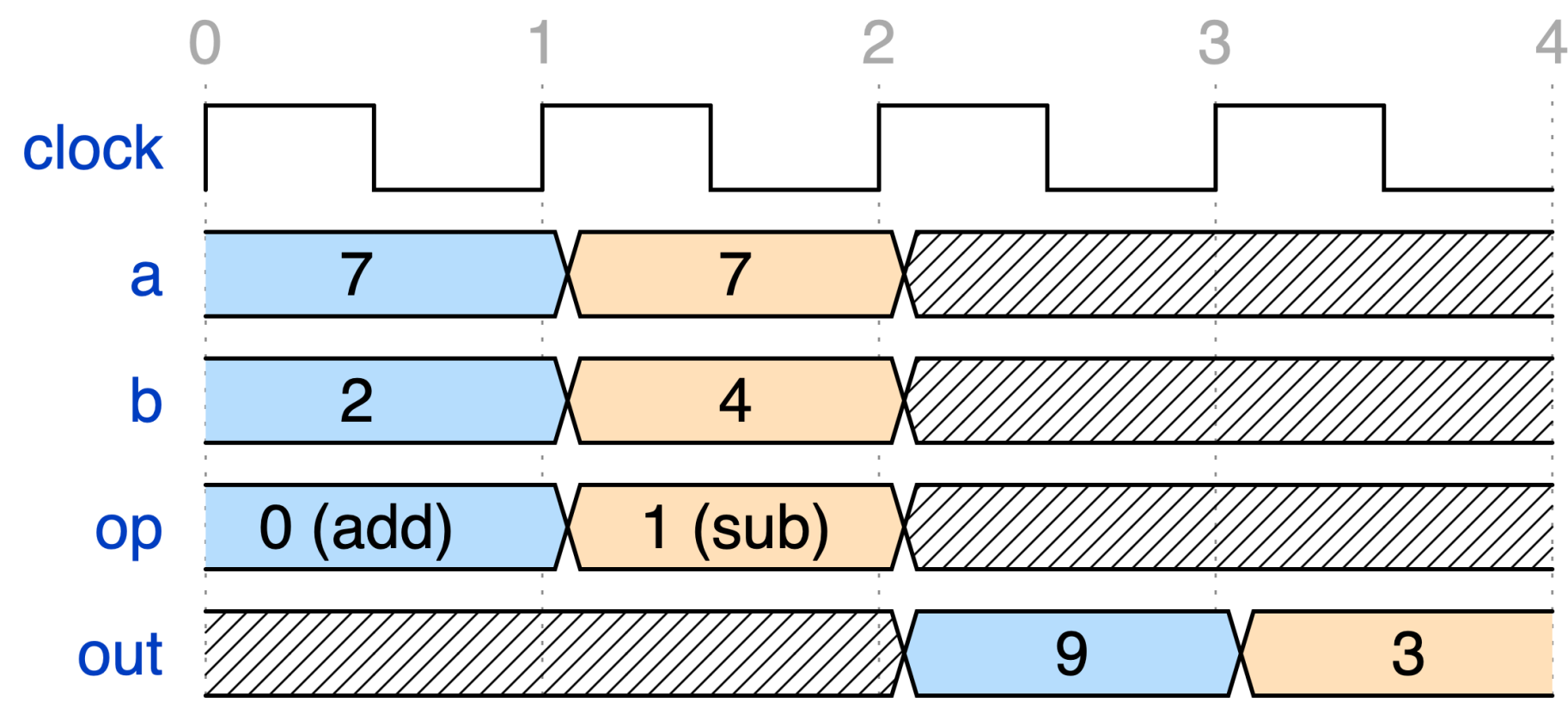
Value of out inferred from waveform

**Transaction complete!**

add(a=7, b=2, out=9)

sub(a=7, b=4, out=3)

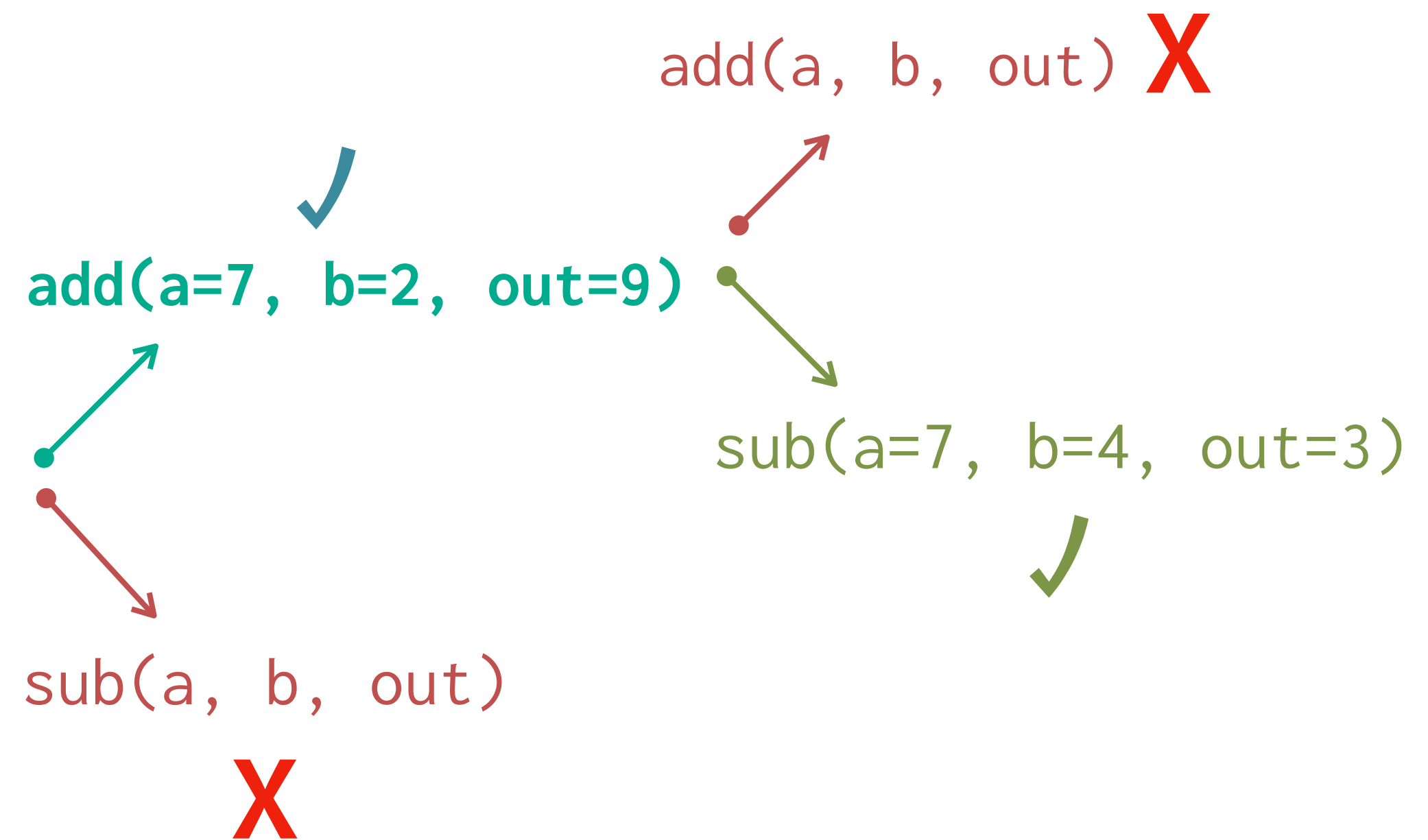
**Transaction complete!**

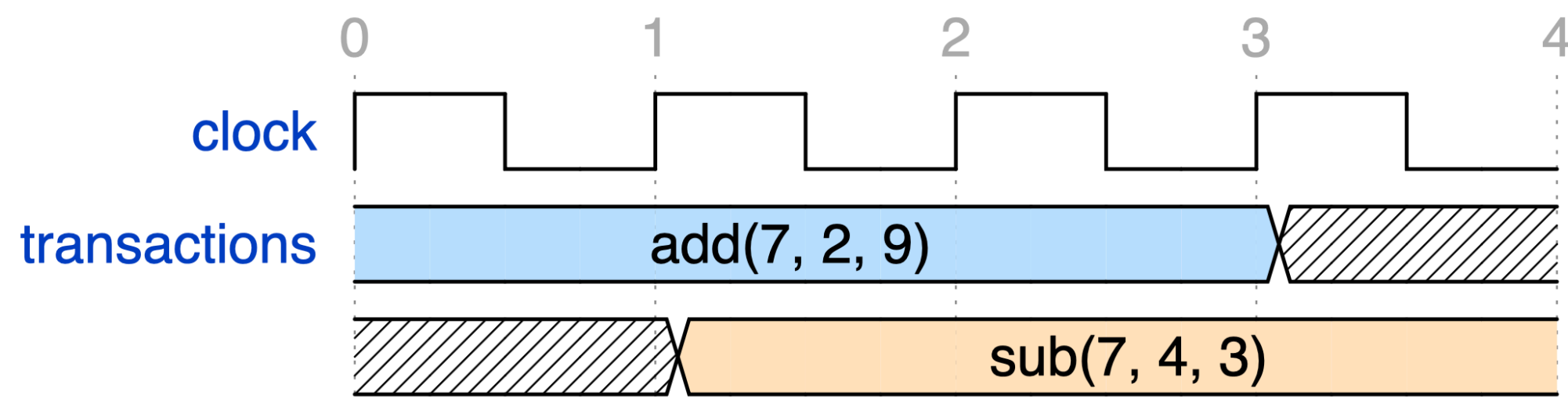


Inferred transaction trace:

`add(7, 2, 9); // cycles 0-3`

`sub(7, 4, 3); // cycles 1-4`

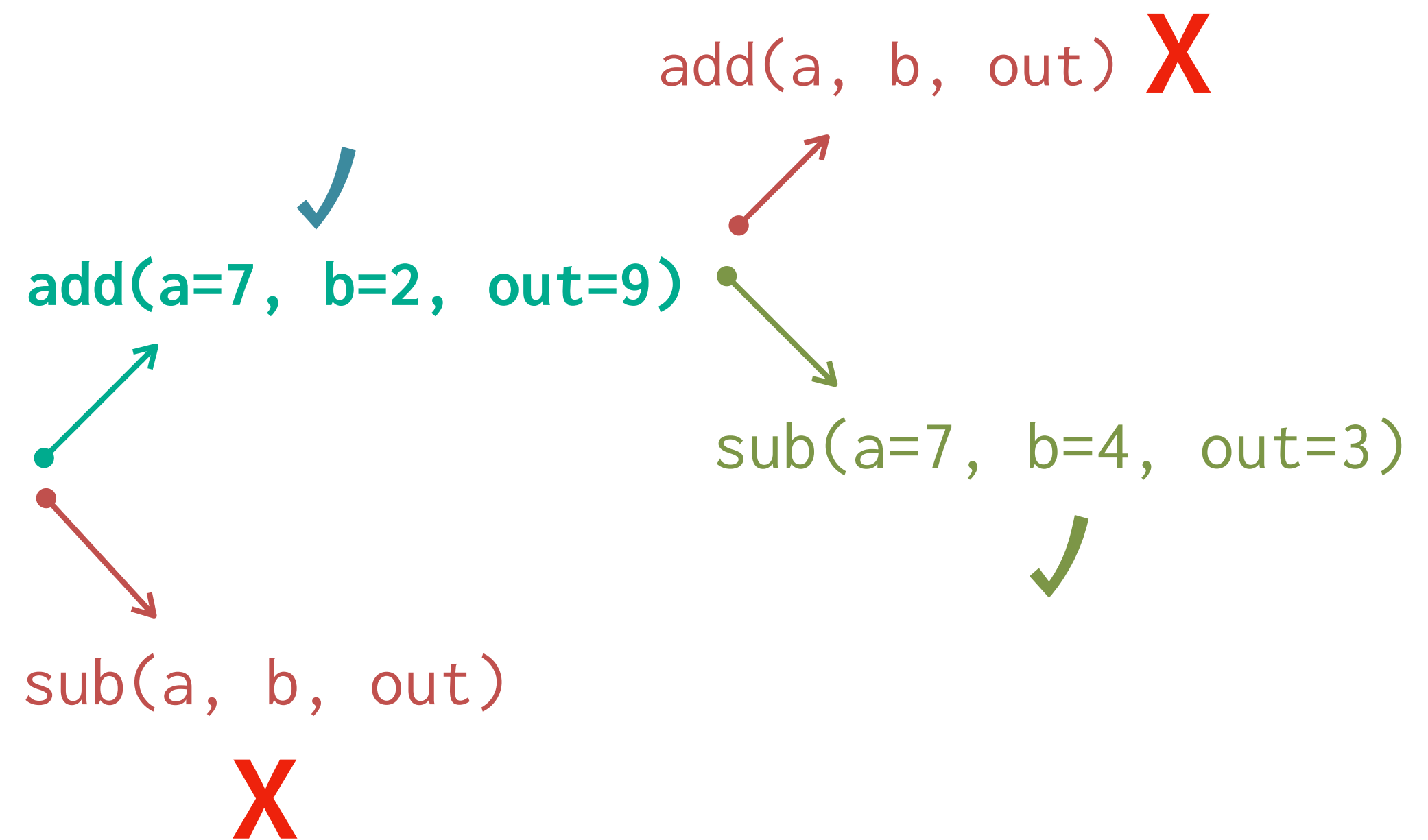




Inferred transaction trace:

`add(7, 2, 9); // cycles 0-3`

`sub(7, 4, 3); // cycles 1-4`



# Ongoing Evaluation

**Goal:** demonstrate that the same Paso spec can be used to both drive designs and infer transactions

# Ongoing Evaluation

**Goal:** demonstrate that the same Paso spec can be used to both drive designs and infer transactions

## AXI-Stream

Ready-valid interface for  
stream processing  
(e.g. for packets)

The ARM logo, consisting of the lowercase letters 'arm' in a bold, blue, sans-serif font.

# Ongoing Evaluation

**Goal:** demonstrate that the same Paso spec can be used to both drive designs and infer transactions

## AXI-Stream

Ready-valid interface for stream processing (e.g. for packets)



## Wishbone

Open-source interconnect for on-chip communication



# Protocol bugs in the wild

**Goal:** demonstrate that the reconstructor can facilitate waveform debugging

Benchmark suite of bugs in  
open-source FPGA designs  
(we focus on protocol violation bugs)

## Debugging in the Brave New World of Reconfigurable Hardware

Jiacheng Ma  
University of Michigan

Gefei Zuo  
University of Michigan

Kevin Loughlin  
University of Michigan

Haoyang Zhang  
University of Michigan

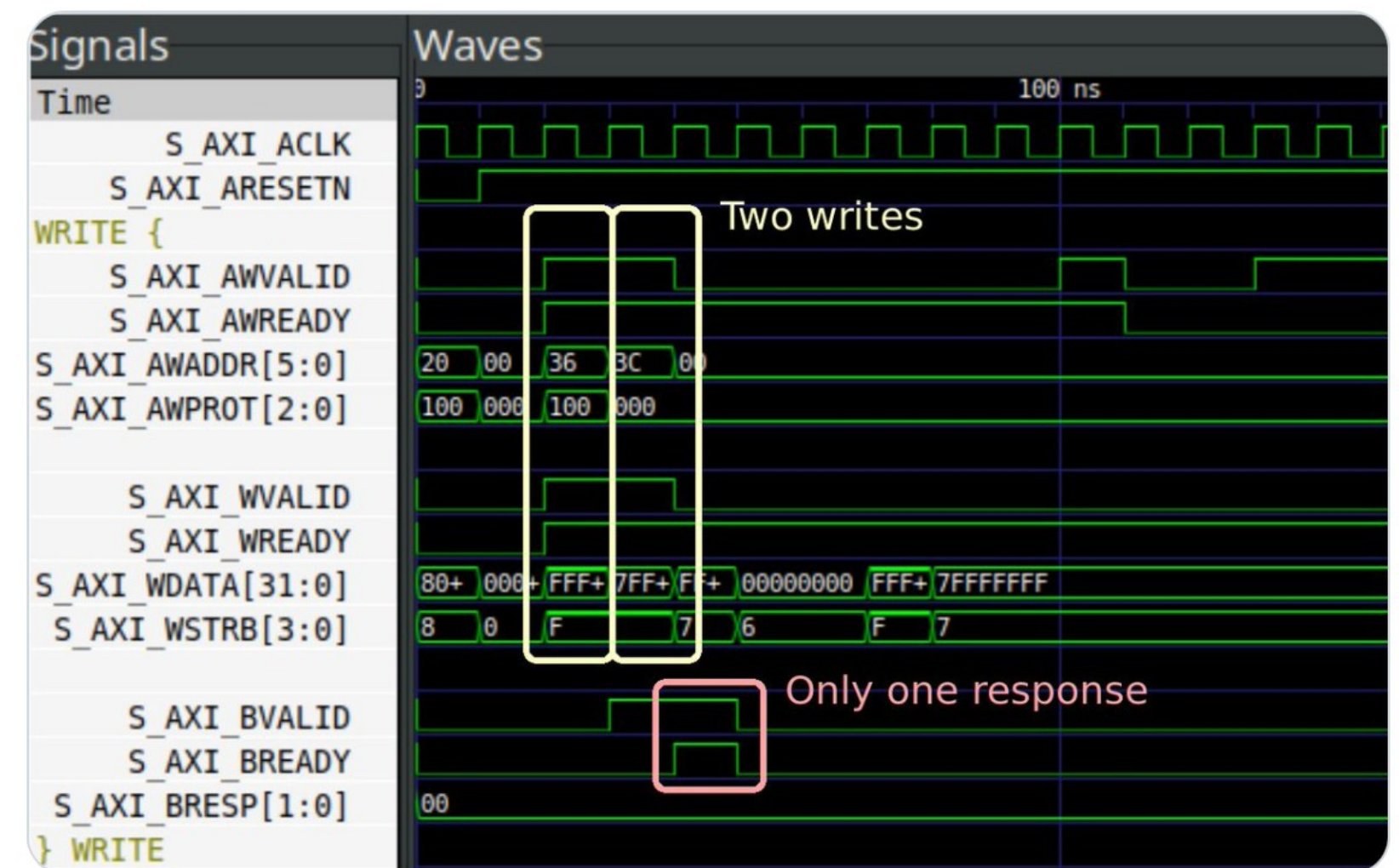
Andrew Quinn  
University of California, Santa Cruz

Baris Kasikci  
University of Michigan

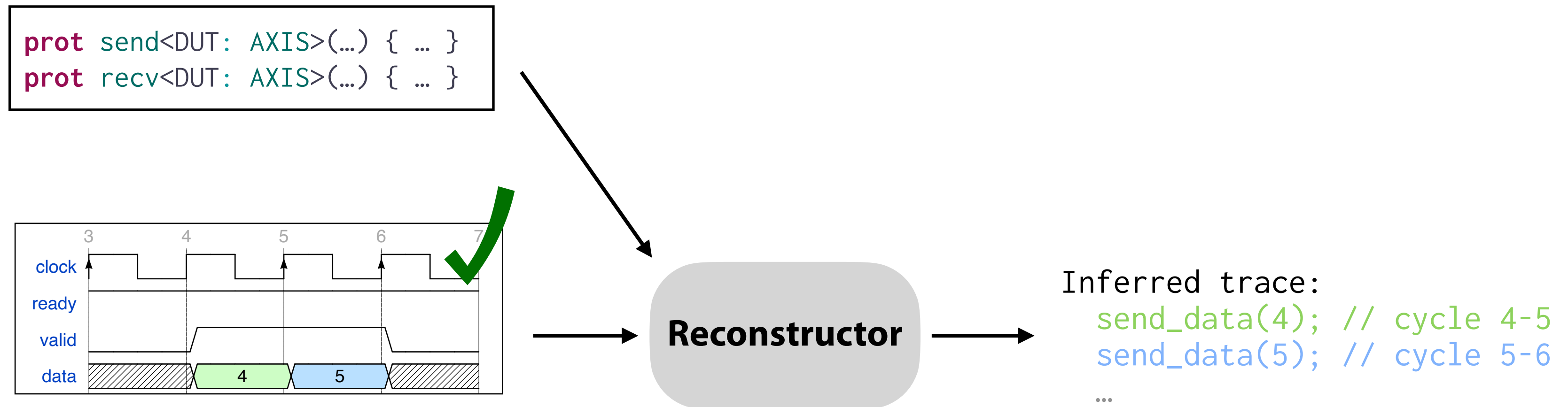
ASPLOS'22



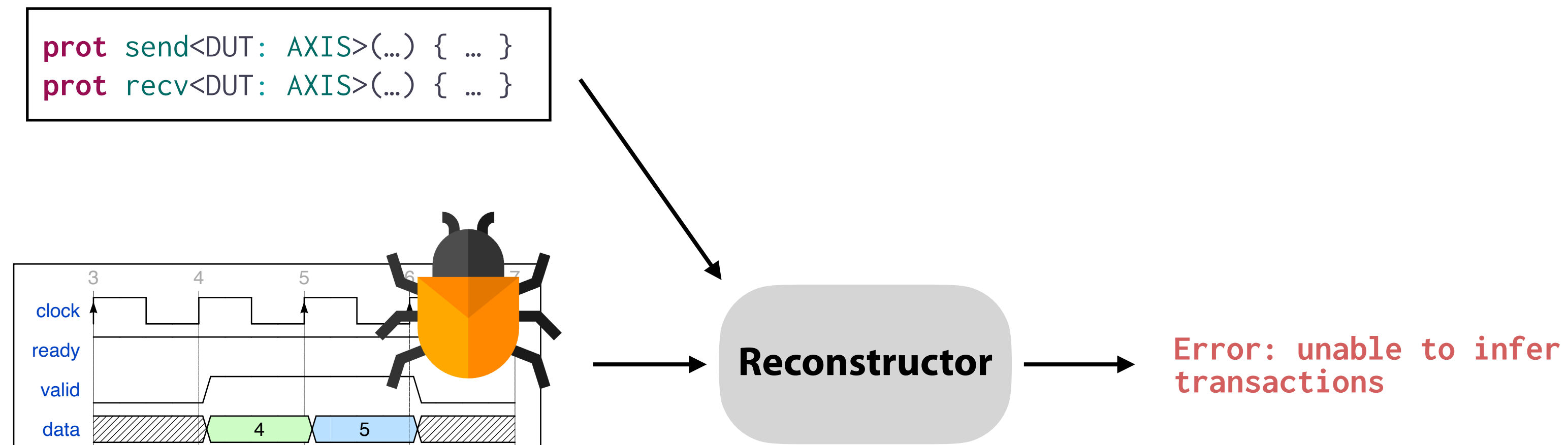
Here's a trace from the Xilinx's "new" AXI-Lite template, as generated by Vivado 2024.2.



# Protocol bugs in the wild



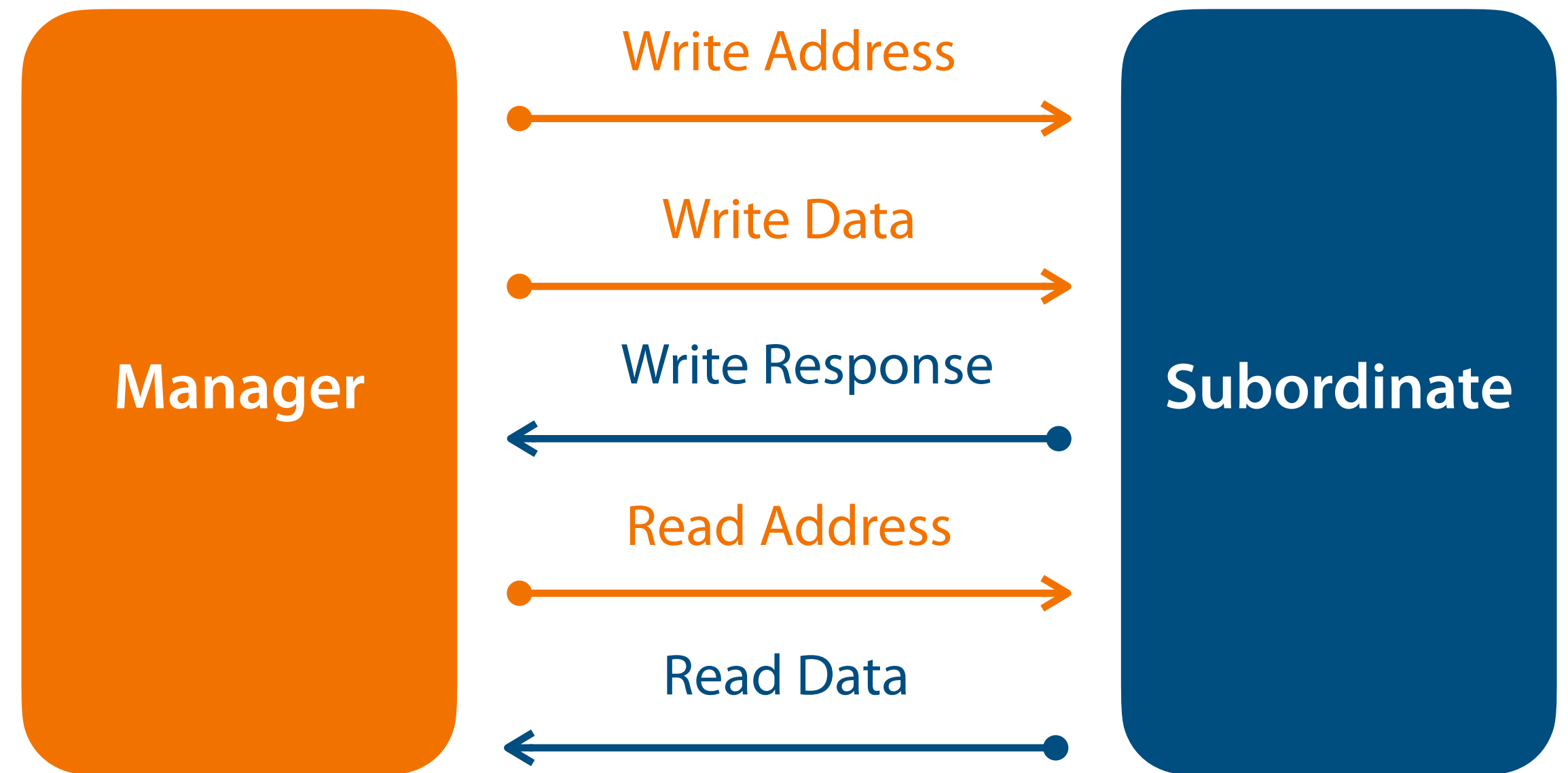
# Protocol bugs in the wild



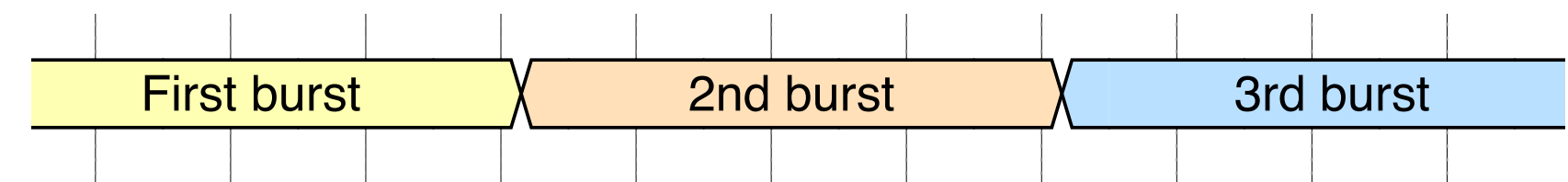
# (Near-Term) Future Work

## Support full AXI / AXI-Lite

Involves reasoning about dependencies between transactions on different channels



## Support Wishbone's Burst Mode



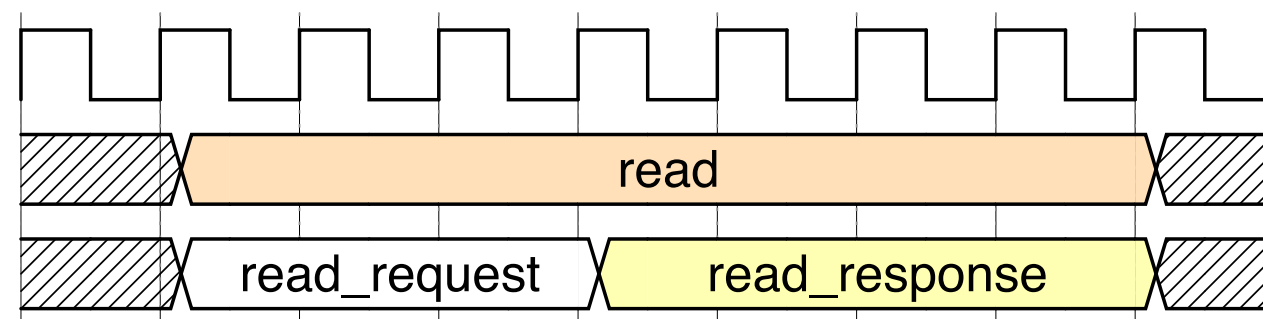
Involves extending our DSL with a sequence data type + associated operations (e.g. iteration)

# Future work: *Transaction-level* visualizations

# Future work: *Transaction-level* visualizations

## **1. Augmented waveform viewer**

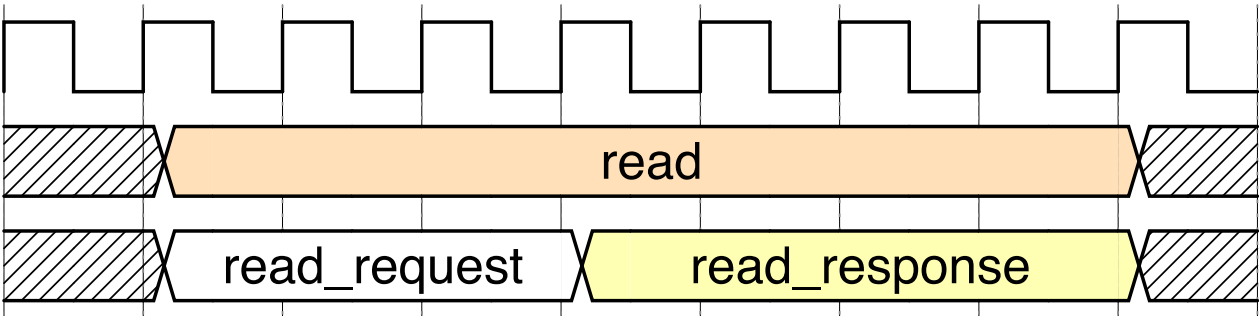
Visualize inferred transactions in the  
Surfer waveform viewer  
[Skarman et al. CAV '25]



# Future work: *Transaction-level* visualizations

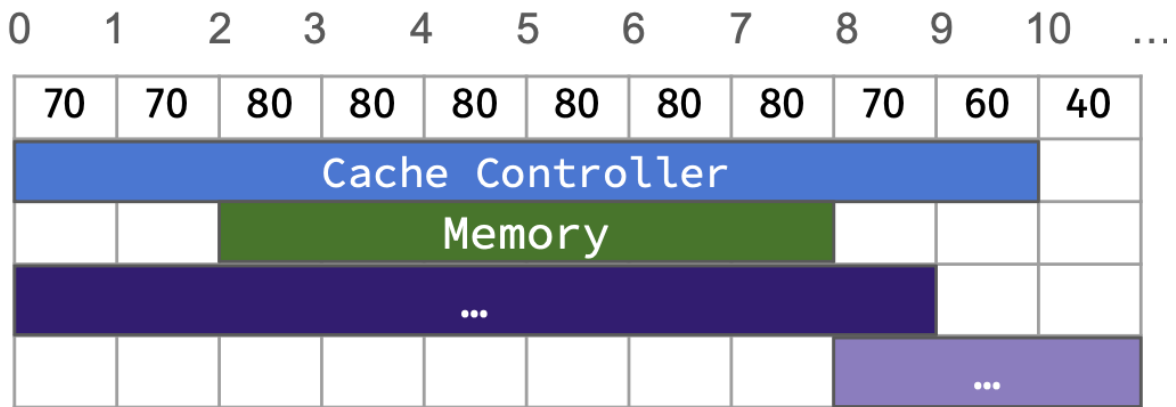
## 1. Augmented waveform viewer

Visualize inferred transactions in the Surfer waveform viewer  
[Skarman et al. CAV '25]



## 2. Transaction profiler

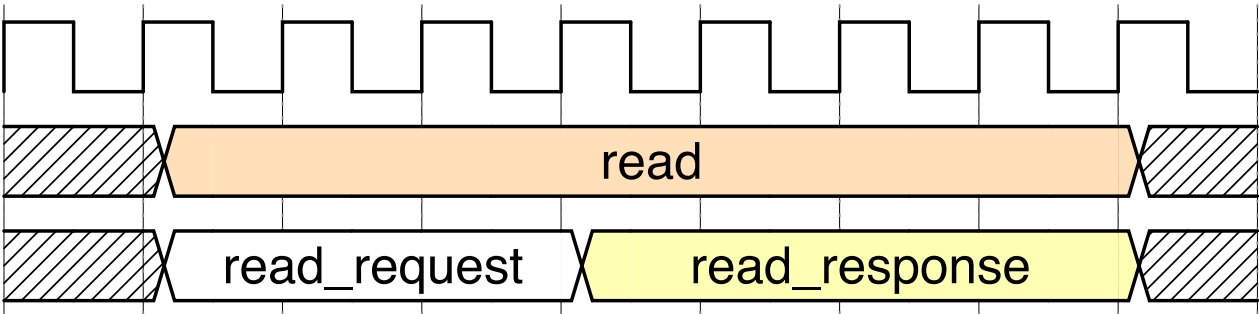
Visualizes performance bottlenecks



# Future work: *Transaction-level* visualizations

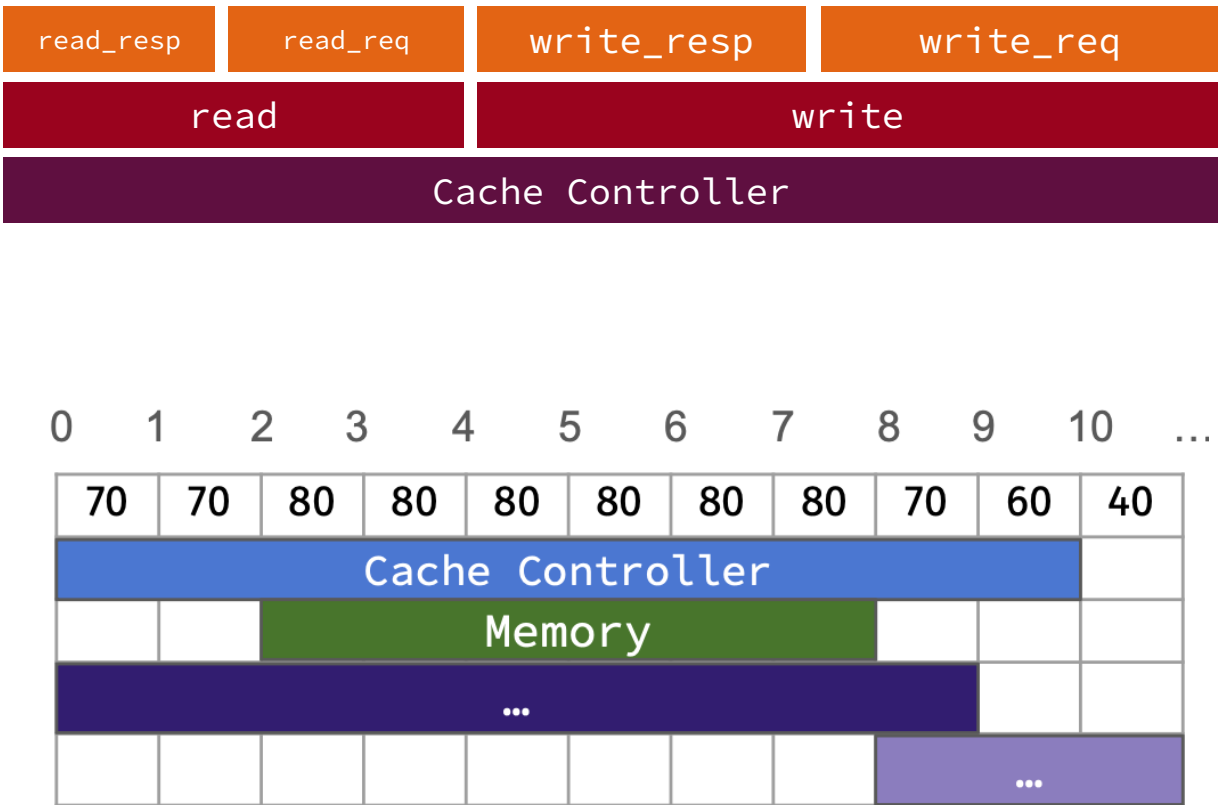
## 1. Augmented waveform viewer

Visualize inferred transactions in the Surfer waveform viewer  
[Skarman et al. CAV '25]



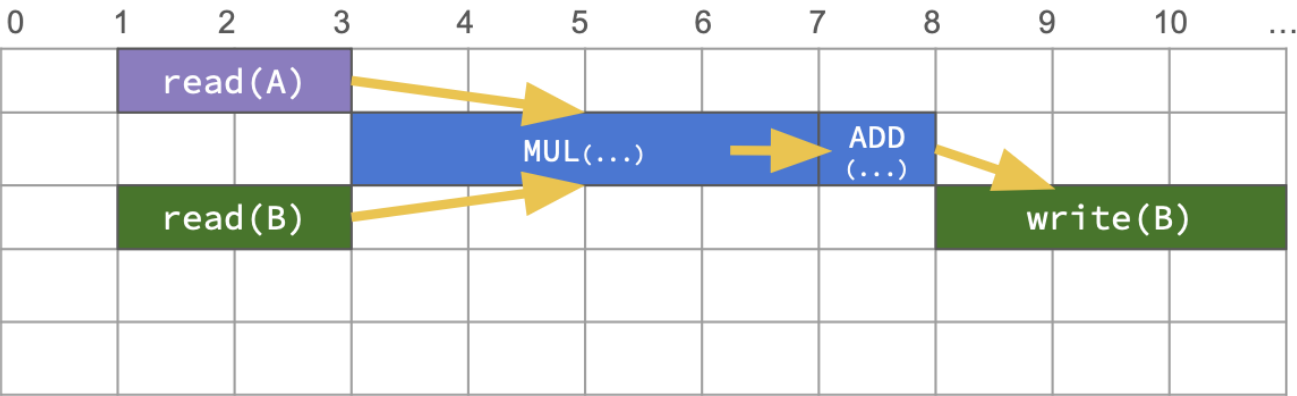
## 2. Transaction profiler

Visualizes performance bottlenecks



## 3. Dependency tracker

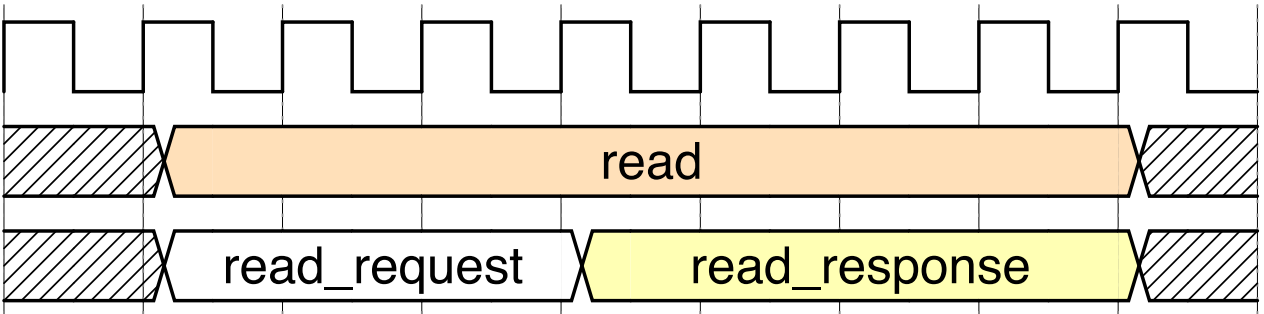
Visualizes & monitors inter-transaction dependencies



# Future work: *Transaction-level* visualizations

## 1. Augmented waveform viewer

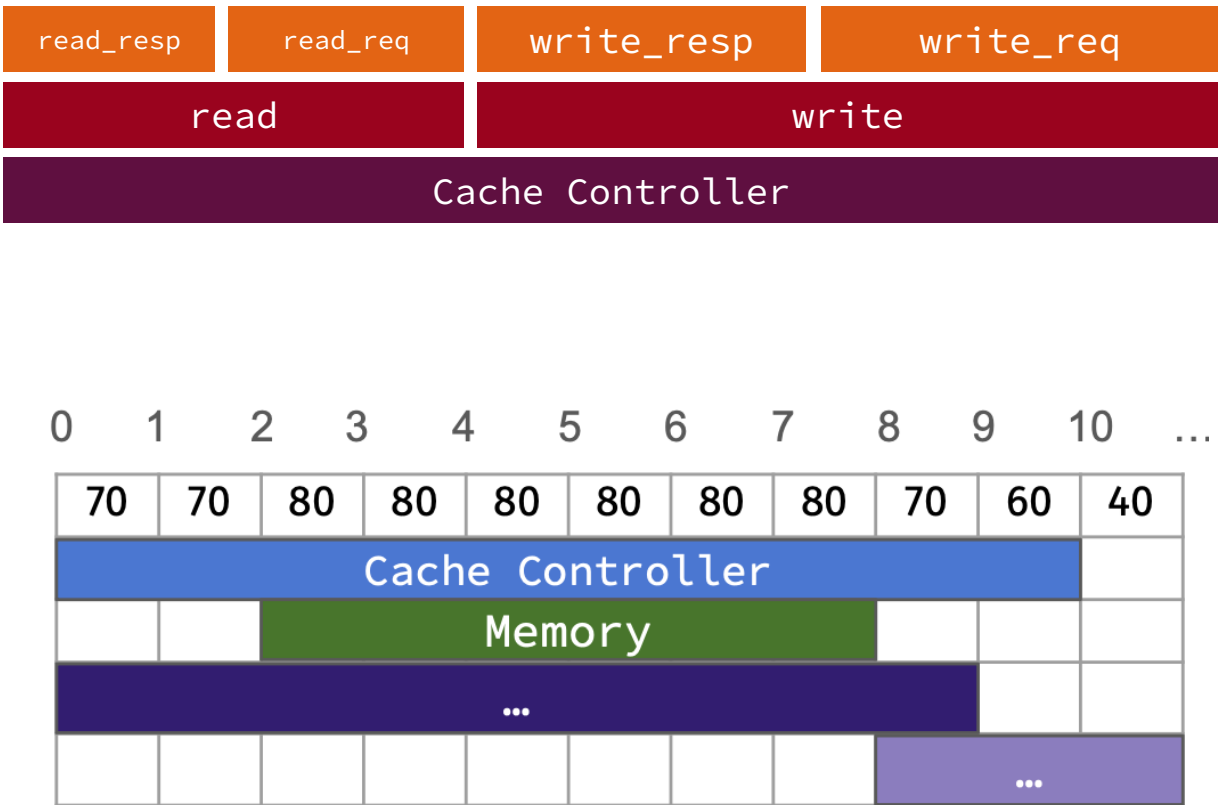
Visualize inferred transactions in the Surfer waveform viewer  
[Skarman et al. CAV '25]



Provides engineers with high-level transaction info while integrating with their existing workflows

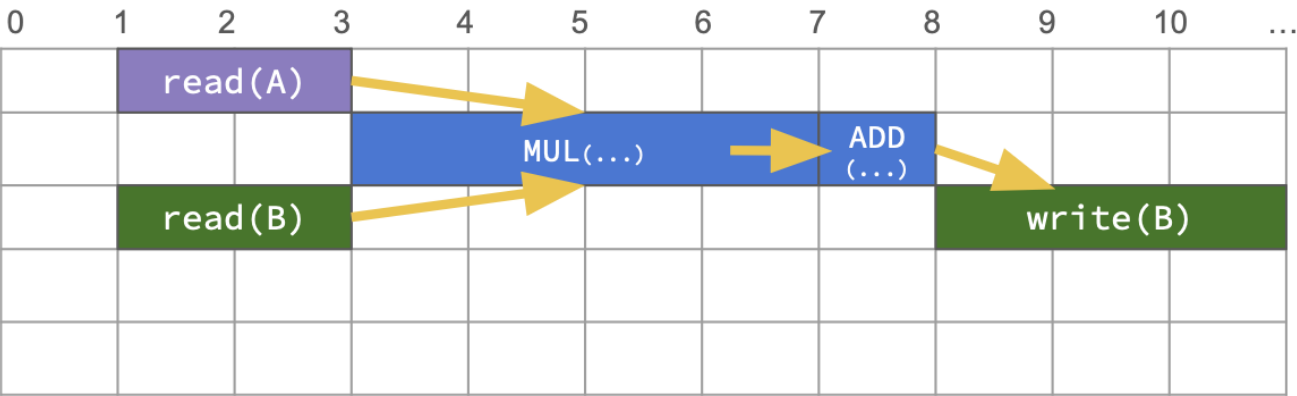
## 2. Transaction profiler

Visualizes performance bottlenecks



## 3. Dependency tracker

Visualizes & monitors inter-transaction dependencies

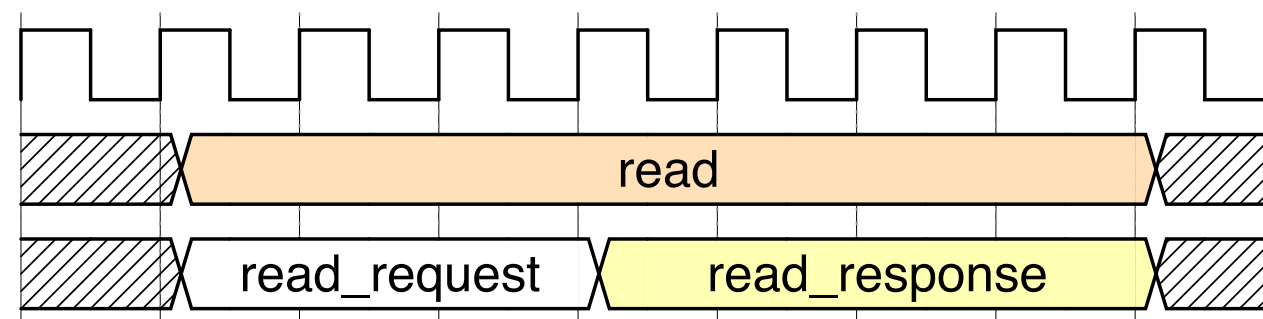


# Future work: *Transaction-level* visualizations

## 1. Augmented waveform viewer

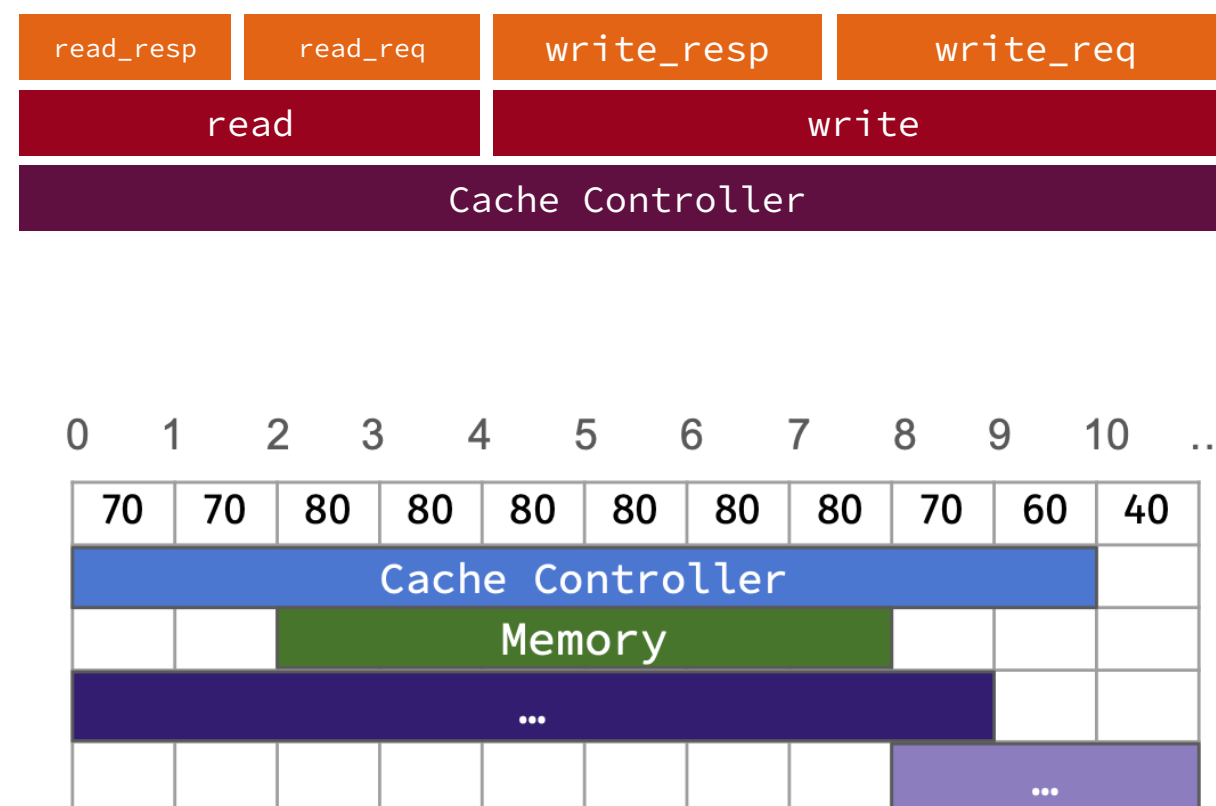
Visualize inferred transactions in the Surfer waveform viewer

[Skarman et al. CAV '25]



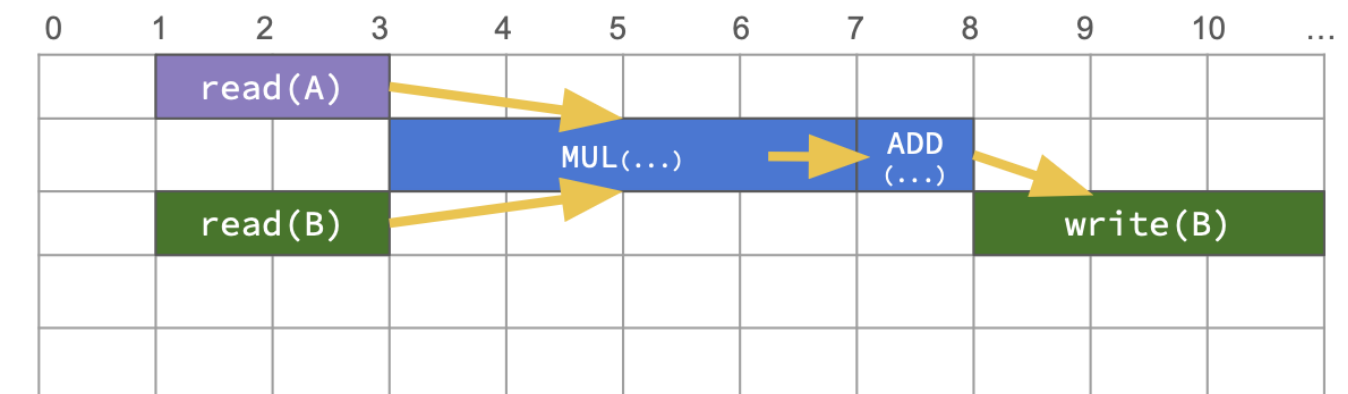
## 2. Transaction profiler

Visualizes performance bottlenecks



## 3. Dependency tracker

Visualizes & monitors inter-transaction dependencies



Provides engineers with high-level transaction info while integrating with their existing workflows

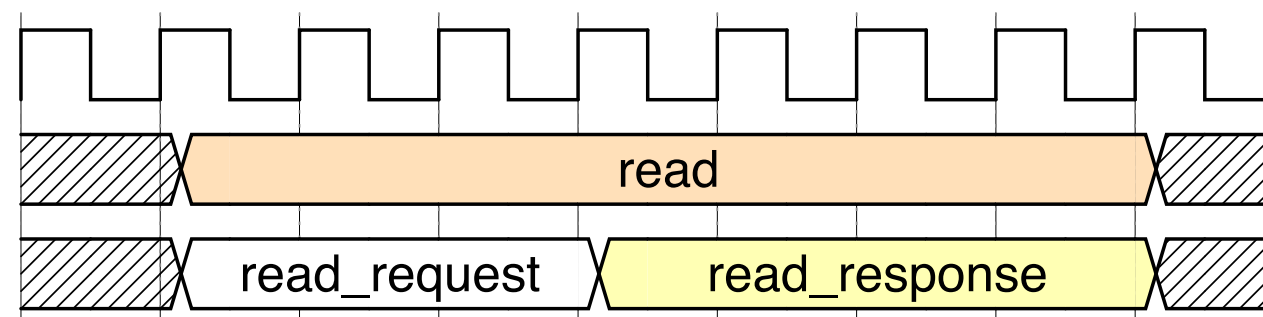
Applicable at any level of hardware description (RTL, HLS)

# Future work: *Transaction-level* visualizations

## 1. Augmented waveform viewer

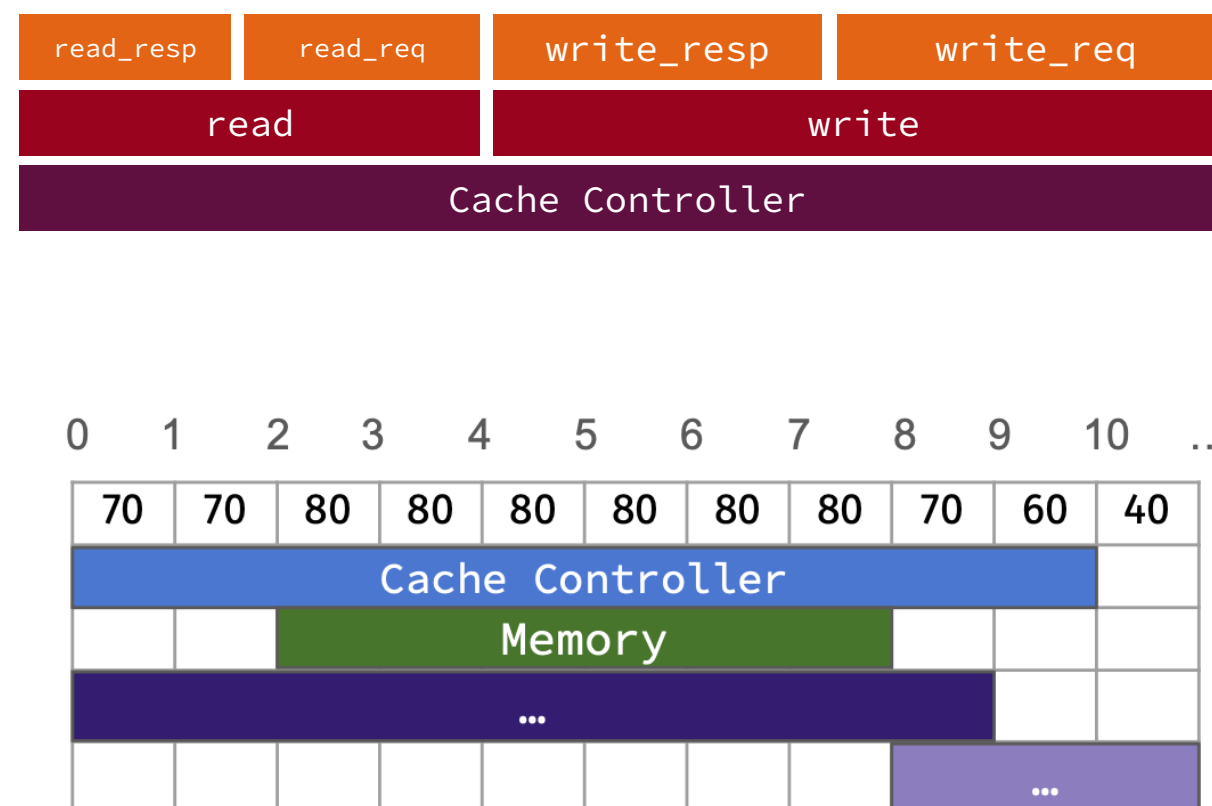
Visualize inferred transactions in the Surfer waveform viewer

[Skarman et al. CAV '25]



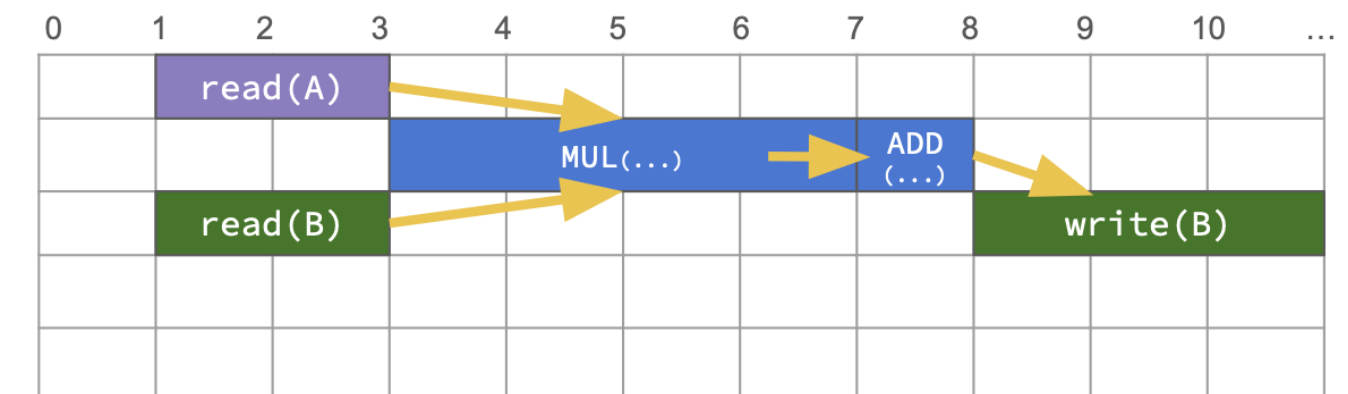
## 2. Transaction profiler

Visualizes performance bottlenecks



## 3. Dependency tracker

Visualizes & monitors inter-transaction dependencies



Provides engineers with high-level transaction info while integrating with their existing workflows

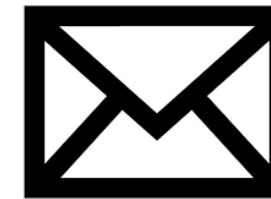
Applicable at any level of hardware description (RTL, HLS)

Assists engineers with dependency tracking at a higher abstraction level

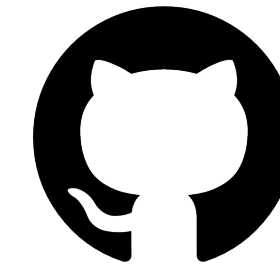
**Conclusion:** Hardware communication protocols can be specified as programs!

**Conclusion:** Hardware communication protocols can be specified as programs!  
Using our DSL, the same protocol spec can be used to both run tests & infer transactions.

ernest  
@cs.cornell.edu



cucapra/protocols



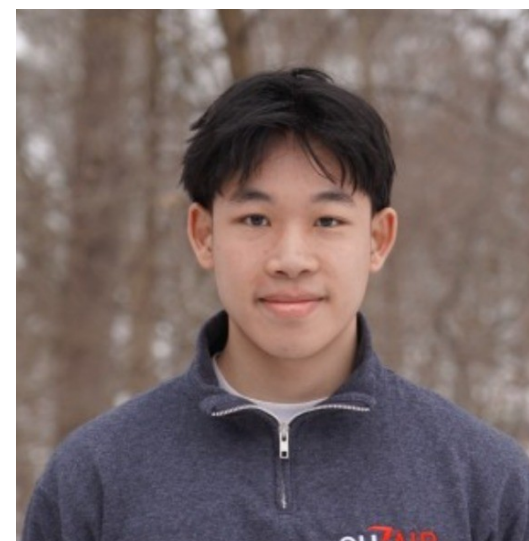
**Conclusion:** Hardware communication protocols can be specified as programs!  
Using our DSL, the same protocol spec can be used to both run tests & infer transactions.



Ernest  
Ng



Nikil  
Shyamsunder



Francis  
Pham



Adrian  
Sampson



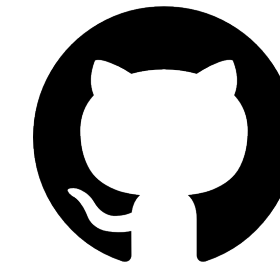
Kevin  
Laeufer

# Thanks!

ernest  
@cs.cornell.edu



cucapra/protocols



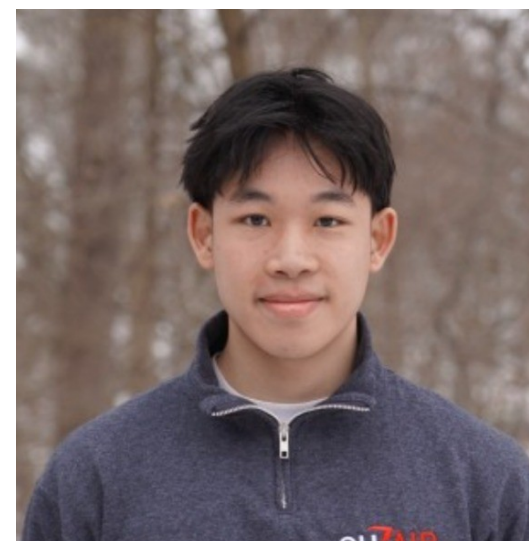
**Conclusion:** Hardware communication protocols can be specified as programs!  
Using our DSL, the same protocol spec can be used to both run tests & infer transactions.



Ernest  
Ng



Nikil  
Shyamsunder



Francis  
Pham



Adrian  
Sampson



Kevin  
Laeuffer